

RTL Design Sherpa

Bridge Micro-Architecture Specification 1.8

October 11, 2026

Table of Contents

1 Bridge Mas Index....31
2 Bridge Hardware Architecture Specification Index....32
2.1 Overview....32
2.2 Related Modules....33
2.3 Navigation....34
2.3.1 Front Matter....34
2.3.2 Chapter 1: Introduction....34
2.3.3 Chapter 2: System Overview....34
2.3.4 Chapter 3: Architecture....34
2.3.5 Chapter 4: Interfaces....34
2.3.6 Chapter 5: Performance....34
2.3.7 Chapter 6: Integration....35
3 Bridge: AXI4 Full Crossbar Generator - Product Requirements Document....36
3.1 Executive Summary....37
3.2 Design Philosophy....38
3.2.1 Core Principles....38
3.2.2 Architecture Philosophy....39
3.3 CRITICAL: RTL Regeneration Requirements....41
3.4 1. Product Overview....42
3.4.1 1.1 Purpose....42
3.4.2 1.2 Target Audience....42
3.4.3 1.3 Success Criteria....42

3.4.4 1.4 Implementation Status and Phases....43
3.5 2. Architecture Overview....45
3.5.1 2.1 AXI4 vs AXIS vs APB....45
3.5.2 2.2 Block Diagram....46
3.5.3 2.3 Key Components....47
3.6 3. Functional Requirements....48
3.6.1 3.1 AXI4 Protocol Compliance....48
3.6.2 3.2 Address Decoding....48
3.6.3 3.3 Arbitration Strategy....49
3.6.4 3.4 Burst Handling....49
3.7 4. Non-Functional Requirements....50
3.7.1 4.1 Performance....50
3.7.2 4.2 Resource Usage (Measured)....50
3.7.3 4.3 Quality Requirements....51
3.8 5. Interface Specifications....52
3.8.1 5.1 AXI4 Master Interfaces (M × 5 channels)....52
3.8.2 5.2 AXI4 Slave Interfaces (S × 5 channels)....53
3.8.3 5.3 Configuration Is Generation-Time, Not Parameters....53
3.9 6. Performance Modeling....55
3.9.1 6.1 Analytical Model....55
3.9.2 6.2 Resource Scaling....55
3.9.3 6.3 Comparison with Other Crossbars....56
3.10 7. Generator Architecture....57
3.10.1 7.1 Python Generator Structure....57

3.10.2 7.2 Address Map Configuration....58
3.11 8. Comparison with APB and Delta....60
3.11.1 8.1 Code Reuse from APB Generator....60
3.11.2 8.2 Code Reuse from Delta Generator....60
3.11.3 8.3 Complexity Comparison....60
3.12 9. Use Cases....62
3.12.1 9.1 Multi-Core Processor Interconnect....62
3.12.2 9.2 DMA + Accelerator System....62
3.12.3 9.3 FPGA System Integration....62
3.13 10. Testing Strategy....63
3.13.1 10.1 FUB (Functional Unit Block) Tests....63
3.13.2 10.2 Integration Tests....63
3.13.3 10.3 Performance Validation....63
3.14 11. Documentation Plan....64
3.14.1 11.1 Specifications (Before Code)....64
3.14.2 11.2 Performance Analysis (Before Implementation)....64
3.14.3 11.3 Code Generation....64
3.14.4 11.4 Verification....64
3.15 12. Success Metrics....65
3.15.1 12.1 Functional Completeness....65
3.15.2 12.2 Performance Targets....65
3.15.3 12.3 Educational Value....65
3.15.4 12.4 Reusability....65
3.16 12.2 PDF Generation Location....66

3.17 13. Future Enhancements....67
3.17.1 13.1 Short-Term (Post-Initial Release)....67
3.17.2 13.2 Long-Term....67
3.18 14. Risk Assessment....68
3.19 15. Project Timeline (Estimated)....69
3.20 16. Glossary....70
3.21 References....71
4 Claude Code Guide: Bridge Subsystem....73
4.1 CRITICAL: Read Architecture Documents First....74
4.2 Quick Context....75
4.3 Target Architecture: Intelligent Width-Aware Routing....76
4.3.1 Efficient Multi-Width Design....76
4.3.2 Why This Architecture....76
4.3.3 Per-Master Output Paths....76
4.3.4 Benefits....77
4.3.5 Implementation Status....77
4.4 CRITICAL RULE #0: RTL Regeneration Requirements....78
4.5 Critical Pitfalls: slave_select_* Reversion After Handshake....79
4.5.1 The Problem: Combinational Address Decode....79
4.5.2 Visible Symptoms....79
4.5.3 The Solution (MANDATORY in All Generators)....79
4.5.4 When This Bug Appears....80
4.6 MANDATORY: Project Organization Pattern....81
4.6.1 Required Directory Structure....81

4.6.2 Testbench Class Location (MANDATORY)....82
4.6.3 Test File Pattern (MANDATORY)....82
4.7 Critical Rules for This Subsystem....83
4.7.1 Rule #0.1: Testbench Architecture - MANDATORY SEPARATION....83
4.7.2 Rule #1: TBBase and the three mandatory methods....83
4.7.3 Rule #3: Use GAXI Components for Protocol Handling....83
4.7.4 Rule #4: Queue-Based Verification....84
4.8 TOML/CSV-Based Bridge Generator (Phase 2 Complete)....85
4.8.1 Overview....85
4.8.2 Quick Start....85
4.8.3 Configuration Format Details....86
4.8.4 Channel-Specific Masters (Phase 2 Feature)....87
4.8.5 Example: RAPIDS-Style Configuration....87
4.8.6 Common User Questions....88
4.8.7 Generator Output Structure....91
4.8.8 Testing Generated Bridges....91
4.9 Bridge Architecture Quick Reference....93
4.9.1 Generated Bridge Crossbar Structure....93
4.9.2 Key Features....93
4.9.3 Address Map....93
4.10 Test Organization....95
4.10.1 Test Levels....95
4.11 Common User Questions and Responses....96
4.11.1 Q: “How does the Bridge work?”....96

4.11.2 Q: “How do I generate a Bridge?” 97
4.11.3 Q: “How do I test a Bridge?” 97
4.12 PDF Generation Location 98
5 Document Information 99
5.1 Overview 99
5.1.1 Bridge Micro-Architecture Specification 99
5.1.2 Document Purpose 99
5.1.3 Intended Audience 99
5.2 Design Notes 100
5.2.1 Revision History 100
5.3 References 106
5.3.1 Related Documents 106
6 Overview 107
6.1 Overview 107
6.1.1 Bridge Micro-Architecture 107
6.1.2 Multi-Protocol Support 107
6.1.3 Channel-Specific Masters 107
6.1.4 Automatic Converters 107
6.2 Functional Description 109
6.2.1 Block Organization 109
6.3 References 110
6.3.1 Related Documentation 110
6.4 Navigation 111
6.4.1 Document Organization 111

7 2.1 Master Adapter....112
7.1 Overview....112
7.1.1 Purpose and Function....112
7.1.2 Figure 2.1: Master Adapter Architecture....112
7.2 Parameters....113
7.2.1 Per-Master Parameters (from TOML/CSV)....113
7.2.2 Global Parameters (affect all adapters)....113
7.3 Ports....114
7.3.1 External Master Interface (per master)....114
7.3.2 Internal Crossbar Interface (per master)....114
7.4 Functional Description....115
7.4.1 Channel Specialization....115
7.4.2 Per-Port Monitor Wrappers (when use_monitor = true)....115
7.4.3 Skid Buffer Architecture....116
7.4.4 Bridge ID Management....117
7.4.5 Protocol Normalization....118
7.4.6 Response Path Slave Selection Tracking....118
7.5 Timing....120
7.5.1 Latency....120
7.5.2 Throughput....120
7.5.3 Critical Paths....120
7.6 Design Notes....121
7.6.1 Resource Utilization....121
7.6.2 Future Enhancements....121

7.7 Related Modules..... 122
7.8 Testing..... 123
7.8.1 Debug and Observability..... 123
8 2.2 Slave Router..... 124
8.1 Overview..... 124
8.1.1 Purpose and Function..... 124
8.1.2 Figure 2.2: Slave Router Architecture..... 124
8.2 Parameters..... 125
8.2.1 Per-Router Parameters..... 125
8.2.2 Global Parameters..... 125
8.3 Functional Description..... 126
8.3.1 Address Decoding Algorithm..... 126
8.3.2 Request Routing..... 128
8.3.3 Out-of-Range Handling..... 129
8.3.4 Default Slave Support..... 131
8.3.5 Address Aliasing..... 131
8.4 Timing..... 132
8.4.1 Decode Latency..... 132
8.4.2 Throughput..... 132
8.4.3 Critical Paths..... 132
8.5 Design Notes..... 133
8.5.1 Resource Utilization..... 133
8.5.2 Performance Optimization..... 133
8.5.3 Future Enhancements..... 134

8.6 Related Modules..... 135
8.7 Testing..... 136
8.7.1 Debug and Observability..... 136
8.7.2 Verification Considerations..... 136
9 2.3 Crossbar Core..... 138
9.1 Overview..... 139
9.1.1 Figure 2.3: Crossbar Core Architecture..... 139
9.2 Parameters..... 140
9.3 Functional Description..... 141
9.3.1 Connectivity Matrix..... 141
9.3.2 Request Path Multiplexing..... 141
9.3.3 Response Path Demultiplexing..... 141
9.3.4 Per-Slave Request Arbitration..... 141
9.3.5 Request Multiplexers with Stable Address Gating..... 142
9.3.6 AW→W Burst Tracking FIFO..... 143
9.3.7 Backpressure Propagation..... 144
9.3.8 Bridge ID Extraction..... 144
9.3.9 CAM-Based Routing (Optional)..... 144
9.3.10 Response Demultiplexers..... 145
9.3.11 Multi-Slave Response Merging..... 145
9.3.12 AXI Ordering Requirements..... 146
9.3.13 Monitor Aggregation at Bridge Top (when variants includes mon)…146
9.4 Timing..... 148
9.4.1 Latency..... 148

9.4.2 Throughput....148
9.4.3 Critical Paths....148
9.5 Design Notes....150
9.5.1 Resource Utilization....150
9.5.2 Debug and Observability....150
9.5.3 Common Issues....151
9.5.4 Future Enhancements....151
9.6 Related Modules....152
9.7 Testing....153
9.7.1 Functional Tests....153
9.7.2 Corner Cases....153
9.7.3 Protocol Compliance....153
10 2.4 Arbitration....154
10.1 Overview....155
10.2 Parameters....156
10.3 Functional Description....157
10.3.1 Per-Slave, Per-Channel Arbiters....157
10.3.2 Round-Robin Arbitration....158
10.3.3 Fixed-Priority Arbitration....159
10.3.4 Weighted Arbitration....160
10.3.5 Burst Handling....161
10.4 Timing....163
10.4.1 Arbitration Latency....163
10.4.2 Critical Paths....163

10.5 Design Notes....164
10.5.1 Resource Utilization....164
10.5.2 Performance Impact....164
10.5.3 Debug and Observability....165
10.5.4 Common Issues....165
10.5.5 Future Enhancements....166
10.6 Related Modules....167
10.7 Testing....168
10.7.1 Test Scenarios....168
10.7.2 Corner Cases....168
11 2.5 ID Management....169
11.1 Overview....170
11.1.1 Figure 2.5: ID Management Architecture....171
11.2 Parameters....172
11.3 Functional Description....173
11.3.1 Bridge ID Concept....173
11.3.2 Bridge ID Width Calculation....173
11.3.3 Multi-Master OR-Merge Gating....174
11.3.4 ID Injection (Request Path)....174
11.3.5 ID Extraction (Response Path)....175
11.3.6 Content Addressable Memory (CAM)....176
11.3.7 CAM Implementation....177
11.3.8 Outstanding Transaction Limits....178
11.4 Timing....179

11.4.1 ID Injection Latency....179
11.4.2 ID Extraction/Routing Latency....179
11.4.3 End-to-End Impact....179
11.5 Design Notes....180
11.5.1 Resource Utilization....180
11.5.2 Debug and Observability....180
11.5.3 Common Issues....181
11.5.4 Future Enhancements....181
11.6 Related Modules....182
11.7 Testing....183
11.7.1 Test Scenarios....183
12 2.6 Width Conversion....184
12.1 2.6.1 Purpose and Function....185
12.2 2.6.2 Internal Data Width....186
12.2.1 64-Bit Standard....186
12.2.2 Width Conversion Locations....186
12.3 2.6.3 Block Diagram....187
12.3.1 Figure 2.6: Width Conversion Architecture....187
12.4 2.6.4 Upsizing (Narrow to Wide)....188
12.4.1 Overview....188
12.4.2 Write Upsizing....188
12.4.3 Read Upsizing....188
12.4.4 Strobe Mapping (Upsizing)....189
12.5 2.6.5 Downsizing (Wide to Narrow)....190

12.5.1 Overview....190
12.5.2 Write Downsizing....190
12.5.3 Read Downsizing....190
12.5.4 Strobe Mapping (Downsizing)....191
12.6 2.6.6 Address Alignment....192
12.6.1 Address Adjustment for Width....192
12.6.2 Unaligned Access Handling....192
12.7 2.6.7 Burst Length Adjustment....193
12.7.1 Length Calculation....193
12.7.2 Odd Burst Lengths....193
12.8 2.6.8 Resource Utilization....194
12.8.1 Per-Converter Resources....194
12.8.2 Scaling....194
12.9 2.6.9 Timing Characteristics....195
12.9.1 Latency....195
12.9.2 Throughput....195
12.10 2.6.10 Configuration Parameters....196
12.10.1 Width Conversion Configuration (TOML)....196
12.11 2.6.11 Debug and Observability....197
12.11.1 Recommended Debug Signals....197
12.11.2 Common Issues and Debug....197
12.12 2.6.12 Verification Considerations....198
12.12.1 Test Scenarios....198
12.13 2.6.13 Performance Considerations....199

12.13.1 Conversion Overhead....199
12.13.2 Optimization Strategies....199
12.14 2.6.14 Future Enhancements....200
12.14.1 Planned Features....200
12.14.2 Under Consideration....200
13 2.7 Protocol Conversion....201
13.1 2.7.1 Purpose and Function....202
13.2 2.7.2 Supported Protocols....203
13.2.1 Current Support (Phase 2)....203
13.2.2 Current Limitation: AXI4-Lite Conversion....203
13.2.3 Future Support (Phase 2+)....204
13.3 2.7.3 Conversion Architecture Overview....205
13.3.1 Figure 2.7.1: Protocol Conversion Architecture....205
13.4 2.7.4 Block Diagram....206
13.4.1 Figure 2.7: Protocol Conversion Architecture....206
13.5 2.7.5 AXI4-Lite to AXI4 Conversion (Master-Side)....207
13.5.1 AXI4-Lite Protocol Overview....207
13.5.2 Conversion Requirements....207
13.5.3 Signal Mapping....207
13.5.4 Implementation....209
13.5.5 Resource Utilization....211
13.5.6 Performance Impact....211
13.5.7 Configuration....212
13.5.8 Common Issues and Debug....212

13.6 2.7.6 AXI4 to APB Conversion (Slave-Side) 213
13.6.1 APB Protocol Overview 213
13.6.2 APB Signals 213
13.6.3 APB State Machine 213
13.6.4 Read Transaction Conversion 214
13.6.5 Write Transaction Conversion 214
13.6.6 Burst Handling 215
13.6.7 Response Mapping 215
13.7 2.7.7 Implementation 216
13.7.1 AXI4-to-APB Converter FSM 216
13.7.2 Address Generation 217
13.8 2.7.8 Resource Utilization 218
13.8.1 APB Converter Resources 218
13.8.2 Scaling 218
13.9 2.7.9 Timing Characteristics 219
13.9.1 Latency 219
13.9.2 Throughput 219
13.10 2.7.10 Configuration Parameters 220
13.10.1 Protocol Conversion Configuration (TOML) 220
13.11 2.7.11 Debug and Observability 221
13.11.1 Recommended Debug Signals 221
13.11.2 Common Issues and Debug 221
13.12 2.7.12 Verification Considerations 222
13.12.1 Test Scenarios 222

13.13 2.7.13 Performance Considerations....223
13.13.1 When to Use APB....223
13.13.2 APB vs. AXI4 Comparison....223
13.14 2.7.14 Mixed Protocol Bridges....224
13.14.1 Example Configuration....224
13.14.2 Routing Optimization....224
13.15 2.7.15 Master-Side vs Slave-Side Conversion....225
13.15.1 Comparison....225
13.15.2 When to Use Each....225
13.16 2.7.16 Generator-Emitted Conversion Shims....226
13.16.1 AXI4 to AXI4-Lite Conversion....226
13.16.2 AXI4 to APB Conversion....226
13.16.3 Master-Side APB Front End....226
13.16.4 Wishbone B4 at Either Boundary....227
13.16.5 AXI3 Boundary Conversion....227
13.16.6 Master-Side AXI5-Lite Sideband....228
13.16.7 Master-Side AXIL→Wider-Slave Alignment....228
13.17 2.7.17 Future Protocol Support....230
13.17.1 Planned Features....230
13.17.2 Under Consideration....230
13.18 2.7.18 Best Practices....231
13.18.1 Design Recommendations....231
13.18.2 Performance Tips....231
14 2.8 Response Routing....232

14.1 2.8.1 Purpose and Function....233
14.2 2.8.2 Block Diagram....234
14.2.1 Figure 2.8: Response Routing Architecture....234
14.3 2.8.3 Stable Response Routing via FIFO-Tracked Slave Select....235
14.3.1 Architecture: FIFO-Tracked Master Routing....235
14.3.2 BID Extraction....235
14.3.3 CAM-Based Extraction....236
14.4 2.8.4 Response Demultiplexing....237
14.4.1 Per-Master Response Paths....237
14.4.2 Demux Implementation....237
14.5 2.8.5 Multi-Slave Response Merging....238
14.5.1 Problem Statement....238
14.5.2 Response Arbitration....238
14.5.3 Response Buffering....239
14.6 2.8.6 Independent Write-Tracker FIFO (W-Channel Deadlock Fix)....240
14.7 2.8.7 Backpressure Management....241
14.7.1 Ready Signal Routing....241
14.7.2 Stall Conditions....241
14.8 2.8.8 Response Path Latency....242
14.8.1 End-to-End R Channel....242
14.8.2 End-to-End B Channel....242
14.9 2.8.9 Error Response Routing....243
14.9.1 Slave Error Responses....243
14.9.2 Out-of-Range Error Responses....243

14.10 2.8.10 Resource Utilization....244
14.10.1 Response Router Resources....244
14.10.2 Scaling....244
14.11 2.8.11 Configuration Parameters....245
14.11.1 Response Routing Configuration (TOML)....245
14.12 2.8.12 Debug and Observability....246
14.12.1 Recommended Debug Signals....246
14.12.2 Performance Counters....246
14.13 2.8.13 Common Issues and Debug....247
14.14 2.8.14 Verification Considerations....248
14.14.1 Test Scenarios....248
14.15 2.8.15 Performance Optimization....249
14.15.1 Techniques....249
14.16 2.8.16 Advanced Features....250
14.16.1 Response Reordering (Future)....250
14.16.2 Response Coalescing (Future)....250
14.16.3 Response QoS (Future)....250
14.17 2.8.17 Timing Considerations....251
14.17.1 Critical Paths....251
14.17.2 Optimization Strategies....251
14.18 2.8.18 Future Enhancements....252
14.18.1 Planned Features....252
14.18.2 Under Consideration....252
15 2.9 Error Handling....253

15.1 2.9.1 Purpose and Function....254
15.2 2.9.2 Error Categories....255
15.2.1 Transaction Errors....255
15.2.2 System Errors....255
15.3 2.9.3 Block Diagram....257
15.3.1 Figure 2.9: Error Handling Architecture....257
15.4 2.9.4 Out-of-Range Address Handling....258
15.4.1 Detection....258
15.4.2 Error Response Generation....258
15.4.3 Configurable Data Patterns....259
15.5 2.9.5 Protocol Violation Handling....260
15.5.1 Common Violations....260
15.5.2 Protocol Checker Implementation....260
15.6 2.9.6 Timeout Detection....262
15.6.1 Per-Transaction Watchdog....262
15.6.2 Timeout Configuration....263
15.7 2.9.7 Error Logging and Reporting....264
15.7.1 Error Status Register....264
15.7.2 Error History Buffer....264
15.7.3 Error Interrupts....265
15.8 2.9.8 Resource Utilization....266
15.8.1 Error Handling Resources....266
15.9 2.9.9 Configuration Parameters....267
15.9.1 Error Handling Configuration (TOML)....267

15.10 2.9.10 Debug and Observability....268
15.10.1 Recommended Debug Signals....268
15.11 2.9.11 Common Issues and Debug....269
15.12 2.9.12 Verification Considerations....270
15.12.1 Test Scenarios....270
15.13 2.9.13 Recovery Mechanisms....271
15.13.1 Automatic Recovery....271
15.13.2 Manual Recovery....271
15.14 2.9.14 Safety and Reliability....272
15.14.1 Error Containment....272
15.14.2 Fail-Safe Behavior....272
15.14.3 Error Injection (Debug/Test)....272
15.15 2.9.15 Future Enhancements....273
15.15.1 Planned Features....273
15.15.2 Under Consideration....273
16 AMBA5 Boundary and Native Sideband....274
16.1 Boundary Wrappers....275
16.2 Native Sideband Through the Structs (A5-2)....276
16.2.1 Data-scaled fields: Memory Tagging and chunking (bridge TASK-007, was BRIDGE-018)....276
16.3 Connectivity Gating (poison, atomic, mte) and Droppable Chunking....277
16.4 Atomic Filter (A5-3a, write-only masters)....278
16.5 Read-Return Atomics (A5-3b, rw masters)....279
16.6 APB5 Slaves (A5-3c)....281

16.7 Lite and APB Requesters (bridge TASK-004, was BRIDGE-014)....282
16.8 Verification Anchors....283
17 Bridge Arbiter Finite State Machines....285
17.1 Overview....285
17.2 AW Channel Arbiter FSM....286
17.2.1 Figure 5.3: AW Arbiter FSM....286
17.3 AR Channel Arbiter FSM....288
17.3.1 Figure 5.4: AR Arbiter FSM....288
17.4 Round-Robin Arbitration Algorithm....290
17.5 Grant Locking Mechanism....291
17.6 Independent Read/Write Arbitration....293
17.7 FSM Instance Breakdown by Configuration....294
17.8 Performance Characteristics....295
17.9 Comparison with Other Crossbar Arbiters....296
17.10 Future Enhancements (Phase 3+)....297
17.11 Related Documentation....298
18 Transaction Tracking FSMs....299
18.1 Overview....299
18.2 Write Data Tracking FSM....300
18.2.1 Purpose....300
18.2.2 States....300
18.2.3 Figure 3.1: Write Data Tracking FSM....300
18.2.4 Implementation....300
18.3 Response Pending Counter....302

18.3.1 Purpose....302
18.3.2 Implementation....302
18.4 Burst Counter FSM....303
18.4.1 Purpose....303
18.4.2 States....303
18.4.3 Implementation....303
18.5 Related Documentation....304
19 CAM Architecture....305
19.1 Overview....305
19.2 Functional Description....306
19.2.1 What the CAM Was Meant to Do....306
19.2.2 Figure 4.1: CAM Entry Format....306
19.2.3 CAM Sizing....306
19.2.4 Example Configuration....306
19.2.5 Insertion (AR/AW Acceptance)....307
19.2.6 Lookup (R/B Response)....307
19.2.7 Duplicate IDs: the Ordering Counter (Mode 2)....307
19.2.8 Deletion (Response Complete)....308
19.2.9 Implementation Options....308
19.2.10 CAM Overflow Handling....309
19.3 Related Modules....310
20 Response Tracking....311
20.1 Overview....311
20.2 Functional Description....312

20.2.1 HISTORICAL – the unbuilt ID-table design....312
20.2.2 Not implemented: the per-slave ID table....312
20.2.3 Per-Slave ID Table (historical)....312
20.2.4 Figure 4.2: ID Table Structure....313
20.2.5 Table Sizing....313
20.2.6 ID Extension....314
20.2.7 ID Extraction....314
20.2.8 Table Operations....314
20.2.9 Multi-ID Considerations....315
20.3 Design Notes....317
20.3.1 Resource Utilization....317
20.4 Related Modules....318
21 Width Converters....319
21.1 Overview....319
21.2 Functional Description....320
21.2.1 Upsize Converter....320
21.2.2 Figure 5.1: Upsize Converter (64-bit to 512-bit)....320
21.2.3 Implementation....320
21.2.4 Burst Length Conversion....321
21.2.5 Downsize Converter....321
21.2.6 Figure 5.2: Downsize Converter (512-bit to 64-bit)....322
21.2.7 Implementation....322
21.2.8 Strobe Handling....323
21.2.9 AXIL-to-Wider-Slave Master-Side Alignment Converters....324

21.3 Design Notes....326
21.3.1 Resource Utilization....326
21.4 Related Modules....327
22 APB Converters....328
22.1 Overview....328
22.2 Functional Description....329
22.2.1 Conversion Requirements....329
22.2.2 Write Conversion....330
22.2.3 Figure 5.3: AXI4 Write to APB Write....330
22.2.4 Read Conversion....332
22.2.5 Figure 5.4: AXI4 Read to APB Read....332
22.2.6 Burst Handling....333
22.2.7 Error Handling....333
22.3 Timing....335
22.3.1 Throughput Impact....335
22.3.2 Latency Impact....335
22.4 Design Notes....336
22.4.1 Resource Utilization....336
22.5 Related Modules....337
23 Module Structure....338
23.1 Overview....338
23.2 Parameters....339
23.2.1 Compile-Time Parameters....339
23.2.2 Per-Port Parameters....339

23.3 Ports....340
23.3.1 Module Declaration....340
23.3.2 Port Organization....340
23.3.3 Master-Side Signals (External)....340
23.3.4 Slave-Side Signals (External)....341
23.4 Functional Description....342
23.4.1 Component Instantiation....342
23.4.2 Internal Signals....343
23.4.3 Reset Style and the Emitted Filelist....344
23.4.4 Generated Variants....344
23.4.5 Generated File Structure....345
23.5 Related Modules....346
24 Signal Naming....347
24.1 Overview....347
24.2 Ports....348
24.2.1 External Port Naming....348
24.2.2 Master Port Examples....348
24.2.3 Slave Port Examples....348
24.2.4 APB Port Naming....349
24.3 Functional Description....350
24.3.1 Internal Signal Naming....350
24.3.2 Debug Signal Naming....350
24.3.3 Verification Pattern Matching....351
24.4 Related Modules....352

25 Test Strategy....353
25.1 Overview....353
25.2 Related Modules....354
25.3 Testing....355
25.3.1 Test Categories....355
25.3.2 Test Parameterization....356
25.3.3 Protocol-BFM-Only Testing with Memory-Backed Slaves....360
25.3.4 Coverage Goals....363
25.3.5 Test Execution....364
26 Debug Guide....366
26.1 Overview....366
26.2 Waveforms....367
26.2.1 Generating Waveforms....367
26.2.2 Key Signals to Observe....367
26.3 Related Modules....368
26.4 Testing....369
26.4.1 Debug Signal Access....369
26.4.2 Common Issues....370
26.4.3 Assertion Failures....371
26.4.4 Performance Debugging....372

List of Figures

Figure 2.1: Master Adapter Architecture....112
Figure 2.2: Slave Router Architecture....124
Figure 2.3: Crossbar Core Architecture....139
Figure 2.5: ID Management Architecture....171
Figure 2.6: Width Conversion Architecture....187
Figure 2.7.1: Protocol Conversion Architecture....205
Figure 2.7: Protocol Conversion Architecture....206
Figure 2.8: Response Routing Architecture....233
Figure 2.9: Error Handling Architecture....256
Figure 5.3: AW Arbiter FSM....285
Figure 5.4: AR Arbiter FSM....287
Figure 3.1: Write Data Tracking FSM....299
Figure 4.1: CAM Entry Format....305
Figure 4.2: ID Table Structure....312
Figure 5.1: Upsize Converter (64-bit to 512-bit)....319
Figure 5.2: Downsize Converter (512-bit to 64-bit)....321
Figure 5.3: AXI4 Write to APB Write....329
Figure 5.4: AXI4 Read to APB Read....331

List of Tables

Table 1: Table 1.1: Supported Protocols....107
Table 2: Table 1.2: Channel-Specific Master Types....107
Table 3: Table 1.3: Document Organization....111
Table 4: Table 5.3: Burst Length Conversion Examples....320
Table 5: Table 5.1: AXI4 vs APB Protocol Comparison....328
Table 6: Table 5.2: APB to AXI4 Error Mapping....332
Table 7: Table 7.1: Test Parameter Matrix....355
Table 8: Table 7.1a: AMBA5 Fixtures....355
Table 9: Table 7.2: Test Level Depth Profile....364
Table 10: Table 7.3: Debug Signals That Exist....368

List of Waveforms

No waveforms in this document.

1 Bridge Mas Index

Generated: 2026-10-11

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Bridge Hardware Architecture Specification Index

2.1 Overview

Version: 1.2 **Date:** 2026-10-11 **Purpose:** High-level hardware architecture specification for Bridge component

---

2.2 Related Modules

- **Bridge MAS** - Micro-Architecture Specification (detailed block-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

2.3 Navigation

Note: Every chapter below is one source file; the document build assembles the spec from these links.

2.3.1 Front Matter

- Document Information

2.3.2 Chapter 1: Introduction

- Purpose and Scope
- Document Conventions
- Definitions and Acronyms

2.3.3 Chapter 2: System Overview

- Use Cases
- Key Features
- System Context

2.3.4 Chapter 3: Architecture

- Block Diagram
- Data Flow
- Protocol Support

2.3.5 Chapter 4: Interfaces

- AXI4 Interface Overview
- APB Interface Overview
- Clock and Reset
- AXI5 and APB5 Interfaces (AMBA5)
- Unmapped-Address Handling
- Wishbone B4 Interface

2.3.6 Chapter 5: Performance

- Throughput Characteristics
- Latency Analysis
- Resources

2.3.7 Chapter 6: Integration

- System Requirements

- Parameter Configuration
- Verification Strategy
- Synthesis and Implementation
- AXI3 Integration

Last Updated: 2026-10-11 **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 Bridge: AXI4 Full Crossbar Generator - Product Requirements Document

Project: Bridge **Version:** 2.2 **Status:** Phase 2 Complete - Simplified Architecture with Hard Limits
Created: 2025-10-18 **Last Updated:** 2026-09-13

3.1 Executive Summary

Bridge is a Python-based AXI4 crossbar generator that produces simple, performant SystemVerilog RTL for connecting multiple AXI4 masters to multiple AXI4 slaves. The name follows the infrastructure theme - bridges connect different regions, enabling communication across divides, just like crossbars connect masters and slaves.

Design Philosophy: A simple AMBA fabric that is performant, but makes no attempt to support all features. ID and address widths are PER PORT, set in the TOML (`id_width`, `addr_width`) – see §5.3, “Configuration Is Generation-Time, Not Parameters”. An earlier revision of this line claimed hard 8-bit/64-bit limits; the generator has never enforced them, and the shipped configs use 2-, 4- and 8-bit IDs.

Key Differentiator from Delta: - **Delta:** AXI-Stream crossbar (streaming data, single channel, simple routing) - **Bridge:** AXI4 full crossbar (memory-mapped, 5 channels, burst support, `bridge_id` sideband routing)

Key Differentiator from Commercial IP: - **Commercial AXI4 Crossbars:** Feature-complete, support every AXI4 edge case, complex configurability - **Bridge:** Simple and performant, supports common use cases, hard limits for simplicity

Target Use Case: High-performance memory-mapped interconnects for multi-core processors, accelerators, and memory controllers where simplicity and performance matter more than spec compliance.

3.2 Design Philosophy

Vision: A simple AMBA fabric that is performant, but makes no attempt to support all features.

3.2.1 Core Principles

1. Simplicity Over Completeness - Focus on common use cases, not edge cases - Implement what's needed for real hardware, not AXI4 spec compliance theater - Prefer straightforward implementations over complex feature sets

2. Performance First - Direct connections where possible (matching widths = zero overhead) - Minimal conversion logic in critical paths - Optimize for throughput and latency, not feature count

3. Hard Limits for Simplicity

We enforce uniform widths on certain parameters to eliminate complexity:

Parameter	Hard Limit	Rationale
ID Width	Per port, set at generation	NOT fixed at 8. The TOML sets `id_width` per port and the generated bridge bakes it in – `bridge_2x2_rw` has 4-bit IDs. This row previously said "8 bits (fixed)", which the generator has never enforced.
Address Width	Per port, set at generation	NOT fixed at 64. `bridge_2x2_rw` is a 32-bit-address bridge. Same correction as the row above.
Data Width	Variable (32b-512b)	Width converters ONLY for data. Commonly needed for bandwidth optimization.

Why This Works: - ID width conversion adds complexity with minimal benefit (nobody needs >256 outstanding transactions per master) - Address width conversion is rarely needed (peripheral addresses fit in 32-bit, but using 64-bit everywhere is simpler) - Data width conversion is the ONLY width conversion that provides real value (bandwidth matching)

4. What We DON'T Support (By Design)

Features intentionally excluded for simplicity: - (Both of these WERE excluded once and no longer are: ID width and address width are per-port generation inputs. Left here as history because the "design differentiator" argument above still cites them.) - ~~No AXI4-Lite protocol variant~~ – `axil` and `axil5` ARE supported, as slave protocols (converted at the boundary by `axi4_to_axil4_{rd,wr}` / `axi4_to_axil5_{rd,wr}`) and, since bridge TASK-004 (was BRIDGE-014), as master protocols (tied-off promotion plus the AXI5-Lite sideband surface). `apb` / `apb5` likewise work on both sides (`axi4_to_apb{4,5}_shim` at a slave, `apb{4,5}_to_axi4` at a master), Wishbone B4 since bridge TASK-008 (was BRIDGE-019) (`axi4_to_wb4` / `wb4_to_axi4`), and AXI3 (`axi4_to_axi3` at a slave, `axi3_to_axi4` at a master). - No ACE protocol extensions (cache coherency) - ~~No AXI5 features~~ – interop-mode AXI5 sideband, native sideband through the structs, atomics of every class and APB5/AXI5-Lite ports were delivered by bridge TASK-002 (was BRIDGE-002); what is NOT built is a fabric that is itself AXI5 (bridge TASK-007, was BRIDGE-018) - QoS, Region and REQUEST-side User (AWUSER/ARUSER/WUSER) are routed since `c2955863`/`2b229516`. RESPONSE-side User is not: `BUSER`/`RUSER` exist as ports but the master adapters tie them to zero (`.fub_axi_buser(1'b0)`, `.fub_axi_ruser(1'b0)`), so a slave's response User never reaches its master. An earlier correction of this line said "end to end", which was true only of the request side.

5. What We DO Support

Core AXI4 features that matter: - Full 5-channel AXI4 protocol (AW, W, B, AR, R) - Burst transactions (INCR, WRAP, FIXED) - In-order completion. Responses are routed by a per-master `bridge_id` sideband and an in-order FIFO, NOT by matching AXI IDs – see FR-2. - Multiple outstanding transactions per master, **to one slave at a time**. The generated master adapters carry a single-outstanding-target gate (`aw_gate_ok` / `ar_gate_ok`): a new AW/AR to a DIFFERENT slave is held off until the outstanding ones drain. - Channel-specific masters (write-only, read-only, read-write) - Data width conversion (32b ↔️ 64b ↔️ 128b ↔️ 256b ↔️ 512b) - Configurable M×N topology (1-32 masters, 1-256 slaves) - Fair round-robin arbitration per slave

3.2.2 Architecture Philosophy

Delivered: Intelligent width-aware routing, not a fixed-width crossbar. The generated adapters carry per-width paths and convert only at a mismatch.

OLD Approach (What We're Moving Away From):

```
Master (64b) → Upsize(64→256) → Fixed 256b Crossbar → Downsize(256→64)
→ Slave (64b)
```

- Two conversions for same-width connections (wasteful!)
- Artificial bandwidth bottleneck
- Unnecessary logic and latency

NEW Approach (Target Architecture):

```
Master (64b) → Direct Connection → Slave (64b)    [0 conversions!]
Master (512b) → Conv(512→64) → Slave (64b)         [1 conversion]
```

- Per-master paths to each unique slave width it connects to
- Direct paths where widths match (zero overhead)
- Converters only where actually needed
- Router selects appropriate path based on address decode

Result: Minimal conversion logic, maximum performance, pragmatic simplicity.

3.3 CRITICAL: RTL Regeneration Requirements

ALL generated RTL files MUST be deleted and regenerated together whenever ANY generator code changes.

Why This Matters: - Generated RTL files may have interdependencies (bridges, wrappers, integrators) - Generator code changes can create version mismatches between files - Partial regeneration creates subtle incompatibilities that cause test failures - Even "small innocuous" generator changes can have cascading effects

The Rule:

```
# WRONG - Partial regeneration
./bridge_generator.py --masters 5 --slaves 3 --output ../rtl/
# Only regenerates bridge_axi4_flat_5x3.sv
# Other files (wrappers, integrators) now mismatched!

# CORRECT - Full regeneration
cd bin
make clean                                  # Delete ALL generated
bridges (rtl/generated/)
make all                                    # Regenerate everything from
bridge_batch.csv
```

Generator Files That Trigger Full Regeneration: - `bin/bridge_generator.py` - Main bridge generator (CSV/TOML driven) - Any Python file in `bin/bridge_pkg/` (components/, generators/, config, csv_parser, ...) - Any Jinja template in `bin/bridge_pkg/jinja_templates/` - `bridge_wrapper_generator.py` - Wrapper generation - **Any** Python file in `projects/components/fabric-gen-ip/bridge/bin/`

Symptoms of Version Mismatch: - Tests that previously passed now fail - Simulation errors about missing signals - Mismatched port widths or counts - Address decode routing to wrong slaves

Think of Generated RTL Like Compiled Code: When you update a compiler, you don't selectively recompile - you rebuild everything. When you update a generator, you don't selectively regenerate - you regenerate everything.

3.4 1. Product Overview

3.4.1 1.1 Purpose

Bridge automates generation of AXI4 crossbar infrastructure: - **Python code generation** - Parameterized RTL generation (similar to APB/Delta) - **Performance modeling** - Analytical + simulation validation - **Flat topology** - Full M×N interconnect matrix - **`bridge_id` sideband routing** - in-order response return (see FR-2) - **Burst optimization** - Pipelined burst transfers

3.4.2 1.2 Target Audience

Primary Users: - RTL designers building SoC interconnect - System architects evaluating interconnect topologies - Verification engineers needing crossbar testbenches - Students learning AXI4 protocol and interconnects

Educational Focus: - Demonstrates AXI4 protocol complexity - Shows arbitration strategies for memory-mapped busses - Teaches ID-based transaction tracking - Illustrates burst optimization techniques

3.4.3 1.3 Success Criteria

Functional: - [x] Generates working AXI4 crossbar RTL (bridge_generator.py) - [x] Generates CSV/TOML-configured bridges (bridge_generator.py + bridge_pkg/; the earlier separate bridge_csv_generator.py was merged in) - [x] Passes Verilator lint - [x] Supports 1-32 masters, 1-256 slaves - [ ] Handles out-of-order completion via IDs. NOT IMPLEMENTED: `bridge_cam.sv` exists but is instantiated by no generated module. The slave adapters run "Bridge ID Tracking - FIFO Mode (In-Order)" and 93 generated files still declare `cam_wr_allocate`/`cam_rd_allocate` without driving them. - [x] Supports burst lengths 1-256 beats - [x] Channel-specific masters (wr/rd/rw) for resource optimization (Phase 2) - [x] APB converter integration – DELIVERED. `axi4_to_apb4_shim` is instantiated by `apb_periph_adapter.sv` in every mixed config, and `bridge_1x2_rw_apb5`, `bridge_1x2_rw_apb5_mon` and the four `mix_*` bridges ship with APB slaves. - [x] AXI3 converter integration – DELIVERED. `axi4_to_axi3` and `axi3_to_axi4` boundary converters are emitted for `protocol = "axi3"` slave and master ports; `bridge_1x2_axi3`, `bridge_2x2_axi3_masters`, `bridge_2x2_axi3_slaves` and `bridge_mix_e` ship as validation fixtures.

Performance: - [ ] Latency ≤ 3 cycles for single-beat transactions. **Unverified for the generated bridge.** The generated path crosses four skid buffers on the round trip (adapter timing wrappers on both sides), and gaxi_skid_buffer registers its output side, so an ideal zero-wait slave still costs >=4 cycles of bridge-added latency before arbitration. The 2-3 cycle figure in 6.1 describes the wrapperless flat crossbar, not what the generator emits. - [ ] Throughput: M×N paths do NOT all transfer concurrently. Each master is held to ONE target slave at a time (`aw_gate_ok`/`ar_gate_ok`, "single-outstanding-target"), a deadlock fix, not an oversight. Concurrency is across MASTERS, not across a master's targets. - [x] Performance models implemented (bridge_model.py - V1 Flat) - [ ] Fmax ≥ 300 MHz on UltraScale+ FPGAs (pending synthesis validation)

Quality: - [x] Complete specifications (PRD) before code - [x] Performance models validate requirements (bridge_model.py) - [x] All generated RTL Verilator verified - [x] Integration examples provided (CSV examples) - [x] Comprehensive documentation (bin/GENERATOR_ARCHITECTURE.md, docs/bridge_has/, docs/bridge_mas/)

3.4.4 1.4 Implementation Status and Phases

Unified Generator (bridge_generator.py):

The bridge generator now supports both TOML/CSV configuration and legacy array-indexed modes:

Configuration Modes: 1. **TOML/CSV Mode (Preferred)** - TOML port configuration (`bridge_name.toml`) - CSV connectivity matrix (`bridge_name_connectivity.csv`) - Custom port prefixes (`rapids_m_axi_`, `cpu_m_axi_`, etc.) - Interface wrapper integration (timing isolation) - Mixed protocols (AXI4 + AXI3 + APB + AXI4-Lite slaves) - Channel-specific masters (wr/rd/rw) - Status: Phase 2 complete, Phase 3 pending

2. **Legacy CSV Mode (Backwards Compatible)**
   - Separate `ports.csv` and `connectivity.csv` files
   - Migration path to TOML format
   - See `bin/test_configs/README.md` for conversion guide

Phase Status:

Phase 1: CSV Configuration COMPLETE - CSV parser (ports.csv, connectivity.csv) - Port generation with custom prefixes - Converter identification logic - Basic crossbar instantiation

Phase 2: Channel-Specific Masters COMPLETE (2025-10-26) - Added `channels` field to PortSpec (rw/wr/rd) - Conditional port generation based on channels - **Resource Optimization:** - wr (write-only): AW, W, B channels only → 39% port reduction - rd (read-only): AR, R channels only → 61% port reduction - rw (full): All 5 channels - Width converter awareness (only generate needed converters) - Example: 4-master bridge saves 35% signals with optimized channels

Phase 3: APB Converter Integration – DELIVERED - AXI2APB converter module: `axi4_to_apb4_shim`, emitted by `axi4_to_apb4_shim_component.py` - APB signal packing/unpacking: in the shim - APB converter instantiation in generated bridges: `apb_periph_adapter.sv` and the `periph5_adapter.sv` variants - Width converter + APB converter chaining: the `*_mon` builds chain the AXI4 timing wrapper into the shim - End-to-end testing with APB slaves: `test_bridge_1x2_rw_apb5*` and the `mix_*` suites, all in the 72-test FULL regression - The “placeholders with TODO comments” status was stale. This block said PENDING while the shim it describes was shipping in six configurations.

Additional Resources: - `models/bridge_model/bridge_model.py` - Performance modeling (V1 Flat implemented) - `rtl/bridge_cam.sv` - CAM for ID tracking. **Not instantiated by any generated module**; kept for the OOO work FR-2 describes as not implemented. - See `bin/GENERATOR_ARCHITECTURE.md` for the generator/build mechanics - See `docs/bridge_has/`

and `docs/bridge_mas/` for architecture diagrams and specs (the older BRIDGE_CURRENT_STATE.md / BRIDGE_ARCHITECTURE_DIAGRAMS.md were superseded)

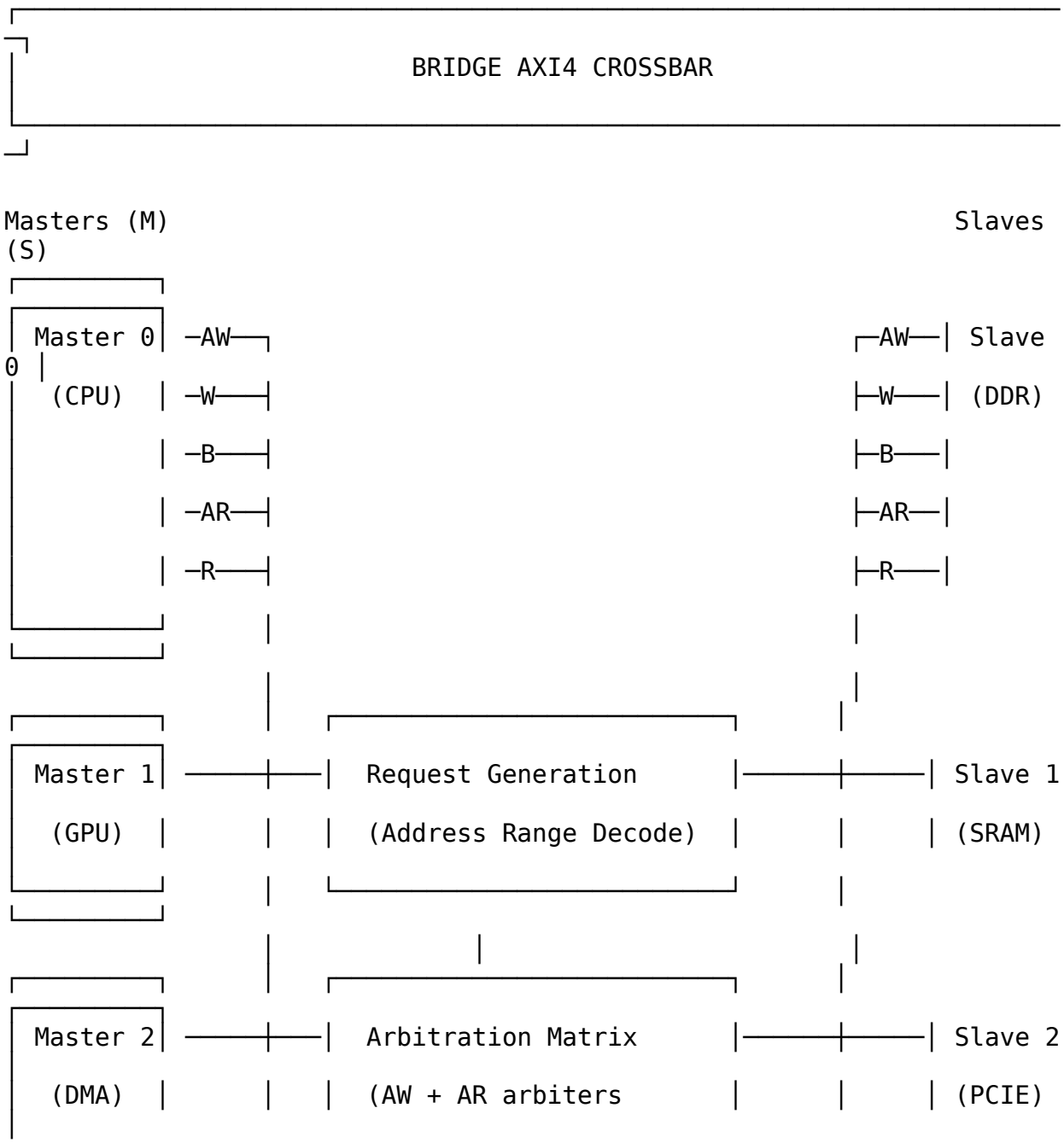
3.5 2. Architecture Overview

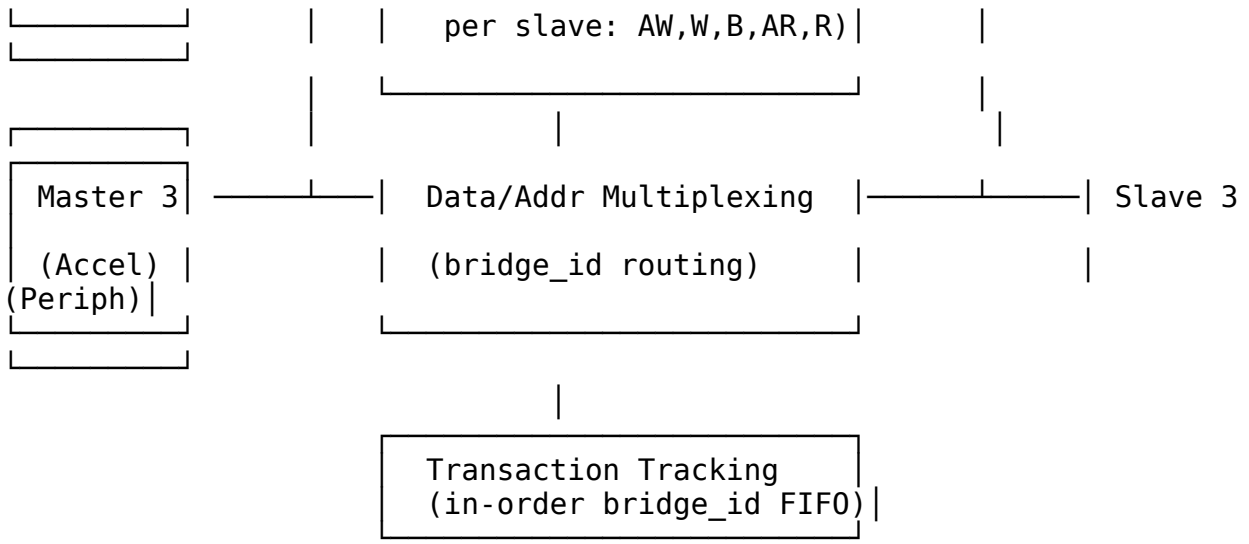
3.5.1 2.1 AXI4 vs AXIS vs APB

Feature	APB	AXI-Stream (Delta)	AXI4 (Bridge)
Protocol Type	Simple register bus	Streaming data	Memory-mapped burst
Channels	1 (address + data)	1 (data stream)	5 (AW, W, B, AR, R)
Request Generation	Address range decode	TDEST decode	Address range decode
Arbitration	Per-slave, per-cycle	Per-slave, packet-locked	Per-slave, per-address-phase
Out-of-Order	No (sequential)	No (streaming order)	No (in-order, `bridge_id` routed)
Burst Support	No	Packet (via TLAST)	Yes (AWLEN/ARLEN)
Complexity	Low	Medium	**High**
Latency	1-2 cycles	2 cycles	2 cycles each way (measured; HAS Table 5.7)
Use Case	Control registers	Data streaming	Memory-mapped I/O

Bridge Complexity Sources: 1. **5 independent channels**, of which TWO are arbitrated. The crossbar instantiates a round-robin arbiter per slave for AW and AR only; W, R and B follow the selection recorded at the address handshake, so they are routed, not arbitrated. 2. **`bridge_id` sideband routing**, in-order (the AXI ID is not used for routing) 3. **Burst handling** with interleaving constraints 4. **Write response tracking** (match AW with B channel) 5. **Address decode** + **ID muxing** for response routing

3.5.2 2.2 Block Diagram





3.5.3 2.3 Key Components

1. Request Generation - Address range decode for each slave - Generates M×S request matrix per channel (AW, AR) - Similar to APB but more complex (2 address channels)

2. Per-Slave Arbitration - **2 arbiters per slave**, not five: - AW channel arbiter (write address) - AR channel arbiter (read address) - W has no arbiter – a W-owner FIFO gives the channel to whichever master's AW the slave accepted first. - B and R have no arbiter – they are muxed combinationally on the `bridge_id` carried alongside the response. - Round-robin, grant held to the ADDRESS handshake – NOT to xlast. The generated arbiters read `lock until handshake`. - Separate read/write paths (no head-of-line blocking)

3. Data Multiplexing - Mux master signals to selected slave - ID-based response routing (B, R channels) - Burst tracking (grant held to the ADDRESS handshake, not to xlast – the generated arbiters read `lock until handshake`; see FR-6)

4. Transaction Tracking - In-order `bridge_id` FIFO per slave adapter (`Bridge ID Tracking - FIFO Mode (In-Order)`), popped in AW/AR order - Track the originating master in a per-slave in-order FIFO, popped on the response. There is no {Master ID, Transaction ID} map – routing is by FIFO POSITION and the AXI ID is not consulted. - Required for routing B/R channels back to correct master - ID tables for OUT-OF-ORDER support are not implemented: `bridge_cam.sv` exists but is instantiated in zero generated bridges

5. ~~Optional Performance Counters~~ – NOT implemented, and no config key enables them - Transaction counts per master/slave - Arbitration conflict counts - Latency histograms

---

3.6 3. Functional Requirements

3.6.1 3.1 AXI4 Protocol Compliance

FR-1: Full AXI4 Protocol Support - Support all 5 AXI4 channels: AW, W, B, AR, R - Comply with AMBA AXI4 specification (ARM IHI 0022) - Support burst lengths: 1-256 beats (AWLEN/ARLEN = 0-255) - Support burst types: INCR, WRAP, FIXED - Support burst sizes: 1-128 bytes (AWSIZE/ARSIZE = 0-7)

FR-2: Response Routing (in-order) - Responses are routed by the `bridge_id` sideband, a static per-master tag, popped from an in-order FIFO. The AXI ID plays no part in routing. - Master IDs pass through unchanged – there is no ID extension. `cpu_adapter` drives `cpu_32b_aw.id = fub_axi_awid`, 4 bits in, 4 bits out. - **Constraint this imposes:** the slave must return responses in the order its requests were accepted. A reordering slave misroutes – the FIFO head names the wrong master, so one master receives another's beats and the second starves. Nothing detects this. - Two masters never alias an ID at a slave: inside the fabric every ID is `{master index, master id}` (bridge TASK-005, was BRIDGE-016), so a slave port carries the widest master's id_width plus `$clog2(NUM_MASTERS)` bits – 8-bit masters behind a 16-master bridge give 12-bit IDs at the slaves. The `bridge_id` sideband still routes responses; the prefix is what makes per-ID tracking sound across masters, and every real AXI slave of a multi-master bridge is tracked by ID in `bridge_cam` (bridge BUG-012, was BRIDGE-015). Single-master bridges add no prefix and keep the in-order FIFO.

FR-3: Atomic Operations (the bridge does NOT implement a monitor – see below) - Pass AWLOCK/ARLOCK through to the slave. **The bridge implements no exclusive monitor**: there is no per-slave exclusive tracking and no EXOKAY generation anywhere in the generated RTL – `awlock` is muxed to the granted slave and nothing more. Exclusives work only if the attached slave implements the monitor itself. 13.1 correctly lists the monitor as future work; this requirement previously read as though the bridge provided it. - ~~Track exclusive monitor per slave~~ – NOT implemented; see the corrected bullet above - ~~Generate BRESP/RRESP errors for failed exclusives~~ – NOT implemented

3.6.2 3.2 Address Decoding

FR-4: Configurable Address Map - Support M masters × S slaves - Each slave has base address and size - Non-overlapping address ranges (verified at generation) - Default slave for unmapped addresses (optional)

FR-5: Address Range Configuration

```
# Example address map
address_map = {
    0: {'base': 0x00000000, 'size': 0x40000000, 'name': 'DDR'},      #
1GB
    1: {'base': 0x40000000, 'size': 0x00100000, 'name': 'SRAM'},     #
1MB
    2: {'base': 0x50000000, 'size': 0x01000000, 'name': 'PCIE'},     #
16MB
```

```
    3: {'base': 0x60000000, 'size': 0x00010000, 'name': 'Peripherals'}
# 64KB
}
```

3.6.3 3.3 Arbitration Strategy

FR-6: Round-Robin Arbitration - Fair bandwidth allocation (no starvation) - Separate arbiters for AW and AR channels - Grant is locked only until the ADDRESS handshake (`awvalid && awready`), not until xlast. The RTL comment reads “lock until handshake”. Back-to-back AWs from different masters can therefore be accepted and their W bursts sequenced by the W-owner FIFO. - Round-robin arbitration, HARD-CODED. There is no TOML key and no parameter to change it; “configurable” was aspirational.

FR-7: Read/Write Independence - Separate read and write paths - No head-of-line blocking between read/write - Concurrent read and write to same slave (if slave supports)

3.6.4 3.4 Burst Handling

FR-8: Burst Optimization - Pipelined burst transfers (overlap address and data) - No artificial burst splitting - Full AXI4 burst protocol support

FR-9: Interleaving Constraints - W channel follows the AW owner via a per-slave W-owner FIFO (not a held grant) - R channel routed by in-order `bridge_id` FIFO position, NOT by transaction ID. IDs are pass-through: the slave port is the same width as the master port. `bridge_cam.sv` exists but is instantiated in zero generated bridges. - ID-based interleaving is NOT supported. Each slave port must return B/R in request order across ALL IDs – see bridge BUG-008 (was BRIDGE-010).

3.7 4. Non-Functional Requirements

3.7.1 4.1 Performance

NFR-1: Latency - **Single-beat read:** ≤ 3 cycles (address decode + arbitration + mux) - **Single-beat write:** ≤ 3 cycles (address decode + arbitration + mux) - **Burst transfer:** No additional latency per beat (pipelined)

NFR-2: Throughput - **Concurrent transfers:** different masters to different slaves, yes; ONE master to several slaves, no – see 1.3 - **Burst efficiency:** Line-rate data transfer after address phase - **No artificial stalls:** Crossbar adds no wait states beyond arbitration

NFR-3: Fmax - **Target:** 300-400 MHz on Xilinx UltraScale+ - **Registered outputs:** All slave outputs registered for timing closure - **Pipelineable:** Optional pipeline stages for >400 MHz

3.7.2 4.2 Resource Usage (Measured)

Post-route, out of context, Vivado 2025.1 (HAS 6.4 flow; full tables in HAS 5.3):

Bridge	Part	LUTs	FFs	BRAM / DSP	Timing
`bridge_2x2_rw` (2 x 2, 32-bit AXI4)	Artix-7 100T -1 @ 10 ns	4,615	3,253	0 / 0	met, +0.12 ns
`bridge_2x2_rw_pipe` (registered crossbar)	Artix-7 100T -1 @ 10 ns	5,646	4,227	0 / 0	met, +1.27 ns
`bridge_2x2_axi5_native` (128-bit, MTE + chunking)	Artix-7 100T -1 @ 10 ns	8,007	6,213	0 / 0	met, +0.04 ns
`bridge_4x4_rw` (4 x 4, 32..256-bit)	Kintex-7 325T -2 @ 6.667 ns	29,719	29,508	0 / 0	met, +0.28 ns

No configuration uses block RAM or DSPs: the per-slave bridge_id FIFOs and the CAM are LUT arrays. Data width sets the cost – the 4 x 4 with width converters is six times the 2 x 2 – and on the Artix-7 at 100 MHz any configuration beyond a plain 2 x 2 needs `xbar_pipeline = true` to close.

3.7.3 4.3 Quality Requirements

NFR-4: Code Generation Quality - Lint-clean SystemVerilog (Verilator) - Synthesizable (Vivado, Yosys, Design Compiler) - No vendor-specific primitives (technology-agnostic) - Clear structure (commented, readable)

NFR-5: Verification - CocoTB testbench framework - Transaction-level verification - ~~Out-of-order test scenarios~~ – not applicable; ordering is structural (see FR-9) - Burst interleaving tests - >95% functional coverage

---

3.8 5. Interface Specifications

3.8.1 5.1 AXI4 Master Interfaces (M × 5 channels)

Write Address Channel (AW):

```
input  logic [ADDR_WIDTH-1:0]   s_axi_awaddr  [NUM_MASTERS];
input  logic [ID_WIDTH-1:0]     s_axi_awid    [NUM_MASTERS];
input  logic [7:0]              s_axi_awlen   [NUM_MASTERS];  // Burst
length-1
input  logic [2:0]              s_axi_awsize  [NUM_MASTERS];  // Burst
size
input  logic [1:0]              s_axi_awburst [NUM_MASTERS];  //
INCR/WRAP/FIXED
input  logic                    s_axi_awlock  [NUM_MASTERS];  //
Exclusive access
input  logic [3:0]              s_axi_awcache [NUM_MASTERS];  // Cache
attributes
input  logic [2:0]              s_axi_awprot  [NUM_MASTERS];  //
Protection type
input  logic [3:0]              s_axi_awqos   [NUM_MASTERS];  // QoS
-- routed, not arbitrated on
input  logic [3:0]              s_axi_awregion[NUM_MASTERS];  //
Region -- passed through
input  logic [UW-1:0]           s_axi_awuser  [NUM_MASTERS];  // USER
-- passed through
input  logic                    s_axi_awvalid [NUM_MASTERS];
output logic                    s_axi_awready [NUM_MASTERS];
```

Write Data Channel (W):

```
input  logic [DATA_WIDTH-1:0]   s_axi_wdata  [NUM_MASTERS];
input  logic [DATA_WIDTH/8-1:0] s_axi_wstrb  [NUM_MASTERS];  // Byte
strobes
input  logic                    s_axi_wlast  [NUM_MASTERS];  // Last
beat
input  logic [UW-1:0]           s_axi_wuser  [NUM_MASTERS];  // USER
-- passed through
input  logic                    s_axi_wvalid [NUM_MASTERS];
output logic                    s_axi_wready [NUM_MASTERS];
```

Write Response Channel (B):

```
output logic [ID_WIDTH-1:0]     s_axi_bid    [NUM_MASTERS];
output logic [1:0]              s_axi_bresp  [NUM_MASTERS];  //
OKAY/SLVERR/DECERR (EXOKAY never generated -- no exclusive monitor)
output logic [UW-1:0]           s_axi_buser  [NUM_MASTERS];  // USER
-- returned to the master
```

```
output logic                    s_axi_bvalid [NUM_MASTERS];
input  logic                    s_axi_bready [NUM_MASTERS];
```

Read Address Channel (AR):

```
input  logic [ADDR_WIDTH-1:0]   s_axi_araddr  [NUM_MASTERS];
input  logic [ID_WIDTH-1:0]     s_axi_arid    [NUM_MASTERS];
input  logic [7:0]              s_axi_arlen   [NUM_MASTERS];
input  logic [2:0]              s_axi_arsize  [NUM_MASTERS];
input  logic [1:0]              s_axi_arburst [NUM_MASTERS];
input  logic                    s_axi_arlock  [NUM_MASTERS];
input  logic [3:0]              s_axi_arcache [NUM_MASTERS];
input  logic [2:0]              s_axi_arprot  [NUM_MASTERS];
input  logic [3:0]              s_axi_arqos   [NUM_MASTERS];  // QoS
-- routed, not arbitrated on
input  logic [3:0]              s_axi_arregion[NUM_MASTERS];  //
Region -- passed through
input  logic [UW-1:0]           s_axi_aruser  [NUM_MASTERS];  // USER
-- passed through
input  logic                    s_axi_arvalid [NUM_MASTERS];
output logic                    s_axi_arready [NUM_MASTERS];
```

Read Data Channel (R):

```
output logic [DATA_WIDTH-1:0]   s_axi_rdata  [NUM_MASTERS];
output logic [ID_WIDTH-1:0]     s_axi_rid    [NUM_MASTERS];
output logic [1:0]              s_axi_rresp  [NUM_MASTERS];
output logic                    s_axi_rlast  [NUM_MASTERS];
output logic [UW-1:0]           s_axi_ruser  [NUM_MASTERS];  // USER
-- returned to the master
output logic                    s_axi_rvalid [NUM_MASTERS];
input  logic                    s_axi_rready [NUM_MASTERS];
```

3.8.2 5.2 AXI4 Slave Interfaces (S × 5 channels)

Mirror of master interfaces, with M → S direction reversed.

3.8.3 5.3 Configuration Is Generation-Time, Not Parameters

The generated bridge exposes no parameters. Everything below is fixed when the generator runs and reaches the module through its package:

```
module bridge_2x2_rw
    import bridge_2x2_rw_pkg::*;
(
    input logic aclk, ...
```

Geometry and widths come from the TOML, per port, and appear inside the generated modules as localparams (`localparam ADDR_WIDTH = 32;`, `localparam ID_WIDTH = 4;` in `cpu_adapter`). To change any of them, regenerate – there is nothing to override at elaboration.

An earlier revision of this section listed a parameter block including `MAX_BURST_LEN`, `PIPELINE_OUTPUTS` and `ENABLE_COUNTERS`. **None of those three exist in any generated module**, and neither do the performance counters that `ENABLE_COUNTERS` implied – there is no counter logic anywhere in the generated set.

3.9 6. Performance Modeling

3.9.1 6.1 Analytical Model

Latency Components (Flat Crossbar):

```
Single-Beat Read Latency:
  1. Address Decode:      0 cycles (combinatorial)
  2. Arbitration:         1 cycle  (AR arbiter decision)
  3. Address Mux:         0 cycles (combinatorial)
  4. Slave Access:        (slave-dependent, not crossbar)
  5. Response Mux:        1 cycle  (R channel ID lookup + mux)
  6. Output Register:     1 cycle  (optional, for timing closure)

  Total (no pipeline):    2 cycles
  Total (pipelined):      3 cycles
```

Burst Transfer Throughput:

```
After address phase completes, data transfer is line-rate:
  - W channel: 1 beat/cycle (routed by the W-owner FIFO)
  - R channel: 1 beat/cycle (ID-routed from slave)

Example: 256-beat burst
  Address phase:  2 cycles (measured, one-time)
  Data phase:     256 cycles (line-rate)
  Total:          258-259 cycles
  Efficiency:     98.8%
```

Concurrent Throughput:

```
Maximum concurrent transfers (4×4 crossbar):
  - Read:  4 concurrent (one per master-slave pair)
  - Write: 4 concurrent (one per master-slave pair)
  - Total: 8 concurrent read+write (separate paths)

Throughput @ 100 MHz, 512-bit data:
  Per path:   100 MHz × 512 bits = 51.2 Gbps
  Total:      8 paths × 51.2 Gbps = 409.6 Gbps theoretical
```

3.9.2 6.2 Resource Scaling

LUT Usage Formula (empirical):

```
LUTs ≈ 500 (base) + 150 × M × S + 20 × FIFO_DEPTH × BRIDGE_ID_WIDTH
```

Example (4×4 crossbar, ID_WIDTH=4, bridge_id FIFO depth 16 per slave):

```
LUTs ≈ 500 + 150 × 16 + 20 × 16 × 4
     ≈ 500 + 2,400 + 1,280
     ≈ 4,180 LUTs
```

3.9.3 6.3 Comparison with Other Crossbars

Crossbar Type	Latency	Throughput	Complexity	Use Case
APB	1-2 cycles	Low (serialized)	Low	Control registers
AXI-Stream (Delta)	2 cycles	High (streaming)	Medium	Data streaming
AXI4 (Bridge)	2 cycles each way	High (burst)	**High**	Memory-mapped I/O
AXI4 + Slices	4-6 cycles	High (burst)	Very High	>400 MHz designs

Bridge Sweet Spot: memory-mapped interconnects that need burst efficiency and mixed protocols. NOT a fit where out-of-order completion matters – the fabric is in-order by construction (FR-9, bridge BUG-008 (was BRIDGE-010)).

3.10 7. Generator Architecture

3.10.17.1 Python Generator Structure

```
class BridgeGenerator:
    """AXI4 crossbar RTL generator"""

    def __init__(self, config):
        self.num_masters = config.num_masters
        self.num_slaves = config.num_slaves
        self.data_width = config.data_width
        self.addr_width = config.addr_width
        self.id_width = config.id_width
        self.address_map = config.address_map

    def generate_address_decode(self) -> str:
        """Generate address range decode logic"""
        # For each master × slave, check if address in range
        # More complex than AXIS (uses address), simpler than full
decode

    def generate_aw_arbiter(self, slave_idx) -> str:
        """Generate write address channel arbiter for one slave"""
        # Round-robin arbiter
        # Grant held to the ADDRESS handshake, not to the B response.
        # (This pseudocode described an abandoned lock-until-response
design.)

    def generate_ar_arbiter(self, slave_idx) -> str:
        """Generate read address channel arbiter for one slave"""
        # Round-robin arbiter
        # Grant held to the ADDRESS handshake, not to RLAST.

    def generate_w_channel_mux(self, slave_idx) -> str:
        """Generate write data channel multiplexer"""
        # W channel follows the AW owner recorded in the per-slave W-
owner
        # FIFO. The arbiter grant itself released at the ADDRESS
handshake.

    def generate_b_channel_demux(self, slave_idx) -> str:
        """Generate write response channel demultiplexer"""
        # Route by FIFO POSITION: pop the per-slave bridge_id FIFO.
        # The returned BID is NOT consulted (see 4.2).

    def generate_r_channel_demux(self, slave_idx) -> str:
```

```
        """Generate read data channel demultiplexer"""
        # Route by FIFO POSITION: pop the per-slave bridge_id FIFO on
RLAST.
        # The returned RID is NOT consulted (see 4.2).

    def generate_bridge_id_fifo(self, slave_idx) -> str:
        """Generate transaction ID tracking table"""
        # Maps {slave, transaction_id} → master_id
        # Indexed on AW/AR grant, looked up on B/R response

    def generate_crossbar(self) -> str:
        """Generate complete crossbar module"""
        # Instantiate all arbiters, muxes, demuxes and the per-slave
        # bridge_id FIFOs (there are no ID tables)
```

3.10.2 7.2 Address Map Configuration

Python Configuration:

```
bridge_config = {
    'num_masters': 4,
    'num_slaves': 4,
    'data_width': 512,
    'addr_width': 64,
    'id_width': 4,
    'address_map': {
        0: {'base': 0x00000000, 'size': 0x40000000, 'name': 'DDR'},
        1: {'base': 0x40000000, 'size': 0x00100000, 'name': 'SRAM'},
        2: {'base': 0x50000000, 'size': 0x01000000, 'name': 'PCIE'},
        3: {'base': 0x60000000, 'size': 0x00010000, 'name':
'Peripherals'}
    },
    # NOTE: 'pipeline_outputs' and 'enable_counters' are NOT real
config
    # keys -- config_validator rejects unknown keys, and 5.3 says the
    # counters do not exist. Removed from this example rather than
left
    # for someone to copy.
}
```

Generated Address Decode:

```
// Address decode for each master
always_comb begin
    for (int s = 0; s < NUM_SLAVES; s++)
        aw_request_matrix[s] = '0;

    for (int m = 0; m < NUM_MASTERS; m++) begin
```

```
        if (s_axi_awvalid[m]) begin
            // Slave 0: DDR (0x00000000 - 0x3FFFFFFF)
            if (s_axi_awaddr[m] >= 64'h00000000 &&
                s_axi_awaddr[m] < 64'h40000000)
                aw_request_matrix[0][m] = 1'b1;

            // Slave 1: SRAM (0x40000000 - 0x400FFFFF)
            if (s_axi_awaddr[m] >= 64'h40000000 &&
                s_axi_awaddr[m] < 64'h40100000)
                aw_request_matrix[1][m] = 1'b1;

            // ... slaves 2-3 ...
        end
    end
end
```

3.11 8. Comparison with APB and Delta

3.11.1 8.1 Code Reuse from APB Generator

Similar Components (~70% reuse): - Address range decode logic (same pattern, different signals) - Round-robin arbiter (same algorithm) - Data multiplexing pattern (same approach) - Backpressure handling (similar to PREADY)

New Components for Bridge: - **2 arbiters per slave** (AW and AR). W is owned via a FIFO, B and R are muxed combinationally – no arbiter on any of the three. - **`bridge_id` sideband routing** (B and R channel demuxing) - **In-order transaction tracking** (a FIFO per direction, not an ID table) - **Burst handling** (grant locked to the ADDRESS handshake, not xlast)

Migration Effort from APB: - ~120 minutes (vs ~75 min for AXIS, due to higher complexity) - Most time: response demuxing and the bridge_id FIFOs

3.11.2 8.2 Code Reuse from Delta Generator

Similar Components (~60% reuse): - Python generation framework - Command-line interface - Performance modeling structure - Arbitration pattern (round-robin with locking)

Key Differences: - **5 channels** vs 1 channel (Delta only has TDATA/TVALID/TREADY/TLAST) - **`bridge_id` sideband routing** vs TDEST-based routing - **Address decode** vs TDEST decode (Bridge more complex) - **Transaction tracking** vs packet atomicity (different mechanisms)

3.11.3 8.3 Complexity Comparison

Metric	APB Crossbar	Delta (AXIS)	Bridge (AXI4)
Channels to arbitrate	1	1	5
Request generation	Address ranges	TDEST decode	Address ranges
Response routing	Grant-based	Grant-based	ID-based
Burst support	No	Packet (TLAST)	Yes (AWLEN/ARLEN)
Out-of-order	No	No	No – in-order `bridge_id` FIFO (`bridge_cam` unused)
Transaction	No	No	Yes

Metric	APB Crossbar	Delta (AXIS)	Bridge (AXI4)
tracking			
Lines of Python	~500	~697	**~900** (est.)
Lines of generated SV	~200 (4×4)	~250 (4×4)	**~400** (4×4) (est.)

= Additional complexity in Bridge

3.12 9. Use Cases

3.12.1 9.1 Multi-Core Processor Interconnect

Scenario: 4 CPU cores + GPU accessing DDR + SRAM + Peripherals

```
Configuration:
  Masters:  5 (4 CPUs, 1 GPU)
  Slaves:   3 (DDR, SRAM, Peripherals)
  Data:     512-bit (cache line width)
  Address:  64-bit (large memory space)
  ID:       4-bit (up to 16 outstanding per master)
```

Benefits: - Concurrent access to all slaves - in-order completion (see FR-9 – out-of-order is NOT implemented) - Burst transfers for cache line fills - Separate read/write paths (no head-of-line blocking)

3.12.2 9.2 DMA + Accelerator System

Scenario: DMA engine + 4 accelerators accessing shared memory

```
Configuration:
  Masters:  5 (1 DMA, 4 accelerators)
  Slaves:   2 (DDR, Control registers)
  Data:     512-bit (high-bandwidth DMA)
  Address:  32-bit (limited address space)
  ID:       2-bit (simple ID space)
```

Benefits: - High-bandwidth memory access for DMA - Fair arbitration prevents accelerator starvation - Control register access doesn't block data transfers

3.12.3 9.3 FPGA System Integration

Scenario: MicroBlaze CPU + custom accelerators + memory controllers

```
Configuration:
  Masters:  8 (1 CPU, 7 accelerators)
  Slaves:   4 (DDR, BRAM, AXI GPIO, AXI DMA)
  Data:     128-bit (AXI4 standard width)
  Address:  32-bit (standard FPGA address space)
  ID:       4-bit (moderate outstanding transactions)
```

Benefits: - Standard AXI4 interfaces (Xilinx IP compatibility) - Scalable to many masters/slaves - Performance counters for profiling. **NOT IMPLEMENTED** – no counter logic exists in any generated module.

3.13 10. Testing Strategy

3.13.1 10.1 FUB (Functional Unit Block) Tests

Address Decode Tests: - Verify all address ranges correctly decoded - Test boundary conditions (base, base+size-1) - Test unmapped addresses (error response)

Arbiter Tests: - Round-robin fairness (all masters get turns) - Grant locking to the ADDRESS handshake (not xlast – see FR-6) - Starvation prevention

bridge_id FIFO Tests: - Correct ID → master mapping - ~~Out-of-order transaction handling~~ – not implemented; see FR-9 - Table full condition

Mux/Demux Tests: - Data integrity through crossbar - Response routing to correct master - Concurrent transfers don't interfere

3.13.2 10.2 Integration Tests

Single-Master, Single-Slave: - Basic read/write transactions - Burst transfers (various lengths) - ~~Out-of-order completions~~ – not applicable (in-order fabric)

Multi-Master, Single-Slave: - Arbitration correctness - Fairness verification - Burst interleaving (if supported)

Multi-Master, Multi-Slave: - Concurrent transfers - No crosstalk between paths - Full M×S matrix coverage

Stress Tests: - All masters active simultaneously - Maximum burst lengths - Full ID space utilization - Back-to-back bursts

3.13.3 10.3 Performance Validation

Latency Measurement: - Single-beat read: measure actual vs analytical - Single-beat write: measure actual vs analytical - Compare with specification (≤3 cycles)

Throughput Measurement: - Burst transfer efficiency (should be ~98%) - Concurrent path throughput (should be line-rate × M×S) - Compare with theoretical maximum

Resource Validation: - Synthesize for Xilinx UltraScale+ - Compare LUT/FF usage with estimates - Verify Fmax ≥ 300 MHz

---

3.14 11. Documentation Plan

3.14.1 11.1 Specifications (Before Code)

- ☒ **PRD.md** - This document (complete requirements)
- ☐ **README.md** - User guide with quick start
- ☐ **BRIDGE_VS_APB_GENERATOR.md** - Migration guide from APB
- ☐ **BRIDGE_VS_DELTA_GENERATOR.md** - Comparison with Delta
- ☐ **AXI4_PROTOCOL_GUIDE.md** - AXI4 primer for students

3.14.2 11.2 Performance Analysis (Before Implementation)

- ☒ **models/bridge_model/bridge_model.py** - Analytical model (V1 Flat implemented)
  - Latency calculations (single-beat, burst)
  - Throughput estimates (concurrent paths)
  - Resource estimates (LUT/FF scaling)
- ☐ **bridge simulator** - Discrete event simulation (optional, not implemented)
  - Cycle-accurate modeling
  - Traffic pattern support
  - Validation against RTL

3.14.3 11.3 Code Generation

- ☐ **bin/bridge_generator.py** - Main RTL generator
  - Command-line interface
  - Address map configuration
  - Generated RTL output

3.14.4 11.4 Verification

- ☐ **dv/tests/** - CocoTB testbenches
  - FUB tests (individual blocks)
  - Integration tests (multi-block)
  - Stress tests (corner cases)

3.15 12. Success Metrics

3.15.1 12.1 Functional Completeness

- ☐ Generates working AXI4 crossbar RTL
- ☐ Passes all CocoTB tests
- ☐ Verilator lint clean
- ☐ Xilinx Vivado synthesis clean

3.15.2 12.2 Performance Targets

- ☐ Latency ≤ 3 cycles (measured in simulation)
- ☐ Throughput = line-rate × M×S paths (measured)
- ☐ Fmax ≥ 300 MHz (post-synthesis)

3.15.3 12.3 Educational Value

- ☐ Complete specifications demonstrating rigor
- ☐ Performance models validate requirements
- ☐ Clear code structure (readable generated RTL)
- ☐ Integration examples provided

3.15.4 12.4 Reusability

- ☐ ~70% code reuse from APB generator (measured by LOC)
- ☐ Similar patterns to Delta generator
- ☐ Can be extended (weighted arbitration, QoS, etc.)

3.16 12.2 PDF Generation Location

IMPORTANT: PDF files should be generated in the docs directory:

`projects/components/fabric-gen-ip/bridge/docs/`

Quick Commands: Use the provided shell scripts:

```
cd projects/components/fabric-gen-ip/bridge/docs
./generate_has_pdf.sh   # Bridge_HAS_v<rev>.docx/.pdf
./generate_mas_pdf.sh   # Bridge_MAS_v<rev>.docx/.pdf
```

The shell scripts will automatically: 1. Use the md_to_docx.py tool from bin/ 2. Process the bridge_has/bridge_mas index files 3. Generate both DOCX and PDF files in the docs/ directory 4. Create table of contents and title page

See: `bin/md_to_docx.py` for complete implementation details

3.17 13. Future Enhancements

3.17.1 13.1 Short-Term (Post-Initial Release)

- ☐ **Optional pipeline stages** - For Fmax >400 MHz
- ☐ **Weighted arbitration** - QoS support
- ☒ **Default slave** - unmapped address handling. Built 2026-09-07: a subtractive catch-all answers DECERR + 0xDEADBEEF, with a sticky status/IRQ cleared by the `unmapped_clear` INPUT PIN. There is no APB access to it – the pin is the whole interface, and wiring it to a register is the integrator's job. See HAS 4.5 and bridge BUG-007 (was BRIDGE-009).
- ☐ **Exclusive monitor** - Full atomic operation support

3.17.2 13.2 Long-Term

- ☐ **Tree topology** - Hierarchical crossbar (like Delta)
- ☐ **AXI4-Lite variant** - Simplified for control registers
- ☐ **ACE support** - Coherent cache interconnect
- ☐ **GUI configurator** - Visual address map setup

3.18 14. Risk Assessment

Risk	Probability	Impact	Mitigation
~~ID table complexity~~	–	–	Moot: no ID tables were built. The realised risk was the opposite – the DOCS described tables for months while the RTL used an in-order FIFO.
Response-ordering assumption	High	High	The fabric REQUIRES each slave to return B/R in request order across IDs and does not check it. Assert on a returned BID/RID that does not match the FIFO head – see bridge BUG-008 (was BRIDGE-010).
Fmax below target	Low	Medium	Optional pipeline stages for timing closure
Resource usage exceeds	Low	Low	Empirical formulas guide expectations
Burst interleaving bugs	Medium	High	Separate test suite for burst scenarios

3.19 15. Project Timeline (Estimated)

Week 1: Specifications and Models - [x] Day 1-2: PRD.md (complete) - [ ] Day 3-4: Performance modeling (analytical) - [ ] Day 5-7: README.md, migration guides

Week 2-3: Core Implementation - [ ] Day 1-3: Address decode + arbiter generation - [ ] Day 4-5: Data mux/demux generation - [x] Day 6-8: response tracking (delivered as a per-slave in-order bridge_id FIFO, not ID tables) - [ ] Day 9-10: Integration and testing

Week 4: Verification and Examples - [ ] Day 1-5: CocoTB testbenches - [ ] Day 6-7: Integration examples

---

3.20 16. Glossary

- **AXI4:** Advanced eXtensible Interface version 4 (AMBA standard)
- **Burst:** Multi-beat transaction (AWLEN/ARLEN > 0)
- **Crossbar:** Full M×N interconnect matrix
- **ID:** AXI transaction identifier. Passed through this bridge unchanged; it is NOT used for response routing here.
- **Out-of-order:** responses returning in a different order than requested. AXI4 permits it between IDs; **this bridge does not support it** and does not detect a slave that does (bridge BUG-008, was BRIDGE-010).
- **xlast:** WLAST (write) or RLAST (read) - last beat indicator

---

Version: 2.1 **Status:** Specification Complete - Ready for Implementation **Next Steps:** Create performance models, then implement generator

Project Bridge - Connecting masters and slaves across the divide

---

3.21 References

- Dally & Towles, *Principles and Practices of Interconnection Networks*, Morgan Kaufmann 2004 — backs the design philosophy (simple-performant, hard limits; class cost discussions and the observation that a full crossbar with buffering is costly), per-slave/channel arbitration as a degenerate separable allocator (ch18 §19.3), round-robin as a starvation-free baseline (ch17 §18.4.2), skid buffers as buffered flow control (ch12), and weighted-arbiter starvation bounds (ch17). (Reference archive)
- Duato/Yalamanchili/Lionel, *Interconnection Networks*, Morgan Kaufmann 2003 — backs independent read/write paths that avoid head-of-line blocking (ch4), deterministic single-stage crossbar deadlock-freedom (ch3), and the need for explicit ID-keyed tables/CAMs to route out-of-order responses correctly (ch3/4 ordering material). (Reference archive)
- Jantsch & Tenhunen (eds.), *Networks on Chip*, Kluwer 2003 — backs the QoS-plus-aging arbitration policy and its bounded starvation argument (ch13). (Reference archive)
- Maaref, *Architecting and Building High-Speed SoCs*, Packt 2022 — backs the AMBA-suite context for read/write channel independence and the AXI ID ordering rules that Bridge's in-order FIFO simplifies (ch3 §4.2). (Reference archive)
- Roy, *Advanced Digital System Design*, Springer 2023 — backs AXI's independent read/write channels (§18.2.2.1) and the decoupling of W from AW with B correlated through the stored transaction ID (§18.2.3). (Reference archive)
- Eschemann, *The FPGA Programming Handbook*, Packt 2023 — backs the secondary, educational framing of AXI4 channel structure and the cost warning that a full crossbar with buffering can become expensive (ch8). (Reference archive)
- ARM, *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022E — the AXI4 protocol authority; backs the ordering rules that permit out-of-order completion across IDs and therefore motivate the CAM-style tracking Bridge documents as future work. (Reference archive)

4 Claude Code Guide: Bridge Subsystem

Version: 2.3 **Last Updated:** 2026-09-13 **Purpose:** AI-specific guidance for working with Bridge subsystem

---

4.1 CRITICAL: Read Architecture Documents First

Before making ANY changes to bridge generator or understanding signal flow:

READ: `projects/components/fabric-gen-ip/bridge/bin/GENERATOR_ARCHITECTURE.md` (generator build mechanics, reconciled against the code 2026-09-29) and `projects/components/fabric-gen-ip/bridge/docs/bridge_mas/` (micro-architecture spec; rendered as `docs/Bridge_MAS_v1.7.pdf`)

These documents contain the **definitive bridge architecture** including: - Correct signal flow (wrappers → decoder → converters → crossbar → slaves) - Component purposes and placement - Common misconceptions that previous agents made - Why there's NO fixed crossbar width

If you skip these documents, you WILL make incorrect assumptions.

4.2 Quick Context

What: Bridge - one AXI4/AXI5 crossbar generator, `bin/bridge_generator.py`, taking TOML or CSV configuration. There is no second generator: the separate `bridge_csv_generator.py` was merged into it and no longer exists. **Status:** Phase 2 Complete - CSV generator with channel-specific masters (wr/rd/rw) **Your Role:** Help users configure CSV files, generate bridges, understand architecture, and create tests

Complete Documentation (Read in This Order): 1. `projects/components/fabric-gen-ip/bridge/bin/GENERATOR_ARCHITECTURE.md` ← **START HERE** (generator build mechanics; the design lives in the MAS below) 2. `projects/components/fabric-gen-ip/bridge/PRD.md` ← Product requirements 3. `vault/Tasks/bridge/INDEX.md` ← Task tracking (open / closed / dropped; the old TASKS.md was folded in 2026-09-10) 4. `projects/components/fabric-gen-ip/bridge/docs/bridge_has/bridge_has_index.md` ← Hardware Architecture Spec (rendered: `docs/Bridge_HAS_v1.7.pdf`) 5. `projects/components/fabric-gen-ip/bridge/docs/bridge_mas/bridge_mas_index.md` ← Micro-Architecture Spec (rendered: `docs/Bridge_MAS_v1.7.pdf`)

(The older BRIDGE_ARCHITECTURE.md, BRIDGE_CURRENT_STATE.md, BRIDGE_ARCHITECTURE_DIAGRAMS.md, CSV_BRIDGE_STATUS.md, and docs/bridge_spec/ documents were superseded by the HAS/MAS above.)

4.3 Target Architecture: Intelligent Width-Aware Routing

Core Principle: Direct connections where possible, converters only where needed, no fixed crossbar width.

4.3.1 Efficient Multi-Width Design

```
Master_A (64b)
  ├─ Direct → Slave_0 (64b)           [0 conversions, minimal latency]
  ├─ Conv(64→128) → Slave_1 (128b)    [1 conversion]
  └─ Conv(64→512) → Slave_2 (512b)    [1 conversion]

Master_B (512b)
  ├─ Conv(512→64) → Slave_0 (64b)     [1 conversion]
  ├─ Conv(512→128) → Slave_1 (128b)   [1 conversion]
  └─ Direct → Slave_2 (512b)          [0 conversions, full bandwidth]
```

Router Logic: Address decoder determines target slave, selects correctly-sized path for that master-slave pair.

4.3.2 Why This Architecture

Naive Fixed-Width Approach (Don't Do This):

```
Master (64b) → Upsize(64→256) → Fixed 256b Crossbar → Downsize(256→64)
→ Slave (64b)
```

- TWO conversions for same-width connections (wasteful!)
- Reduced bandwidth on narrow paths
- Unnecessary logic and area
- Higher latency

Intelligent Routing (Target):

```
Master (64b) → Router → Direct Connection → Slave (64b)
```

- ZERO conversions for matching widths
- Full native bandwidth
- Minimal logic
- Lowest latency

4.3.3 Per-Master Output Paths

Each master has N output paths (one per unique slave width it connects to):

```
// Master_A connects to slaves at 64b, 128b, 512b
// Generate 3 output paths:
```

```
logic [63:0]  master_a_64b_wdata;   // For 64b slaves (direct)
logic [127:0] master_a_128b_wdata;  // For 128b slaves (via converter)
logic [511:0] master_a_512b_wdata;  // For 512b slaves (via converter)

// Router selects based on address decode:
always_comb begin
    case (decoded_slave_id)
        SLAVE_0: select master_a_64b_wdata;   // Slave_0 is 64b
        SLAVE_1: select master_a_128b_wdata;  // Slave_1 is 128b
        SLAVE_2: select master_a_512b_wdata;  // Slave_2 is 512b
    endcase
end
```

4.3.4 Benefits

1. **Resource Efficient** - Only instantiate converters actually needed
2. **Maximum Performance** - Direct paths have zero conversion overhead
3. **Optimal Bandwidth** - No artificial width bottlenecks
4. **Lower Latency** - Minimal logic in critical path for matching widths
5. **Scalable** - Works for any combination of master/slave widths

4.3.5 Implementation Status

Current State: Intelligent per-master multi-width routing – DELIVERED. `cpu_master_adapter.sv` in `bridge_4x4_rw` carries separate 32b, 64b, 128b and 256b paths and converts only where a master and its slave disagree. There is no fixed internal crossbar width.

This block described the Phase-1 fixed-width crossbar as current and the per-width routing as a future migration “requiring generator architecture rework”. That rework has happened; the block outlived it.

4.4 CRITICAL RULE #0: RTL Regeneration Requirements

Change any generator, delete every generated bridge, regenerate all of them. The rule itself, why partial regeneration fails silently, and the symptom list are the root `/CLAUDE.md` Rule #0 and `vault/handbook/design/generated-rtl-discipline.md`. What is bridge-specific:

Generators whose change forces a full regeneration - `bin/bridge_generator.py` - **any** Python file under `bin/bridge_pkg/` (components/, generators/, config, csv_parser, signal_naming, ...) - any Jinja template in `bin/bridge_pkg/jinja_templates/`

The workflow

```
cd projects/components/fabric-gen-ip/bridge/bin
make clean          # removes rtl/generated/
make all            # regenerates from bridge_batch.csv

cd ../dv/tests
make clean-all && make run-all-func    # ALL tests; bridge runs
serial, ~1GB each
```

`clean-all` is not optional here. You have just regenerated every `.sv`, so a stale build directory reports green for the old design – precisely the failure this workflow exists to prevent (`vault/handbook/dv/running-regressions.md`).

Never regenerated: `rtl/bridge_cam.sv` (hand-written, and instantiated by zero generated bridges) and anything in `rtl/` outside `rtl/generated/`.

Identifying a generated file takes five strings, not one. Each generator class stamps its own header, and `bridge_generator.py` appears in none of them:

File	Header
`<name>.sv`	`// Generated by: BridgeModuleGenerator`
`<name>_pkg.sv`	`// Generated by: PackageGenerator`
`<name>_xbar.sv`	`// Generated by: CrossbarGenerator`
`<master>_adapter.sv`	`// Generated by: AdapterGenerator`
`<slave>_adapter.sv`	`// Generated by: SlaveAdapterGenerator`

Grepping for `bridge_generator.py` finds nothing; grepping only for `BridgeModuleGenerator` finds one file in five.

4.5 Critical Pitfalls: slave_select_* Reversion After Handshake

4.5.1 The Problem: Combinational Address Decode

The master adapter's R/B response MUX and the crossbar's AR/AW gating both relied on `<master>_slave_select_{ar,aw}` — a combinational decode of `fub_axi_{ar,aw}addr`. This signal reverts immediately when the address port is freed:

1. AR/AW handshake completes (arvalid && arready, awvalid && awready)
2. Wrapper pops skid buffer and releases the address port
3. fub_axi_araddr / fub_axi_awaddr reverts to next transaction or zero
4. Decode flips to different slave
5. Response MUX is now gated on WRONG slave
6. Response path closes, transaction hangs

This happens BEFORE the converter even finishes pushing to the crossbar, and ALWAYS before response arrives.

4.5.2 Visible Symptoms

- Fails: Multi-slave multi-master bridges hang (timeout)
- Fails: Multi-width masters fail basic connectivity
- Fails: APB/AXIL slaves consistently timeout (longer converter latency = more decode changes)
- Works: Single-slave or single-master systems (no decode change)

4.5.3 The Solution (MANDATORY in All Generators)

For Crossbar AR/AW Gating: - Replace `<master>_slave_select_ar/aw` gates with **inline address re-decode** on stable m_axi buses - Address on m_axi is held stable across the entire AXI handshake

For Master Adapter Response MUX: - Add **per-channel response-tracking FIFOs** that capture slave index at request time - Use FIFO head for response MUX instead of live combinational decode

Crossbar Code Pattern:

```
// WRONG (old implementation):
assign gate_ar_s0 = m0_slave_select_ar && m0_arvalid;
//                  ^^^^^^^^^^^^^^^^^^ Reverts after handshake!

// CORRECT (new implementation):
assign gate_ar_s0 = ((m0_m_axi_araddr >= S0_BASE) &&
                     (m0_m_axi_araddr < S0_END) &&
```

```
                        m0_arvalid);  // Re-decode on stable m_axi
address
```

Master Adapter Code Pattern:

```
// WRONG (old implementation):
assign r_slave_select = comb_slave_select_ar;  // Stale after pop!

// CORRECT (new implementation):
// Capture at request time
always_ff @(posedge aclk) begin
    if (fub_axi_arvalid && fub_axi_arready) begin
        ar_trk_fifo <= comb_slave_select_ar;
    end
end

// Use stable value for response MUX
assign r_slave_select = ar_trk_fifo_out;
```

4.5.4 When This Bug Appears

- Works: Single-master, single-slave systems (no address re-evaluation)
- Fails: Multi-master to same slave (multiple addresses decode differently)
- Fails: Multi-slave same master (address decode changes per transaction)
- Fails: Multi-width paths (W beats re-evaluate AW decode mid-burst)

4.6 MANDATORY: Project Organization Pattern

THIS SUBSYSTEM MUST FOLLOW THE RAPIDS/AMBA ORGANIZATIONAL PATTERN - NO EXCEPTIONS

4.6.1 Required Directory Structure

```
projects/components/fabric-gen-ip/bridge/
├── bin/                            # Bridge-specific tools
(generators, scripts)
│   ├── bridge_generator.py       # AXI4 crossbar generator (CSV/TOML
driven)
│   ├── bridge_pkg/                 # Generator implementation package
│   ├── test_configs/               # TOML port configs + CSV
connectivity
│   └── Makefile                    # make all / make single / make
clean
├── docs/                           # Design documentation
(bridge_has/, bridge_mas/, PDFs)
├── dv/                             # Design Verification (MANDATORY
structure)
│   ├── tbclasses/                  # Testbench classes (MANDATORY - TB
classes here!)
│   │   ├── __init__.py
│   │   └── bridge2x2_rw_tb.py      # Per-config generated TB classes
│   ├── components/                 # Bridge-specific BFMs (if needed)
│   ├── scoreboards/                # Bridge-specific scoreboards (if
needed)
│   ├── testplans/                  # Test plans
│   └── tests/                      # All test files (test runners
only, flat)
│       ├── conftest.py             # Pytest configuration (MANDATORY)
│       └── test_bridge_*.py        # Per-config tests
(test_bridge_2x2_rw.py, ...)
├── rtl/                            # RTL source files
│   ├── bridge_cam.sv               # Hand-written CAM (unused by every
generated bridge)
│   ├── filelists/                  # Generated filelists
│   └── generated/                  # Generated bridges (one subdir per
config)
├── CLAUDE.md                       # This file
├── PRD.md                          # Product requirements
└── TASKS.md                        # Task history and status
```

4.6.2 Testbench Class Location (MANDATORY)

TB classes live in `dv/tbclasses/`, never inside a test runner. The rule and the reasoning are in `vault/handbook/dv/tb-structure.md` and `/GLOBAL_REQUIREMENTS.md` 2.1. The bridge layout that satisfies it is the tree under Rule #0.1 below.

4.6.3 Test File Pattern (MANDATORY)

Pattern B – `cocotb_test_*` functions with pytest wrappers at the bottom, each naming its `testcase=` – is the `test-patterns` skill and `/GLOBAL_REQUIREMENTS.md` 3.2. Read the real thing rather than a template: `dv/tests/test_bridge_2x2_rw.py`. Tests and TB classes are generated per bridge configuration alongside the RTL, so the generated file is the pattern.

4.7 Critical Rules for This Subsystem

4.7.1 Rule #0.1: Testbench Architecture - MANDATORY SEPARATION

TB classes live in `dv/tbclasses/`, one per config (`bridge2x2_rw_tb.py`); runners stay flat in `dv/tests/`. That layout is the tree above – it is not repeated here. Why the separation matters for bridge specifically is that one TB is shared by the per-config test, the `*_mon_monitor.py` stress test and external imports: `vault/handbook/dv/tb-structure.md`.

4.7.2 Rule #1: TBBase and the three mandatory methods

(Former Rule #2 is folded in here; #3 and #4 keep their numbers so existing references to them still resolve.)

Every TB inherits `TBBase` and implements `setup_clocks_and_reset` / `assert_reset` / `deassert_reset`. The requirement is `/GLOBAL_REQUIREMENTS.md` 2.2 and 2.3; what TBBase provides and why the three methods are mandatory is `vault/handbook/dv/tb-structure.md`.

Bridge-specific: the class is `BridgeAXI4FlatTB(TBBase)`, the clock is `aclk` and reset is `aresetn` (active-low), so `assert_reset` drives `self.dut.aresetn.value = 0`.

4.7.3 Rule #3: Use GAXI Components for Protocol Handling

Use the framework BFMs; never hand-drive AXI4 valid/ready. The map and trap list are in `vault/handbook/dv/bfm-usage.md`. Bridge wires GAXI onto the AXI4 channels with the `s0_axi4_` prefix:

```
from CocoTBFramework.components.gaxi.gaxi_master import GAXIMaster
from CocoTBFramework.components.gaxi.gaxi_slave import GAXISlave
from CocoTBFramework.components.axi4.axi4_field_configs import
AXI4FieldConfigHelper

# CORRECT: Use GAXI for AXI4 channels
self.aw_master = GAXIMaster(
    dut=dut,
    title="AW_M0",
    prefix="s0_axi4_",
    clock=clock,

field_config=AXI4FieldConfigHelper.create_aw_field_config(id_width,
addr_width, 1),
    pkt_prefix="aw",
    multi_sig=True,
```

```
    log=log
)
```

4.7.4 Rule #4: Queue-Based Verification

For simple in-order verification, use direct monitor queue access:

```
# CORRECT: Direct queue access
aw_pkt = self.aw_slave._recvQ.popleft()
w_pkt = self.w_slave._recvQ.popleft()

# Verify
assert aw_pkt.addr == expected_addr
assert w_pkt.data == expected_data

# WRONG: Memory model for simple test
memory_model = MemoryModel()  # Unnecessary complexity
```

When to Use Memory Models: - Simple in-order tests → Use queue access - Single-master systems → Use queue access - Complex out-of-order scenarios → not supported; slaves must respond in request order - Multi-master with address overlap → Memory model tracks state

---

4.8 TOML/CSV-Based Bridge Generator (Phase 2 Complete)

4.8.1 Overview

The Bridge generator creates parameterized SystemVerilog crossbars from TOML configuration files with CSV connectivity matrices, eliminating manual RTL editing for complex interconnects.

Key Benefits: - **Human-readable configuration** - TOML for ports, CSV for connectivity - **Custom signal prefixes** - Each port has unique prefix (rapids_m_axi_, apb0_, etc.) - **Channel-specific masters** - Write-only (wr), read-only (rd), or full (rw) - **Interface modules** - Timing isolation via axi4_master/slave wrappers with configurable skid depths - **Automatic converters** - Width and protocol conversion inserted automatically - **Resource efficient** - Only generates needed channels and converters

4.8.2 Quick Start

1. Create bridge_mybridge.toml:

```
[bridge]
  name = "bridge_mybridge"
  description = "Custom bridge example"

  # Default skid buffer depths (can be overridden per port)
  defaults.skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}

  masters = [
    {name = "cpu", prefix = "cpu_m_axi", id_width = 4, addr_width =
32, data_width = 64, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "dma", prefix = "dma_m_axi", id_width = 4, addr_width =
32, data_width = 512, user_width = 1,
     interface = {type = "axi4_master", skid_depths = {ar = 4, r = 8,
aw = 4, w = 8, b = 4}}}
  ]

  slaves = [
    {name = "ddr", prefix = "ddr_s_axi", id_width = 4, addr_width =
32, data_width = 512, user_width = 1,
     base_addr = 0x00000000, addr_range = 0x80000000, interface =
{type = "axi4_slave"}},
    {name = "sram", prefix = "sram_s_axi", id_width = 4, addr_width =
32, data_width = 256, user_width = 1,
     base_addr = 0x80000000, addr_range = 0x80000000, interface =
{type = "axi4_slave"}}
  ]
```

2. Create bridge_mybridge_connectivity.csv:

```
master\slave,ddr,sram
cpu,1,1
dma,1,1
```

3. Generate bridge:

```
cd projects/components/fabric-gen-ip/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_mybridge.toml
# Auto-finds bridge_mybridge_connectivity.csv

# Or use bulk generation
python3 bridge_generator.py --bulk bridge_batch.csv
```

Result: Complete SystemVerilog module with: - Custom port prefixes per port - Timing isolation via interface wrappers - Only needed AXI4 channels (wr/rd/rw optimized) - Width converters for data mismatches - Internal crossbar instantiation - APB/AXI4-Lite converter integration points

4.8.3 Configuration Format Details

TOML Port Configuration:

The primary format is now **TOML** (preferred over CSV for better structure and interface configuration):

Port Specifications: - `name` - Unique identifier (cpu, dma, ddr, etc.) - `prefix` - Signal prefix (cpu_m_axi_, ddr_s_axi_, etc.) - `id_width` - AXI4 ID width in bits - `addr_width` - Address width in bits - `data_width` - Data width in bits (32, 64, 128, 256, 512) - `user_width` - AXI4 user signal width - `base_addr` - Slave base address (slaves only) - `addr_range` - Slave address range (slaves only) - `interface` - Interface wrapper configuration (optional) - `type` - “axi4_master”, “axi4_slave”, “axi4_master_mon”, “axi4_slave_mon”, or omit for direct connection - `skid_depths` - Per-channel buffer depths: {ar, r, aw, w, b}. Valid range is **2..8 inclusive, any integer** – odd depths are legal. `gaxi_skid_buffer` says so at its parameter: “Must be 2..8 inclusive”. An earlier `{2, 4, 6, 8}` here was an inference, not the contract.

CSV Connectivity Matrix:

```
master\slave,slave0,slave1,slave2
master0,1,0,1
master1,0,1,1
```

- `1` = connected, `0` = not connected
- Partial connectivity supported (not all masters to all slaves)
- Auto-detected based on TOML filename: `bridge_name.toml` → `bridge_name_connectivity.csv`

Legacy CSV Format:

For backwards compatibility, the generator still supports CSV port files: - See `bin/test_configs/README.md` for migration guide from CSV to TOML - TOML is now preferred for new bridges (better structure, interface config support)

4.8.4 Channel-Specific Masters (Phase 2 Feature)

Why Channel-Specific? Real hardware often has dedicated read or write masters. Generating all 5 AXI4 channels wastes resources:

Traditional (wasteful):

```
// Write-only master gets unused read channels
input  logic [63:0]  rapids_descr_m_axi_awaddr,  // USED
input  logic [511:0] rapids_descr_m_axi_wdata,   // USED
output logic [7:0]   rapids_descr_m_axi_bid,     // USED
input  logic [63:0]  rapids_descr_m_axi_araddr,  // UNUSED (50%
waste!)
output logic [511:0] rapids_descr_m_axi_rdata,   // UNUSED
```

Channel-Specific (optimized):

```
rapids_descr_wr,master,axi4,wr,rapids_descr_m_axi_,512,64,8,N/A,N/A
```

Generated:

```
// Write-only master - only AW, W, B channels
input  logic [63:0]  rapids_descr_m_axi_awaddr,
input  logic [511:0] rapids_descr_m_axi_wdata,
output logic [7:0]   rapids_descr_m_axi_bid,
// NO READ CHANNELS (araddr, rdata, etc.)
```

Resource Savings: - 40-60% fewer ports for dedicated masters - Only necessary width converters instantiated - Channel-aware direct connection wiring - Faster synthesis, smaller netlists

4.8.5 Example: RAPIDS-Style Configuration

RAPIDS Architecture: - Descriptor write master (wr) - Writes descriptors to memory - Sink write master (wr) - Writes incoming packets to memory - Source read master (rd) - Reads outgoing packets from memory - CPU master (rw) - Full access for configuration

TOML Configuration (bridge_rapids.toml):

```
[bridge]
  name = "bridge_rapids"
  description = "RAPIDS accelerator bridge"
  defaults.skid_depths = {ar = 2, r = 4, aw = 2, w = 4, b = 2}

  masters = [
    {name = "rapids_descr_wr", prefix = "rapids_descr_m_axi", channels
= "wr",
```

```
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "rapids_sink_wr", prefix = "rapids_sink_m_axi", channels =
"wr",
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "rapids_src_rd", prefix = "rapids_src_m_axi", channels =
"rd",
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     interface = {type = "axi4_master"}},
    {name = "cpu", prefix = "cpu_m_axi", channels = "rw",
     id_width = 4, addr_width = 32, data_width = 64, user_width = 1,
     interface = {type = "axi4_master"}}
  ]

  slaves = [
    {name = "ddr", prefix = "ddr_s_axi", protocol = "axi4",
     id_width = 8, addr_width = 64, data_width = 512, user_width = 1,
     base_addr = 0x80000000, addr_range = 0x80000000,
     interface = {type = "axi4_slave"}},
    {name = "apb_periph", prefix = "apb0_", protocol = "apb",
     addr_width = 32, data_width = 32,
     base_addr = 0x00000000, addr_range = 0x00010000}
  ]
```

Connectivity CSV (bridge_rapids_connectivity.csv):

```
master\slave,ddr,apb_periph
rapids_descr_wr,1,0
rapids_sink_wr,1,0
rapids_src_rd,1,0
cpu,1,1
```

Generated RTL Features: - rapids_descr_wr: write channels only - rapids_sink_wr: write channels only - rapids_src_rd: read channels only

The specific counts this list used to give (37 wr / 24 rd against 61 full, for 39% and 61% reductions) reproduce from no port structure in the tree and are not recoverable – the RAPIDS bridges they describe are not in the generated set here. For scale, counted on the shipped variants: an AXI4 `rw` master port is 44 signals, a write-only master 24, a read-only master 20. Channel trimming is real and worth doing; the old percentages were not measurements. - cpu_master: Width converters for 64b→512b upsize (both wr and rd converters) - ddr_controller: Direct 512b connection (no conversion) - apb_periph0: APB slave – `axi4_to_apb4_shim`, a real conversion

4.8.6 Common User Questions

Q: “How do I generate a bridge?”

A: Three steps:

1. Create TOML port configuration file
2. Create CSV connectivity matrix
3. Run bridge_generator.py

```
# Create bridge_mybridge.toml (port configuration)
# Create bridge_mybridge_connectivity.csv (connectivity matrix)

cd projects/components/fabric-gen-ip/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_mybridge.toml
# Auto-finds bridge_mybridge_connectivity.csv

# Or use bulk generation for multiple bridges
python3 bridge_generator.py --bulk bridge_batch.csv
```

Q: "What's the difference between wr/rd/rw channels?"

A: Number of AXI4 channels generated:

- **rw** (read/write) - All 5 channels: AW, W, B, AR, R
- **wr** (write-only) - 3 channels: AW, W, B (no read channels)
- **rd** (read-only) - 2 channels: AR, R (no write channels)

Benefits: 40-60% fewer ports for dedicated masters, less logic, faster synthesis

Q: "Can I add timing isolation to ports?"

A: Yes, use interface configuration:

```
masters = [
  {name = "cpu", prefix = "cpu_m_axi", ...,
   interface = {type = "axi4_master", skid_depths = {ar = 2, r = 4, aw
= 2, w = 4, b = 2}}}
]
```

Available interface types: - `"axi4_master"` - Timing isolation on master port - `"axi4_slave"` - Timing isolation on slave port - `"axi4_master_mon"` - Timing + monitoring - `"axi4_slave_mon"` - Timing + monitoring - Omit `interface` field for direct connection

Q: "Can I mix AXI4 and APB slaves?"

A: Yes, use protocol field in TOML:

```
slaves = [
  {name = "ddr", prefix = "ddr_s_axi", protocol = "axi4", ...},
  {name = "apb0", prefix = "apb0_", protocol = "apb", ...}
]
```

Generator inserts `axi4_to_apb4_shim` automatically. This is a real, complete conversion – burst decomposition, width slicing and response mapping – not a Phase-3 placeholder.

Q: "Can ports be AMBA5 (AXI5 / APB5)?"

A: Yes — per port; every wrapper feature rides the fabric natively (bridge TASK-002 (was BRIDGE-002), bridge TASK-004 (was BRIDGE-014), bridge TASK-007 (was BRIDGE-018)):

```
[[bridge.masters]]
protocol = "axi5"
axi5_features = ["trace", "atomic"]  #
nsaid/trace/mpam/mecid/unique/chunking are
                                     # droppable sideband;
poison/atomic/mte are
                                     # connectivity-gated (every
connected path
                                     # must be AXI5 + feature + width-
matched);
                                     # mte and chunking need 128-bit
ports

[[bridge.slaves]]
protocol = "apb5"   # APB4 rules apply (rw-only, 32-bit); adds the
APB5 pins
```

Sideband passes natively on width-matched AXI5<->AXI5 paths and terminates with a generation-time warning elsewhere. Read-return atomics forward natively on rw master ports (per-ID return tracker at the slave adapter); on write-only ports the boundary's `axi5_atomic_filter` answers them with DECERR. AXI5-Lite and APB5 are legal as MASTER protocols too (front-end converters). Details: `docs/bridge_has/ch04_interfaces/04_axi5_apb5_interfaces.md` and `docs/bridge_mas/ch02_blocks/10_amba5_boundary.md`.

Q: "Wishbone?" `protocol = "wb4"` on either side (bridge TASK-008 (was BRIDGE-019), HAS 4.6): `channels = "rw"`, `id_width = 0`, 8/16/32/64-bit. One Wishbone transfer is one single-beat AXI4 transaction; AXI bursts decompose to single Wishbone transfers; burst hints are carried, not formed. Best effort by decision – do not propose burst formation or a WB4 CDC port without a consumer.

Q: "What if data widths don't match?"

A: Generator inserts width converters automatically:

```
masters = [
  {name = "cpu", data_width = 64, ...},  # 64b master
]
slaves = [
  {name = "ddr", data_width = 512, ...}  # 512b slave
]
# Generator creates width converter: 64b → 512b
```

Q: "Do all masters need to connect to all slaves?"

A: No, use partial connectivity in CSV:

```
master\slave,ddr,sram,apb
rapids_descr,1,1,0     # Connects to ddr and sram only
cpu,1,1,1              # Connects to all three
```

4.8.7 Generator Output Structure

Generated File Contains:

1. **Module Header** - NOT parameterized. The generator resolves every width and count at emission time, so the top module has no NUM_MASTERS / NUM_SLAVES parameters to override.
2. **Port Declarations** - custom prefix per port, channel-specific signals
3. **Internal Signals** - crossbar interface nets
4. **Width Converters** - master-side upsize instances (channel-aware)
5. **Crossbar Instance** - internal AXI4 crossbar
6. **Slave Adapters** - one per slave, carrying the in-order `bridge_id` FIFO
7. **Subtractive slave** - `axi4_subtractive_slave`, ALWAYS emitted, wired to the decode `else`; answers unmapped addresses with DECERR (HAS 4.5)
8. **Protocol shims** - `axi4_to_apb4_shim` / `axi4_to_axil4_*` for apb/axil slaves. These are real conversions, not the Phase-3 TODO placeholders this list used to claim.
9. **Monitor subsystem** (monitored variants) - per-port wrappers, a `monbus_arbiter` tree and one `monbus_axil4_axil4_group`

Example Output Size: - Simple 2x2 bridge: 1,237 lines in the top file; 4,651 across all generated files for that variant (measured, not estimated). Formerly "~400" (plus per-master and per-slave adapters, a crossbar and a package). The "~400" this line used to claim predates monitors, the cfg subsystem and sideband routing. - Complex 5x3 with mixed protocols: 1,776 in the top file; 7,556 across all files. The old "~900" was smaller than the 2x2 figure beside it, which alone showed both were guesses. - Includes comprehensive comments and structure

4.8.8 Testing Generated Bridges

For Pure AXI4 Bridges (Working Now):

```
# Generate bridge (outputs to rtl/generated/<bridge_name>/)
python3 bridge_generator.py --ports test_configs/bridge_2x2_rw.toml

# Run its test (per-config TB classes live in dv/tbclasses/, e.g.
bridge2x2_rw_tb.py)
cd ../dv/tests
```

```
make clean-all && make run-bridge_2x2_rw-gate-parallel   # or -func / -full, [-waves]
```

Tests are GENERATED too. `make regen` in `bin/` regenerates RTL only; the tests and TB classes come from `bridge_generator.py --bulk bridge_batch.csv --generate-tests`, which writes into `dv/tests` and `dv/tbclasses` per the manifest's own output columns (the `--output-*` flags do not override a bulk run). Regenerate them the same way you regenerate RTL: delete every file marked "Generated by bridge test generator", regenerate all, never hand-edit one. Hand-written tests for a generated DUT live in their own file beside it (`test_bridge_2x2_rw_tracking.py`, `test_bridge_*_sideband*.py`, `*_atomics.py`, `*_bfm5.py`).

For Mixed AXI4/APB: the APB path is complete: `axi4_to_apb4_shim` does real burst decomposition, width slicing and response mapping, and six configurations ship APB slaves with tests in the FULL regression. This line claimed placeholders long after the converter shipped.

4.9 Bridge Architecture Quick Reference

4.9.1 Generated Bridge Crossbar Structure

Illustrative shape (see `rtl/generated/bridge_2x2_rw/bridge_2x2_rw.sv` for a real example):

The generated top takes **no parameters at all** – every width is fixed at generation time and reaches the module through its package:

```
module bridge_2x2_rw
    import bridge_2x2_rw_pkg::*;
(
    input  logic aclk,
    input  logic aresetn,

    // Master-side interfaces (slave ports on bridge)
    // AW, W, B, AR, R channels for each master

    // Slave-side interfaces (master ports on bridge)
    // AW, W, B, AR, R channels for each slave
);
```

4.9.2 Key Features

1. **5-Channel Implementation:** Complete AW, W, B, AR, R routing
2. **Round-Robin Arbitration:** Per-slave arbitration with grant locking
3. **Response routing:** a small in-order FIFO per direction in each slave adapter, holding the `bridge_id` of the master whose request was accepted. Not a RAM, and not indexed by AXI ID.
4. **`bridge_id` sideband routing:** responses return IN ORDER. The slave must complete in the order its requests were accepted; a reordering slave misroutes silently (see FR-2 in the PRD).
5. **Configurable Parameters:** NxM topology, data/addr/ID widths

4.9.3 Address Map

There is no default address map and no 0x10000000 stride. Windows come from each slave's `base_addr` / `addr_range` in the TOML and are baked into the decode. `bridge_2x2_rw` splits the space in half:

- Slave 0 (ddr): 0x00000000 - 0x7FFFFFFF
- Slave 1 (sram): 0x80000000 - 0xFFFFFFFF

Probing 0x1000_0000 expecting "slave 1" hits slave 0.

4.10 Test Organization

Runners are flat in `dv/tests/`: `test_bridge_<config>.py` per configuration, `test_bridge_*_mon_monitor.py` for monitor-stress builds, plus `monitor_stress_common.py` and `conftest.py`. `make list` enumerates them.

4.10.1 Test Levels

Three real levels, both mechanisms, every test (45 of 45 under `bin/review/check_test_levels.py`): each wrapper's `generate_bridge_levels()` branches on `REG_LEVEL` (GATE 72 / FUNC 144 / FULL 216 cells) and exports `TEST_LEVEL` + `SEED` per cell; the TB reads the depth profile from `dv/tbclasses/bridge_levels.py`. Do NOT stamp `TEST_LEVEL` into `os.environ` from a conftest – cocotb_test lets the environment override `extra_env`, and that ran all 216 cells at one depth once (tooling BUG-004, was TOOL-016).

Per-Config Functional Tests (generated): basic connectivity, boundary probe, arbitration (multi-master). Every AXI5 master port carries an `AXI5ComplianceChecker`; every test asserts zero violations.

Monitor Stress Tests (`*_mon_monitor.py`) (generated): bridges built with monitor interfaces, monbus traffic / WRITE_BP / ERR_BP phases scaled by level.

Hand-written: bridge BUG-009 (was BRIDGE-011) tracking + latency on 2x2; the AMBA5 sign-off tests (sideband values, sideband through arbitration, sideband across a width converter, atomics incl. Compare, AXI5 read BFM sign-off).

4.11 Common User Questions and Responses

4.11.1 Q: "How does the Bridge work?"

A: Direct answer:

The Bridge AXI4 crossbar connects multiple AXI4 masters to multiple slaves:

1. **Address Decode (AW/AR):** Routes master requests to appropriate slave based on address
2. **Write Path:** AW → arbitration → slave, W follows locked grant
3. **Read Path:** AR → arbitration → slave, R returns via the slave adapter's in-order `bridge_id` FIFO
4. **Arbitration:** Per-slave round-robin, AW and AR only, with the grant locked until the ADDRESS handshake – not until burst complete. W is owned via a FIFO; B and R have no arbiter.
5. **Response Routing:** each slave adapter keeps an IN-ORDER FIFO of the originating master's `bridge_id`, pushed on the address handshake and popped on the response. Routing is keyed on FIFO POSITION, not on the returned BID/RID – IDs are pass-through and never widened. There are no ID lookup tables; `bridge_cam.sv` is instantiated in zero generated bridges.

Key Features: - Configurable NxM topology - One fabric clock; an AXI4 slave port declared `cdc = true` runs on its own clock behind `axi4_cdc_{wr,rd}` (HAS 4.5a), and an `apb`/`apb5` slave crosses domains inside its own shim (two async FIFOs) - Crossbar options: `xbar_pipeline = true` (registered slave-side channels – needed on the Artix-7 at 100 MHz beyond a plain 2x2, HAS 6.4) and `arbitration = "qos"` with `qos_aging_shift` (HAS 6.8a) - Out-of-context synthesis of any fixture: `projects/components/fabric-gen-ip/bridge/fpga/` (`make synth BRIDGE=<fixture> PART=<part> CLK_NS=<ns>`), measured tables in HAS 5.3 - Subtractive catch-all: an unmapped address gets DECERR + 0xDEADBEEF and a sticky status/IRQ instead of hanging the master (HAS 4.5)

Deliberately NOT supported – these were claimed here for months and none of them is built: - Out-of-order responses. Ordering is structural: one target slave per master at a time (`aw_gate_ok`), and each slave port must return B/R in request order across all IDs (bridge BUG-008, was BRIDGE-010). - Burst-locked arbitration. The grant releases at the address handshake, as item 4 above already said – this bullet contradicted it directly. - Configurable arbitration policy. Round-robin is hard-coded.

See: - `projects/components/fabric-gen-ip/bridge/PRD.md` - Complete specification - `projects/components/fabric-gen-ip/bridge/bin/bridge_generator.py` - Generator implementation

4.11.2 Q: "How do I generate a Bridge?"

A: Use the bridge_generator.py tool:

```
cd projects/components/fabric-gen-ip/bridge/bin
python3 bridge_generator.py --ports test_configs/bridge_2x2_rw.toml
# Auto-finds test_configs/bridge_2x2_rw_connectivity.csv
# Or regenerate everything: make all  (bulk from bridge_batch.csv)
```

Generated files: - RTL: `projects/components/fabric-gen-ip/bridge/rtl/generated/<bridge_name>/` (top module, crossbar, adapters, package) - Contains complete 5-channel crossbar with ID tracking

4.11.3 Q: "How do I test a Bridge?"

Import the generated TB for that configuration; the runner is generated beside it.

```
cd projects/components/fabric-gen-ip/bridge/dv/tests
make clean-all && make run-bridge_2x2_rw-gate   # serial by design,
~1GB per test
```

`dv/tests/test_bridge_2x2_rw.py` is the full shape.

4.12 PDF Generation Location

Both books build into `docs/` through the wrapper scripts, which apply the house style:

```
cd projects/components/fabric-gen-ip/bridge/docs
./generate_has_pdf.sh    # Bridge_HAS_v<rev> from bridge_has/
./generate_mas_pdf.sh    # Bridge_MAS_v<rev> from bridge_mas/
```

They call `bin/md_to_docx.py`, which follows the index links, demotes headings and builds the ToC. Pipeline detail: `vault/handbook/authoring/doc-pipeline.md`.

Maintained By: RTL Design Sherpa Project

(The version and date for this file are in the header at the top. A second stamp here said 1.0 / 2025-10-18 while the header said 2.1 / 2025-11-03 – two stamps in one file guarantee one of them is wrong.)

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

5 Document Information

5.1 Overview

5.1.1 Bridge Micro-Architecture Specification

Property	Value
Document Title	Bridge Micro-Architecture Specification
Version	1.7
Date	September 8, 2026
Status	Released
Classification	Open Source - MIT License

5.1.2 Document Purpose

This Micro-Architecture Specification (MAS) documents how the Bridge component is built:

- Block-level architecture and internal structure
- FSM state diagrams and transition tables
- ID management (and the CAM architecture that was never built)
- Signal-level interface details
- Width and protocol converter implementation
- Timing diagrams and debug information

Yes, the CAM gets called out in its own bullet — it was documented, never built, and the pages say so rather than letting you go looking for it in the RTL. For high-level architecture, system integration, and performance overview, refer to the companion **Bridge Hardware Architecture Specification (HAS)**.

5.1.3 Intended Audience

- RTL designers implementing or modifying Bridge
- Verification engineers developing testbenches
- System integrators debugging Bridge behavior
- Technical staff requiring implementation details

5.2 Design Notes

5.2.1 Revision History

Version	Date	Author	Description
1.0	2026-01-03	RTL Design Sherpa	Initial release - restructured from single spec
1.1	2026-06-04	RTL Design Sherpa	Content sync to RTL state at 2026-06-04. Adds (1) monitor-aggregation discussion in the crossbar-core chapter — per-port `axi4_{master,slave}_{rd,wr}_mon` wrappers feed a `monbus_arbiter` tree into a single bridge-top `monbus_axil4_axil4_group`, packet+timestamp carried atomically through a 192-bit skid; references the canonical 128-bit packet spec at `docs/markdown/rtl-amba/includes/monitor_package_spec.md`; (2) generator-emitted protocol-conversion shims at the slave

Version	Date	Author	Description
			boundary — `axi4_to_axil4_{rd,wr}` for AXIL slaves, chained with the AXIL→APB shim for APB slaves; (3) AXIL→wider-slave master-side alignment converters in the width-converters chapter (`axil_to_axi4_wide_align_{rd,wr}` — partial-word alignment, not protocol bridging); (4) generated-RTL chapter now lists monitor-variant module emission and per-port `(UNIT_ID, AGENT_ID)` assignment conventions; (5) verification-strategy chapter rewritten for protocol-BFM-only TBs with memory-backed slaves, the new `boundary_probe` test pattern, and

Version	Date	Author	Description
			the four `test_bridge_*_monitor_*.py` canonical patterns (smoke/capture/error_inject/irq); (6) response-routing chapter notes the independent W-tracker FIFO + split AW/W-ready MUX fix (commit `d24bd617`) that closed an interleaved-master deadlock.
1.2	2026-09-08	RTL Design Sherpa	Correctness and voice pass. Four external qc rounds (65/54/53/57 findings, all triaged against rtl/generated and the generator) removed the planned-but-never-built ID-table/CAM out-of-order scheme, the fixed 64-bit internal path, response arbitration/FIFOs and the AXI4-Lite-as-future-work status from the

Version	Date	Author	Description
			text; annotated the TOML keys the loader has never known and its three-tier unknown-key behaviour; replaced asserted latency figures with measured ones. Four RTL defects found by the rounds are fixed and mutation-checked (bridge BUG-009 (was BRIDGE-011) response-FIFO overrun, axi4_subtractive_slave simultaneous AW+AR fault loss, axi5_atomic_filter duplicate B and B-before-WLAST). Prose then voice-passed with all correctness annotations preserved.
1.3	2026-09-11	RTL Design Sherpa	bridge TASK-004 (was BRIDGE-014): AXI5-Lite and APB/APB5 as master protocols (front-end

Version	Date	Author	Description
			converters); bridge TASK-005 (was BRIDGE-016) slave ID widening
1.4	2026-09-11	RTL Design Sherpa	bridge TASK-008 (was BRIDGE-019): Wishbone B4 on both sides of the fabric
1.5	2026-09-11	RTL Design Sherpa	bridge TASK-006 (was BRIDGE-017): measured throughput and latency, registered crossbar, QoS arbitration, CDC slave ports, out-of-context synthesis flow and measured resources, CAM ordering-count fix
1.6	2026-09-13	RTL Design Sherpa	bridge TASK-007 (was BRIDGE-018): native AXI5 fabric – Memory Tagging and read-data chunking through the structs
1.7	2026-09-13	RTL Design Sherpa	Document sync: one fabric clock plus CDC ports throughout, Wishbone best-effort stance, out-of-order chunk

Version	Date	Author	Description
			proof

5.3 References

5.3.1 Related Documents

- **Bridge HAS** - Hardware Architecture Specification (high-level)
- **PRD.md** - Product requirements and overview
- **CLAUDE.md** - AI development guide

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

6 Overview

6.1 Overview

6.1.1 Bridge Micro-Architecture

Bridge is a multi-protocol AXI4 crossbar generator: it turns CSV/TOML configuration files into parameterized SystemVerilog RTL. This document describes what that RTL looks like on the inside.

6.1.2 Multi-Protocol Support

Bridge supports three AMBA protocols:

Table 1.1: Supported Protocols

Protocol	Use Case	Conversion
AXI4 Full	High-bandwidth memory	Direct or width convert
AXI4-Lite	Register access	Protocol downgrade
APB	Low-speed peripherals	Full conversion

6.1.3 Channel-Specific Masters

Masters that only read or only write don't pay for the channels they never use:

Table 1.2: Channel-Specific Master Types

Type	Channels	Signals	Use Case
Full (rw)	AW, W, B, AR, R	100%	CPU, config master
Write-only (wr)	AW, W, B	~60%	DMA write engine
Read-only (rd)	AR, R	~40%	DMA read engine

6.1.4 Automatic Converters

Bridge inserts converters automatically:

- **Width converters** - Handle data width mismatches
- **Protocol converters** - AXI4 to APB conversion

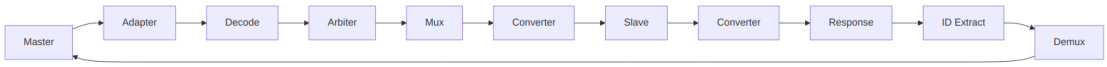
- **Per-path optimization** - Only where needed

You never instantiate these by hand; the generator looks at each master/slave pairing and drops in exactly the conversion that path needs.

6.2 Functional Description

6.2.1 Block Organization

```
Bridge NxM
├── Master Adapters (M instances)
│   ├── Skid buffers
│   └── sideband bridge_id (AXI IDs pass through unchanged)
├── Crossbar Core
│   ├── Address decode
│   ├── Per-slave arbiters
│   └── Channel muxes
├── Slave Adapters (N instances)
│   ├── Protocol shims (axi4_to_apb4_shim / axi4_to_axil4_*)
│   └── In-order bridge_id FIFO (response tracking)
├── Subtractive slave (always present, internal)
│   └── Answers unmapped addresses: DECERR + 0xDEADBEEF + sticky
status
└── Response Routing
    ├── bridge_id FIFO head selects the master
    └── Master demux (the AXI ID is passed through untouched)
```



Signal Flow

6.3 References

6.3.1 Related Documentation

- **Bridge HAS** - High-level architecture and integration
- **PRD.md** - Product requirements
- **Generator source** - `bin/bridge_generator.py` (with `bin/bridge_pkg/`)

6.4 Navigation

6.4.1 Document Organization

Table 1.3: Document Organization

Chapter	Content
Ch 2	Block Descriptions
Ch 3	FSM Design
Ch 4	ID Management
Ch 5	Converters
Ch 6	Generated RTL
Ch 7	Verification

7 2.1 Master Adapter

7.1 Overview

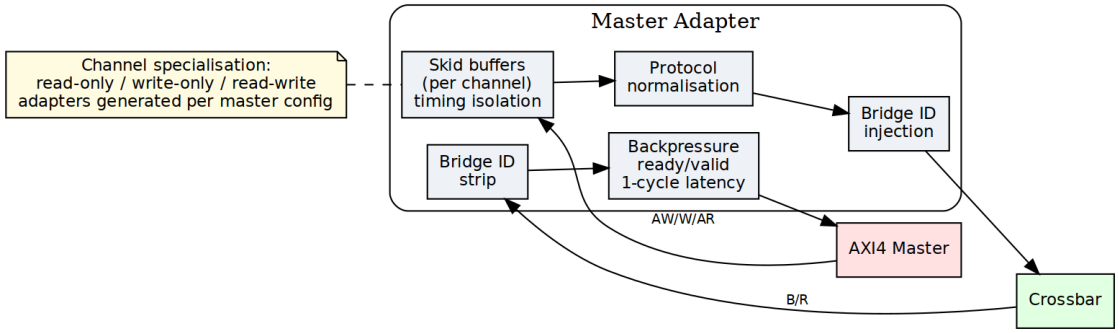
Every master port in the bridge gets its own Master Adapter. The adapter sits between the master and the crossbar core, and it exists to make the crossbar's life simple: timing is isolated at the boundary, channels are specialized to what the master actually uses, and every transaction carries a Bridge ID before it enters the fabric.

7.1.1 Purpose and Function

Concretely, the adapter does five things:

1. **Timing Isolation**: Inserts pipeline registers (skid buffers) to break combinatorial paths from master to crossbar
2. **Channel Specialization**: Separates read-only, write-only, and read-write masters for optimal resource utilization
3. **Bridge ID Injection**: Adds internal tracking IDs to transactions for response routing
4. **Protocol Normalization**: Ensures all transactions meet crossbar assumptions and constraints
5. **Backpressure Management**: Handles ready/valid handshaking with single-cycle latency

7.1.2 Figure 2.1: Master Adapter Architecture



Master Adapter Architecture

The figure shows Master Adapter architecture with per-master skid buffers, bridge ID injection/stripping, and channel specialization for rd/wr/rw configurations.

7.2 Parameters

Everything here comes straight out of the TOML/CSV configuration — the generator does the arithmetic, you just declare the ports.

7.2.1 Per-Master Parameters (from TOML/CSV)

```
[[bridge.masters]]
name = "cpu"
channels = "rw"              # "rd", "wr", or "rw"
arid_width = 4               # External ID width
awid_width = 4               # Can differ from ARID
addr_width = 32              # Address bus width
data_width = 64              # Data bus width
# pipeline_depth: NOT A KEY. Skid depth is set per channel by the
generator;
# the contract is 2..8 inclusive, so 1 is rejected.
```

7.2.2 Global Parameters (affect all adapters)

```
[bridge]
# internal_data_width: NOT A KEY -- there is no fixed internal
crossbar width.
enable_width_conversion = true
bid_width = 2                # Calculated: clog2(num_masters)
```

7.3 Ports

Two interfaces per adapter: one facing the master, one facing the fabric. Note the ID widths change across that boundary — wider on the inside.

7.3.1 External Master Interface (per master)

```
Input Channels (from master):
- ARVALID, ARREADY, ARADDR, ARBURST, ARCACHE, ARID, ARLEN,
  ARLOCK, ARPROT, ARQOS, ARSIZE

- AWVALID, AWREADY, AWADDR, AWBURST, AWCACHE, AWID, AWLEN,
  AWLOCK, AWPROT, AWQOS, AWSIZE

- WVALID, WREADY, WDATA, WSTRB, WLAST

Output Channels (to master):
- RVALID, RREADY, RDATA, RID, RLAST, RRESP

- BVALID, BREADY, BID, BRESP
```

7.3.2 Internal Crossbar Interface (per master)

```
Output Channels (to crossbar):
- AR* signals + Internal ARID (wider)
- AW* signals + Internal AWID (wider)
- W* signals (unchanged)

Input Channels (from crossbar):
- R* signals + Internal RID (wider)
- B* signals + Internal BID (wider)
```

7.4 Functional Description

7.4.1 Channel Specialization

The Bridge generator creates three types of master adapters based on the channel usage specified in the configuration:

7.4.1.1 Read-Only Masters

Channels: AR (Address Read), R (Read Data)

```
Configuration: "rd" or "read"
RTL Generated: adapter_master_rd_<id>.sv

Optimizations:
- No write address channel logic
- No write data channel logic
- No write response channel logic
- Reduced arbitration participation (read path only)
```

7.4.1.2 Write-Only Masters

Channels: AW (Address Write), W (Write Data), B (Write Response)

```
Configuration: "wr" or "write"
RTL Generated: adapter_master_wr_<id>.sv

Optimizations:
- No read address channel logic
- No read data channel logic
- Reduced arbitration participation (write path only)
```

7.4.1.3 Read-Write Masters

Channels: AR, R, AW, W, B (Full AXI4 interface)

```
Configuration: "rw" or "readwrite"
RTL Generated: adapter_master_rw_<id>.sv

Full Functionality:
- All five AXI4 channels implemented
- Participates in both read and write arbitration
- Maximum flexibility but larger resource footprint
```

7.4.2 Per-Port Monitor Wrappers (when `use_monitor = true`)

When the bridge TOML configuration sets `use_monitor = true` for a master port, the generator instantiates per-direction monitor wrappers immediately downstream of the master adapter:

- **axi4_master_rd_mon**: Wraps the AR/R channels, emits packets with UNIT_ID=1
- **axi4_master_wr_mon**: Wraps the AW/W/B channels, emits packets with UNIT_ID=1

Each wrapper: - Samples all channel signals and timestamp at handshake points - Emits 128-bit monitor packets on internal monbus - Passes signals through transparently in the common case, but it CAN stall: `axi4_master_rd_mon` gates the address channel on the monitor's back-pressure, `assign w_gated_arvalid = fub_axi_arvalid & (w_block_ready | ~cfg_monitor_enable);` so a saturated transaction table holds off new addresses until it drains. With `cfg_monitor_enable` low the gate is bypassed and the path really is transparent. - Is instantiated only when requested (generator-time configuration)

The generated per-port monbus streams are later aggregated by a tree of `monbus_arbiter` instances at the bridge top.

7.4.3 Skid Buffer Architecture

Each AXI4 channel passes through a skid buffer for timing isolation. The skid buffer implements:

7.4.3.1 Registered Forward Path

```
// Simplified skid buffer concept
always_ff @(posedge clk) begin
    if (!rst_n) begin
        valid_reg <= 1'b0;
    end else if (!valid_reg || ready_out) begin
        valid_reg <= valid_in;
        data_reg  <= data_in;
    end
end
```

7.4.3.2 Single-Cycle Backpressure Response

- When downstream is not ready, accepts one additional beat into holding register
- Signals ready_in = 0 in same cycle to upstream
- No combinatorial paths between upstream and downstream

7.4.3.3 Pipeline Depth

- Default: 2 (the `gaxi_skid_buffer` minimum)
- Range: **2..8 inclusive, any integer**. The value is the skid buffer's `DEPTH`, and its elaboration guard rejects anything outside that: `if (DEPTH < 2 || DEPTH > 8) $error(...)`. A depth of 1 does not elaborate, so the "default 1" this line used to claim was never buildable.
- Trade-off: Latency vs. timing closure

Performance Impact: - Adds 1 cycle latency per channel - Enables higher clock frequencies - Prevents critical paths through crossbar

7.4.4 Bridge ID Management

> **Not built.** IDs are NOT injected, extended or stripped anywhere in this bridge. `cpu_m_axi_awid` and `ddr_s_axi_awid` are both 4 bits in `bridge_2x2_rw`: the master's ID passes through untouched and comes back untouched. The originating master travels as a SEPARATE SIDEBAND signal (`xbar_bridge_id_aw/_ar`) into a per-slave in-order FIFO, and responses are routed by that FIFO's POSITION – the returned BID/RID is never consulted. The section below describes the scheme that was replaced. See ch04 `02_id_tracking.md`.

7.4.4.1 ID Width Calculation

```
BID_WIDTH = clog2(num_masters)

Examples:
- 2 masters  → BID_WIDTH = 1
- 4 masters  → BID_WIDTH = 2
- 8 masters  → BID_WIDTH = 3
- 16 masters → BID_WIDTH = 4
```

7.4.4.2 ID Injection (Request Path)

For each master adapter, a unique constant Bridge ID is appended to the AXI transaction ID:

```
AR Channel:
  Internal ARID = {MASTER_BID[BID_WIDTH-1:0], External
ARID[ARID_WIDTH-1:0]}
  Internal ARID Width = BID_WIDTH + ARID_WIDTH

AW Channel:
  Internal AWID = {MASTER_BID[BID_WIDTH-1:0], External
AWID[AWID_WIDTH-1:0]}
  Internal AWID Width = BID_WIDTH + AWID_WIDTH
```

Example: 4-master system, external ARID_WIDTH = 4

```
Master 0: BID = 2'b00, External ID = 4'h3 → Internal ID = 6'b00_0011
Master 1: BID = 2'b01, External ID = 4'h3 → Internal ID = 6'b01_0011
Master 2: BID = 2'b10, External ID = 4'h5 → Internal ID = 6'b10_0101
Master 3: BID = 2'b11, External ID = 4'hA → Internal ID = 6'b11_1010
```

7.4.4.3 ID Stripping (Response Path)

The Bridge ID is removed from responses before returning to the master:

```
R Channel:
  External RID = Internal RID[ARID_WIDTH-1:0]
  Bridge routes based on Internal RID[BID_WIDTH+ARID_WIDTH-
1:ARID_WIDTH]

B Channel:
  External BID = Internal BID[AWID_WIDTH-1:0]
  Bridge routes based on Internal BID[BID_WIDTH+AWID_WIDTH-
1:AWID_WIDTH]
```

This ensures: - Master sees original transaction IDs - Bridge internally tracks which master originated each transaction - Responses route back to correct master even with ID reuse across masters

7.4.5 Protocol Normalization

The Master Adapter enforces several protocol requirements:

7.4.5.1 Valid Address Ranges

- Checks addresses against slave address maps
- Flags out-of-range accesses for error handling
- Prevents deadlocks from illegal addresses

7.4.5.2 Burst Constraints

- Validates ARLEN/AWLEN vs. 4KB boundary rules
- Ensures burst types are supported (FIXED, INCR, WRAP)
- Checks SIZE vs. DATA_WIDTH compatibility

7.4.5.3 Signal Defaults

- Provides default values for unused signals
- Ensures ARCACHE/AWCACHE have legal values
- Sets ARPROT/AWPROT based on configuration

7.4.6 Response Path Slave Selection Tracking

7.4.6.1 Problem: Stale Slave Select Decode

Here's the part that bites people. The master adapter decodes the incoming address combinationally to pick a target slave. When the wrapper's skid buffer pops and `fub_axi_awaddr`/`fub_axi_araddr` reverts, the decode changes even though the converter is still in flight pushing the request to the crossbar. This causes the response MUX to route read/write responses from the wrong slave.

7.4.6.2 Solution: Per-Channel Response-Tracking FIFOs

Added separate FIFOs for AR and AW channels that capture the slave index (`comb_slave_select_ar/aw`) at the moment of the FUB-level handshake:

```
// At request time (when fub_axi_arvalid && fub_axi_arready)
ar_trk_push <= comb_slave_select_ar;  // Capture decoded slave index

// At response time (when m_axi_rvalid && m_axi_rready && m_axi_rlast)
r_slave_select <= ar_trk_pop;  // Stable slave index for response MUX
```

Benefits: - R/B responses route correctly even after address reverts - Supports multi-slave masters spanning different widths - Stable path for both data width and target slave

Implementation: - One FIFO per (master, AW/AR channel) - Width: `clog2(NUM_SLAVES)` bits (4 bits typical for 16-slave systems) - Depth: Matches maximum outstanding transactions (8-16 typical)

7.5 Timing

7.5.1 Latency

Request Path (Master → Crossbar): 1 cycle – one registered skid stage.

Response Path (Crossbar → Master): 1 cycle – one registered skid stage.

Total: 2 cycles each way through the bridge (one skid in the master adapter, one in the slave adapter). Measured, not derived: see HAS Table 5.7 and `test_bridge_2x2_rw_latency`, which asserts 2 exactly.

> **Depth is not latency.** This block read "with 4-stage pipeline: 4 cycles" and a round trip of "2-8 cycles depending on pipeline depth". A `gaxi_skid_buffer` presents its output one cycle after the write regardless of DEPTH; the depth sets how many beats it can ABSORB under backpressure, not how long a beat takes to cross it. There is also no ID injection or stripping to cost anything – IDs pass through untouched.

7.5.2 Throughput

- **Maximum**: 1 transaction per cycle per channel (fully pipelined)
- **Backpressure**: Propagates with 1-cycle latency
- **Burst Performance**: No degradation; bursts flow continuously when ready

7.5.3 Critical Paths

Typical critical paths (if skid buffers not used): - Master ARVALID → Crossbar arbitration logic - Crossbar grant → Master ARREADY - Slave RDATA → Master RDATA

Solution: Skid buffers break all these paths at the cost of 1-cycle latency.

7.6 Design Notes

7.6.1 Resource Utilization

7.6.1.1 Per-Master Adapter Resources (Typical)

Read-Write Master (64-bit data, 32-bit addr, 4-bit ID):

```
Logic Elements:  ~200-400 (depending on ID width and features)
Registers:       ~150-250 (pipeline stages + control)
Block RAM:       0 (no CAM anywhere in the design)

Breakdown:
- Skid buffers (5 channels × ~30 regs each): ~150 regs
- ID manipulation logic: ~50 LEs
- Control FSMs: ~100 LEs
- Routing logic: ~50 LEs
```

Read-Only Master: ~60% of read-write resources
Write-Only Master: ~65% of read-write resources

7.6.1.2 Scaling Considerations

Resource usage scales with: - Number of masters (linear scaling, one adapter per master) - ID width (logarithmic: +BID_WIDTH bits per transaction) - Pipeline depth (linear: deeper pipelines = more registers) - Data width (linear: wider data = wider skid buffers)

7.6.2 Future Enhancements

7.6.2.1 Planned Features

- **Dynamic Pipeline Depth**: Adjust depth based on operating frequency
- **FIFO Mode**: Deeper buffering (16-256 entries) for burst-intensive masters
- **QoS Support**: Priority-based arbitration hints
- **Performance Counters**: Transaction counts, stall cycles, utilization metrics

7.6.2.2 Under Consideration

- **Clock Domain Crossing**: Per-master clock domains with async FIFOs
- **Width Conversion at Adapter**: Move width logic to adapters for distributed conversion
- **Outstanding Transaction Tracking**: Windowing for improved OOO performance

7.7 Related Modules

- Section 2.3: Crossbar Core (interconnect architecture)
- Section 2.4: Arbitration (how adapters compete for slaves)
- Section 2.5: ID Management (sideband bridge_id; the CAM it describes was never built)
- HAS ch04_interfaces/01_axi4_interface.md (port signals)

7.8 Testing

7.8.1 Debug and Observability

7.8.1.1 Recommended Debug Signals

For ILA or waveform capture:

```
- Master adapter input valid/ready (all channels)
- Master adapter output valid/ready (all channels)
- Bridge ID assignments per transaction
- Pipeline stage occupancy
- Backpressure events (ready = 0 while valid = 1)
```

7.8.1.2 Common Issues and Debug

Symptom: Master hangs waiting for READY
Check: - Is adapter receiving grants from arbiters? - Is slave responding to requests? - Is response path clear?

Symptom: Incorrect data returned to master
Check: - Bridge ID extraction logic - ID width mismatches - Response routing in crossbar

Symptom: Timing violations
Check: - Increase pipeline_depth parameter - Verify clock frequency vs. design complexity - Check for combinatorial loops

8 2.2 Slave Router

8.1 Overview

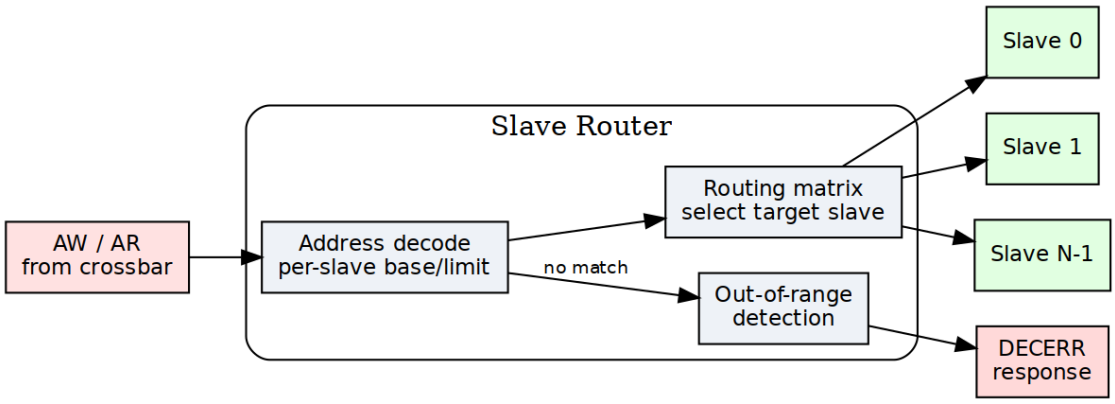
Every master gets its own Slave Router. The router examines each request address, steers the transaction to one of the configured slaves, and produces the error response itself when the address lands nowhere.

8.1.1 Purpose and Function

The router does five things:

1. **Address Decoding**: Matches request addresses against configured slave address ranges
2. **Request Routing**: Directs AR/AW/W channels to the selected slave
3. **Out-of-Range Detection**: Identifies addresses that don't map to any slave
4. **Error Response Generation**: Creates DECERR responses for invalid addresses
5. **Default Slave**: unmapped addresses go to an ALWAYS-PRESENT internal subtractive slave (`axi4_subtractive_slave`), which answers DECERR and returns `0xDEADBEEF` on reads. It is not optional and there is no `default = true` TOML key – the generator inserts it into every bridge.

8.1.2 Figure 2.2: Slave Router Architecture



Slave Router Architecture

Slave router architecture showing address decoding, routing matrix, and out-of-range detection for AW and AR channels.

8.2 Parameters

8.2.1 Per-Router Parameters

```
# Router behavior is defined by slave configurations

[[bridge.slaves]]
name = "ddr_memory"
base_address = 0x8000_0000
size = 0x4000_0000           # 1 GB
default = false
# (no oor_data_pattern knob: READ_FILL is a module parameter,
0xDEADBEEF)
```

8.2.2 Global Parameters

```
[bridge]
enable_default_slave = false      # Allow default slave
strict_address_decode = true      # Flag overlapping ranges as errors
# (no oor_response_latency knob: the responder answers as fast as the
#  handshake allows; there is nothing to tune)
```

8.3 Functional Description

8.3.1 Address Decoding Algorithm

8.3.1.1 Configuration-Based Address Maps

Each slave is configured with:

```
[[bridge.slaves]]
name = "memory"
base_address = 0x4000_0000
size = 0x1000_0000        # 256 MB
default = false           # Not a default slave
```

The router generates address ranges:

```
Start Address = base_address
End Address   = base_address + size - 1
```

8.3.1.2 Decoding Priority

When multiple slaves have overlapping address ranges, the router uses **first-match priority**:

```
Priority Order = Order in configuration file (top to bottom)

Example:
  Slave 0: 0x0000_0000 - 0x0FFF_FFFF  (checked first)
  Slave 1: 0x1000_0000 - 0x1FFF_FFFF  (checked second)
  Slave 2: 0x2000_0000 - 0x2FFF_FFFF  (checked third)

Address 0x1000_5000 → Matches Slave 1
```

8.3.1.3 Range Checking Logic

For each address, the router performs:

```
// Simplified address decode logic
logic [NUM_SLAVES-1:0] slave_match;

for (int i = 0; i < NUM_SLAVES; i++) begin
    slave_match[i] = (addr >= SLAVE_BASE[i]) &&
                     (addr <= SLAVE_END[i]);
end

// Priority encode: Select first matching slave
logic [$clog2(NUM_SLAVES)-1:0] slave_select;
always_comb begin
    slave_select = 0;
    for (int i = 0; i < NUM_SLAVES; i++) begin
```

```
        if (slave_match[i]) begin
            slave_select = i;
            break;  // First match wins
        end
    end
end

// Out-of-range detection
logic oor = ~(|slave_match);  // No slaves matched
```

8.3.1.4 Subtractive decode: the `else` that removes the hang

The pseudocode above computes an out-of-range flag; the generated RTL goes further and gives that case a destination. The emitted decode chain ends in a bare `else` selecting an internal catch-all slave:

```
comb_slave_select_aw = '0;
if      (fub_axi_awaddr <= 32'h3FFFFFFF)  comb_slave_select_aw[0] =
1'b1;  // ddr
else if (fub_axi_awaddr >= 32'h40000000)  comb_slave_select_aw[1] =
1'b1;  // scratch
else                                      comb_slave_select_aw[3] =
1'b1;  // subtractive
```

So the one-hot is **never all-zero**. Before 2026-09-07 there was no `else`: an unmapped address selected nothing, the AW-ready MUX fell through to `default: // No slave selected` with `awready` low, and the master hung forever. The hang is now impossible by construction rather than handled.

8.3.1.5 Three decoders, not one

The catch-all is a synthetic slave appended last to the slave list, so it reuses the crossbar's routing rather than needing a new datapath. What made that harder than it sounds is that **three independent places derive the slave list for themselves**, and each needed telling:

Where	What it derives	What the catch-all needed
master adapter	the if/else decode above	nothing – subtractive semantics come free from the `else`
crossbar	its own per-slave `_to_` terms, from address RANGES	the term must be `!(other ranges)`; a full-span slave otherwise reduces to `1'b1`
cfg regblock	per-slave configuration registers	exclusion – it has no monitor wrapper to

Where	What it derives	What the catch-all needed
		configure

The crossbar case is the instructive one. Its decode is derived from each slave's address range independently of the adapter's chain, so a full-span catch-all became `wire cpu_32b_aw_to_subtractive = 1'b1;` – matching every address. Every transaction then routed to the real slave *and* the catch-all at once, and because the payload muxes OR their inputs, the master read back `addr = real | catchall`. Silent corruption, which is worse than the hang it replaced. The term is now the negation of the other ranges, **inlined** rather than referencing the sibling `<master>_<suffix>_<channel>_to_<slave>` wires: those are emitted per channel, so a slave with no read path from a given master has no `ar_to_` wire at all and naming it does not compile.

The cfg regblock case cost a whole debug cycle for a subtler reason: emitting per-slave cfg for the catch-all added 52 fields and **renumbered every register after them**, which moved the monitor-enable bits. One monbus stress test saw `pkts=0` while all 69 other tests passed, because only that test depended on register offsets. A register map is an ABI.

8.3.1.6 Power-of-Two Optimization

For slaves with power-of-two sizes starting at aligned addresses, simplified decode:

```
// Optimized decode for: base=0x8000_0000, size=0x1000_0000 (256MB)
// Mask off lower bits: Only check upper bits
logic match = (addr[31:28] == 4'h8);  // Much simpler than range check
```

The generator automatically detects and applies this optimization.

8.3.2 Request Routing

8.3.2.1 AR Channel Routing

Read address routing flow: 1. **Decode**: Determine target slave from ARADDR 2. **Check OOR**: If no slave matches, flag for error response 3. **Route**: Send AR transaction to selected slave's arbiter 4. **Track**: Remember routing decision for R channel responses

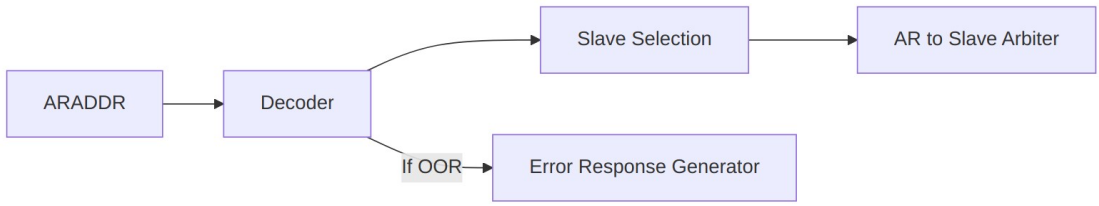


Diagram 2

8.3.2.2 AW/W Channel Routing

Write transactions require coordinated routing:

1. **AW Phase**:
   - Decode AWADDR to determine target slave
   - Route AW transaction to selected slave's arbiter
   - **Store routing decision** for subsequent W beats
2. **W Phase**:
   - **Follow AW routing** (W channel has no address)
   - Route all W beats to same slave as AW
   - Continue until WLAST = 1

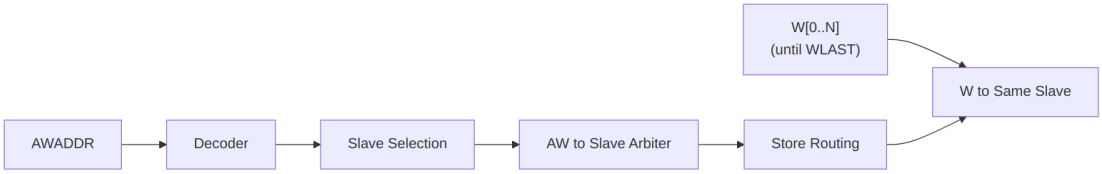


Diagram 3

8.3.2.3 Write Data Tracking FSM

```
State Machine for W Channel Routing:

IDLE:
  - Wait for AWVALID && AWREADY
  - On handshake: Store target slave, goto WRITING

WRITING:
  - Route W beats to stored slave
  - On WVALID && WREADY && WLAST: goto IDLE

Error Handling:
  - If AW was OOR: Discard W beats, generate BRESP error
```

8.3.3 Out-of-Range Handling

8.3.3.1 Detection

An address is out-of-range if it matches no slave's range. The decode chain ends in an `else`, so such an address is not "detected and handled" – it is *routed*, to an internal subtractive slave that always answers.

8.3.3.2 Error Response Generation

The responder is `rtl/amba/axi4/axi4_subtractive_slave.sv`, instantiated by the generator as the last (internal) slave. It emits no top-level pins of its own; its behaviour is fixed, not configurable.

Read (AR -> R)

```
1. Accept ARVALID (ARREADY high while no read is active)
2. Return exactly ARLEN+1 beats:
     RID   = ARID, unmodified (IDs are pass-through -- nothing is
prepended)
     RDATA = 0xDEADBEEF, replicated to the data width
     RRESP = 2'b11 (DECERR)
     RLAST = 1 on the FINAL beat only
```

RLAST on every beat – which an earlier revision of this page specified – would make a burst master terminate early. The RTL asserts it as `r_r_active && (r_beats_left == 0)`, and the module test checks that it appears on the last beat and no other.

Write (AW/W -> B)

```
1. WREADY is unconditionally high -- W beats are sunk whether or not
AW has
   arrived yet. AXI4 permits write data before its address, and gating
WREADY
   on having seen AW deadlocks such a master.
2. One B per AW:
     BID   = AWID, unmodified
     BRESP = 2'b11 (DECERR)
3. Write data is discarded. There is nowhere for it to go.
```

Status. The first unmapped access latches a sticky flag with its address and a saturating count, raised on `unmapped_irq` and, on cfg-regblock builds, readable and clearable via `SUBTRACTIVE_STATUS` / `SUBTRACTIVE_ADDR`. See HAS 4.5.

Reachability. The catch-all is the decode `else`, so it can only fire if the slave ranges leave a gap. Of the 22 generated bridges, 18 tile the address space completely and the branch is unreachable logic that synthesis removes.

8.3.3.3 Read data pattern

`READ_FILL`, a module parameter, defaults to `32'hDEAD_BEEF` and is replicated to the bus width. It is not run-time configurable and there is no menu of options: an earlier revision of this page offered four (zeros, a debug signature, an address echo, a master/slave ID indicator), none of which was ever built.

The value matters more than the choice. Zeros are indistinguishable from real memory, so an all-zero error return reads as a plausible value and the fault stays hidden; `0xDEADBEEF` in a dump is unambiguous. Address-echo variants sound useful but the address is already captured in `SUBTRACTIVE_ADDR`, where software can read it without decoding it out of the data bus.

8.3.4 Default Slave Support

8.3.4.1 Configuration

```toml
[[bridge.slaves]]
name = "error_responder"
```

```
base_address = 0x0          # Ignored for default slave
size = 0x0                  # Ignored for default slave
default = true              # Catch-all for unmapped addresses
```

8.3.4.2 Behavior

When a default slave is configured: - Addresses that don't match any specific slave → Routed to default slave - Default slave typically returns DECERR but with configurable response - Useful for prototyping (accept all addresses initially) - Can implement memory-mapped debug register for address capture

Note: Only ONE default slave allowed per bridge.

8.3.5 Address Aliasing

8.3.5.1 Multiple Slaves, Same Address

If configuration has overlapping ranges:

```
[[bridge.slaves]]
name = "fast_cache"
base_address = 0x8000_0000
size = 0x1000_0000

[[bridge.slaves]]
name = "slow_memory"
base_address = 0x8000_0000  # Same base!
size = 0x4000_0000
```

Result: First-match priority applies. All accesses go to `fast_cache`. The `slow_memory` range 0x9000_0000-0xBFFF_FFFF is **unreachable** from this master.

Warning: Generator can optionally flag this as error in DRC mode.

8.3.5.2 Intentional Aliasing Use-Cases

Legitimate uses of overlapping ranges: 1. **Cache hierarchy**: Small fast cache shadows larger slow memory 2. **Memory remapping**: Different views of same physical memory 3. **Peripheral mirroring**: Register block appears at multiple addresses

8.4 Timing

8.4.1 Decode Latency

Combinatorial Decode (Default): - Address → Slave selection: 0 cycles (combinatorial) - Critical path: ARADDR → Slave arbiter request - May limit maximum frequency in large systems

Registered Decode (Optional): - Address → Slave selection: 1 cycle (registered) - Adds latency but breaks critical path - Recommended for >8 slaves or >300 MHz operation

8.4.2 Throughput

- **Maximum**: 1 transaction per cycle per master
- **No blocking**: Router does not stall; arbiters handle conflicts
- **Pipelining**: AR and AW decode in parallel (independent paths)

8.4.3 Critical Paths

Typical critical paths: - ARADDR → Address comparators → Priority encoder → Arbiter request - w/4 slaves: ~10-15 logic levels (FPGA-dependent) - w/8 slaves: ~12-18 logic levels

Mitigation: - Enable registered decode mode (+1 cycle latency) - Use power-of-two slave sizes (simplified compare) - Synthesizer optimization directives

8.5 Design Notes

8.5.1 Resource Utilization

8.5.1.1 Per-Router Resources (Typical)

4-slave configuration (32-bit address):

```
Logic Elements:  ~150-200 LEs
Registers:       ~50 regs
Block RAM:       0

Breakdown:
- Address comparators (4 slaves × ~30 LEs): ~120 LEs
- Priority encoder: ~20 LEs
- W channel FSM: ~30 regs, ~20 LEs
- OOR error generator: ~20 regs, ~10 LEs
```

8-slave configuration:

```
Logic Elements:  ~250-350 LEs (scales roughly with slave count)
Registers:       ~60 regs
```

8.5.1.2 Scaling Considerations

Resource usage scales with: - **Number of slaves**: Linear (each slave adds comparator logic) - **Address width**: Minimal impact (wider comparators, but same structure) - **Optimizations**: Power-of-two sizes reduce logic significantly

8.5.2 Performance Optimization

1. Registered Decode (High Frequency)

```
Trade-off: +1 cycle latency for timing closure
Best for: >8 slaves, >300 MHz targets
```

2. Simplified Address Decode (Power-of-2)

```
Optimization: Mask-based compare instead of range check
Best for: Slaves with aligned, power-of-2 sizes
Savings: ~50% reduction in comparator logic
```

3. Parallel Decode (Low Logic)

```
Implementation: Separate decoders per address bit-field
Best for: Many non-overlapping slaves
Savings: Reduces logic depth for priority encoding
```

4. CAM-Based Decode (Many Slaves)

```
Trade-off: Uses block RAM for address lookup table
Best for: >16 slaves, non-contiguous ranges
Note: Requires RAM resources
```

8.5.3 Future Enhancements

8.5.3.1 Planned Features

- **Dynamic Address Remapping**: Runtime-configurable slave ranges
- **Transaction Filtering**: Block certain address ranges per-master
- **Priority Hints**: QoS-based prioritization (beyond first-match)
- **Address Translation**: Offset/mask transformations before slave routing

8.5.3.2 Under Consideration

- **Multi-region Slaves**: Slave spans multiple non-contiguous ranges
- **Secure Address Spaces**: Per-master access control lists
- **Debug Address Capture**: Log invalid addresses to register

8.6 Related Modules

- Section 2.1: Master Adapter (upstream from router)
- Section 2.3: Crossbar Core (downstream arbitration)
- Section 2.4: Arbitration (how routed requests compete)
- HAS ch04_interfaces/01_axi4_interface.md (slave port signals)

8.7 Testing

8.7.1 Debug and Observability

8.7.1.1 Recommended Debug Signals

```
- Address decode outputs (which slave matched)
- OOR flags (per channel)
- Default slave hit counter
- W channel FSM state
- Routing decision storage (for W tracking)
```

8.7.1.2 Common Issues and Debug

Symptom: Reads return all zeros
Check: - Is address out-of-range? - Check slave base/size configuration - Verify address decode logic in waveform

Symptom: Write data goes to wrong slave
Check: - W channel tracking FSM state - Did AW routing complete before W started? - Check for AWVALID/AWREADY handshake

Symptom: Unexpected DECERR responses
Check: - Address decode configuration - Overlapping slave ranges (wrong priority) - Off-by-one in size calculations

8.7.2 Verification Considerations

8.7.2.1 Address Decode Tests

Critical test scenarios: 1. **Boundary conditions**: base_address, base_address + size - 1 2. **Just out-of-range**: base_address - 1, base_address + size 3. **Each slave**: Verify routing to correct slave 4. **Overlapping ranges**: Verify priority encoding 5. **Default slave**: Unmapped addresses route correctly

8.7.2.2 Write Tracking Tests

W channel FSM testing: 1. **Simple write**: Single AW, single W (WLAST=1) 2. **Burst write**: AW with AWLEN=7, eight W beats 3. **Back-to-back writes**: New AW before previous W completes 4. **Interleaved masters**: Multiple masters writing simultaneously (if supported)

8.7.2.3 OOR Error Tests

```
Test: Read from unmapped address
- Send AR to 0xFFFF_FFFF (assuming unmapped)
- Verify R response with RRESP = DECERR
- Verify RID matches ARID
- Check response latency

Test: Write to unmapped address
```

```
- Send AW to invalid address
- Send W data
- Verify B response with BRESP = DECERR
- Verify BID matches AWID
```

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 2.3 Crossbar Core

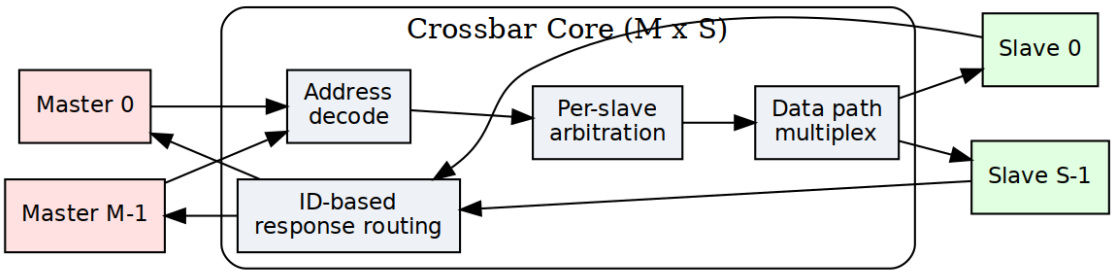
The crossbar core is the fabric itself — any master can reach any slave through it. Arbitration, request routing, and response management, the machinery that makes any-to-any work, all live here.

9.1 Overview

The core does five things:

1. **Full Connectivity**: Provides complete N×M master-to-slave interconnect matrix
2. **Independent Arbitration**: Per-slave arbiters allow parallel access to different slaves
3. **Response Routing**: Directs slave responses back to originating masters using Bridge IDs
4. **Transaction Ordering**: Maintains AXI ordering requirements within address dependencies
5. **Backpressure Management**: Handles flow control across multiple concurrent transactions

9.1.1 Figure 2.3: Crossbar Core Architecture



Crossbar Core Architecture

Crossbar core architecture showing complete M×S switching fabric with address decode, per-slave arbitration, data path multiplexing, and ID-based response routing.

9.2 Parameters

The crossbar-level knobs come straight from the TOML. Read the comments as carefully as the keys — several entries you'd expect to find here don't exist, and the file says so:

```
[bridge]
num_masters = 4
num_slaves = 3
# internal_data_width: NOT A KEY -- the crossbar has no fixed internal
width;
# each path carries its own port width and converters sit at the
boundaries.
arbiter_type = "round_robin"       # "round_robin", "fixed_priority",
"weighted"
registered_mux = false             # true = +1 cycle, better timing
registered_demux = false           # true = +1 cycle, better timing
# NOTE: there is no enable_cam key -- the loader does not know it, and
no CAM exists
# cam_depth: NOT A KEY. No CAM exists and the loader does not know
this name.
```

9.3 Functional Description

9.3.1 Connectivity Matrix

The bridge implements a **non-blocking crossbar** where: - N masters can each access different slaves simultaneously - Conflicts occur only when multiple masters target the same slave - Independent read and write paths increase parallelism

Example: 4 masters, 3 slaves

```
        S0   S1   S2
    M0  ●    ●    ●
    M1  ●    ●    ●
    M2  ●    ●    ●
    M3  ●    ●    ●
```

● = Connection available

9.3.2 Request Path Multiplexing

For each slave, requests from all masters are multiplexed:

```
Slave 0 Request Inputs:
  - M0 → S0 (AR, AW, W channels)
  - M1 → S0 (AR, AW, W channels)
  - M2 → S0 (AR, AW, W channels)
  - M3 → S0 (AR, AW, W channels)

Arbiter selects one master per channel per cycle
  → Grants propagate through MUX
  → Selected master's transaction forwarded to slave
```

9.3.3 Response Path Demultiplexing

Responses from slaves are demultiplexed back to masters:

```
Slave 0 Response Outputs (R, B channels):
  - Extract Bridge ID from RID/BID
  - Route to corresponding master (M0, M1, M2, or M3)
  - Strip Bridge ID before delivering to master adapter

Routes responses by the position of a per-slave in-order bridge_id
FIFO (no CAM)
```

9.3.4 Per-Slave Request Arbitration

Each slave has **independent arbiters** for: 1. **AR Channel**: Read address arbitration (all masters competing) 2. **AW/W Channels**: Write address/data arbitration (all masters competing)

This separation allows: - Simultaneous read and write to same slave (if slave supports it) - Independent grant decisions for AR vs. AW channels - Better throughput for mixed read/write workloads

Policy (`[bridge] arbitration`, bridge TASK-006 (was BRIDGE-017)). `"rr"` (default) is round-robin among the requesting masters, grant locked until the slave-side handshake. `"qos"` picks the requester with the highest *effective* priority: `AxQOS` plus an age term that climbs one level every `2**qos_aging_shift` cycles a request waits (default 16), saturating at 15, cleared on grant; equals share round-robin. A QoS-0 requester behind a QoS-15 one is therefore served after at most `15 * 2**shift` cycles of waiting – delayed, never starved. Priority reorders only what is waiting at the arbiter: a slave that accepts an AW every cycle takes each master's AW as it arrives, faster than any master issues them, so no two requests are ever pending together and the W order is arrival order whatever the QoS values say. It takes effect where it is meant to, at a slave whose outstanding-write depth is full and whose AW channel backpressures, because that is when a backlog forms and every AW slot is a decision. Measured on `bridge_2x2_rw_qos` (`test_bridge_2x2_rw_qos_arb`) against a port that accepts one AW per 13 cycles: a QoS-8 stream against a QoS-0 stream takes >= 75% of the beats in either orientation, the QoS-0 stream's longest wait for an AW grant stays inside the aging bound, and equal QoS splits the port like the baseline. `AxQOS` itself is still passed through to the slave unchanged.

9.3.5 Request Multiplexers with Stable Address Gating

After arbitration, multiplexers select the granted master's signals. AR/AW gating uses **inline address re-decode** on the stable m_axi address bus (held stable across the full handshake):

```
// Inline address re-decode for stable gating (replaces slave_select_*
gating)
// The address on m_axi is held stable from fub_axi handshake through
m_axi handshake
always_comb begin
    // Re-decode each master's address to determine target slave
    m0_targets_s0 = (m0_m_axi_araddr >= S0_BASE) && (m0_m_axi_araddr <
S0_END);
    m1_targets_s0 = (m1_m_axi_araddr >= S0_BASE) && (m1_m_axi_araddr <
S0_END);
    m2_targets_s0 = (m2_m_axi_araddr >= S0_BASE) && (m2_m_axi_araddr <
S0_END);
    m3_targets_s0 = (m3_m_axi_araddr >= S0_BASE) && (m3_m_axi_araddr <
S0_END);
end

// Gate with valid signals
assign m0_ar_s0_valid = m0_arvalid && m0_targets_s0;
assign m1_ar_s0_valid = m1_arvalid && m1_targets_s0;
assign m2_ar_s0_valid = m2_arvalid && m2_targets_s0;
assign m3_ar_s0_valid = m3_arvalid && m3_targets_s0;
```

```
// Simplified AR channel MUX for Slave 0
always_comb begin
    case (ar_grant_s0)
        2'b00: begin  // Master 0 granted
            s0_arvalid = m0_arvalid;
            s0_araddr  = m0_m_axi_araddr;  // Stable across m_axi
handshake
            s0_arid    = m0_arid;    // Includes Bridge ID
            // ... other AR signals
        end
        2'b01: begin  // Master 1 granted
            s0_arvalid = m1_arvalid;
            s0_araddr  = m1_m_axi_araddr;  // Stable across m_axi
handshake
            s0_arid    = m1_arid;
            // ... other AR signals
        end
        // ... cases for M2, M3
    endcase
end
```

Why this matters: The address on `m_axi_*` is held stable for the entire AXI handshake (arvalid && arready). This allows address re-decoding to determine the target slave even as the master's internal address state changes.

9.3.6 AW→W Burst Tracking FIFO

For masters with multiple slave connections, W beats must follow the slave that the AW was driven to. A per-(master, slave) FIFO tracks which slave the AW was routed to:

```
// Track which slave the AW was driven to
always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        aw_trk_slave_id <= '0;
        aw_trk_valid <= 1'b0;
    end else if (m0_awvalid && m0_m_axi_awready && s_selected_aw ==
S0) begin
        aw_trk_slave_id <= S0;  // Track that AW went to slave 0
        aw_trk_valid <= 1'b1;
    end else if (m0_wvalid && m0_m_axi_wready && m0_m_axi_wlast) begin
        aw_trk_valid <= 1'b0;  // AW->W burst complete
    end
end

// Gate W path based on tracked slave
logic w_to_s0, w_to_s1, w_to_s2;
always_comb begin
    w_to_s0 = m0_wvalid && (aw_trk_slave_id == S0);
    w_to_s1 = m0_wvalid && (aw_trk_slave_id == S1);
```

```
    w_to_s2 = m0_wvalid && (aw_trk_slave_id == S2);
end
```

Why this matters: Without tracking, W beats would re-evaluate address decode which may have stale or different values, routing to the wrong width converter or slave.

9.3.7 Backpressure Propagation

Ready signals flow back from slave through arbiter to the granted master:

```
Slave S0 ARREADY → Arbiter → Granted Master ARREADY
                           ↘ Other Masters ARREADY = 0
```

This ensures: - Only granted master sees READY from slave - Non-granted masters see READY = 0 (backpressure) - No combinatorial loops in ready path (registered arbitration)

9.3.8 Bridge ID Extraction

> **Not built.** Nothing is extracted from the BID. IDs pass through untouched (`cpu_m_axi_awid` and `ddr_s_axi_awid` are both 4 bits in `bridge_2x2_rw`), and the originating master travels as a separate sideband into a per-slave in-order FIFO whose POSITION selects the return path. The returned BID/RID is never consulted. The section below describes the replaced scheme; see ch04 `02_id_tracking.md`.

Response routing uses the Bridge ID embedded in transaction IDs:

```
R Channel:
  Slave RID = {BID[BID_WIDTH-1:0], Original_ID[ID_WIDTH-1:0]}
  Extract: Master_Index = BID
  Route R response to Master[Master_Index]

B Channel:
  Slave BID = {BID[BID_WIDTH-1:0], Original_ID[ID_WIDTH-1:0]}
  Extract: Master_Index = BID
  Route B response to Master[Master_Index]
```

9.3.9 CAM-Based Routing (Optional)

> **Not built.** No generated bridge contains a CAM – `bridge_cam.sv` is instantiated in zero of them. Responses are routed by the POSITION of an in-order per-slave FIFO holding a sideband master id, so out-of-order completion is not supported and the section below describes an option that was never implemented. See ch04 `02_id_tracking.md`.

For large master counts or OOO responses, a CAM tracks outstanding transactions:

```
CAM Entry Structure:
  - Internal ID (with BID)
  - Master index
  - Transaction type (read/write)
  - Timestamp (for timeout detection)

Lookup:
  Input: RID or BID from slave
  Output: Master index for routing
  Latency: 1 cycle (registered CAM)
```

Benefit (of the unbuilt CAM): would have handled ID reordering and burst interleaving

9.3.10 Response Demultiplexers

Based on the extracted Bridge ID, responses are routed:

```
// Simplified R channel DEMUX from Slave 0
logic [BID_WIDTH-1:0] master_id;
assign master_id = s0_rid[TOTAL_ID_WIDTH-1:ID_WIDTH];  // Extract BID

always_comb begin
    // Default: No masters receive response
    m0_rvalid = 1'b0;
    m1_rvalid = 1'b0;
    m2_rvalid = 1'b0;
    m3_rvalid = 1'b0;

    // Route to indicated master
    case (master_id)
        2'b00: begin
            m0_rvalid = s0_rvalid;
            m0_rdata  = s0_rdata;
            m0_rid    = s0_rid[ID_WIDTH-1:0];  // Strip BID
            // ... other R signals
        end
        2'b01: begin
            m1_rvalid = s0_rvalid;
            // ... route to M1
        end
        // ... M2, M3
    endcase
end
```

9.3.11 Multi-Slave Response Merging

When multiple slaves can respond simultaneously, arbitration ensures: - Only one slave's response delivered per master per cycle - Fair arbitration if multiple slaves have responses for same master - No response loss (responses queued until master ready)

9.3.12 AXI Ordering Requirements

The crossbar maintains AXI ordering rules.

9.3.12.1 Read-After-Write (RAW) Ordering

Rule: Read from address must see data from earlier write to same address

Crossbar Behavior: - Ordering enforced at slave level (slave handles RAW within itself) - Crossbar does NOT reorder transactions to same slave from same master - Different masters to same slave: No ordering guaranteed (slave must handle)

9.3.12.2 Write-After-Write (WAW) Ordering

Rule: Writes to overlapping addresses must complete in issue order

Crossbar Behavior: - Same master to same slave: Order preserved by arbiter (FIFO grant queue) - Different masters to same slave: Slave responsible for write ordering

9.3.12.3 Out-of-Order (OOO) Completion

> **Not built.** No generated bridge contains a CAM – `bridge_cam.sv` is instantiated in zero of them. Responses are routed by the POSITION of an in-order per-slave FIFO holding a sideband master id, so out-of-order completion is not supported and the section below describes an option that was never implemented. See ch04 `02_id_tracking.md`.

Allowed: - Read responses must return IN ORDER; a slave that reorders between RIDs misroutes (bridge BUG-008, was BRIDGE-010) - Reads to different slaves can complete in any order - Writes to different slaves can complete in any order

Crossbar Support: - Bridge ID tracking enables OOO response routing - Master sees responses in slave-determined order - Multi-master OOO requires careful slave design

9.3.13 Monitor Aggregation at Bridge Top (when `variants` includes `mon`)

When monitor collection is enabled, per-port monitor streams from `axi4_master_{rd,wr}_mon` and `axi4_slave_{rd,wr}_mon` wrappers are aggregated through a tree of `monbus_arbiter` instances. The final aggregated stream feeds a single `monbus_axil4_axil4_group` instance at the bridge top level:

Aggregation Hierarchy:

```
Master-side monitors → monbus_arbiter tree (if >2 masters)
Slave-side monitors  → monbus_arbiter tree (if >2 slaves)
                          ↓
                 monbus_axil4_axil4_group
                          ↓
          s_mon_axil (slave), m_axil_mon (master), stream_irq
```

The monbus_axil4_axil4_group instance: - Provides a 64-bit slave AXIL interface for CPU read access (s_mon_axil_*) - Provides a master AXIL interface for DMA writes to system memory (m_axil_mon_*) - Contains an error FIFO tracking packets with error flags - Exposes stream_irq (asserted when error FIFO has records) - Includes an internal 64-bit free-running timestamp counter sampled at each packet arrival

Reference: See docs/markdown/rtl-amba/_book_monitor_index.md (monitor documentation book, with per-module pages under docs/markdown/rtl-amba/monitor/) for complete monitor design-surface documentation and docs/markdown/rtl-amba/includes/monitor_package_spec.md for the packet layout.

9.4 Timing

9.4.1 Latency

Request Path (Master → Slave): - Arbiter decision: 1 cycle (registered) - MUX selection: 0 cycles (combinatorial) or +1 cycle with `xbar_pipeline` - **Total**: 1-2 cycles through crossbar (measured end to end: 2 combinational, 3 registered – HAS Table 5.7)

Response Path (Slave → Master): - BID extraction: 0 cycles (combinatorial) - DEMUX routing: 0 cycles (combinatorial) or +1 cycle with `xbar_pipeline` - **Total**: 0-1 cycles through crossbar (measured end to end: 2 combinational, 3 registered)

End-to-End (Master adapter → Slave adapter): - Adapters: 2 cycles (skid buffers) - Router: 0-1 cycles (decode) - Crossbar: 1-2 cycles (arbitration + MUX) - **Total**: 3-5 cycles minimum latency

9.4.2 Throughput

Best Case (no conflicts): - Each master to different slave: N transactions/cycle (N = master count) - Maximum aggregate bandwidth: N × data_width bits/cycle

Arbitration Limits: - Multiple masters to one slave: 1 transaction/cycle to that slave - Other slaves remain available for parallel access

Burst Performance: - Once granted, bursts flow at 1 beat/cycle - Grant held until burst completes (RLAST or WLAST)

9.4.3 Critical Paths

The paths that will bite you first:

1. **Arbiter Request → Grant**:
   - All masters' VALID signals → Arbiter logic → Grant decision
   - Depth: ~5-8 logic levels for 4 masters
2. **Grant → Ready Backpressure**:
   - Slave READY → Arbiter → Selected master READY
   - Depth: ~4-6 logic levels
3. **Response Demux**:
   - Slave RDATA/RID → BID extraction → Master select → Master RDATA
   - Depth: ~6-10 logic levels for 64-bit data

Mitigation Strategies:

1. **Registered crossbar** – BUILT (bridge TASK-006 (was BRIDGE-017), `[bridge] xbar_pipeline = true`). Every slave-side channel gets a 2-deep `gaxi_skid_buffer` inside the xbar: the request stages (AW, W, AR) sit

between the arbitrated mux and the slave port and carry the bridge id in their payload; the response stages (B, R) sit between the slave port and the OR-merge and carry the slave adapter's `bid/rid_bridge_id` and route-open flag WITH the beat, so the merge keys on the staged id and the stage's own valid. Both cones – decode -> arbiter -> N-way mux, and the response OR-merge – end at a register. The skid's ready is registered too, so both directions are cut and throughput is unchanged: `test_bridge_2x2_rw_perf` measures the same 1.00 beat/cycle on `bridge_2x2_rw_pipe` as on `bridge_2x2_rw`, and `test_bridge_2x2_rw_pipe_latency` pins the cost at exactly one cycle each way (3/3 against the baseline's 2/2). The routing and mux emitters are untouched; they drive `xs_<slave>_axi_*` nets and `_generate_pipeline_stages()` joins those to the ports. Off by default.

2. **Pipelined Arbitration**: Multi-cycle arbiter for >8 masters (not built)
3. **Hierarchical Crossbar**: For >16 masters, use tree structure (not built)

9.5 Design Notes

9.5.1 Resource Utilization

4 masters × 3 slaves configuration (64-bit data, 32-bit addr):

```
Logic Elements:  ~2000-3500 LEs
Registers:       ~800-1200 regs
Block RAM:       0 (no CAM is built)

Breakdown per slave:
- Arbiter (4 masters, RR):        ~200 LEs, ~50 regs
- Request MUX (AR/AW/W):          ~400 LEs, ~100 regs
- Response DEMUX (R/B):           ~300 LEs, ~80 regs
- Control FSMs:                   ~100 LEs, ~50 regs

Total for 3 slaves: 3 × 1000 LEs = ~3000 LEs
Plus routing overhead: +500 LEs
```

Scaling with Masters and Slaves — linear: - Adding 1 master: +~500 LEs per slave (new arbiter input) - Adding 1 slave: +~1000 LEs (complete new slave port)

Example: 8 masters × 6 slaves

```
Estimated:  ~12,000 LEs, ~3000 regs
Block RAM:  0 (no CAM is built)
```

Optimization Techniques:

1. **Read-Only/Write-Only Masters**: Reduces arbiter complexity by 40-50%
2. **Power-of-Two Master Count**: Simplifies BID width and routing logic
3. **Pipeline Stages**: Trading latency for frequency (deeper pipelines)

9.5.2 Debug and Observability

Recommended debug signals:

```
Per Slave:
- Arbiter grants (which master granted)
- Arbiter requests (which masters requesting)
- Request MUX outputs (VALID, READY, ADDR, ID)
- Response DEMUX inputs (VALID, READY, DATA, ID)

Global:
- Active transactions count
- Stall counters (arbiter conflicts)
- BID extraction errors
- bridge_id FIFO occupancy (wr_ptr/rd_ptr)
```

Useful metrics for profiling:

```
- Transactions per slave (read, write separate)
- Arbiter conflict rate (requests denied due to grant  contention)
- Average grant latency
- Utilization per master (% cycles active)
- Utilization per slave (% cycles busy)
```

9.5.3 Common Issues

Symptom: Master hangs with VALID=1, READY=0
Check: - Is another master holding grant to this slave? - Is arbiter logic functioning (check grant signals)? - Is slave responding with READY?

Symptom: Response goes to wrong master
Check: - Bridge ID values (verify correct BID per master) - bridge_id FIFO contents (wr_fifo/rd_fifo) - BID extraction logic (check bit positions)

Symptom: Throughput lower than expected
Check: - Arbiter conflicts (multiple masters to same slave?) - Pipeline depth (excessive latency reducing effective bandwidth?) - Burst efficiency (are bursts being granted properly?)

9.5.4 Future Enhancements

Planned Features: - **Weighted Round-Robin**: QoS support with configurable priorities - **Slave-Side Arbitration Policies**: Per-slave arbiter configuration - **Grant Prediction**: Speculative grant for lower latency - **Congestion Control**: Throttling to prevent hotspots

Under Consideration: - **Partial Crossbar**: Configurable master-to-slave connectivity (not full mesh) - **Multi-Tier Hierarchy**: For 32+ masters/slaves - **Virtual Channels**: Separate channels for different traffic classes - **Register Slicing**: Automatic pipeline insertion for timing

9.6 Related Modules

- Section 2.1: Master Adapter (request sources)
- Section 2.2: Slave Router (address decode before arbitration)
- Section 2.4: Arbitration (detailed arbiter algorithms)
- Section 2.5: ID Management (sideband bridge_id tracking; its CAM was never built)
- Section 3.1: Top-Level Integration (crossbar instantiation)

9.7 Testing

9.7.1 Functional Tests

1. **Single Master to Each Slave**: Verify basic connectivity
2. **All Masters to One Slave**: Stress arbiter fairness
3. **All Masters to All Slaves**: Maximum parallelism test
4. **OOO Responses**: Issue transactions with different latencies
5. **Burst Interleaving**: Multiple masters with bursts to same slave

9.7.2 Corner Cases

```
- Back-to-back grants (no idle cycles)
- Grant held for maximum burst length (256 beats)
- Rapid master switching (each gets 1 transaction then switches)
- Response while request in progress (pipelining)
- All masters requesting same slave simultaneously
```

9.7.3 Protocol Compliance

- AXI4 protocol checker at each master/slave interface
- Verify no READY → VALID dependencies (AXI violation)
- Check ID preservation through crossbar (modulo Bridge ID)
- Verify LAST signal handling

10 2.4 Arbitration

When several masters ask for the same slave in the same cycle, something has to pick a winner — that's the arbiter. Each slave gets dedicated arbiters for its read and write channels, so the pick is fair and nothing queues behind the wrong channel.

10.1 Overview

The arbiter does five things:

1. **Conflict Resolution**: Selects one master when multiple masters request same slave
2. **Fairness**: Ensures all masters eventually get access (starvation-free)
3. **Throughput Optimization**: Minimizes idle cycles and maximizes utilization
4. **Grant Management**: Maintains grants for burst transactions
5. **Priority Enforcement**: Supports priority-based access policies (optional)

10.2 Parameters

Arbiter configuration from the TOML — though here the comments carry the real story, since two of the three policies documented later in this chapter exist nowhere in the generated RTL:

```
[bridge]
# arbiter_type: NOT A KEY. Every generated arbiter is ROUND-ROBIN with
lock-
# until-handshake; fixed_priority and weighted exist nowhere in
bin/bridge_pkg/
# or rtl/generated/.

[bridge.arbitration]
pipeline_stages = 1              # 1-3 (more = better timing, higher
latency)
# enable_priority_aging: NOT A KEY -- there is no aging logic.
aging_threshold = 1000           # Cycles before priority boost

# Weighted arbitration does not exist.
[[bridge.arbitration.weights]]
master = "cpu"
weight = 4                       # Relative weight (1-255)

[[bridge.arbitration.weights]]
master = "dma"
weight = 2

[[bridge.arbitration.weights]]
master = "periph"
weight = 1
```

10.3 Functional Description

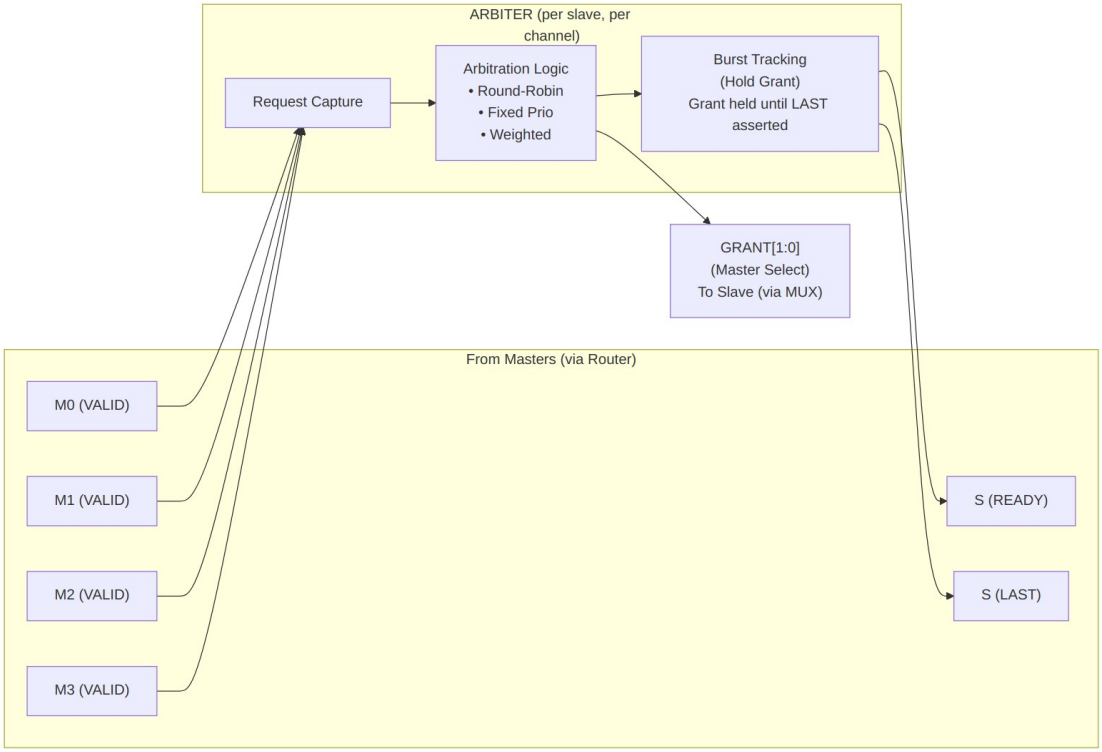
10.3.1 Per-Slave, Per-Channel Arbiters

Each slave has **two independent arbiters**:

```
Slave 0:
  - AR Arbiter (Read Address Channel)
    * Inputs: M0_ARVALID, M1_ARVALID, M2_ARVALID, M3_ARVALID
    * Output: Grant[1:0] → Selects one master

  - AW Arbiter (Write Address Channel)
    * Inputs: M0_AWVALID, M1_AWVALID, M2_AWVALID, M3_AWVALID
    * Output: Grant[1:0] → Selects one master
    * Coordinates with W channel data flow
```

Independence: AR and AW arbiters operate independently, allowing simultaneous read and write to same slave (if slave supports full-duplex operation).



Block Diagram

10.3.2 Round-Robin Arbitration

10.3.2.1 Algorithm

The **Round-Robin (RR)** arbiter cycles through masters in order, giving each master one opportunity to access the slave:

```
Priority Order (rotates after each grant):
  Cycle 0: M0 > M1 > M2 > M3
  Cycle 1: M1 > M2 > M3 > M0  (M0 was granted, now lowest priority)
  Cycle 2: M2 > M3 > M0 > M1  (M1 was granted)
  Cycle 3: M3 > M0 > M1 > M2  (M2 was granted)
  Cycle 4: M0 > M1 > M2 > M3  (M3 was granted, cycle repeats)
```

10.3.2.2 Implementation

```
// Simplified Round-Robin arbiter (4 masters)
logic [1:0] last_grant;      // Which master was granted last
logic [3:0] request;         // Current requests (VALID signals)
logic [1:0] grant;           // Grant decision

always_ff @(posedge clk) begin
    if (!rst_n) begin
        last_grant <= 2'b00;
    end else if (|request && slave_ready) begin
        // Update last_grant when transaction completes
        if (grant_accepted) begin
            last_grant <= grant;
        end
    end
end

always_comb begin
    // Default: No grant
    grant = 2'b00;

    // Priority encode starting after last grant
    case (last_grant)
        2'b00: begin  // Last grant to M0, try M1, M2, M3, M0
            if (request[1])      grant = 2'b01;
            else if (request[2]) grant = 2'b10;
            else if (request[3]) grant = 2'b11;
            else if (request[0]) grant = 2'b00;
        end
        2'b01: begin  // Last grant to M1, try M2, M3, M0, M1
            if (request[2])      grant = 2'b10;
            else if (request[3]) grant = 2'b11;
            else if (request[0]) grant = 2'b00;
            else if (request[1]) grant = 2'b01;
```

```
        end
        // ... similar for M2, M3
    endcase
end
```

10.3.2.3 Characteristics

Fairness: Excellent - Each master gets equal access over time
Latency: Bounded - Maximum wait = (N-1) × transaction_time
Throughput: Optimal - No idle cycles when requests pending
Starvation: None - Every master eventually gets grant

Best Use Cases: - Peers with similar priority - General-purpose interconnects - Default choice for most bridges

10.3.3 Fixed-Priority Arbitration

10.3.3.1 Algorithm

The **Fixed-Priority** arbiter always grants to the highest-priority requesting master:

```
Priority Order (never changes):
  M0 > M1 > M2 > M3  (M0 always highest priority)

Grant Decision:
  if (M0_VALID) → Grant = M0
  else if (M1_VALID) → Grant = M1
  else if (M2_VALID) → Grant = M2
  else if (M3_VALID) → Grant = M3
```

10.3.3.2 Implementation

```
// Fixed-priority arbiter (4 masters)
logic [3:0] request;
logic [1:0] grant;

always_comb begin
    // Priority encoder: M0 highest, M3 lowest
    if (request[0])      grant = 2'b00;  // M0 highest priority
    else if (request[1]) grant = 2'b01;
    else if (request[2]) grant = 2'b10;
    else if (request[3]) grant = 2'b11;  // M3 lowest priority
    else                 grant = 2'b00;  // Default (no requests)
end
```

10.3.3.3 Characteristics

Fairness: Poor - Low-priority masters can starve
Latency: Variable - High-priority: minimal, Low-priority: unbounded

Throughput: Optimal - No idle cycles when requests pending
Starvation: Possible for low-priority masters

Best Use Cases: - Real-time systems with priority requirements - CPU (high) vs. DMA (low) scenarios - Time-critical masters need guaranteed access

10.3.3.4 Starvation Prevention

Optional enhancement: **Priority aging**

```
- Track cycles since last grant per master
- Temporarily boost priority of starved masters
- Reset age counter after grant

Example:
  M0 age = 100 cycles → Boost M0 above normal priority
  M1 age = 5 cycles   → Normal priority
```

10.3.4 Weighted Arbitration

10.3.4.1 Algorithm

The **Weighted** arbiter assigns different access rates to masters based on weights:

```
Configuration:
  M0 weight = 4  (50% of bandwidth)
  M1 weight = 2  (25% of bandwidth)
  M2 weight = 1  (12.5% of bandwidth)
  M3 weight = 1  (12.5% of bandwidth)

Grant Sequence (repeating pattern):
  M0, M0, M0, M0, M1, M1, M2, M3  (8 cycles total)
  M0 gets 4/8, M1 gets 2/8, M2 gets 1/8, M3 gets 1/8
```

10.3.4.2 Implementation

```
// Weighted arbiter using credit counter
logic [7:0] credits [0:3];  // Credit counters per master
logic [1:0] grant;

always_ff @(posedge clk) begin
    if (!rst_n) begin
        credits[0] <= 8'd4;  // M0 weight
        credits[1] <= 8'd2;  // M1 weight
        credits[2] <= 8'd1;  // M2 weight
        credits[3] <= 8'd1;  // M3 weight
    end else if (grant_accepted) begin
        // Decrement granted master's credits
        credits[grant] <= credits[grant] - 1;
```

```
        // Reload when all credits exhausted
        if (all_credits_zero) begin
            credits[0] <= 8'd4;
            credits[1] <= 8'd2;
            credits[2] <= 8'd1;
            credits[3] <= 8'd1;
        end
    end
end

always_comb begin
    // Grant to master with highest credits and active request
    grant = select_highest_credit_with_request(credits, request);
end
```

10.3.4.3 Characteristics

Fairness: Configurable - Proportional to weights
Latency: Bounded - Depends on weight ratio
Throughput: Near-optimal - Small overhead for credit tracking
Starvation: None - All masters get share per weight

Best Use Cases: - QoS requirements (guaranteed bandwidth) - Mixed workload (video + CPU + DMA) - Service-level agreements

10.3.5 Burst Handling

10.3.5.1 Grant Locking

Once granted, a master **holds the grant** until its burst completes:

```
AR Channel Burst:
  1. Master asserts ARVALID with ARLEN = 7 (8-beat burst)
  2. Arbiter grants to this master
  3. Grant held until slave returns RLAST
  4. Other masters blocked during this time

AW/W Channel Burst:
  1. Master asserts AWVALID with AWLEN = 15 (16-beat burst)
  2. Arbiter grants to this master
  3. Grant held until master asserts WLAST
  4. Other masters blocked during W data transfer
```

10.3.5.2 Implementation

```
// Burst grant locking
logic grant_locked;
logic [1:0] locked_master;
```

```
always_ff @(posedge clk) begin
    if (!rst_n) begin
        grant_locked <= 1'b0;
    end else begin
        if (grant_accepted && !last_beat) begin
            // Lock grant for burst
            grant_locked <= 1'b1;
            locked_master <= grant;
        end else if (last_beat) begin
            // Release grant when burst completes
            grant_locked <= 1'b0;
        end
    end
end

assign grant = grant_locked ? locked_master : arbiter_decision;
```

10.3.5.3 Fairness Considerations

Long bursts can block other masters: - Maximum AXI burst: 256 beats - At 1 cycle/beat: Up to 256 cycles blocked - Impacts latency for other masters

Mitigation: - Limit burst lengths in master configuration - Use burst interleaving (complex, not implemented in Phase 1) - Monitor arbiter stall counters

10.4 Timing

10.4.1 Arbitration Latency

Single-Cycle Arbitration (default):

```
Request → Grant Decision: 1 cycle
Grant Decision → MUX Output: 0 cycles (combinatorial)
Total: 1 cycle
```

Multi-Cycle Arbitration (high frequency):

```
Request → Grant Decision: 2-3 cycles (pipelined)
Improves timing at cost of latency
```

10.4.2 Critical Paths

Typical critical paths in the arbiter: 1. **Request collection**: All master VALID signals → Arbiter input 2. **Priority encode**: Compare all requests → Grant decision 3. **Grant propagate**: Grant → MUX select → Slave interface

Path Depth: - 4 masters: ~6-8 logic levels - 8 masters: ~8-12 logic levels - 16 masters: ~12-16 logic levels

10.5 Design Notes

10.5.1 Resource Utilization

Round-Robin (4 masters):

```
Logic Elements:  ~150 LEs
Registers:       ~40 regs

Breakdown:
- Priority encoder:        ~80 LEs
- Last grant tracking:     ~10 regs
- Request capture:         ~20 regs
- Grant FSM:               ~40 LEs, ~10 regs
```

Fixed-Priority (4 masters):

```
Logic Elements:  ~80 LEs
Registers:       ~20 regs

Simpler than RR (no rotation logic)
```

Weighted (4 masters):

```
Logic Elements:  ~200 LEs
Registers:       ~60 regs

Additional credit counters and comparison logic
```

Scaling with Master Count — resource usage grows approximately **O(N²)** where N = master count:

```
2 masters:  ~50 LEs
4 masters:  ~150 LEs
8 masters:  ~400 LEs
16 masters: ~1200 LEs
```

The N² scaling comes from: - Priority encoder: Compares all pairs of masters - Grant MUX: N-to-1 multiplexing increases with N

10.5.2 Performance Impact

Round-Robin: - **Pros**: Fair, starvation-free, predictable latency - **Cons**: Cannot prioritize time-critical masters - **Use**: General-purpose, peer masters

Fixed-Priority: - **Pros**: Guaranteed low latency for high-priority masters - **Cons**: Low-priority can starve, unpredictable for low-priority - **Use**: Real-time, priority-sensitive systems

Weighted: - **Pros**: Configurable bandwidth allocation, QoS support - **Cons**: More complex, small overhead - **Use**: Mixed workload, SLA requirements

Arbiter Efficiency — assuming all masters request the same slave continuously:

```
Best Case: 100% efficiency (one master)
  - No arbitration conflicts
  - Zero idle cycles

Typical Case: 25-50% per-master efficiency (4 masters, round-robin)
  - Each master gets ~25% of time grants
  - Load balancing to other slaves improves

Worst Case: Heavy contention
  - 4 masters to 1 slave: 25% efficiency per master
  - Solution: Add slaves or use more slaves in parallel
```

10.5.3 Debug and Observability

Recommended debug signals and counters:

```
Per Arbiter:
- Request vector (all masters requesting)
- Grant decision (which master granted)
- Grant locked status (burst in progress)
- Last grant (for RR rotation tracking)

Performance Counters:
- Grants per master (utilization)
- Denied requests per master (contention)
- Average grant latency per master
- Arbiter conflict cycles (multiple requests, one grant)
```

10.5.4 Common Issues

Symptom: Master never gets grant (starvation)
Check: - Arbiter type (fixed-priority can starve low-priority masters) - Other masters continuously requesting? - Enable priority aging if using fixed-priority

Symptom: Unfair access (one master dominates)
Check: - Arbiter type (should be round-robin for fairness) - Burst lengths (long bursts block others) - Request patterns (one master requesting more frequently)

Symptom: Low throughput despite requests
Check: - Arbiter conflicts (too many masters to one slave) - Pipeline depth (excessive arbitration latency) - Burst efficiency (short bursts waste cycles)

10.5.5 Future Enhancements

Planned Features: - **Dynamic Weight Adjustment**: Runtime-configurable weights via registers - **QoS Classes**: Multiple priority levels with configurable policies - **Deadline-Based Arbitration**: Grant based on transaction deadlines - **Predictive Arbitration**: Speculative grants to reduce latency

Under Consideration: - **Multi-Level Arbitration**: Hierarchical for >16 masters - **Token Bucket**: Rate limiting per master - **History-Based**: Learn access patterns and optimize grants - **ECC Protection**: For arbitration state (safety-critical systems)

10.6 Related Modules

- Section 2.3: Crossbar Core (arbiter integration)
- Section 2.1: Master Adapter (request sources)
- Section 2.5: ID Management (transaction tracking during grants)
- Chapter 6: Performance (arbiter impact on throughput)

10.7 Testing

10.7.1 Test Scenarios

1. **Fair Access Test** (Round-Robin):

```
- All masters continuously request same slave
- Verify each master gets equal grants over 1000 cycles
- Expected: Each of 4 masters gets ~250 grants
```

2. **Priority Test** (Fixed-Priority):

```
- High-priority master requests intermittently
- Low-priority masters request continuously
- Verify high-priority always wins when requesting
```

3. **Burst Locking Test**:

```
- Master issues 16-beat burst
- Other masters request during burst
- Verify grant held until LAST
- Verify other masters blocked during burst
```

4. **Weighted Bandwidth Test**:

```
- Configure M0=4, M1=2, M2=1, M3=1
- All masters request continuously for 10000 cycles
- Verify grants proportional: M0≈50%, M1≈25%, M2≈12.5%, M3≈12.5%
```

10.7.2 Corner Cases

```
- Single master requesting (trivial grant)
- All masters requesting simultaneously (worst-case arbitration)
- Grant released and new grant same cycle (back-to-back)
- Master drops request after arbiter latency (stale grant)
- Maximum length burst (256 beats blocking)
```

11 2.5 ID Management

ID Management is how the bridge remembers which master sent each outstanding transaction, so the response can find its way back.

11.1 Overview

Before anything else in this chapter, read this — it colors everything below:

> **What the generated bridge actually does.** There is no CAM, no ID table and no ID injection. `bridge_cam.sv` exists in the tree and is instantiated in **zero** generated bridges. AXI IDs pass through UNTOUCHED and at equal width – `cpu_m_axi_awid` and `ddr_s_axi_awid` are both 4 bits in `bridge_2x2_rw`, with nothing prepended and nothing stripped.
>
> The originating master travels as a SIDEBAND signal (`xbar_bridge_id_aw` / `xbar_bridge_id_ar`) beside the transaction. Each slave adapter pushes it into an in-order FIFO on the address handshake, pops it on the response, and routes the response by FIFO POSITION – never by the returned BID/RID.
>
> Two consequences follow, and both are tracked: * each slave port REQUIRES responses in request order across all IDs; a slave that reorders between IDs misroutes here (bridge BUG-008 (was BRIDGE-010), now detected by a simulation-only check that compares the returned BID/RID against the FIFO head); * the FIFO is fixed-depth, so the address handshake is gated on it being not-full (bridge BUG-009, was BRIDGE-011).
>
> Out-of-order response routing is therefore NOT supported and NOT implemented.

> **Everything below this line describes a design that was never built.** It is retained because the mechanism that replaced it is easy to mistake for it.

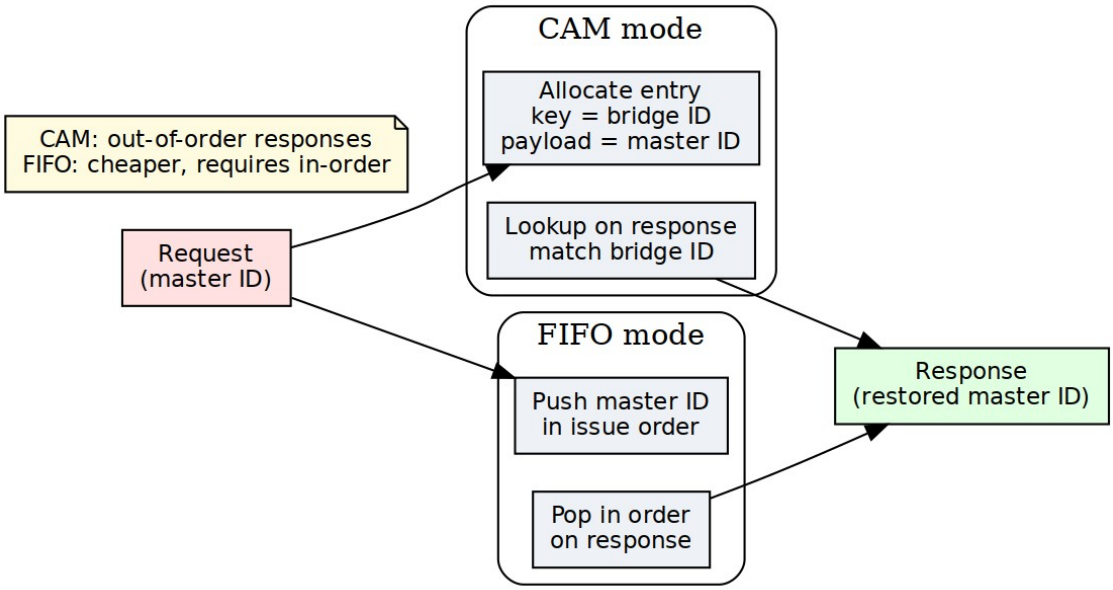
The unbuilt design described three pieces: Bridge ID injection, Content Addressable Memory (CAM) structures, and ID translation logic.

The ID management system does five things:

1. **Transaction Tracking**: Maintains association between requests and originating masters
2. **Response Routing**: Directs responses to correct master using embedded IDs

3. **Out-of-Order Support**: Handles responses returning in different order than issued
4. **ID Space Isolation**: Prevents ID conflicts between multiple masters
5. **Burst Management**: Tracks multi-beat bursts through the bridge

11.1.1 Figure 2.5: ID Management Architecture



ID Management Architecture

ID management architecture showing CAM and FIFO modes for transaction tracking and response routing.

11.2 Parameters

```
[bridge]
num_masters = 4
enable_cam = false              # Use CAM for ID tracking
cam_type = "parallel"           # "parallel", "bram", "sequential"
cam_depth = 16                  # Outstanding transaction capacity

# NONE of the keys below exist. There is no [bridge.id_management]
table,
# no ID injection/extraction (IDs are pass-through) and no timeout
anywhere
# in the bridge -- see 2.9. config_validator rejects unknown keys, so
a TOML
# written from this block does not load.
#
# The real per-port keys are id_width / addr_width / data_width under
# [[bridge.masters]] and [[bridge.slaves]]:

[[bridge.masters]]
name = "cpu"
id_width = 4
max_outstanding_reads = 8       # CAM allocation limit
max_outstanding_writes = 8
```

11.3 Functional Description

11.3.1 Bridge ID Concept

Problem Statement — without ID management, the bridge cannot determine which master originated a transaction:

```
Scenario: Two masters with overlapping IDs
  Master 0: Issues read with ARID = 4'h5
  Master 1: Issues read with ARID = 4'h5 (same ID!)

When slave responds with RID = 4'h5, which master gets the response?
  → Ambiguous! Need additional information.
```

Solution: Bridge ID Injection — the bridge adds a **Bridge ID (BID)** to each transaction ID:

```
Internal ID = {Bridge_ID, Original_ID}

Master 0, ARID = 4'h5:
  Internal ARID = {2'b00, 4'h5} = 6'b00_0101

Master 1, ARID = 4'h5:
  Internal ARID = {2'b01, 4'h5} = 6'b01_0101

Now responses with RID = 6'b00_0101 → Route to Master 0
                    RID = 6'b01_0101 → Route to Master 1
```

11.3.2 Bridge ID Width Calculation

Formula:

```
BID_WIDTH = clog2(NUM_MASTERS)

Where clog2(x) = ceiling(log2(x))
```

Examples:

```
2 masters:  clog2(2)  = 1 bit   (BID: 0, 1)
3 masters:  clog2(3)  = 2 bits  (BID: 00, 01, 10)
4 masters:  clog2(4)  = 2 bits  (BID: 00, 01, 10, 11)
5 masters:  clog2(5)  = 3 bits  (BID: 000-100)
8 masters:  clog2(8)  = 3 bits  (BID: 000-111)
16 masters: clog2(16) = 4 bits  (BID: 0000-1111)
```

ID Width Growth:

```
Configuration: 4 masters, external ARID_WIDTH = 4

Internal ARID_WIDTH = BID_WIDTH + External ARID_WIDTH
```

```
                    = 2 + 4
                    = 6 bits

Slave sees 6-bit IDs
Master sees 4-bit IDs
Bridge translates between them
```

11.3.3 Multi-Master OR-Merge Gating

When multiple masters drive address and ID signals toward a slave, an OR-merge combines them. To prevent idle masters with stale addresses from corrupting the routing tag, every OR-merge term is gated on both the slave_select condition AND the valid signal:

```
// Wrong (deprecated):
assign s0_bridge_id_aw = m0_bridge_id_aw | m1_bridge_id_aw |
m2_bridge_id_aw;
//                       ↓ Master 1 contributes even if awvalid=0 and
address is stale!

// Correct:
assign s0_bridge_id_aw = (m0_slave_select_aw && m0_awvalid ?
m0_bridge_id_aw : '0) |
                         (m1_slave_select_aw && m1_awvalid ?
m1_bridge_id_aw : '0) |
                         (m2_slave_select_aw && m2_awvalid ?
m2_bridge_id_aw : '0);
```

Why this matters: An idle master with stale address (fub_axi_awaddr = 0) still decodes to slave 0 if that's the address map. Without valid gating, its zero bridge_id corrupts the slave's routing tag via OR-merge. When the slave responds, the B-response gets routed to the idle master instead of the actual requester.

Implementation: - All AW/AR/W data signals gated by `{slave_select && <channel_valid>}` - bridge_id_aw/ar signals gated by corresponding `<channel>valid` - Prevents any idle master from affecting the OR-merge result

11.3.4 ID Injection (Request Path)

11.3.4.1 Read Address Channel (AR)

```
// ID injection at master adapter
logic [BID_WIDTH-1:0] master_bid;         // Constant per master
logic [ARID_WIDTH-1:0] external_arid;     // From master
logic [TOTAL_ARID_WIDTH-1:0] internal_arid;  // To crossbar

assign master_bid = MASTER_INDEX;  // M0=0, M1=1, M2=2, M3=3
assign internal_arid = {master_bid, external_arid};
```

```
// Example: Master 2, ARID = 4'h7
// internal_arid = {2'b10, 4'h7} = 6'b10_0111
```

11.3.4.2 Write Address Channel (AW)

```
// ID injection for write transactions
logic [BID_WIDTH-1:0] master_bid;
logic [AWID_WIDTH-1:0] external_awid;
logic [TOTAL_AWID_WIDTH-1:0] internal_awid;

assign internal_awid = {master_bid, external_awid};

// Note: AWID and ARID widths can differ per master
```

11.3.4.3 Injection Timing

Combinatorial (default): - ID concatenation done in same cycle as VALID assertion - Zero latency overhead - May contribute to critical path in high-frequency designs

Registered (optional): - Add pipeline register after ID injection - +1 cycle latency - Breaks critical path

11.3.5 ID Extraction (Response Path)

11.3.5.1 Read Data Channel (R)

```
// ID extraction at crossbar response router
logic [TOTAL_RID_WIDTH-1:0] internal_rid;   // From slave
logic [BID_WIDTH-1:0] extracted_bid;        // Upper bits
logic [RID_WIDTH-1:0] external_rid;         // Lower bits

// Extract Bridge ID
assign extracted_bid = internal_rid[TOTAL_RID_WIDTH-1:RID_WIDTH];

// Strip Bridge ID for master
assign external_rid = internal_rid[RID_WIDTH-1:0];

// Route based on BID
always_comb begin
    case (extracted_bid)
        2'b00: master0_rvalid = slave_rvalid;
        2'b01: master1_rvalid = slave_rvalid;
        2'b10: master2_rvalid = slave_rvalid;
        2'b11: master3_rvalid = slave_rvalid;
    endcase
end
```

11.3.5.2 Write Response Channel (B)

```
// Similar extraction for write responses
logic [TOTAL_BID_WIDTH-1:0] internal_bid;
logic [BID_WIDTH-1:0] extracted_bid;
logic [BID_WIDTH-1:0] external_bid;

assign extracted_bid = internal_bid[TOTAL_BID_WIDTH-1:BID_WIDTH];
assign external_bid = internal_bid[BID_WIDTH-1:0];

// Route to originating master
```

11.3.6 Content Addressable Memory (CAM)

When to Use CAM:

Simple Configurations (No CAM needed): - Single master (BID unnecessary) - Direct ID mapping suffices - Lower resource usage

Complex Configurations (CAM beneficial): - Many masters (>8) - Out-of-order responses common - Multiple outstanding transactions per master - ID reordering within slave

CAM Structure:

```
Entry Format:
```

Internal ID (6-12 bits	Master Index (2-4 bits)	Transaction Type (R/W)	Timestamp (counter)	Valid (1b)
10b	3b	1b	16b	1b

```
Total per entry: ~30 bits
CAM depth: 16-64 entries typical
```

CAM Operations:

Allocation (Request Path):

```
1. Master issues AR/AW transaction
2. Find free CAM entry (Valid = 0)
3. Write entry:
   - Internal ID = {BID, External_ID}
   - Master Index = BID
   - Transaction Type = Read or Write
   - Timestamp = Current cycle count
   - Valid = 1
4. Forward transaction to slave
```

Lookup (Response Path):

```
1. Slave returns R/B response with Internal ID
2. CAM lookup: Search for matching Internal ID
3. Extract Master Index from matching entry
4. Route response to Master[Master Index]
5. If LAST: Deallocate entry (Valid = 0)
```

Associative Search: - All entries checked in parallel - 1-cycle lookup (registered CAM) - Match found → Returns master index - No match → Error condition (protocol violation)

11.3.7 CAM Implementation

11.3.7.1 Parallel CAM (Fast, Resource-Intensive)

```
// Parallel CAM structure (16 entries)
typedef struct packed {
    logic valid;
    logic [TOTAL_ID_WIDTH-1:0] internal_id;
    logic [BID_WIDTH-1:0] master_idx;
    logic txn_type;  // 0=read, 1=write
    logic [15:0] timestamp;
} cam_entry_t;

cam_entry_t cam [0:15];

// Parallel search
logic [15:0] match;
logic [3:0] match_idx;

for (genvar i = 0; i < 16; i++) begin
    assign match[i] = cam[i].valid &&
                      (cam[i].internal_id == response_id);
end

// Priority encode first match
assign match_idx = find_first_set(match);
assign routed_master = cam[match_idx].master_idx;
```

11.3.7.2 Sequential CAM (Slow, Resource-Efficient)

```
// Sequential CAM search (multi-cycle)
logic [3:0] search_idx;
logic searching;

always_ff @(posedge clk) begin
    if (search_start) begin
        search_idx <= 0;
        searching <= 1'b1;
    end else if (searching) begin
        if (cam[search_idx].valid &&
```

```
            cam[search_idx].internal_id == response_id) begin
            // Match found
            routed_master <= cam[search_idx].master_idx;
            searching <= 1'b0;
        end else if (search_idx == 15) begin
            // No match found (error)
            searching <= 1'b0;
            error <= 1'b1;
        end else begin
            search_idx <= search_idx + 1;
        end
    end
end
```

Trade-off: - Parallel: 1-cycle, ~2000 LEs for 16 entries - Sequential: 16 cycles, ~200 LEs for 16 entries

11.3.8 Outstanding Transaction Limits

Configuration:

```
[bridge]
enable_cam = true
# cam_depth: NOT A KEY. No CAM exists and the loader does not know
this name.

[[bridge.masters]]
name = "cpu"
max_outstanding_reads = 8   # Per-master limit
max_outstanding_writes = 8
```

Enforcement — when the CAM is full:

```
1. Master issues new request
2. Check CAM for free entry
3. If full:
   - ARREADY/AWREADY = 0 (backpressure)
   - Wait for response to free entry
4. When entry freed (RLAST/BVALID):
   - Accept new request
```

Sizing Guidelines:

```
CAM Depth = Σ(max_outstanding per master) + Safety margin

Example: 4 masters, 4 outstanding each
  Minimum CAM depth = 4 × 4 = 16 entries
  Recommended depth = 20 entries (25% margin)
```

11.4 Timing

11.4.1 ID Injection Latency

Combinatorial (default): - 0 cycles overhead - Part of adapter pipeline stage

Registered: - +1 cycle latency - Breaks timing path

11.4.2 ID Extraction/Routing Latency

Direct Decode (no CAM): - 0 cycles (combinatorial BID extraction)

Parallel CAM: - 1 cycle (registered search)

Sequential CAM: - 1-N cycles (where N = CAM depth) - Average: N/2 cycles

11.4.3 End-to-End Impact

```
Configuration: No CAM, combinatorial ID management
  Request:  Master → +0 cycles → Crossbar
  Response: Slave → +0 cycles → Master
  Total: 0 cycles overhead

Configuration: Parallel CAM, registered
  Request:  Master → +1 cycle (allocation) → Crossbar
  Response: Slave → +1 cycle (lookup) → Master
  Total: 2 cycles overhead
```

11.5 Design Notes

11.5.1 Resource Utilization

Per Master (ID injection/extraction only, no CAM):

```
Logic Elements:  ~20-50 LEs
Registers:       ~10 regs

Simple bit concatenation and extraction
Minimal overhead
```

CAM Resources (16 entries, 6-bit IDs):

```
Parallel CAM:
  Logic Elements:  ~2000 LEs
  Registers:       ~500 regs
  Block RAM:       0 (distributed)

BRAM-Based CAM:
  Logic Elements:  ~500 LEs
  Registers:       ~100 regs
  Block RAM:       1-2 KB

Sequential CAM:
  Logic Elements:  ~200 LEs
  Registers:       ~100 regs
  Block RAM:       0
```

Scaling:

```
CAM Depth      Parallel      BRAM          Sequential
16 entries     ~2000 LEs     ~500 LEs      ~200 LEs
32 entries     ~4000 LEs     ~800 LEs      ~250 LEs
64 entries     ~8000 LEs     ~1200 LEs     ~300 LEs
```

11.5.2 Debug and Observability

Recommended debug signals:

```
ID Injection:
- Master BID assignments (constant per master)
- External IDs (from masters)
- Internal IDs (after injection)

ID Extraction:
- Internal IDs (from slaves)
- Extracted BIDs
- External IDs (to masters)
```

```
CAM (if enabled):
- CAM occupancy (number of valid entries)
- Allocation/deallocation events
- Search hits/misses
- Timeout events
```

Performance counters:

```
- Total transactions tracked
- CAM hit rate
- CAM miss rate (should be 0)
- Average CAM occupancy
- Peak CAM occupancy
- Timeout errors
- ID width overhead (bits added per transaction)
```

11.5.3 Common Issues

Symptom: Response goes to wrong master
Check: - BID assignment (verify each master has unique BID) - ID width calculation (BID_WIDTH correct?) - Bit slicing in extraction logic - CAM contents (if used)

Symptom: CAM full, transactions stalling
Check: - CAM depth vs. outstanding transaction count - Are responses being received? (slave responsive?) - Timeout threshold (too short?) - Leaked entries (not deallocated after LAST)

Symptom: CAM miss errors
Check: - Slave returning incorrect IDs - ID corruption in transit - Race condition (deallocation before response complete)

11.5.4 Future Enhancements

Planned Features: - **Dynamic CAM Depth**: Runtime adjustment based on utilization - **Per-Master CAM Partition**: Guaranteed entries per master - **ID Compression**: Reduce internal ID width for slaves with limited ID support - **Transaction Ordering**: CAM tracks issue order for reordering enforcement

Under Consideration: - **Multi-Level CAM**: Hierarchical for large transaction counts - **TCAM Support**: Partial ID matching (wildcards) - **Error Injection**: Debug mode to test error handling - **CAM Mirrors**: Redundant CAMs for fail-safe operation

11.6 Related Modules

- Section 2.1: Master Adapter (BID injection location)
- Section 2.3: Crossbar Core (BID extraction, response routing)
- Section 2.8: Response Routing (detailed routing logic)
- HAS ch04_interfaces/01_axi4_interface.md (ID widths – note IDs are pass-through)

11.7 Testing

11.7.1 Test Scenarios

1. **Basic ID Injection/Extraction**:

```
- Single master, single transaction
- Verify BID added correctly
- Verify BID stripped before response delivery
- Check external ID unchanged
```

2. **Multi-Master ID Isolation**:

```
- Multiple masters with same external IDs
- Verify responses route to correct master
- Check BID uniqueness prevents collisions
```

3. **CAM Capacity**:

```
- Issue max_outstanding transactions
- Verify backpressure when CAM full
- Issue responses to free entries
- Verify new transactions accepted
```

4. **Out-of-Order Responses**:

```
- Issue transactions: ID=1, ID=2, ID=3
- Return responses: ID=3, ID=1, ID=2
- Verify CAM correctly routes each
```

5. ~~**CAM Timeout**~~ – not testable: there is no timeout to trigger and no CAM in the generated bridges (`bridge_cam.sv` is instantiated in zero of them). A hung slave stalls its path indefinitely; the system-level answer is a watchdog outside the bridge.

12 2.6 Width Conversion

Masters and slaves rarely agree on data width, and width conversion is how the bridge copes. There is NO fixed internal width — each path carries its port's own width, and converters sit only at the boundaries where two ports disagree. `bridge_4x4_rw` alone instantiates paths at 32, 64, 128 and 256 bits.

12.1 2.6.1 Purpose and Function

Width conversion has five jobs:

1. **Data Path Adaptation**: Converts between different data widths (8, 16, 32, 64, 128, 256 bits)
2. **Burst Splitting**: Divides wide transactions into multiple narrow transactions
3. **Burst Merging**: Combines multiple narrow beats into fewer wide beats
4. **Strobe Mapping**: Translates write strobes across width boundaries
5. **Address Alignment**: Adjusts addresses for width differences

12.2 2.6.2 Internal Data Width

12.2.1 64-Bit Standard

The bridge uses **64-bit (8-byte) internal data width** for the crossbar:

```
Rationale:
- Balance between resource usage and performance
- Common width for modern embedded processors
- Efficient for both 32-bit and 64-bit masters
- Reasonably sized multiplexers in crossbar
```

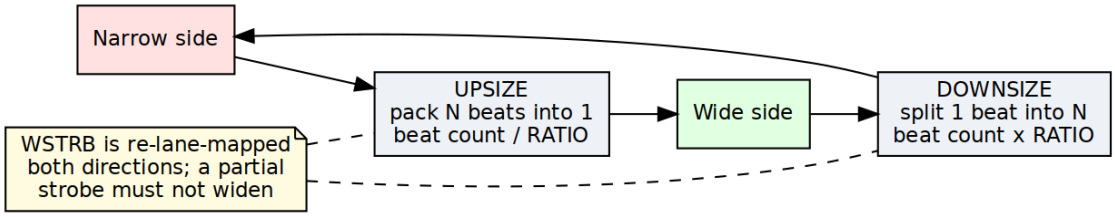
12.2.2 Width Conversion Locations

Converters live at the port boundaries, so a path only pays for conversion when its two ends actually disagree:

```
Master (32-bit) → [Upsizer] → Crossbar (64-bit) → [Downsizer] → Slave
(32-bit)
Master (64-bit) → [No conversion] → Crossbar (64-bit) → [No
conversion] → Slave (64-bit)
Master (128-bit) → [Downsizer] → Crossbar (64-bit) → [Upsizer] → Slave
(128-bit)
```

12.3 2.6.3 Block Diagram

12.3.1 Figure 2.6: Width Conversion Architecture



Width Conversion Architecture

Width conversion architecture showing data upsizing and downsizing with beat count adjustment and strobe handling.

12.4 2.6.4 Upsizing (Narrow to Wide)

12.4.1 Overview

Upsizing converts narrow data to wide data by buffering multiple narrow beats into a single wide beat:

```
Example: 32-bit master → 64-bit crossbar
  Master beat 0: 32'hAAAA_BBBB (bytes 3:0)
  Master beat 1: 32'hCCCC_DDDD (bytes 7:4)
  → Crossbar beat: 64'hCCCC_DDDD_AAAA_BBBB
```

12.4.2 Write Upsizing

Burst of 4 (32-bit) → Burst of 2 (64-bit) — two master beats pack into each crossbar beat, and the burst length halves:

```
Master AW: AWADDR = 0x1000, AWLEN = 3, AWSIZE = 2 (4 bytes)
Master W beats:
  W[0]: WDATA = 0xAAAA_BBBB, WSTRB = 4'b1111, WLAST = 0
  W[1]: WDATA = 0xCCCC_DDDD, WSTRB = 4'b1111, WLAST = 0
  W[2]: WDATA = 0xEEEE_FFFF, WSTRB = 4'b1111, WLAST = 0
  W[3]: WDATA = 0x1111_2222, WSTRB = 4'b1111, WLAST = 1

Crossbar AW: AWADDR = 0x1000, AWLEN = 1, AWSIZE = 3 (8 bytes)
Crossbar W beats:
  W[0]: WDATA = 0xCCCC_DDDD_AAAA_BBBB, WSTRB = 8'b1111_1111, WLAST = 0
  W[1]: WDATA = 0x1111_2222_EEEE_FFFF, WSTRB = 8'b1111_1111, WLAST = 1
```

12.4.3 Read Upsizing

Burst of 8 (32-bit) → Burst of 4 (64-bit) — the same trick in reverse, where each wide crossbar beat fans back out into two narrow master beats:

```
Master AR: ARADDR = 0x2000, ARLEN = 7, ARSIZE = 2 (4 bytes)

Crossbar AR: ARADDR = 0x2000, ARLEN = 3, ARSIZE = 3 (8 bytes)

Crossbar R beats:
  R[0]: RDATA = 0x1111_2222_3333_4444, RLAST = 0
  R[1]: RDATA = 0x5555_6666_7777_8888, RLAST = 0
  R[2]: RDATA = 0x9999_AAAA_BBBB_CCCC, RLAST = 0
  R[3]: RDATA = 0xDDDD_EEEE_FFFF_0000, RLAST = 1

Master R beats (split from crossbar):
  R[0]: RDATA = 0x3333_4444, RLAST = 0  (from R[0] low)
  R[1]: RDATA = 0x1111_2222, RLAST = 0  (from R[0] high)
  R[2]: RDATA = 0x7777_8888, RLAST = 0  (from R[1] low)
```

```
  R[3]: RDATA = 0x5555_6666, RLAST = 0  (from R[1] high)
  R[4]: RDATA = 0xBBBB_CCCC, RLAST = 0  (from R[2] low)
  R[5]: RDATA = 0x9999_AAAA, RLAST = 0  (from R[2] high)
  R[6]: RDATA = 0xFFFF_0000, RLAST = 0  (from R[3] low)
  R[7]: RDATA = 0xDDDD_EEEE, RLAST = 1  (from R[3] high)
```

12.4.4 Strobe Mapping (Upsizing)

Strobes accumulate right alongside the data — each narrow WSTRB lands in the half of the wide word its beat occupied:

```
32-bit WSTRB → 64-bit WSTRB:

Beat 0 (lower 32 bits):
  32-bit WSTRB = 4'b1010 → 64-bit WSTRB = 8'b0000_1010

Beat 1 (upper 32 bits):
  32-bit WSTRB = 4'b1111 → 64-bit WSTRB = 8'b1111_0000

Combined:
  64-bit WSTRB = 8'b1111_1010
```

12.5 2.6.5 Downsizing (Wide to Narrow)

12.5.1 Overview

Downsizing converts wide data to narrow data by splitting a single wide beat into multiple narrow beats:

```
Example: 128-bit master → 64-bit crossbar
  Master beat: 128'h1111_2222_3333_4444_5555_6666_7777_8888
  → Crossbar beat 0: 64'h5555_6666_7777_8888 (lower)
  → Crossbar beat 1: 64'h1111_2222_3333_4444 (upper)
```

12.5.2 Write Downsizing

Burst of 2 (128-bit) → Burst of 4 (64-bit):

```
Master AW: AWADDR = 0x3000, AWLEN = 1, AWSIZE = 4 (16 bytes)
Master W beats:
  W[0]: WDATA = 0xAAAA...(128 bits), WSTRB = 16'hFFFF, WLAST = 0
  W[1]: WDATA = 0xBBBB...(128 bits), WSTRB = 16'hFFFF, WLAST = 1

Crossbar AW: AWADDR = 0x3000, AWLEN = 3, AWSIZE = 3 (8 bytes)
Crossbar W beats:
  W[0]: WDATA = 0xAAAA_low(64), WSTRB = 8'hFF, WLAST = 0
  W[1]: WDATA = 0xAAAA_high(64), WSTRB = 8'hFF, WLAST = 0
  W[2]: WDATA = 0xBBBB_low(64), WSTRB = 8'hFF, WLAST = 0
  W[3]: WDATA = 0xBBBB_high(64), WSTRB = 8'hFF, WLAST = 1
```

12.5.3 Read Downsizing

Burst of 1 (128-bit) → Burst of 2 (64-bit):

```
Master AR: ARADDR = 0x4000, ARLEN = 0, ARSIZE = 4 (16 bytes)

Crossbar AR: ARADDR = 0x4000, ARLEN = 1, ARSIZE = 3 (8 bytes)

Crossbar R beats:
  R[0]: RDATA = 0x1111_2222_3333_4444, RLAST = 0
  R[1]: RDATA = 0x5555_6666_7777_8888, RLAST = 1

Master R beat (merged):
  R[0]: RDATA = 0x5555_6666_7777_8888_1111_2222_3333_4444, RLAST = 1
```

12.5.4 Strobe Mapping (Downsizing)

A 128-bit bus has 16 strobe bits, one per byte lane; a 64-bit bus has 8. On a 2:1 downsize each wide beat becomes two narrow beats, and the strobes split along the same boundary as the data — the low 8 bits go with beat 0, the high 8 with beat 1. No bit is recomputed; the slice is positional.

```
128-bit WSTRB → 64-bit WSTRB (2:1 split):

  wide WSTRB = 16'b1111_0000_0000_1111   (bytes 15:12 and 3:0 written)

  Beat 0 (bytes  7:0) = wide[7:0]  = 8'b0000_1111
  Beat 1 (bytes 15:8) = wide[15:8] = 8'b1111_0000
```

The example this replaces was arithmetically impossible: it wrote a 32-bit value (`16'hF0F0_0FF0`) into a 16-bit literal, then gave both beats the same result (`8'hF0`) from different bit patterns, one of them an 8-bit literal holding three hex digits (`8'h0F0`). Nothing in the RTL was needed to see it was wrong.

12.6 2.6.6 Address Alignment

12.6.1 Address Adjustment for Width

Change the width and the address has to align to the new beat size:

```
32-bit (4-byte) aligned address: 0x1004
Upsized to 64-bit (8-byte): 0x1000 (round down to 8-byte boundary)

Narrow access at 0x1004 within 64-bit word:
  - Byte offset = 0x1004 & 0x7 = 4
  - Data at bytes [7:4] of 64-bit word
  - Lower bytes [3:0] unused (WSTRB = 8'b1111_0000)
```

12.6.2 Unaligned Access Handling

When a master issues an access that isn't aligned to its own width, there are three ways to play it:

```
Master: 32-bit, unaligned access at 0x1002
Problem: Not aligned to 4-byte boundary

Options:
1. Error response (strict mode)
2. Round down address, use WSTRB for actual bytes
3. Split into multiple aligned accesses

Bridge default: Option 2 (use WSTRB)
```

12.7 2.6.7 Burst Length Adjustment

12.7.1 Length Calculation

The AXI length field is beats-minus-one, so the conversion works out like this:

```
Formula:
  New_Length = (Old_Length + 1) × (Old_Width / New_Width) - 1

Example: Upsizing 32→64
  Old: AWLEN = 7 (8 beats), WIDTH = 32
  New: AWLEN = (7+1) × (32/64) - 1 = 8 × 0.5 - 1 = 3 (4 beats)

Example: Downsizing 128→64
  Old: ARLEN = 3 (4 beats), WIDTH = 128
  New: ARLEN = (3+1) × (128/64) - 1 = 4 × 2 - 1 = 7 (8 beats)
```

12.7.2 Odd Burst Lengths

When the burst doesn't divide evenly, the last wide beat runs partial and the strobes say which bytes are real:

```
Example: 3 beats of 32-bit → 64-bit
  3 beats × 4 bytes = 12 bytes total
  12 bytes / 8 bytes per beat = 1.5 beats

Solution:
  - Beat 0: Full 64-bit (8 bytes)
  - Beat 1: Partial 64-bit (4 bytes, WSTRB = 8'b0000_1111)
  - AWLEN = 1 (2 beats)
```

12.8 2.6.8 Resource Utilization

12.8.1 Per-Converter Resources

32→64 Upsizer:

```
Logic Elements:  ~300 LEs
Registers:       ~100 regs (buffering + FSM)
Block RAM:       0

Breakdown:
- Data buffer (32 bits): ~32 regs
- Strobe buffer: ~4 regs
- Beat counter: ~8 regs
- Control FSM: ~50 LEs, ~20 regs
- MUX logic: ~150 LEs
```

64→32 Downsizer:

```
Logic Elements:  ~250 LEs
Registers:       ~120 regs
Block RAM:       0

Similar structure but includes split logic
```

128→64 Downsizer:

```
Logic Elements:  ~400 LEs
Registers:       ~180 regs
Block RAM:       0

Larger data paths, more complex MUX
```

12.8.2 Scaling

Three things drive the cost:

- **Width ratio**: 2:1 vs. 4:1 vs. 8:1 conversion
- **Data width**: 128-bit vs. 256-bit buffers
- **Buffering depth**: Single vs. multi-beat buffering

12.9 2.6.9 Timing Characteristics

12.9.1 Latency

Upsizing (combining beats): - Buffering latency: 1-2 cycles (wait for N narrow beats) - First beat output: After receiving required narrow beats - Example: 32→64 requires 2 master beats before 1 crossbar beat

Downsizing (splitting beats): - Minimal latency: 1 cycle (register stage) - First beat output: Immediately (split from wide beat) - Example: 128→64 outputs first 64-bit beat immediately

12.9.2 Throughput

Upsizing Throughput:

```
32→64 upsizing:
  Master: 32 bits/cycle = 4 bytes/cycle
  Crossbar: 64 bits per 2 cycles = 4 bytes/cycle (same)
  No throughput loss
```

Downsizing Throughput:

```
128→64 downsizing:
  Master: 128 bits/cycle = 16 bytes/cycle
  Crossbar: 64 bits/cycle = 8 bytes/cycle
  Throughput halved (crossbar becomes bottleneck)
```

Upsizing is free; downsizing costs you the width ratio. Keep that asymmetry in mind when you pick port widths.

12.10 2.6.10 Configuration Parameters

12.10.1 Width Conversion Configuration (TOML)

```toml
[bridge]
# internal_data_width: NOT A KEY -- there is no fixed internal
crossbar width.
enable_width_conversion = true  # Allow width differences

[[bridge.masters]]
name = "cpu_32bit"
data_width = 32                 # Narrower than crossbar (upsizing)
addr_width = 32

[[bridge.masters]]
name = "dma_64bit"
data_width = 64                 # Matches crossbar (no conversion)
addr_width = 32

[[bridge.masters]]
name = "gpu_128bit"
data_width = 128                # Wider than crossbar (downsizing)
addr_width = 36

[[bridge.slaves]]
name = "memory_64bit"
data_width = 64                 # Matches crossbar (no conversion)

[[bridge.slaves]]
name = "periph_32bit"
data_width = 32                 # Narrower than crossbar (downsizing)
```

12.11 2.6.11 Debug and Observability

12.11.1 Recommended Debug Signals

When a conversion path misbehaves, these are the signals to pull into the waveform first:

```
Upsizer:
- Beat accumulator (partial wide beat being assembled)
- Beat counter (position in burst)
- Buffer valid (accumulated beats ready)
- Strobe accumulation

Downsizer:
- Beat splitter state (which sub-beat currently outputting)
- Remaining beats to output
- Source data register
```

12.11.2 Common Issues and Debug

Symptom: Data corruption after width conversion
Check: - Byte ordering (endianness) - Strobe mapping (correct bytes enabled) - Address alignment - Burst length calculation

Symptom: Stalls after width conversion
Check: - Buffer depths (upsizer needs buffering) - Backpressure propagation - LAST signaling

Symptom: Incorrect burst lengths
Check: - Length calculation formula - Odd burst handling - SIZE field matching width

12.12 2.6.12 Verification Considerations

12.12.1 Test Scenarios

1. **Power-of-2 Width Ratios**:

```
- 32→64 (2:1)
- 64→128 (1:2)
- 32→128 (4:1)
```

2. **Various Burst Lengths**:

```
- Single beat (ALEN=0)
- Even bursts (ALEN=3, 7, 15)
- Odd bursts (ALEN=1, 5, 9)
```

3. **Sub-Word Accesses**:

```
- Byte writes (WSTRB with individual bytes)
- Half-word, word accesses
- Unaligned accesses
```

4. **Mixed Widths**:

```
- 32-bit master → 64-bit crossbar → 32-bit slave
- 128-bit master → 64-bit crossbar → 32-bit slave
- All width combinations
```

12.13 2.6.13 Performance Considerations

12.13.1 Conversion Overhead

Upsizing Overhead: - Buffering latency: +1-2 cycles - Throughput maintained - Best for burst-oriented masters

Downsizing Overhead: - Split latency: +1 cycle - Throughput reduced by width ratio - Can bottleneck wide masters

12.13.2 Optimization Strategies

1. **Match Common Widths**: match a master to its slave's width, so no converter is inserted on that path. There is no single internal width to match.
2. **Burst Sizing**: Use appropriate burst lengths for width ratios
3. **Parallel Paths**: Multiple 32-bit masters can aggregate to 64-bit crossbar bandwidth
4. **Selective Conversion**: Only convert where necessary

12.14 2.6.14 Future Enhancements

12.14.1 Planned Features

- **Configurable Internal Width**: Support 32, 64, 128, 256-bit crossbar
- **Multi-Beat Buffering**: Deeper upsizing buffers for smoother flow
- **Byte Rotation**: Handle unaligned access more efficiently
- **Pipelined Conversion**: Multi-stage for high frequency

12.14.2 Under Consideration

- **Asymmetric Widths**: Different widths for read vs. write paths
- **Sparse Data Optimization**: Skip unused bytes in conversions
- **Tagged Data**: Metadata preservation through width conversion
- **Zero-Copy Bypass**: Direct routing for matching widths

Related Sections: - Section 2.1: Master Adapter (upsizing location) - Section 2.3: Crossbar Core (internal data width) - HAS ch06_integration/02_parameters.md (per-port data_width) - HAS ch06_integration/02_parameters.md (per-port data_width)

---

13 2.7 Protocol Conversion

The crossbar speaks AXI4 internally. Protocol conversion is what lets the bridge reach slaves that speak something simpler — APB (Advanced Peripheral Bus) for low-bandwidth peripherals being the common case.

13.1 2.7.1 Purpose and Function

Protocol conversion has five jobs:

1. **Protocol Translation**: Converts AXI4 transactions to target protocol (e.g., APB)
2. **Handshake Mapping**: Translates ready/valid to protocol-specific handshakes
3. **Burst Decomposition**: Breaks AXI bursts into single-beat target transactions
4. **Response Mapping**: Converts protocol-specific responses back to AXI responses
5. **Timing Adaptation**: Handles different timing requirements between protocols

13.2 2.7.2 Supported Protocols

13.2.1 Current Support (Phase 2)

AXI4 (Native): - Full AXI4 protocol - No conversion required - Maximum performance

AXI4-Lite (Master-Side): - Simplified AXI4 subset - Single-beat transactions only (ARLEN=0, AWLEN=0) - Converts to full AXI4 for crossbar - Common for control/status registers

APB (Slave-Side): - APB3 and APB4 support - For low-bandwidth peripherals - Simplified handshaking

APB and APB5 (Master-Side, bridge TASK-004 (was BRIDGE-014)): - The bridge is the APB completer; `apb4_to_axi4` / `apb5_to_axi4` (converters component) turn each transfer into one single-beat AXI4 transaction in front of the ordinary master timing wrapper - SLVERR and DECERR both fold to PSLVERR (APB has one error bit) - APB5: `PAUSER[0]`/`PWUSER[0]` ride the fabric's USER bit; `PWAKEUP` is accepted and terminated

AXI5-Lite (Master-Side, bridge TASK-004 (was BRIDGE-014)): - The AXI4-Lite path plus the AXI5-Lite sideband on the boundary - `exclusive` -> `AxLOCK` and `user` -> the 1-bit USER fields ride the fabric; every other group is terminated at the bridge top

Wishbone B4 (both sides, bridge TASK-008 (was BRIDGE-019)): - Slave port: `axi4_to_wb4` (the AXI4-Lite decomposers + `axil4_to_wb4`), one Wishbone transfer per AXI4 beat, ACK -> OKAY, ERR/RTY -> SLVERR - Master port: `wb4_to_axi4` (`wb4_to_axil4` + the single-beat AXI4 promotion) in front of the ordinary timing wrapper; SLVERR/DECERR -> ERR - B4 pipelined; `CTI`/`BTE` carried as CLASSIC/LINEAR, never acted on

AXI3 (both sides, GitHub #97): - Slave port: `axi4_to_axi3` converts the crossbar's AXI4 master face to an AXI3 slave face; same data width only. - Master port: `axi3_to_axi4` converts the external AXI3 master face to an AXI4 slave face, then the ordinary AXI4 timing wrapper takes over. - Long egress bursts are chopped, WID is stamped/stripped, and lock mapping follows the converter's documented encoding — see "AXI3 Boundary Conversion" below and the converters MAS §3.15/§3.16.

13.2.2 Current Limitation: AXI4-Lite Conversion

Superseded. This paragraph said AXIL slaves were treated as full AXI4 internally with no real conversion; the very next paragraph, and the RTL, say otherwise – `axi4_to_axil4_{rd,wr}.sv` perform genuine burst decomposition into single-beat AXI4-Lite transactions. Kept only so the contradiction is not silently deleted. The old text read: - Use the standard AXI4 timing wrapper (same as AXI4 slaves) - Expose full 5-channel AXI4 interface at the bridge boundary - Are documented as "axil" for user reference only

The bridge now emits real AXI4-to-AXIL4 conversion shims (`axi4_to_axil4_{rd,wr}.sv`) at the slave boundary when `protocol = "axil"` is specified in the TOML. These shims downgrade bursts to single beats and enforce AXIL protocol constraints transparently. See "Generator-Emitted Conversion Shims" below.

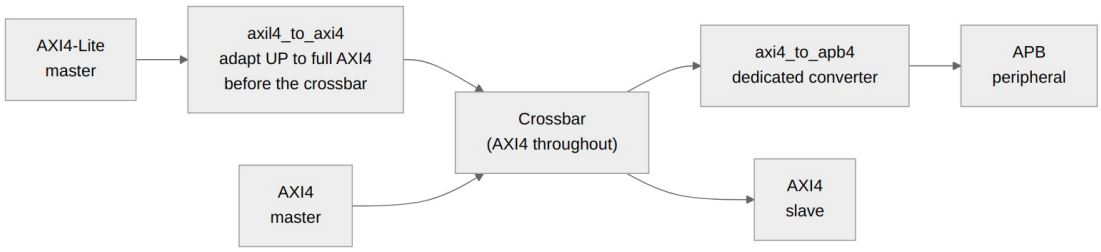
13.2.3 Future Support (Phase 2+)

AXI4-Lite conversion was listed here as future work while the section directly above states it is built and names the shims. It is built; the stale entry has been removed.

- **AHB**: Advanced High-performance Bus
- **Wishbone**: Open-source bus standard
- **Custom**: User-defined protocols

13.3 2.7.3 Conversion Architecture Overview

13.3.1 Figure 2.7.1: Protocol Conversion Architecture

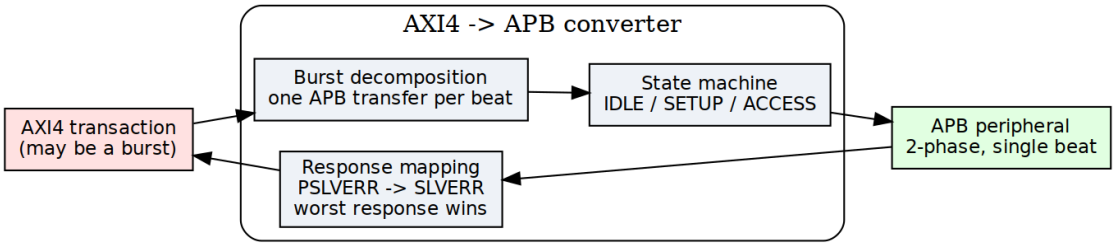


Protocol Conversion Architecture

The diagram shows the complete protocol conversion flow: AXI4-Lite masters are adapted to full AXI4 before entering the crossbar, while APB peripherals receive AXI4 transactions through a dedicated converter.

13.4 2.7.4 Block Diagram

13.4.1 Figure 2.7: Protocol Conversion Architecture



Protocol Conversion Architecture

Protocol conversion showing AXI4 to APB conversion with state machine, burst decomposition, and response mapping.

13.5 2.7.5 AXI4-Lite to AXI4 Conversion (Master-Side)

13.5.1 AXI4-Lite Protocol Overview

AXI4-Lite is a simplified subset of AXI4 aimed at simple control/status register access:

```
Key Differences from Full AXI4:
- FIXED burst length: ARLEN/AWLEN always = 0 (single beat)
- FIXED burst size: No SIZE field, always full data width
- FIXED burst type: No BURST field, always INCR
- NO exclusive access: No LOCK support
- NO unaligned transfers: Address must be aligned
- Simpler ID: Typically 1-4 bits (fewer outstanding transactions)

Similarities to AXI4:
- Same 5 channels: AR, R, AW, W, B
- Same valid/ready handshaking
- Response codes: OKAY, SLVERR, DECERR. NOT EXOKAY -- AXI4-Lite has no
  exclusive access, so the code has no meaning on this interface.
- Same data widths: 32 or 64 bits typically
```

13.5.2 Conversion Requirements

Adapting an AXI4-Lite master to the full AXI4 crossbar takes three things:

1. **Add Missing Signals**: Provide default values for burst-related signals
2. **Validate Constraints**: Ensure single-beat assumption holds
3. **Pass-Through Simplicity**: Most signals connect directly

13.5.3 Signal Mapping

```
// AXI4-Lite to AXI4 Signal Mapping

// AR Channel (Read Address)
// AXI4-Lite Input          AXI4 Crossbar Output
axi4lite_arvalid      →     axi4_arvalid
axi4lite_arready      ←     axi4_arready
axi4lite_araddr       →     axi4_araddr
axi4lite_arprot       →     axi4_arprot
axi4lite_arid (opt)   →     axi4_arid

// Added by adapter (constants):
                            axi4_arlen    = 8'h00      // Always 1
beat
                            axi4_arsize   = log2(DW/8) // Full width
                            axi4_arburst  = 2'b01      // INCR
                            axi4_arlock   = 1'b0       // No lock
```

```
                            axi4_arcache  = 4'b0000    // Device non-
buf
                            axi4_arqos    = 4'h0       // No QoS
                            axi4_arregion = 4'h0       // Region 0

// R Channel (Read Data)
// AXI4 Crossbar Input      AXI4-Lite Output
axi4_rvalid           →     axi4lite_rvalid
axi4_rready           ←     axi4lite_rready
axi4_rdata            →     axi4lite_rdata
axi4_rresp            →     axi4lite_rresp
axi4_rid (opt)        →     axi4lite_rid (opt)

// Discarded by adapter:
axi4_rlast                  // Always 1 for single beat

// AW Channel (Write Address)
axi4lite_awvalid      →     axi4_awvalid
axi4lite_awready      ←     axi4_awready
axi4lite_awaddr       →     axi4_awaddr
axi4lite_awprot       →     axi4_awprot
axi4lite_awid (opt)   →     axi4_awid

// Added by adapter:
                            axi4_awlen    = 8'h00
                            axi4_awsize   = log2(DW/8)
                            axi4_awburst  = 2'b01
                            axi4_awlock   = 1'b0
                            axi4_awcache  = 4'b0000
                            axi4_awqos    = 4'h0
                            axi4_awregion = 4'h0

// W Channel (Write Data)
axi4lite_wvalid       →     axi4_wvalid
axi4lite_wready       ←     axi4_wready
axi4lite_wdata        →     axi4_wdata
axi4lite_wstrb        →     axi4_wstrb

// Added by adapter:
                            axi4_wlast    = 1'b1       // Always last

// B Channel (Write Response)
axi4_bvalid           →     axi4lite_bvalid
axi4_bready           ←     axi4lite_bready
axi4_bresp            →     axi4lite_bresp
axi4_bid (opt)        →     axi4lite_bid (opt)
```

13.5.4 Implementation

The adapter itself is almost embarrassingly simple — mostly wires, with constants tied off for everything AXI4-Lite doesn't have:

```
// AXI4-Lite to AXI4 Adapter (simplified)
module axi4lite_to_axi4_adapter #(
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 64,
    parameter ID_WIDTH = 4          // Optional, often 0 for AXI4-Lite
) (
    input logic clk,
    input logic rst_n,

    // AXI4-Lite Master Interface
    input  logic                    lite_arvalid,
    output logic                    lite_arready,
    input  logic [ADDR_WIDTH-1:0]   lite_araddr,
    input  logic [2:0]              lite_arprot,
    input  logic [ID_WIDTH-1:0]     lite_arid,    // Optional

    output logic                    lite_rvalid,
    input  logic                    lite_rready,
    output logic [DATA_WIDTH-1:0]   lite_rdata,
    output logic [1:0]              lite_rresp,
    output logic [ID_WIDTH-1:0]     lite_rid,     // Optional

    input  logic                    lite_awvalid,
    output logic                    lite_awready,
    input  logic [ADDR_WIDTH-1:0]   lite_awaddr,
    input  logic [2:0]              lite_awprot,
    input  logic [ID_WIDTH-1:0]     lite_awid,    // Optional

    input  logic                    lite_wvalid,
    output logic                    lite_wready,
    input  logic [DATA_WIDTH-1:0]   lite_wdata,
    input  logic [DATA_WIDTH/8-1:0] lite_wstrb,

    output logic                    lite_bvalid,
    input  logic                    lite_bready,
    output logic [1:0]              lite_bresp,
    output logic [ID_WIDTH-1:0]     lite_bid,     // Optional

    // Full AXI4 Crossbar Interface
    output logic                    axi4_arvalid,
    input  logic                    axi4_arready,
    output logic [ADDR_WIDTH-1:0]   axi4_araddr,
    output logic [7:0]              axi4_arlen,
    output logic [2:0]              axi4_arsize,
```

```
    output logic [1:0]              axi4_arburst,
    output logic                    axi4_arlock,
    output logic [3:0]              axi4_arcache,
    output logic [2:0]              axi4_arprot,
    output logic [3:0]              axi4_arqos,
    output logic [3:0]              axi4_arregion,
    output logic [ID_WIDTH-1:0]     axi4_arid,

    input  logic                    axi4_rvalid,
    output logic                    axi4_rready,
    input  logic [DATA_WIDTH-1:0]   axi4_rdata,
    input  logic [1:0]              axi4_rresp,
    input  logic                    axi4_rlast,
    input  logic [ID_WIDTH-1:0]     axi4_rid,

    // ... (AW, W, B channels similar)
);

    // AR Channel: Pass-through with constants
    assign axi4_arvalid  = lite_arvalid;
    assign lite_arready  = axi4_arready;
    assign axi4_araddr   = lite_araddr;
    assign axi4_arprot   = lite_arprot;
    assign axi4_arid     = lite_arid;

    // Constants for single-beat burst
    assign axi4_arlen    = 8'h00;                   // 1 beat
    assign axi4_arsize   = $clog2(DATA_WIDTH/8);    // Full width
    assign axi4_arburst  = 2'b01;                   // INCR
    assign axi4_arlock   = 1'b0;                    // No lock
    assign axi4_arcache  = 4'b0000;                 // Device non-buf
    assign axi4_arqos    = 4'h0;                    // No QoS
    assign axi4_arregion = 4'h0;                    // Region 0

    // R Channel: Pass-through, ignore rlast
    assign lite_rvalid = axi4_rvalid;
    assign axi4_rready = lite_rready;
    assign lite_rdata  = axi4_rdata;
    assign lite_rresp  = axi4_rresp;
    assign lite_rid    = axi4_rid;
    // axi4_rlast ignored (always 1 for single beat)

    // AW Channel: Similar to AR
    assign axi4_awvalid  = lite_awvalid;
    assign lite_awready  = axi4_awready;
    assign axi4_awaddr   = lite_awaddr;
    assign axi4_awprot   = lite_awprot;
    assign axi4_awid     = lite_awid;
```

```
    assign axi4_awlen    = 8'h00;
    assign axi4_awsize   = $clog2(DATA_WIDTH/8);
    assign axi4_awburst  = 2'b01;
    assign axi4_awlock   = 1'b0;
    assign axi4_awcache  = 4'b0000;
    assign axi4_awqos    = 4'h0;
    assign axi4_awregion = 4'h0;

    // W Channel: Pass-through, add wlast
    assign axi4_wvalid = lite_wvalid;
    assign lite_wready = axi4_wready;
    assign axi4_wdata  = lite_wdata;
    assign axi4_wstrb  = lite_wstrb;
    assign axi4_wlast  = 1'b1;                          // Always last

    // B Channel: Pass-through
    assign lite_bvalid = axi4_bvalid;
    assign axi4_bready = lite_bready;
    assign lite_bresp  = axi4_bresp;
    assign lite_bid    = axi4_bid;

endmodule
```

13.5.5 Resource Utilization

AXI4-Lite Adapter Resources:

```
Logic Elements:  ~50-100 LEs (minimal, mostly wiring)
Registers:       ~50 regs (if skid buffers added)
Block RAM:       0

Breakdown:
- Signal pass-through: ~20 LEs (buffering)
- Constant generation: ~10 LEs
- Optional skid buffers: ~50 regs (for timing)

Note: Most implementations are purely combinatorial wire
      assignments with optional pipeline registers.
```

13.5.6 Performance Impact

Latency: - **Zero-latency** (combinatorial) if no pipeline stages - **1-2 cycles** if skid buffers added for timing - No protocol conversion overhead

Throughput: - **1 transaction per cycle** (same as native AXI4) - No degradation for single-beat transactions - Limited by AXI4-Lite's single-beat constraint

13.5.7 Configuration

```
[[bridge.masters]]
name       = "control_processor"
prefix     = "cpu_axil_"       # required -- prefixes every external
signal
protocol   = "axil"            # NOT "axi4lite"; see the protocol
table in the HAS
channels   = "rw"              # "rw" | "rd" | "wr"
id_width   = 0                 # AXI4-Lite carries no ID
addr_width = 32
data_width = 32
user_width = 1
```

Mind the field names: they are `id_width` / `addr_width` / `data_width` – one `id_width` per port, not the `arid_width` / `awid_width` pair an earlier revision of this page showed, which the loader does not read. The table is `[[bridge.masters]]`, not `[[masters]]`. Compare `bin/test_configs/bridge_1x5_wr_axil.toml` for a config that actually generates.

13.5.8 Common Issues and Debug

Issue 1: Burst Detected on AXI4-Lite

```
Symptom: ARLEN/AWLEN != 0 on AXI4-Lite interface
Cause: Master not properly configured as AXI4-Lite
Check: Verify master only issues single-beat transactions
```

Issue 2: Unaligned Addresses

```
Symptom: ARADDR/AWADDR not aligned to data width
Cause: AXI4-Lite requires full-width aligned access
Check: Address[log2(DW/8)-1:0] should be zero
```

Issue 3: rlast/wlast Handling

```
Symptom: Master expects rlast/wlast but doesn't have them
Cause: True AXI4-Lite interface omits these signals
Solution: Adapter provides wlast=1 to crossbar, strips rlast
```

13.6 2.7.6 AXI4 to APB Conversion (Slave-Side)

13.6.1 APB Protocol Overview

APB is a simple, low-power bus protocol — it earns its keep on area and power, not speed:

```
Characteristics:
- Single address phase
- Single data phase
- No burst support (one transfer per operation)
- Minimal logic
- Low power consumption
- Suitable for peripherals: UARTs, timers, GPIOs
```

13.6.2 APB Signals

```
Address Phase:
  PADDR[N-1:0]  - Address bus
  PSEL          - Slave select
  PENABLE       - Enable (2nd cycle of transfer)
  PWRITE        - Write direction (1=write, 0=read)

Data Phase (Write):
  PWDATA[N-1:0] - Write data
  PSTRB[N/8-1:0]- Write strobes (APB4 only)

Data Phase (Read):
  PRDATA[N-1:0] - Read data

Response:
  PREADY        - Slave ready (can extend transfer)
  PSLVERR       - Slave error (APB3+)
```

13.6.3 APB State Machine

Every APB transfer is a 2-phase handshake — SETUP, then ACCESS:

```
IDLE:
  - Wait for AXI request (ARVALID or AWVALID)
  - PSEL = 0, PENABLE = 0

SETUP:
  - Assert PSEL = 1
  - Drive PADDR, PWRITE, PWDATA (if write)
  - PENABLE = 0
  - Duration: 1 cycle

ACCESS:
  - Assert PENABLE = 1
```

```
  - Wait for PREADY = 1
  - Capture PRDATA (if read) or PSLVERR
  - Can extend multiple cycles if PREADY = 0

Complete:
  - De-assert PSEL, PENABLE
  - Return to IDLE or SETUP (if more beats)
```

13.6.4 Read Transaction Conversion

AXI4 Read:

```
Cycle 0: ARVALID=1, ARADDR=0x100, ARLEN=3 (4 beats)
Cycle 1: ARREADY=1
Cycles 2-5: R beats returning
```

Converted to APB (4 separate APB reads):

```
Beat 0:
  Cycle 0: PSEL=1, PENABLE=0, PADDR=0x100, PWRITE=0 (SETUP)
  Cycle 1: PSEL=1, PENABLE=1, wait PREADY (ACCESS)
  Cycle 2: PREADY=1, capture PRDATA → First R beat

Beat 1:
  Cycle 3: PSEL=1, PENABLE=0, PADDR=0x104 (SETUP)
  Cycle 4: PSEL=1, PENABLE=1, wait PREADY (ACCESS)
  Cycle 5: PREADY=1, capture PRDATA → Second R beat

... (beats 2 and 3 similar)
```

Latency: 2-3 cycles per beat (SETUP + ACCESS + ready)

13.6.5 Write Transaction Conversion

AXI4 Write:

```
Cycle 0: AWVALID=1, AWADDR=0x200, AWLEN=1 (2 beats)
Cycle 1: AWREADY=1
Cycle 1: WVALID=1, WDATA=0xAAAA_BBBB, WLAST=0
Cycle 2: WREADY=1
Cycle 2: WVALID=1, WDATA=0xCCCC_DDDD, WLAST=1
Cycle 3: WREADY=1
Cycle 4: BVALID=1, BRESP=OKAY
```

Converted to APB (2 separate APB writes):

```
Beat 0:
  Cycle 0: PSEL=1, PENABLE=0, PADDR=0x200, PWRITE=1,
PWDATA=0xAAAA_BBBB
  Cycle 1: PSEL=1, PENABLE=1, wait PREADY
```

```
  Cycle 2: PREADY=1 → First write complete

Beat 1:
  Cycle 3: PSEL=1, PENABLE=0, PADDR=0x204, PWRITE=1,
PWDATA=0xCCCC_DDDD
  Cycle 4: PSEL=1, PENABLE=1, wait PREADY
  Cycle 5: PREADY=1 → Second write complete, return B
```

13.6.6 Burst Handling

APB has no burst concept, so the converter decomposes:

```
AXI Burst: AWLEN = 15 (16 beats)
→ 16 separate APB transfers
→ Address increments per AWBURST type:
   - INCR: Addr += SIZE each beat
   - WRAP: Wrapping within boundary
   - FIXED: Same address each beat
```

13.6.7 Response Mapping

```
APB → AXI Response Translation:

PREADY=1, PSLVERR=0 → RRESP/BRESP = 2'b00 (OKAY)
PREADY=1, PSLVERR=1 → RRESP/BRESP = 2'b10 (SLVERR)

PREADY stuck at 0:
  The converter WAITS. There is no timeout and no DECERR.
```

There is no PREADY timeout anywhere in the path. `apb4_master` waits unconditionally in ACCESS (`if (m_apb_PREADY) ...` with no counter), and neither `axi4_to_apb4_convert`, `axi4_to_apb4_shim` nor `axi4_to_apb5_shim` contains a threshold register or cycle counter. A slave that never asserts PREADY stalls that path indefinitely, and the stall propagates back through the crossbar as backpressure. Budget for it in the system, or put a watchdog outside the bridge; the bridge will not manufacture an error response.

13.7 2.7.7 Implementation

13.7.1 AXI4-to-APB Converter FSM

```
// Simplified AXI4-to-APB converter
typedef enum logic [2:0] {
    IDLE,
    AR_SETUP,
    AR_ACCESS,
    AW_SETUP,
    W_ACCESS,
    B_RESPONSE
} state_t;

state_t state, next_state;

always_ff @(posedge clk) begin
    if (!rst_n) state <= IDLE;
    else state <= next_state;
end

always_comb begin
    next_state = state;

    case (state)
        IDLE: begin
            if (arvalid) next_state = AR_SETUP;
            else if (awvalid) next_state = AW_SETUP;
        end

        AR_SETUP: begin
            next_state = AR_ACCESS;  // 1 cycle SETUP
        end

        AR_ACCESS: begin
            if (pready) begin
                if (more_beats) next_state = AR_SETUP;  // Next beat
                else next_state = IDLE;
            end
        end

        AW_SETUP: begin
            if (wvalid) next_state = W_ACCESS;
        end

        W_ACCESS: begin
            if (pready) begin
                if (!wlast) next_state = AW_SETUP;  // Next beat
```

```
                else next_state = B_RESPONSE;
            end
        end

        B_RESPONSE: begin
            if (bready) next_state = IDLE;
        end
    endcase
end

// APB signal generation
assign psel = (state != IDLE);
assign penable = (state == AR_ACCESS || state == W_ACCESS);
assign pwrite = (state == AW_SETUP || state == W_ACCESS);
```

13.7.2 Address Generation

```
// Address increment for burst
logic [ADDR_WIDTH-1:0] current_addr;
logic [7:0] beat_count;

always_ff @(posedge clk) begin
    if (state == IDLE) begin
        current_addr <= arvalid ? araddr : awaddr;
        beat_count <= 0;
    end else if ((state == AR_ACCESS || state == W_ACCESS) && pready)
begin
        beat_count <= beat_count + 1;

        case (burst_type)
            2'b01: current_addr <= current_addr + (1 << size);  //
INCR
            2'b10: current_addr <= wrap_address(current_addr);  //
WRAP
            2'b00: current_addr <= current_addr;                //
FIXED
        endcase
    end
end

assign paddr = current_addr;
```

13.8 2.7.8 Resource Utilization

13.8.1 APB Converter Resources

Per APB Slave Interface:

```
Logic Elements:  ~400 LEs
Registers:       ~150 regs
Block RAM:       0

Breakdown:
- FSM control:           ~100 LEs, ~20 regs
- Address generation:    ~80 LEs, ~40 regs
- Burst counter:         ~50 LEs, ~20 regs
- Response accumulation: ~80 LEs, ~30 regs
- Data path MUX:         ~90 LEs, ~40 regs
```

13.8.2 Scaling

Adding APB slaves: - Linear scaling: +~400 LEs per APB slave - Shared address decoder logic - Independent per-slave FSMs

13.9 2.7.9 Timing Characteristics

13.9.1 Latency

APB Read Latency (per beat):

```
Best case: 2 cycles (SETUP + ACCESS with PREADY=1)
Typical: 3-5 cycles (if slave extends with PREADY=0)
Worst case: unbounded -- the converter waits for PREADY with no
timeout

For 8-beat AXI burst:
  Total: 8 × 3 = 24 cycles typical
```

APB Write Latency (per beat):

```
Similar to read: 2-5 cycles per beat
```

13.9.2 Throughput

Severely Limited:

```
APB: ~0.3-0.5 transactions/cycle (due to 2-3 cycle protocol)
AXI4: 1 transaction/cycle (burst mode)

APB suitable only for low-bandwidth peripherals
```

No amount of buffering fixes that — the 2-3 cycle protocol itself is the bottleneck.

13.10 2.7.10 Configuration Parameters

13.10.1 Protocol Conversion Configuration (TOML)

```toml
[[bridge.slaves]]
name = "uart_peripheral"
protocol = "apb"            # "axi4", "apb", "ahb" (future)
base_address = 0xF000_0000
size = 0x1000
data_width = 32

[[bridge.slaves]]
name = "ddr_memory"
protocol = "axi4"           # Native, no conversion
base_address = 0x8000_0000
size = 0x4000_0000
data_width = 64
```

13.11 2.7.11 Debug and Observability

13.11.1 Recommended Debug Signals

```
APB Converter:
- FSM state
- APB phase (SETUP, ACCESS)
- Beat counter (progress through burst)
- Response accumulation (for burst)

APB Bus:
- PSEL, PENABLE, PWRITE
- PADDR, PWDATA, PRDATA
- PREADY, PSLVERR
```

13.11.2 Common Issues and Debug

Symptom: APB slave not responding (timeout)
Check: - PREADY signal (stuck at 0?) - APB slave clock/reset - PSEL assertion (There is no timeout threshold to check – the converter waits forever.)

Symptom: Data corruption on APB
Check: - SETUP phase duration (should be 1 cycle) - PENABLE assertion timing - Data sampling on correct cycle

Symptom: Burst to APB takes too long
Check: - Burst length (consider limiting ARLEN/AWLEN) - APB slave response time (PREADY) - Alternative: Use AXI4 slave instead

13.12 2.7.12 Verification Considerations

13.12.1 Test Scenarios

1. **Single APB Transfer**:

```
- AXI ARLEN=0 (1 beat) → 1 APB read
- Verify SETUP → ACCESS sequence
- Check PREADY handling
```

2. **APB Burst Decomposition**:

```
- AXI AWLEN=7 (8 beats) → 8 APB writes
- Verify address increment
- Check each beat completes before next
```

3. **APB PREADY Extension**:

```
- Slave holds PREADY=0 for N cycles
- Verify converter waits
- Check no data corruption
```

4. **APB Error Response**:

```
- Slave asserts PSLVERR
- Verify mapped to AXI SLVERR
- Check error propagated to master
```

5. **APB stall (no timeout to test)**:

```
- Slave never asserts PREADY
- Verify the path stalls and backpressures cleanly, with no lost or
  duplicated beats once PREADY finally arrives
- There is NO timeout and NO DECERR to check for -- see 2.7.6
```

13.13 2.7.13 Performance Considerations

13.13.1 When to Use APB

Good Use Cases: - Low-speed peripherals (UART, GPIO, timers) - Infrequent accesses - Simple register interfaces - Power-sensitive designs

Poor Use Cases: - High-bandwidth devices - Burst-intensive masters - Performance-critical paths - Memory interfaces

13.13.2 APB vs. AXI4 Comparison

Feature	APB	AXI4
Complexity	Simple	Complex
Throughput	Low (~0.3/cyc)	High (1/cyc burst)
Latency/beat	2-5 cycles	1 cycle
Burst Support	No	Yes (up to 256)
Resources	~400 LEs	Native (no converter)
Power	Very low	Moderate
Use Case	Peripherals	Memory, DMA

13.14 2.7.14 Mixed Protocol Bridges

13.14.1 Example Configuration

```toml
# Bridge with mixed protocols
[bridge]
num_masters = 2
num_slaves = 3

[[bridge.masters]]
name = "cpu"
protocol = "axi4"

[[bridge.masters]]
name = "dma"
protocol = "axi4"

[[bridge.slaves]]
name = "ddr_memory"
protocol = "axi4"          # High bandwidth

[[bridge.slaves]]
name = "sram"
protocol = "axi4"          # Medium bandwidth

[[bridge.slaves]]
name = "peripherals"
protocol = "apb"           # Low bandwidth, simple
```

13.14.2 Routing Optimization

Walk the routes and you can see exactly where the conversion cost lands:

```
CPU → DDR Memory: AXI4-to-AXI4 (native, fast)
CPU → Peripherals: AXI4-to-APB (converted, slower)
DMA → SRAM: AXI4-to-AXI4 (native, fast)
DMA → Peripherals: AXI4-to-APB (rare, acceptable slowdown)
```

13.15 2.7.15 Master-Side vs Slave-Side Conversion

13.15.1 Comparison

Feature	Master-Side (AXI4-Lite)	Slave-Side (APB)
Complexity	Very Simple	Complex
Resource Usage	~50-100 LEs	~400 LEs
Latency Added	0-1 cycles	2-5 cycles/beat
Throughput Impact	None	Severe (3x slower)
Burst Handling	Single beat only	Decompose to singles
State Machine	None (combinatorial)	Multi-state FSM
Buffering Required	Optional (timing)	Essential
Use Case	Control registers	Peripherals

13.15.2 When to Use Each

AXI4-Lite Master Adapter: - Simple control/status register interfaces - Low-complexity masters (MCUs, simple CPUs) - Minimal resource overhead acceptable - No burst performance needed

APB Master Front End (`apb{4,5}_to_axi4`, bridge TASK-004 (was BRIDGE-014)): - An APB requester that needs to reach AXI4 or AXI4-Lite completers - One-outstanding by nature of APB: one transfer per fabric round trip - Same lane behaviour as an AXI4-Lite master toward wider slaves (the aligner is shared)

APB Slave Converter: - Legacy peripheral integration - Very simple slave devices (GPIO, timers) - Low-bandwidth acceptable - Power optimization critical

13.16 2.7.16 Generator-Emitted Conversion Shims

You don't instantiate any of these converters by hand. The bridge generator automatically emits protocol conversion shims at the slave boundary based on the TOML configuration. These shims are instantiated between the crossbar core (uniformly AXI4 internally) and the external slave port.

13.16.1 AXI4 to AXI4-Lite Conversion

When `protocol = "axil"` is specified in the slave TOML: - Generator emits `axi4_to_axil4_rd.sv` (read path) and `axi4_to_axil4_wr.sv` (write path) - Shims downgrade bursts to single beats (set ARLEN/AWLEN = 0 on output) - Enforce single-beat semantics transparently - Data width remains unchanged (upsizing/downsizing done separately) - Shims are inserted **between crossbar and slave port** in the generated top-level module

Modules:
`projects/components/utility-ip/converters/rtl/axi4_to_axil4_{rd,wr}.sv`

13.16.2 AXI4 to APB Conversion

When `protocol = "apb"` is specified in the slave TOML: - Generator emits ONE shim, `axi4_to_apb4_shim` – a direct full-AXI4-to-APB4 converter. There is no AXI4-Lite stage: the shim does its own burst decomposition (`r_burst_count` + `axi_gen_addr`), so nothing upstream has to reduce the burst first. - `protocol = "apb5"` emits `axi4_to_apb5_shim`, a sideband wrapper over that same shim – see AMBA5 Boundary. - Slave port externally presents APB signals; internally the crossbar is AXI4

Modules: `projects/components/utility-ip/converters/rtl/axi4_to_apb4_shim.sv` (`axi4_to_apb5_shim.sv` for `apb5`)

An earlier revision of this page described the path as an `axi4_to_axil4` shim followed by an internal AXIL-to-APB bridge chain. No such chain exists, and no generated APB slave adapter instantiates `axi4_to_axil4`.

13.16.3 Master-Side APB Front End

When `protocol = "apb"` or `"apb5"` is specified on a **master** (bridge TASK-004, was BRIDGE-014): - The master adapter's external surface is the APB completer set (the requester's `PSEL/PENABLE/PADDR/PWRITE/PWDATA/PSTRB/PPROT` are inputs; `PREADY/PRDATA/PSLVERR` outputs; `apb5` adds `PAUSER/PWUSER/PWAKEUP` in and `PRUSER/PBUSER` out). - The adapter instantiates `apb4_to_axi4` / `apb5_to_axi4` on an internal AXI4 face (`apbx_axi_*`) and feeds that to the same `axi4_slave_{wr,rd}` timing wrapper – `_mon` in the monitored variant – an AXI4 master port gets. From the wrapper onward the port is an AXI4-Lite-shaped single-beat requester: decode, the width converters and the wide-slave aligner, and the response mux are untouched. - The fabric ID is the master index alone (`id_width = 0`, bridge TASK-005 (was BRIDGE-016)).

Modules: `projects/components/utility-ip/converters/rtl/apb4_to_axi4.sv`, `apb5_to_axi4.sv`, `apb_cmdrsp_to_axi4.sv` – see the converters MAS.

13.16.4 Wishbone B4 at Either Boundary

When `protocol = "wb4"` is specified (bridge TASK-008, was BRIDGE-019): - On a **slave**, the adapter instantiates `axi4_to_wb4` through the `Axi4ToWb4Shim` component – the same channel wiring, bridge BUG-009 (was BRIDGE-011) not-full gate and `converter_*` response intercepts as the APB shim, and the same `axi4_master_*_mon` sandwich in the monitored variant. The external surface is the thirteen-signal B4 requester set from `bridge_pkg/wb4_signals.py`, the one table the bridge top, the adapter and the instance component all read. - On a **master**, the adapter's external surface is the B4 completer set (directions reversed) and `wb4_to_axi4` drives an internal `wbx_axi_*` AXI4 face into the same `axi4_slave_{wr,rd}` wrapper an AXI4 master gets. From there the port is an AXI4-Lite-shaped single-beat requester and takes the wide-slave aligner toward wider slaves.

Modules: `projects/components/utility-ip/converters/rtl/axi4_to_wb4.sv`, `wb4_to_axi4.sv` – see the converters MAS.

13.16.5 AXI3 Boundary Conversion

When `protocol = "axi3"` is specified, a same-width protocol converter sits at the boundary and the crossbar behind it stays AXI4:

- **Slave boundary:** the generated adapter instantiates `axi4_to_axi3` between the crossbar core and the external AXI3 slave port. Egress AXI4 bursts longer than 16 beats are chopped into sequential AXI3 bursts; `WID` is stamped per beat; `QoS`/`REGION` are dropped; lock is mapped as the converter documents it.
- **Master boundary:** the adapter exposes an AXI3 completer surface and instantiates `axi3_to_axi4` in front of the ordinary AXI4 timing wrapper. `WID` is stripped, `QoS`/`REGION` are driven to zero, and lock is mapped as the converter documents it.

The bridge does not add width conversion inside the protocol converter. When an AXI3 port's external width differs from a target slave width, the existing internal width-adaptation path (`dwidth_converter` or the wide-slave aligner) operates on the AXI4 side behind the converter.

Converter-sourced obligations (for full wording see converters MAS §3.15 *AXI4 to AXI3 Converter* and §3.16 *AXI3 to AXI4 Converter*):

1. An AXI3 master must not interleave W data across IDs. `axi3_to_axi4_wr` asserts `ASSERT_A3_WID_STABLE` if `WID` changes mid-burst or differs from the active AWID.
2. Egress bursts longer than 16 beats are chopped by address regeneration; `INCR` and `FIXED` are supported, `WRAP` longer than 16 beats is rejected.
3. `QoS` and `REGION` are absent on AXI3 pins.

4. The converters are same-width only; width adaptation is the bridge's internal concern.
5. Lock mapping is exactly as the converter documents:
   - `axi4_to_axi3`: `1'b0 → 2'b00`, `1'b1 → 2'b10`.
   - `axi3_to_axi4`: `2'b10 → 1'b1`, all other values → `1'b0`.

Validation fixtures: `bridge_1x2_axi3`, `bridge_2x2_axi3_masters`, `bridge_2x2_axi3_slaves`, `bridge_mix_e`.

Modules: `projects/components/utility-ip/converters/rtl/axi4_to_axi3.sv`, `axi4_to_axi3_rd.sv`, `axi4_to_axi3_wr.sv`, `axi3_to_axi4.sv`, `axi3_to_axi4_rd.sv`, `axi3_to_axi4_wr.sv`.

13.16.6 Master-Side AXI5-Lite Sideband

When `protocol = "axil5"` is specified on a **master** (bridge TASK-004, was BRIDGE-014), the bridge top exposes the AXI4-Lite set plus every AXI5-Lite sideband group (from `bridge_pkg/axil5_sideband.py`, the one table the slave side also reads, with the directions flipped for a requester). The enabled forwardable groups join the Lite surface and are wired into the adapter's AXI4 face (`exclusive` -> `awlock`/`arlock`, `user` -> `aw/w/ar user`); the rest is terminated at the top – requester-driven inputs consumed by a reduction into an `_unused_<master>_axil5_sb` wire, completer-driven outputs driven `'0`. Response-side USER (`buser`/`ruser`) is connected but reads 0, because the master adapters tie response USER (PRD).

13.16.7 Master-Side AXIL→Wider-Slave Alignment

When an AXI4-Lite master interfaces with a wider AXI4 slave (e.g., 32-bit AXIL master to 64-bit AXI4 slave) – and likewise an AXI5-Lite or APB master, which present the same single-beat stream: - Generator emits `axil_to_axi4_wide_align_rd.sv` (read) and `axil_to_axi4_wide_align_wr.sv` (write) at the **master adapter output** (before crossbar core) - These modules handle **width alignment** (not protocol conversion — that's done at the slave boundary if needed) - Preserves AXIL's single-beat constraint on the master side - Properly aligns partial-word reads/writes on the wide slave side

Modules: `projects/components/utility-ip/converters/rtl/axil_to_axi4_wide_align_{rd,wr}.sv`

Example: 32-bit AXIL master → 64-bit AXI4 slave - Master writes to addr 0x04 with data 0xAABBCCDD. The shim selects the lane from `addr[2]` (= 1), so the data lands on the UPPER half: the slave sees `wdata[63:32] = 0xAABBCCDD`, `WSTRB = 0xF0`, at row-aligned address `0x00` – the lane bits move out of the address and into the strobe. Worked through in Width Converters. - Shim is inserted after master adapter, before crossbar

13.17 2.7.17 Future Protocol Support

13.17.1 Planned Features

AXI4-Lite – NOT future work; BUILT. `axi4_to_axil4_{rd,wr}` shims are emitted when `protocol = "axil"`, and `bridge_1x2_rw_axil5` and the `mix_*` configurations ship AXIL slaves with tests in the FULL regression. It was listed here as planned while the same page describes the shims that implement it.

AHB (AMBA High-performance Bus): - More capable than APB - Pipeline support - Burst support - Suitable for moderate-bandwidth peripherals

Wishbone – NOT future work; BUILT (bridge TASK-008 (was BRIDGE-019), 2026-09-11). `wb4` is a legal `protocol` on master and slave ports alike; see "Wishbone B4 at Either Boundary" above. Not built: B4 standard (classic) mode from the TOML, and registered-feedback bursts (`CTI`/`BTE` are carried, not generated).

13.17.2 Under Consideration

- **Custom Protocol**: User-defined through configuration
- **Stream Interface**: AXI4-Stream for data streaming
- **PCIe TLP**: For PCIe endpoint integration
- **CHI**: ARM's Coherent Hub Interface

13.18 2.7.18 Best Practices

13.18.1 Design Recommendations

1. **Limit APB Burst Lengths**: Configure masters to use short bursts to APB slaves
2. **Watchdog Outside the Bridge**: the converter has no PREADY timeout, so a wedged APB slave stalls its path forever. If the system needs to survive that, the watchdog belongs upstream.
3. **Protocol Matching**: Use native AXI4 where possible, APB only when necessary
4. **Address Map Planning**: Group APB peripherals together for efficient decoding
5. **Width Matching**: Match APB data width to peripheral requirements

13.18.2 Performance Tips

```
Inefficient: 256-beat AXI burst → APB
  256 beats × 3 cyc/beat = 768 cycles

Better: Limit to 4-beat bursts → APB
  64 bursts of 4 beats each
  Still long, but more manageable

Best: Use AXI4-Lite slave for registers
  No burst, but native protocol
```

Related Sections: - Section 2.3: Crossbar Core (protocol integration point) - HAS ch04_interfaces/01_axi4_interface.md (port signals) - Chapter 4: Programming (configuring protocol conversion) - Appendix A: Generator Deep Dive (protocol converter generation)

---

14 2.8 Response Routing

Requests are easy to route — the address says where they go. Responses are harder: read data and write acknowledgments have to find their way back to whichever master asked, and by then the address that launched the transaction is ancient history. Response routing does that with Bridge IDs, demultiplexing logic, and optional CAM structures.

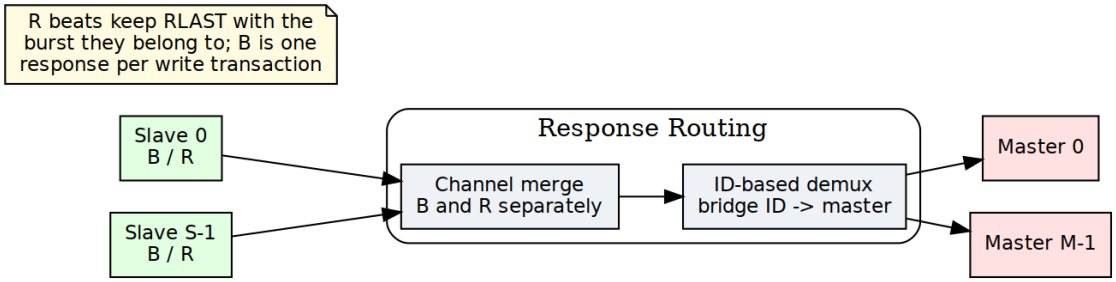
14.1 2.8.1 Purpose and Function

Response routing does five things:

1. **Response Direction**: Routes R and B channel responses to correct master
2. **Position-Based Routing**: pops a per-slave in-order FIFO of sideband master ids; the returned BID/RID is not used
3. **Multi-Slave Merging**: Combines responses from multiple slaves to single master
4. **Flow Control**: Manages backpressure from master on response channels
5. **Error Propagation**: Ensures error responses reach originating master

14.2 2.8.2 Block Diagram

14.2.1 Figure 2.8: Response Routing Architecture



Response Routing Architecture

Response routing architecture showing B and R channel merging from slaves and ID-based demultiplexing to masters.

14.3 2.8.3 Stable Response Routing via FIFO-Tracked Slave Select

14.3.1 Architecture: FIFO-Tracked Master Routing

For multi-slave masters, responses are routed based on a **stable slave-select value** captured at request time, not the current address decode:

```
// At AW/AR time: Capture slave index
assign aw_slave_select = address_decode(fub_axi_awaddr);
always_ff @(posedge aclk) begin
    if (fub_axi_awvalid && fub_axi_awready) begin
        aw_trk_fifo <= aw_slave_select;  // Capture now
    end
end

// At B time: Use captured slave from FIFO (not current decode)
assign b_slave_select = aw_trk_fifo_out;  // Stable value
logic [NUM_MASTERS-1:0] master_bvalid;
always_comb begin
    master_bvalid = '0;
    master_bvalid[extracted_bid] = slave_bvalid && (b_slave_select ==
current_slave);
end
```

Benefits: - B-responses route to correct master even after address reverts - Handles multi-slave masters spanning different widths - Prevents response loss due to stale address decodes

14.3.2 BID Extraction

> **Not built.** Nothing is extracted from the BID. IDs pass through the bridge untouched at equal width, and the return path is chosen by the POSITION of a per-slave in-order FIFO holding a sideband master id. The returned BID/RID is never consulted – which is exactly why a slave that reorders between IDs misroutes here (bridge BUG-008, was BRIDGE-010).

For ID-based routing, Bridge ID is extracted from response:

```
// Extract Bridge ID from response ID
logic [TOTAL_RID_WIDTH-1:0] slave_rid;     // From slave
logic [BID_WIDTH-1:0] extracted_bid;       // Master index
logic [RID_WIDTH-1:0] external_rid;        // To master

// Upper bits = Bridge ID, lower bits = original ID
assign extracted_bid = slave_rid[TOTAL_RID_WIDTH-1:RID_WIDTH];
assign external_rid = slave_rid[RID_WIDTH-1:0];

// Route response based on BID (combined with slave select tracking
```

```
above)
logic [NUM_MASTERS-1:0] master_rvalid;
always_comb begin
    master_rvalid = '0;  // Default: no master selected
    master_rvalid[extracted_bid] = slave_rvalid && (r_slave_select ==
current_slave);
end
```

14.3.3 CAM-Based Extraction

Built (bridge BUG-012 (was BRIDGE-015)/016, 2026-09-10). Every real AXI slave of a multi-master bridge, and any slave declared `enable_ooo = true`, gets `bridge_cam` for both channels and routes responses by ID; single-master bridges, shim slaves and the subtractive slave keep the in-order per-slave FIFO. Per-ID tracking is sound across masters because fabric IDs are `{master index, master id}`; the FIFO would require the slave to complete in request order across ALL IDs, which a compliant AXI slave need not do. Fixture `bridge_2x2_ooo` exercises reordering slaves with two masters sharing IDs; `test_bridge_cam.py` covers the CAM alone. The mode had been unbuildable since c64660f47. See ch04 `02_id_tracking.md`.

For complex configurations with OOO or ID reordering:

```
// CAM lookup for response routing
logic [TOTAL_RID_WIDTH-1:0] response_id;
logic [BID_WIDTH-1:0] master_index;
logic cam_hit;

// Search CAM for matching transaction
assign {cam_hit, master_index} = cam_lookup(response_id);

// Route response to found master
if (cam_hit) begin
    master_rvalid[master_index] = slave_rvalid;
end else begin
    // Error: No matching transaction (protocol violation)
    error_response();
end
```

14.4 2.8.4 Response Demultiplexing

14.4.1 Per-Master Response Paths

Each master has dedicated response channels from the demultiplexer:

```
Master 0 Response:
  - M0_RVALID, M0_RREADY, M0_RDATA, M0_RID, M0_RRESP, M0_RLAST
  - M0_BVALID, M0_BREADY, M0_BID, M0_BRESP

Source: Responses from any slave with BID=0
```

14.4.2 Demux Implementation

```
// Response demultiplexer (4 masters, 3 slaves)
// Combines responses from all slaves, routes to correct master

// Slave responses (multiple sources)
logic s0_rvalid, s1_rvalid, s2_rvalid;
logic [63:0] s0_rdata, s1_rdata, s2_rdata;
logic [5:0] s0_rid, s1_rid, s2_rid;  // Includes BID

// Master responses (multiple destinations)
logic m0_rvalid, m1_rvalid, m2_rvalid, m3_rvalid;
logic [63:0] m0_rdata, m1_rdata, m2_rdata, m3_rdata;
logic [3:0] m0_rid, m1_rid, m2_rid, m3_rid;  // BID stripped

// Demux logic per slave
for (genvar s = 0; s < 3; s++) begin
    logic [1:0] bid;
    assign bid = slave_rid[s][5:4];  // Extract BID

    always_comb begin
        case (bid)
            2'b00: begin
                if (slave_rvalid[s] && !m0_busy) begin
                    m0_rvalid = 1'b1;
                    m0_rdata = slave_rdata[s];
                    m0_rid = slave_rid[s][3:0];  // Strip BID
                end
            end
            2'b01: /* Route to M1 */
            2'b10: /* Route to M2 */
            2'b11: /* Route to M3 */
        endcase
    end
end
```

14.5 2.8.5 Multi-Slave Response Merging

14.5.1 Problem Statement

When multiple slaves can respond to the same master in the same cycle:

```
Scenario:
  Master 0 has outstanding transactions to Slave 0 and Slave 1
  Both slaves return R responses in same cycle

Problem: Master 0 can only accept one R response per cycle
Solution: Arbitrate between slave responses
```

14.5.2 Response Arbitration

> **Not built.** Responses are not arbitrated and there are no response FIFOs. The crossbar instantiates round-robin arbiters for AW and AR only; B and R follow the selection recorded at the address handshake. Searching the generated crossbar for a B or R arbiter, or for a response FIFO, returns nothing.

```
// Arbitrate between multiple slave responses for same master
logic [2:0] slave_has_response;  // Which slaves have responses for M0
logic [1:0] selected_slave;      // Which slave wins arbitration

// Detect responses destined for M0
for (genvar s = 0; s < 3; s++) begin
    assign slave_has_response[s] = slave_rvalid[s] &&
                                   (extract_bid(slave_rid[s]) ==
2'b00);
end

// Round-robin arbiter for fairness
always_ff @(posedge clk) begin
    if (!rst_n) begin
        selected_slave <= 0;
    end else if (m0_rready && (|slave_has_response)) begin
        // Rotate selection for fairness
        if (slave_has_response[selected_slave])
            ; // Keep current
        else
            selected_slave <= find_next_requesting_slave();
    end
end

// MUX selected slave's response to master
assign m0_rvalid = slave_has_response[selected_slave];
```

```
assign m0_rdata = slave_rdata[selected_slave];
assign m0_rid = strip_bid(slave_rid[selected_slave]);
```

14.5.3 Response Buffering

Optional: Add FIFOs to buffer pending responses:

```
Purpose:
  - Prevent response loss during arbitration conflicts
  - Allow slaves to return responses even if master busy
  - Smooth out bursty response patterns

Configuration:
  per_master_response_fifo_depth = 4-16 entries

Trade-off:
  + No response loss
  + Better throughput
  - Additional resources (~200 LEs per FIFO)
  - +1-2 cycle latency
```

14.6 2.8.6 Independent Write-Tracker FIFO (W-Channel Deadlock Fix)

Prior to commit `d24bd617`, the AW and W ready signals shared a single multiplexer at the write-address arbiter output. Under interleaved master traffic patterns, this could cause deadlock: a master's W channel could back up waiting for a non-granted write-address slot, preventing the arbiter from servicing other masters' AW transactions.

The corrected design (current) uses an **independent W-tracker FIFO** alongside the AW tracker:

```
// AW tracker FIFO (per master)
always_ff @(posedge aclk) begin
    if (master_awvalid && master_awready) begin
        aw_trk_fifo <= address_decode(master_awaddr);  // Capture
slave
    end
end

// Independent W tracker and ready MUX
logic aw_ready_mux, w_ready_mux;  // Separate paths (prior: shared)

assign master_awready = aw_ready_mux;   // AW gets its own ready
assign master_wready = w_ready_mux;      // W gets independent ready

// W-tracker: Gate W valid against tracked slave select
always_comb begin
    w_ready_mux = (aw_trk_fifo != 0) && slave_wready;  // W
independent
end
```

Result: AW and W channels can assert independently without blocking each other, preventing deadlock during interleaved multi-master write bursts.

14.7 2.8.7 Backpressure Management

14.7.1 Ready Signal Routing

The master's RREADY/BREADY has to find its way back to the correct slave:

```
// Route master ready back to slaves
logic m0_rready;  // From master
logic [2:0] slave_rready;  // To slaves

// Only selected slave sees master's ready
for (genvar s = 0; s < 3; s++) begin
    assign slave_rready[s] = (selected_slave == s) ? m0_rready : 1'b0;
end

// Non-selected slaves see ready = 0 (backpressure)
```

14.7.2 Stall Conditions

Response path can stall when:

```
1. Master not ready (M_RREADY = 0)
   → Selected slave stalls (S_RREADY = 0)
   → Other slaves buffer or stall

2. Arbitration conflict (multiple slaves ready)
   → Non-selected slaves stalled
   → Selected slave proceeds

3. bridge_id tracking FIFO full -- the address handshake is gated on
it (bridge BUG-009, was BRIDGE-011); there is no response DATA FIFO
   → Slave stalled until space available
```

14.8 2.8.8 Response Path Latency

14.8.1 End-to-End R Channel

```
Slave R response → Bridge → Master R channel

Components:
1. Slave response valid
2. FIFO head read (0 cycles, combinatorial) -- no BID extraction
3. Demux routing (0-1 cycles)
4. No response arbitration: B and R follow the address-phase selection
5. Optional FIFO (0-1 cycles)
6. Master adapter (1 cycle, skid buffer)

Total: 2-4 cycles typical
```

14.8.2 End-to-End B Channel

```
Similar to R channel:
  Slave B → Extraction → Demux → Arbitration → Master B
  Total: 2-4 cycles
```

14.9 2.8.9 Error Response Routing

14.9.1 Slave Error Responses

When slave returns error:

```
RRESP = 2'b10 (SLVERR) or 2'b11 (DECERR)
BRESP = 2'b10 (SLVERR) or 2'b11 (DECERR)

Routing:
  - Extract BID normally
  - Route to originating master
  - Preserve error code
  - Master sees error response
```

14.9.2 Out-of-Range Error Responses

When router generates error for OOR address:

```
Process:
1. Router captures OOR request
2. Generates internal error response
   - Uses cached request ID (with BID)
   - Sets RRESP/BRESP = DECERR
3. Injects into response path
4. Routes to master using BID
```

14.10 2.8.10 Resource Utilization

14.10.1 Response Router Resources

4 masters, 3 slaves (64-bit data):

```
Logic Elements:  ~1500-2000 LEs
Registers:       ~400-600 regs
Block RAM:       0 (unless FIFOs enabled)

Breakdown:
- BID extraction (3 slaves):     ~150 LEs
- Demux logic (4 masters):        ~600 LEs, ~150 regs
- Response arbitration:           ~300 LEs, ~100 regs
- Backpressure routing:           ~200 LEs, ~50 regs
- Control FSMs:                   ~250 LEs, ~100 regs

Optional FIFOs (4 × 8 entries):  +800 LEs, +2KB BRAM
```

14.10.2 Scaling

```
Resource usage scales with:
- Number of masters: Linear (one demux path per master)
- Number of slaves: Linear (one extraction per slave)
- Data width: Linear (wider data = wider demux)
- FIFO depth: Linear (deeper = more BRAM)
```

14.11 2.8.11 Configuration Parameters

14.11.1 Response Routing Configuration (TOML)

```toml
[bridge]
num_masters = 4
num_slaves = 3

[bridge.response_routing]
enable_response_fifos = false       # Buffer responses per master
fifo_depth = 8                      # Entries per FIFO
response_arbiter_type = "round_robin"  # "round_robin",
"fixed_priority"
registered_demux = false            # Register demux output (+1 cycle)

# Per-master response credits (optional)
[[bridge.masters]]
name = "cpu"
max_response_credits = 16           # Outstanding responses allowed
```

14.12 2.8.12 Debug and Observability

14.12.1 Recommended Debug Signals

```
BID Extraction:
- Slave RID/BID (from each slave)
- Extracted BID (master index)
- External RID/BID (to master, BID stripped)

Response Demux:
- Demux select signals (which master per slave)
- Response valid per master
- Response valid per slave

Arbitration:
- Conflict detection (multiple responses for same master)
- Arbiter grant (which slave wins)
- Stall counters

FIFOs (if enabled):
- FIFO occupancy per master
- FIFO full/empty flags
- FIFO overflow errors
```

14.12.2 Performance Counters

```
- Responses routed per master
- Arbitration conflicts (cycles with multiple responses to same
master)
- Response stall cycles (master not ready)
- Average response latency per master
- FIFO overflow counts
- bridge_id FIFO push/pop counts
```

14.13 2.8.13 Common Issues and Debug

Symptom: Response not reaching master
Check: - BID extraction (correct bit slicing?) - Master index (BID → master mapping) - Demux select logic - Backpressure (is master READY?)

Symptom: Response goes to wrong master
Check: - BID values (verify each master has unique BID) - BID width calculation (clog2 correct?) - bridge_id FIFO contents (wr_fifo/rd_fifo) - ID corruption in transit

Symptom: Response ordering incorrect
Check: - FIFO head (routing is by position, so order IS the mechanism) - Arbitration fairness (round-robin working?) - Multiple outstanding transactions per master

Symptom: Response loss
Check: - FIFO overflows (enable FIFOs if needed) - Dropped responses during arbitration conflicts - Protocol violations (slave not following AXI rules)

14.14 2.8.14 Verification Considerations

14.14.1 Test Scenarios

1. **Single Master, Single Slave**:

```
- Basic routing test
- Verify BID extraction and stripping
- Check response reaches correct master
```

2. **Multiple Masters, Single Slave**:

```
- Interleaved responses
- Verify each response routes to correct master
- Check BID uniqueness prevents collisions
```

3. **Single Master, Multiple Slaves**:

```
- Simultaneous responses from multiple slaves
- Verify arbitration fairness
- Check no response loss
```

4. **All Masters, All Slaves**:

```
- Maximum stress test
- All masters issuing to all slaves
- Responses returning in various orders
- Verify correct routing under heavy load
```

5. **OOO Responses**:

```
- Issue transactions: ID=1, ID=2, ID=3
- Return responses: ID=3, ID=1, ID=2
- Verify the FIFO head routes each
- Check response order at master
```

6. **Backpressure Test**:

```
- Master asserts RREADY=0
- Verify slave stalls (RREADY=0 propagated)
- Master asserts RREADY=1
- Verify responses flow
```

14.15 2.8.15 Performance Optimization

14.15.1 Techniques

1. Registered Demux:

```
Trade-off: +1 cycle latency for timing closure
Best for: High-frequency designs, many masters
```

2. Response FIFOs:

```
Trade-off: +2KB BRAM, +1-2 cycle latency
Benefit: Prevent response loss, smooth conflicts
Best for: High-throughput systems
```

3. Distributed Arbitration:

```
Implementation: Per-master arbiters instead of global
Benefit: Reduced logic depth, better timing
Best for: >8 masters
```

4. Priority Response Routing:

```
Implementation: High-priority masters get preference in conflicts
Benefit: Guaranteed low latency for critical masters
Best for: Real-time systems
```

14.16 2.8.16 Advanced Features

14.16.1 Response Reordering (Future)

Built (bridge BUG-012 (was BRIDGE-015)/016, 2026-09-10). Every real AXI slave of a multi-master bridge, and any slave declared `enable_ooo = true`, gets `bridge_cam` for both channels and routes responses by ID; single-master bridges, shim slaves and the subtractive slave keep the in-order per-slave FIFO. Per-ID tracking is sound across masters because fabric IDs are `{master index, master id}`; the FIFO would require the slave to complete in request order across ALL IDs, which a compliant AXI slave need not do. Fixture `bridge_2x2_ooo` exercises reordering slaves with two masters sharing IDs; `test_bridge_cam.py` covers the CAM alone. The mode had been unbuildable since c64660f47. See ch04 `02_id_tracking.md`.

Allow bridge to reorder responses for efficiency:

```
Scenario:
  Master issues: Req A (slow slave), Req B (fast slave)
  Fast slave responds first
  Bridge can deliver B before A (if IDs different)

Benefit: Reduced latency for fast responses
Requirement: CAM to track outstanding in-order constraints
```

14.16.2 Response Coalescing (Future)

Combine multiple responses into fewer beats:

```
Scenario:
  Multiple single-beat reads to narrow slave
  Bridge coaleses into fewer wide beats

Benefit: Reduced overhead
Complexity: High (requires buffering and alignment)
```

14.16.3 Response QoS (Future)

Prioritize responses based on master QoS:

```
Feature: High-QoS masters get response priority
Implementation: Weighted arbitration in response path
Use case: RT masters need bounded response latency
```

14.17 2.8.17 Timing Considerations

14.17.1 Critical Paths

Common critical paths in response routing:

```
1. BID extraction → Demux select → Master RDATA
   Depth: ~8-12 logic levels

2. Slave RVALID → Arbitration → Master RVALID
   Depth: ~6-10 logic levels

3. Master RREADY → Backpressure → Slave RREADY
   Depth: ~5-8 logic levels
```

14.17.2 Optimization Strategies

```
- Register demux outputs (+1 cycle, breaks path)
- Pipeline arbitration (+1-2 cycles, multi-stage)
- Complex ID scenarios are unsupported: routing is positional, not ID-
keyed
- Limit response buffering depth (less MUX levels)
```

14.18 2.8.18 Future Enhancements

14.18.1 Planned Features

- **Dynamic Response Buffering**: Adjust FIFO depth based on utilization
- **Response Ordering Hints**: Masters specify ordering requirements
- **Multi-Cycle Arbitration**: Pipelined for >16 masters
- **Response Compression**: Pack narrow responses into wide beats

14.18.2 Under Consideration

- **Response Speculation**: Predictive routing before full ID available
- **Virtual Response Channels**: Separate paths for different QoS classes
- **Response Mirroring**: Duplicate responses for redundancy
- **Cross-Clock Response**: Async response paths with CDC

Related Sections: - Section 2.3: Crossbar Core (response path architecture) - Section 2.5: ID Management (BID extraction details) - Section 2.4: Arbitration (response arbitration algorithms) - HAS ch04_interfaces/01_axi4_interface.md (response channel signals)

---

15 2.9 Error Handling

What error handling this bridge actually has. Measured against `rtl/generated/`, not designed on paper. The complete inventory:

Mechanism	Where	What it does
`DECERR` on unmapped access	`axi4_subtractive_slave`	answers, so an unmapped address cannot hang the master
`o_hit_irq` / `o_hit_count`	same	sticky flag and saturating count, both cleared by `i_hit_clear`
`o_hit_addr`	same	FIRST fault address; deliberately RETAINED across a clear so the evidence survives
monbus error packet	`*_mon` builds only	reports the fault to a monitor bus
response-ordering check	simulation only	`$error` on a BID/RID that does not match the FIFO head (bridge BUG-008, was BRIDGE-010)

Everything else this chapter describes – a protocol checker, a per-transaction watchdog, an error status register, an error history buffer, error interrupt logic, and a `[bridge.error_handling]` TOML table – is NOT BUILT. Searches for `protocol_check`, `error_status`, `error_history` and `error_irq` across every generated file return nothing, and the config loader does not know the `error_handling` table name, so such a section is silently ignored. Those parts are retained as design intent and each is marked.

Error handling is everything the bridge does when something goes wrong: detecting the problem, answering with the right AXI error response, logging what happened, and keeping the rest of the system running. The covered failure modes are protocol violations, out-of-range addresses, timeout conditions, and configuration errors.

15.1 2.9.1 Purpose and Function

Error handling does five things:

1. **Error Detection**: Identifies violations and abnormal conditions
2. **Error Response Generation**: Creates appropriate AXI error responses
3. **Error Reporting**: Logs and signals errors for debug
4. **Graceful Degradation**: Maintains system stability during errors
5. **Recovery Support**: Enables system recovery after errors

15.2 2.9.2 Error Categories

15.2.1 Transaction Errors

Out-of-Range (OOR) Addresses:

```
Master requests address not mapped to any slave
Response: DECERR (Decode Error)
Action: Bridge generates error response internally
```

Protocol Violations:

```
Master violates AXI4 protocol rules
Examples:
  - VALID deasserted while READY=0
  - Incorrect burst parameters
  - ID width mismatches
Response: Configurable (error log, SLVERR, or ignore)
```

Slave Errors:

```
Slave returns SLVERR or DECERR response
Action: Bridge forwards error to master unchanged
```

15.2.2 System Errors

Timeout Conditions: there are none. The bridge has NO transaction timeout – not on the AXI path and not in the APB shim, where 2.7 states it three times (“There is no PREADY timeout anywhere in the path”). A slave that never responds stalls that path indefinitely; nothing in the fabric detects or breaks the stall.

This block previously described a configurable timeout returning DECERR. No such mechanism was ever built, and believing in it is worse than knowing it is absent – an integrator counting on the fabric to time out will not add the watchdog that actually protects the system.

```
(retained for contrast -- NOT implemented)
Slave fails to respond within configured timeout
Response: DECERR after timeout expires
Action: Force transaction completion
```

CAM Overflow:

> **Not built.** No generated bridge contains a CAM – `bridge_cam.sv` is instantiated in zero of them. Responses are routed by the POSITION of an in-order per-slave FIFO holding a sideband master id, so out-of-order completion is not supported and the section below describes an option that was never implemented. See ch04 `02_id_tracking.md`.

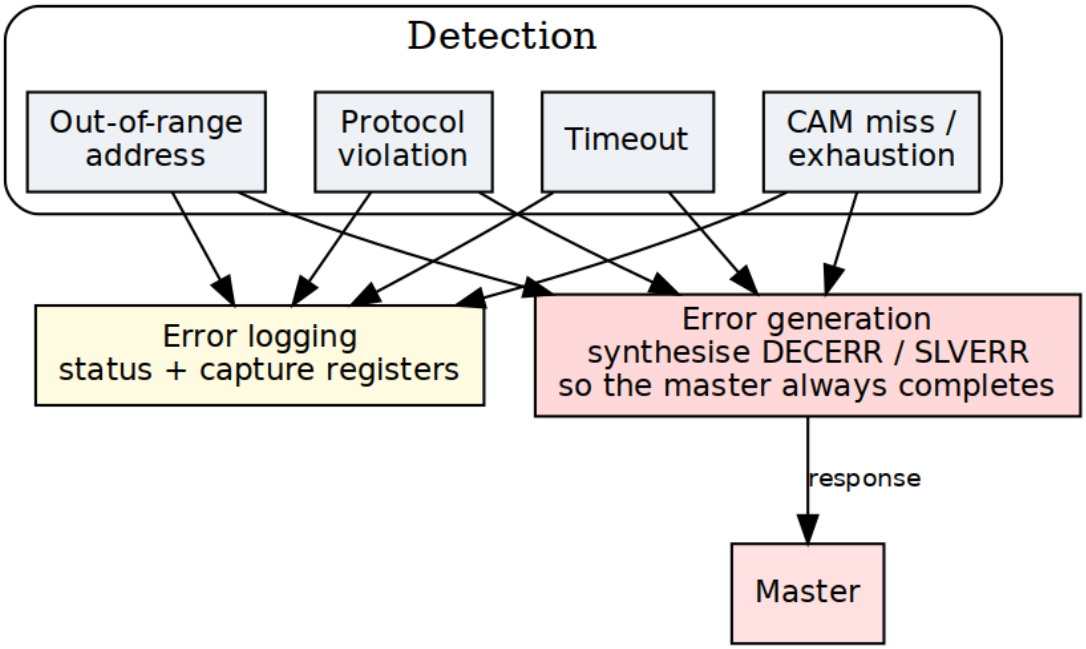
```
Too many outstanding transactions (CAM full)
Response: Backpressure master (ARREADY/AWREADY=0)
Action: Wait for responses to free entries
```

Configuration Errors:

```
Invalid bridge configuration detected at generation time
Examples:
  - Overlapping slave address ranges
  - Invalid data width combinations
  - Illegal parameter values
Response: Generation error, fail early
```

15.3 2.9.3 Block Diagram

15.3.1 Figure 2.9: Error Handling Architecture



Error Handling System

Error handling system showing detection (OOR, protocol, timeout, bridge_id FIFO), generation, and logging subsystems.

15.4 2.9.4 Out-of-Range Address Handling

15.4.1 Detection

Router identifies OOR when no slave address range matches:

```
// Address decode with OOR detection
logic [NUM_SLAVES-1:0] slave_match;
logic oor_detected;

for (genvar i = 0; i < NUM_SLAVES; i++) begin
    assign slave_match[i] = (addr >= SLAVE_BASE[i]) &&
                            (addr <= SLAVE_END[i]);
end

assign oor_detected = ~(|slave_match) && ~default_slave_en;
```

15.4.2 Error Response Generation

Read OOR Response:

```
// Generate R response for OOR read
always_ff @(posedge clk) begin
    if (ar_oor_detected && arvalid && arready) begin
        // Capture transaction details
        oor_arid <= arid;
        oor_arlen <= arlen;
        oor_burst_count <= 0;
        oor_generating <= 1'b1;
    end else if (oor_generating) begin
        // Generate R beats
        rvalid <= 1'b1;
        rdata <= OOR_DATA_PATTERN;  // Configurable pattern
        rid <= oor_arid;
        rresp <= 2'b11;  // DECERR
        rlast <= (oor_burst_count == oor_arlen);

        if (rready) begin
            oor_burst_count <= oor_burst_count + 1;
            if (rlast) oor_generating <= 1'b0;
        end
    end
end
```

Write OOR Response:

```
// Generate B response for OOR write
always_ff @(posedge clk) begin
    if (aw_oor_detected && awvalid && awready) begin
```

```
            // Capture write ID
            oor_awid <= awid;
            oor_w_count <= 0;
            oor_sinking_w <= 1'b1;
        end else if (oor_sinking_w) begin
            // Sink W data (accept and discard)
            wready <= 1'b1;

            if (wvalid && wlast) begin
                oor_sinking_w <= 1'b0;
                oor_gen_bresp <= 1'b1;
            end
        end else if (oor_gen_bresp) begin
            // Generate B response
            bvalid <= 1'b1;
            bid <= oor_awid;
            bresp <= 2'b11;  // DECERR

            if (bready) begin
                oor_gen_bresp <= 1'b0;
            end
        end
    end
```

15.4.3 Configurable Data Patterns

Not built. The read fill value is a module PARAMETER of `axi4_subtractive_slave` (`READ_FILL`, default `32'hDEAD_BEEF`). The generator does not override it – `bridge_2x2_rw.sv` passes only the width, `UNIT_ID` and `AGENT_ID` parameters – and no TOML key sets it. There is no `[bridge.error_handling]` table; the loader does not know that name, so a config that contains one is silently ignored. To change the pattern today, override the parameter at instantiation.

```
[bridge.error_handling]
oor_read_data_pattern = 0xDEADCAFE  # Pattern for OOR reads
oor_response_latency = 2            # Cycles to generate response
```

15.5 2.9.5 Protocol Violation Handling

15.5.1 Common Violations

VALID Dropped While READY=0:

```
AXI4 Rule: Once VALID asserted, must remain until READY
Violation: Master deasserts VALID before handshake
Detection: Track VALID history per channel
Action: Log error, optionally assert error signal
```

Incorrect Burst Parameters:

```
Violations:
  - SIZE > DATA_WIDTH
  - LEN > 255
  - Address not aligned to SIZE
  - Burst crosses 4KB boundary
Detection: Parameter checking logic
Action: SLVERR response or reject transaction
```

ID Width Mismatch:

```
Violation: Master uses ID wider than configured
Detection: Check ID values against expected width
Action: Truncate or error (configurable)
```

15.5.2 Protocol Checker Implementation

> **Not built.** No generated bridge contains a protocol checker: a search for `protocol_check` / `proto_check` / `protocol_viol` across every file in `rtl/generated/` returns nothing. The section below describes hardware that was never implemented. Protocol violations by an attached master or slave are not detected – with one exception, the bridge BUG-008 (was BRIDGE-010) response-ordering check, which is simulation-only and cannot fire in silicon.

```
// AXI4 protocol checker (simplified)
module axi_protocol_checker #(
    parameter ID_WIDTH = 4,
    parameter ADDR_WIDTH = 32,
    parameter DATA_WIDTH = 64
) (
    input logic clk, rst_n,
    // AXI signals
    input logic arvalid, arready,
    input logic [ADDR_WIDTH-1:0] araddr,
    input logic [7:0] arlen,
```

```
    input logic [2:0] arsize,
    // Error outputs
    output logic protocol_error,
    output logic [7:0] error_code
);

    // Check: VALID held until READY
    logic arvalid_prev;
    always_ff @(posedge clk) begin
        arvalid_prev <= arvalid;
        if (arvalid_prev && !arvalid && !aready) begin
            protocol_error <= 1'b1;
            error_code <= 8'h01;  // VALID_DROPPED
        end
    end

    // Check: SIZE <= DATA_WIDTH
    always_comb begin
        if (arvalid && (arsize > $clog2(DATA_WIDTH/8))) begin
            protocol_error = 1'b1;
            error_code = 8'h02;  // INVALID_SIZE
        end
    end

    // Check: Address alignment
    always_comb begin
        if (arvalid && (araddr[arsize-1:0] != 0)) begin
            protocol_error = 1'b1;
            error_code = 8'h03;  // UNALIGNED_ADDR
        end
    end

    // Additional checks...
endmodule
```

15.6 2.9.6 Timeout Detection

15.6.1 Per-Transaction Watchdog

Not built. As described, The bridge has NO per-transaction watchdog: it never terminates a stuck transaction and never manufactures a response for one. Non-monitor builds contain zero timeout logic – `bridge_2x2_rw` and `bridge_mix_a` have no `timeout` signal at all.

What DOES exist, and only in `*_mon` builds, is observability: `cfg_wr_timeout_enable` / `cfg_wr_timeout_cycles` / `cfg_wr_axi_timeout_mask` (and the `rd` equivalents) are inputs to the AXI MONITOR, which emits a timeout EVENT PACKET on monbus when a transaction outlives the programmed cycle count. It reports; it does not intervene. A hung slave still hangs the bridge – the monitor just tells you it happened.

```
// Timeout detector for outstanding transactions
typedef struct packed {
    logic valid;
    logic [TOTAL_ID_WIDTH-1:0] id;
    logic [15:0] timestamp;
    logic is_read;  // 1=read, 0=write
} timeout_entry_t;

timeout_entry_t watchdog [0:15];  // Track up to 16 transactions
logic [15:0] global_timer;

always_ff @(posedge clk) begin
    if (!rst_n) begin
        global_timer <= 0;
    end else begin
        global_timer <= global_timer + 1;

        // Check for timeouts
        for (int i = 0; i < 16; i++) begin
            if (watchdog[i].valid) begin
                logic [15:0] elapsed;
                elapsed = global_timer - watchdog[i].timestamp;

                if (elapsed > TIMEOUT_THRESHOLD) begin
                    // Timeout detected
                    timeout_error <= 1'b1;
                    timeout_id <= watchdog[i].id;
                    timeout_type <= watchdog[i].is_read ? "READ" :
"WRITE";
```

```
                    // Force completion with DECERR
                    force_error_response(watchdog[i]);
                    watchdog[i].valid <= 1'b0;
                end
            end
        end
    end
end
```

15.6.2 Timeout Configuration

> **Not built.** No watchdog exists to configure. In `*_mon` builds the `cfg_*_timeout_*` inputs configure the AXI MONITOR's reporting threshold, not any bridge behaviour.

There is none. `[bridge.error_handling]` is not a table the loader knows, and `enable_timeout` / `timeout_cycles` / `timeout_action` / `per_slave_timeout` are not keys.

What the loader actually does with a key it does not know – measured by feeding it a config carrying `cam_depth`, `pipeline_depth` and `internal_data_width`, not inferred from reading it. Three different behaviours, which is why other pages disagree:

Where	Behaviour
`[bridge.mon_group]`	HARD ERROR – `ValueError` naming the offending keys and listing the valid ones
`[bridge.defaults]`	WARNS: "`[bridge.defaults]` is not implemented – section ignored"
anywhere else	SILENTLY ACCEPTED and ignored; nothing warns

So a TOML written from the block this section used to contain loads fine and does nothing. An earlier revision of this paragraph claimed `config_validator` rejects unknown keys and the config "does not load at all"; that is true only of `mon_group`.

A hung slave stalls its path indefinitely. If the system needs to survive that, the watchdog belongs outside the bridge.

15.7 2.9.7 Error Logging and Reporting

15.7.1 Error Status Register

Not built. No generated file contains an error status register. The nearest real thing is the subtractive slave's `o_hit_irq` / `o_hit_addr` / `o_hit_count`, which are module PORTS, not a memory-mapped register.

```
Error Status Register (Read/Clear):
```

Bits	Description
0	OOR Read Error
1	OOR Write Error
2	Protocol Violation
3	Timeout Error
4	bridge_id FIFO overflow (gated off)
5	Slave Error (SLVERR received)
6	Decode Error (DECERR received)
7	ID Match Error
15:8	Reserved
31:16	Error Count (saturating)

15.7.2 Error History Buffer

Not built. There is no history buffer. Only the FIRST fault address is retained (`o_hit_addr`), plus a saturating count.

```
// Circular buffer for error history
typedef struct packed {
    logic [7:0] error_type;
    logic [31:0] address;
    logic [TOTAL_ID_WIDTH-1:0] id;
    logic [31:0] timestamp;
    logic [7:0] master_index;
    logic [7:0] slave_index;
} error_entry_t;

error_entry_t error_log [0:15];  // 16-entry history
logic [3:0] error_wr_ptr;

always_ff @(posedge clk) begin
    if (error_detected) begin
        error_log[error_wr_ptr] <= {
            error_type,
            error_addr,
            error_id,
```

```
            global_timer,
            error_master,
            error_slave
        };
        error_wr_ptr <= error_wr_ptr + 1;
    end
end
```

15.7.3 Error Interrupts

Not built as a block. The only interrupt is `o_hit_irq` from the subtractive slave: one sticky bit, cleared by `i_hit_clear`. There is no interrupt controller, mask register or priority logic.

```
Optional interrupt generation:

Interrupt Enable Register:
  - OOR_INT_EN
  - TIMEOUT_INT_EN
  - PROTOCOL_INT_EN
  - etc.

Interrupt when enabled error occurs
Clear on status register read or explicit clear
```

15.8 2.9.8 Resource Utilization

15.8.1 Error Handling Resources

Not built. These figures price a protocol checker, watchdog and error registers that do not exist. The real cost is the subtractive slave plus a few status flops.

```
Logic Elements:  ~800-1200 LEs
Registers:       ~400-600 regs
Block RAM:       0-2 KB (if error logging enabled)

Breakdown:
- OOR detection/response:     ~300 LEs, ~100 regs
- Protocol checkers:           ~250 LEs, ~50 regs
- Timeout watchers:            ~200 LEs, ~150 regs
- Error logging:               ~150 LEs, ~100 regs
- Status registers:            ~100 LEs, ~50 regs

Optional error log (16 entries): +2KB BRAM
```

15.9 2.9.9 Configuration Parameters

15.9.1 Error Handling Configuration (TOML)

Not built. The loader does not know a `[bridge.error_handling]` table. A config containing one is silently ignored – nothing warns. The read fill pattern is the `READ_FILL` parameter of `axi4_subtractive_slave`, which the generator never overrides.

```toml
[bridge.error_handling]
# OOR handling
# NOT REAL KEYS. Out-of-range handling is fixed, not configured: the
# subtractive slave always answers DECERR with READ_FILL (0xDEADBEEF),
there
# is no timeout anywhere in the bridge, and there is no protocol
checker.
# Kept visible rather than deleted because several pages once
described these
# as configuration, and a reader who saw them elsewhere should find
out here
# that they do not exist.
#
#   enable_oor_errors / oor_response_type / oor_read_pattern
#   enable_timeout / timeout_cycles / timeout_response
#   enable_protocol_checker
protocol_checker_mode = "log"     # "log", "error", "ignore"

# Error logging
enable_error_log = true
error_log_depth = 16              # Entries in circular buffer

# Error reporting
enable_error_interrupts = false
error_status_register_addr = 0xFFFF_FFF0  # Optional mapped register
```

15.10 2.9.10 Debug and Observability

15.10.1 Recommended Debug Signals

```
Error Detection:
- OOR flags (per master)
- Protocol violation flags
- Timeout counters
- bridge_id FIFO occupancy

Error Response:
- Error response generation FSM states
- Active error responses
- Error data patterns

Error Logging:
- Error count
- Last error type
- Last error address
- Error log write pointer
```

15.11 2.9.11 Common Issues and Debug

Symptom: Unexpected DECERR responses
Check: - Address map configuration (gaps?) - Slave address ranges (overlaps?) - Default slave configuration - Timeout thresholds (too short?)

Symptom: System hangs on certain addresses
Check: - Timeout detection enabled? - Slave responsiveness - Protocol violations upstream - bridge_id FIFO overflow (prevented by the not-full gate, bridge BUG-009 (was BRIDGE-011))

Symptom: Error log filling quickly
Check: - What errors are occurring? - Master behavior (bad addresses?) - Slave configuration - Timeout thresholds

15.12 2.9.12 Verification Considerations

15.12.1 Test Scenarios

1. **OOR Address Tests**:

```
- Read from unmapped address
- Write to unmapped address
- Burst to partially mapped range
- Verify DECERR response
```

2. **Timeout Tests**:

```
- Slave never responds
- Verify timeout after N cycles
- Check error logged
- Verify forced completion
```

3. **Protocol Violation Tests**:

```
- Drop VALID before READY
- Invalid burst parameters
- Misaligned addresses
- Verify detection and response
```

4. **Error Recovery Tests**:

```
- Error occurs
- System continues operating
- Subsequent transactions succeed
- Error status readable
```

5. **Error Logging Tests**:

```
- Generate multiple errors
- Verify all logged
- Check circular buffer wrap
- Verify timestamps
```

15.13 2.9.13 Recovery Mechanisms

15.13.1 Automatic Recovery

```
OOR Errors:
  - Generate error response
  - Continue normal operation
  - No intervention needed

Timeout Errors:
  - Force transaction completion
  - Pop the bridge_id FIFO
  - Allow new transactions

Protocol Violations:
  - Log error
  - Complete/reject transaction
  - Continue with next transaction
```

15.13.2 Manual Recovery

```
bridge_id FIFO stuck:
  - Software reset via control register
  - Reset the bridge_id FIFO pointers
  - Restart affected masters

Persistent Errors:
  - Read error log
  - Identify root cause
  - Reconfigure or reset subsystem
```

15.14 2.9.14 Safety and Reliability

15.14.1 Error Containment

```
Goal: Prevent error propagation

Mechanisms:
- Isolate error to originating master
- Don't corrupt other masters' transactions
- Maintain crossbar integrity
- Log for post-mortem analysis
```

15.14.2 Fail-Safe Behavior

```
On Critical Error:
1. Generate safe error response (DECERR)
2. Log error details
3. Assert error signal (optional)
4. Continue operation if possible
5. Halt only if configured (safety-critical mode)
```

15.14.3 Error Injection (Debug/Test)

> **Not built.** The loader knows no `bridge.debug` table and no `error_injection` – searching `config_loader.py` for either returns nothing. A config containing this section is accepted and silently ignored, which is worse than an error: the build looks configured and injects nothing.

```toml
[bridge.debug]
enable_error_injection = true

[[bridge.debug.error_injection]]
type = "oor"
trigger_address = 0xBAD_ADDR
action = "decerr"

[[bridge.debug.error_injection]]
type = "timeout"
trigger_after_n_cycles = 100
target_slave = 2
```

15.15 2.9.15 Future Enhancements

15.15.1 Planned Features

- **Error Recovery FSM**: Automatic recovery from certain error conditions
- **Error Rate Limiting**: Prevent error storm from misbehaving master
- **Detailed Error Telemetry**: Capture full transaction context on error
- **Error Prediction**: Detect patterns indicating impending failures

15.15.2 Under Consideration

- **ECC Protection**: Error correction for internal state
- **Redundant Checking**: Duplicate checkers for safety-critical
- **Watchdog Timers**: System-level health monitoring
- **Error Severity Levels**: Classify errors by impact

Related Sections: - Section 2.2: Slave Router (OOR detection) - Section 2.5: ID Management (bridge_id FIFO depth) - Section 2.8: Response Routing (error response paths) - Chapter 5: Verification (error testing strategies)

---

16 AMBA5 Boundary and Native Sideband

How AXI5/APB5 ports ride the AXI4 fabric (bridge TASK-002 (was BRIDGE-002) phases A5-1 through A5-3). One spec table drives every generator: `bin/bridge_pkg/sideband.py::SIDEBAND_FIELDS` maps each feature to its per-channel struct fields, widths, and wrapper port bases — package, adapter, crossbar, and slave-adapter emission all iterate it in the same order, so struct layout and wiring cannot drift apart.

16.1 Boundary Wrappers

- AXI5 **master** port → `axi5_slave_{wr,rd}[_mon]` at the master adapter (the bridge is a slave to the external master).
- AXI5 **slave** port → `axi5_master_{wr,rd}[_mon]` at the slave adapter.
- Both families carry every feature signal through their skid path, gated by `ENABLE_<FEATURE>` parameters; the generator binds every pin (enabled features connect through, the rest tie/open).

16.2 Native Sideband Through the Structs (A5-2)

The per-bridge `_pkg` channel structs gain feature fields as the **union** of features on any AXI5 port. Pure-AXI4 bridges emit no fields, so their RTL stays byte-identical — the zero-drift invariant.

- **Master adapter:** packs its own enabled fields from the wrapper's `fub_axi_*` sideband on the **direct (width-matched) arm only**; converter arms pack `'0` — per-beat and per-transaction sideband cannot traverse the dwidth-converter IP. Because non-qualifying sources are guaranteed `'0`, the crossbar forwards request fields unconditionally.
- **Crossbar:** explodes struct fields into discrete `<slave>_axi_<sig>` nets for feature-enabled AXI5 slaves; response fields (`b.trace`, `r.trace`, `r.poison`) mux from qualifying slaves and default `'0` (they must be driven for every master since union fields exist in every struct).
- **Slave adapter:** rides `xbar_<slave>_axi_<sig>` nets into the wrapper via the component's `native_sideband` flag, which stops terminating enabled features' fub ports (fall-through to the connector prefix).

The struct field for AWUNIQUE/ARUNIQUE is named `uniq` (`unique` is an SV keyword).

16.2.1 Data-scaled fields: Memory Tagging and chunking (bridge TASK-007, was BRIDGE-018)

The last two wrapper features, `mte` and `chunking`, joined the table on 2026-09-13 and closed the native-AXI5 gap. Their fields are the only ones whose width follows the data bus: a tag bus is 4 bits per 16 bytes (`aw/w/b/rtag`), `wtagupdate` and `rchunkstrb` one bit per 16 bytes / 128-bit chunk. `sideband.py` carries them as symbolic widths (`WIDTH_TAGS`, `WIDTH_NTAGS`, `WIDTH_CHUNKSTRB`) and resolves them with `field_width(width, data_width)`:

- the per-width `w`/`r` structs size them for their own width;
- the width-independent `aw`/`ar`/`b` structs size them for the **widest port in the bridge**, and the adapters and crossbar fit a narrower port's value with an explicit zero-extension or slice (`fit_expr`) – widths decided in Python, so the emitted RTL carries no casts;
- slave ports and adapter pins are sized for that port's own width.

The fabric carries these fields and never interprets them: tags are payload routed with their beat, `btagmatch` returns through the B mux by bridge id like `bresp`, and a chunked burst's R beats are routed by ID and released on `RLAST` whatever order the completer sends the chunks in. Both features require a port of 128 bits or more (`validate_axi5_wide_features`).

16.3 Connectivity Gating (poison, atomic, mte) and Droppable Chunking

`AXI5_CONNECTIVITY_GATED_FEATURES` in the validator: these features are legal only when **every** connected path is AXI5-both-ends, feature-enabled, and width-matched — otherwise a config error naming the offending pair. `mte` is in the set because a dropped tag operation changes what the write means. Droppable sideband that terminates mid-path is legal but prints a generation-time warning per (master, slave, feature); `chunking` is droppable because `ARCHUNKEN` is permission, not demand – a slave that cannot chunk answers with ordered data and `RCHUNKV` low, which every chunking requester must accept.

16.4 Atomic Filter (A5-3a, write-only masters)

A **write-only** atomic-enabled master's wr path inserts `axi5_atomic_filter` between the boundary wrapper and the fabric:

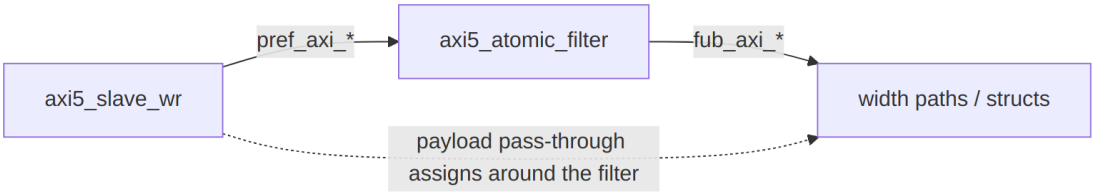


Diagram 5

Handshakes (AW/W/B) and the B payload (`bid`/`bresp`) go **through** the filter; all other payload gets `pref → fub` pass-through assigns. Store- class ATOP and plain writes forward; read-return classes are swallowed (W burst consumed) and answered with a local DECERR B. See the module doc: `docs/markdown/rtl-amba/axi5/axi5_atomic_filter.md`. The filter exists because a port with no read path has nowhere to deliver the R beat those classes answer with. A port that has one gets the next section instead.

16.5 Read-Return Atomics (A5-3b, rw masters)

An **rw** atomic-enabled master has no filter. AtomicLoad (`10xxxx`), AtomicSwap and AtomicCompare (`11000x`) ride the AW path to the slave like any write; the slave performs the operation and answers on **both** B and R, the R beat carrying the location's original data under the **AW's ID**. Two things in this fabric had to learn about that beat, because every R-return tracker in it is fed by AR handshakes:

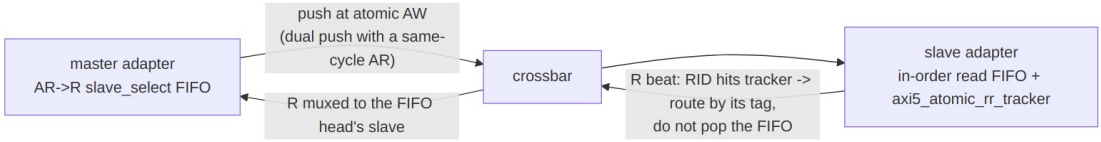


Diagram 6

Master adapter. The AR->R tracking FIFO that drives `r_slave_select` takes an entry at the atomic AW handshake as well as at AR. An AR and an atomic AW can handshake in the same cycle, so the FIFO has a dual push: AR takes the first slot, the AW the second, and the AW's own gate (`aw_rr_gate_ok`) demands two free slots and the same single-outstanding-target rule the read side already applies. The AW is the last entry pushed, so it becomes the active target. The B/W paths are unchanged. The trace tracker mirrors the dual push so the R beat echoes the AW's trace (bridge BUG-010, was BRIDGE-012).

Out of range. A read-return atomic that decodes to the subtractive slave would otherwise wedge the port: that slave answers DECERR on B and knows nothing about R, while the master adapter is holding an R-return slot for the beat. The tracker entry therefore carries a `local` flag and the AW's ID, and when it reaches the head the R mux presents a single DECERR beat with that ID itself, holding every slave's `rready` off (`r_path_active && !r_local_head`) for that cycle. B and R both report DECERR, and the port keeps working. This is bridge BUG-007 (was BRIDGE-009)'s rule applied to the atomic path.

The invariant underneath. The crossbar's response mux is an OR-merge and is a mux only while at most one connected slave's tracker head belongs to a given master; the master adapter's single-outstanding-target gate is what guarantees that. The atomic AW's gate therefore yields when an AR to a different slave handshakes in the same cycle (the one way a dual push could hold two targets), and every generated crossbar now carries a simulation- only `$countones` guard on each master's response-mux select vector that reports the cycle the invariant slips.

Slave adapter. `axi5_atomic_rr_tracker` (module doc: `docs/markdown/rtl-amba/axi5/axi5_atomic_rr_tracker.md`) records (AWID -> requester index) at every AW handshake with `AWATOP[5]` set, and answers combinationally from `RID`. An R beat it claims is routed by its tag and does **not** pop the in-order read FIFO; every other beat routes by FIFO position as before. The atomic AW's `awready` is held while the tracker is full. The bridge BUG-008 (was BRIDGE-010) in-order check skips tracked beats and reports, in simulation, an R beat that matches both a tracked atomic and the FIFO head: that is two requesters aliasing one ID at this slave, which this fabric does not disambiguate.

Validator. An rw atomic master's connected atomic slaves must be `rw` (a write-only slave cannot return read data). The return tracker sits beside whichever read tracker the slave uses, the in-order FIFO or the `enable_ooo` CAM (bridge BUG-012 (was BRIDGE-015) repaired that mode). A write-only atomic master is not subject to the rule; it keeps the filter.

Filelist. `axi5_atomic_filter.f` is pulled only when a write-only atomic master exists; `axi5_atomic_rr_tracker.f` only when an rw atomic slave exists. Pure-AXI4 bridges and the A5-3a fixture are byte-identical to before this change.

16.6 APB5 Slaves (A5-3c)

`protocol = "apb5"` reuses the entire APB4 path with two deltas: the slave adapter instantiates `axi4_to_apb5_shim` (a sideband wrapper over the APB4 shim — see the converters MAS), and the external surface adds `PAUSER/PWUSER` out (driven `'0`) and `PWAKEUP/PRUSER/PBUSER` in (terminated). The generated TB drives the port with the APB4 BFM: APB5 keeps the APB4 transfer protocol.

16.7 Lite and APB Requesters (bridge TASK-004, was BRIDGE-014)

Every protocol value is legal on a master port. Two of them needed work beyond the AXI4-Lite master path that already existed:

- **`axil5` master.** The AXI4-Lite promotion (AXI4 extras tied at the top, `axi4_slave_*` wrappers, the wide-slave aligner toward wider slaves) plus the whole AXI5-Lite sideband on the boundary, directions flipped for a requester. `exclusive` and `user` – the two groups with an AXI4 destination – join the Lite surface and reach the adapter's AXI4 face; the rest is terminated at the top. `validate_axil5_features` applies to masters and slaves alike, so a config cannot name a group the design drops.
- **`apb` / `apb5` master.** The bridge is the APB completer. `apb4_to_axi4` / `apb5_to_axi4` (converters: `apb{4,5}_slave` + `apb_cmdrsp_to_axi4`) sit inside the master adapter in front of the ordinary timing wrapper, so monitoring, decode, width adaptation and the response mux are untouched. One transfer -> one single-beat AXI4 transaction; SLVERR and DECERR fold to PSLVERR; `PAUSER[0]`/`PWUSER[0]` ride the fabric USER bit (visible at an `axil5` slave with `user`). `PWAKEUP` is requester-driven and terminated.

Validator rules: Lite masters `id_width = 0`; APB masters `addr_width = 32` (a requester addresses the whole fabric, unlike an APB slave port's window offset). The generated TB drives `axil5` masters with the AXIL5 BFMs and `apb`/`apb5` masters with `APBMaster`/`APB5Master`; `master_read`/ `master_write` unwrap the APB transaction and turn `PSLVERR` into `AxiResponseError(resp=2)` – APB cannot tell SLVERR from DECERR, so 2 is the honest code.

16.8 Verification Anchors

- Generator unit tests: `bin/tests/test_generator_pkg.py` (feature gating, poison/atomic connectivity rules, generation smoke).
- Sideband **values** end-to-end, driven per transaction by the AXI5 master BFM: `dv/tests/test_bridge_1x2_{rd,wr}_axi5n_sideband.py` (native path and drop path; the AXI5 slave BFM echoes trace on B/R).
- Sideband **through the arbiter**: `dv/tests/test_bridge_2x2_axi5_sideband_arb.py` – two AXI5 masters with distinct NSAIDs contend for the AXI5 slave; every slave-side AW/AR NSAID must belong to its issuing master and the counts must match.
- Sideband **across a width converter**: `dv/tests/test_bridge_1x2_rd_axi5w_sideband.py` – 32b AXI5 master into a 64b AXI4 slave; data round-trips, trace returns 0.
- Atomics, filtered: `dv/tests/test_bridge_1x2_wr_axi5a_atomics.py` (store forwards; load, swap AND compare answered DECERR by the filter, via `atomic_operation`) and `val/amba/test_axi5_atomic_filter.py`.
- Atomics, read-return: `dv/tests/test_bridge_1x2_rw_axi5a_atomics.py` – load ADD/SET/UMAX, swap, compare (match and mismatch) on both slaves, each checked three ways (R data is the pre-op value, memory holds the post-op value, a plain read agrees), then reads and atomics in flight together across and within slaves. The slave BFM performs the operation; the AXI5 checker treats a read-return atomic as an outstanding read and flags any R beat nobody requested.
  `val/amba/test_axi5_atomic_rr_tracker.py` covers the tracker alone. Mutation-checked: with the tracker's `hit` removed from `rid_valid`, the R beat is never routed and the test fails on a read-return timeout.
- Lite requesters: `dv/tests/test_bridge_2x2_lite_req_sideband.py` on `bridge_2x2_lite_req` (AXI4-Lite + AXI5-Lite masters, 64b AXI4 + AXI5-Lite slaves) – `user`/`lock` from the AXI5-Lite master seen at the AXI5-Lite slave and absent from the AXI4-Lite master's traffic, terminated groups reading 0, both halves of a 64-bit row from both Lite masters (the aligner), and both ID-less masters in flight at one slave.
- APB requesters: `dv/tests/test_bridge_2x3_apb_req_paths.py` on `bridge_2x3_apb_req` (APB4 + APB5 masters, AXI4 + AXI5-Lite + APB4 slaves) – PSLVERR for unmapped addresses with the port working after, PPROT at the AXI4 slave, the APB5 USER bit at the AXI5-Lite slave and 0 from the APB4 master, APB in / APB out through the fabric from both

requesters interleaved. The converters alone: `projects/components/utility-ip/converters/dv/tests/test_apb{4,5}_to_axi4.py`.

- AXI5 compliance at the boundary: every generated TB arms an `AXI5ComplianceChecker` on each AXI5 master port and every generated test asserts zero violations before PASSED; `dv/tests/test_bridge_1x2_rd_axi5_bfm5.py` is the read-side sign-off.

---

17 Bridge Arbiter Finite State Machines

17.1 Overview

The Bridge AXI4 crossbar arbitrates per slave, not per master — each slave owns two dedicated arbiters:

- **AW Arbiter** - Write Address channel arbitration
- **AR Arbiter** - Read Address channel arbitration

Total FSM Count: `2 × NUM_SLAVES`

Each arbiter is a simple 2-state FSM. Round-robin access keeps any master from starving, and two states keep the behavior easy to reason about — there's real value in an arbiter you can hold in your head.

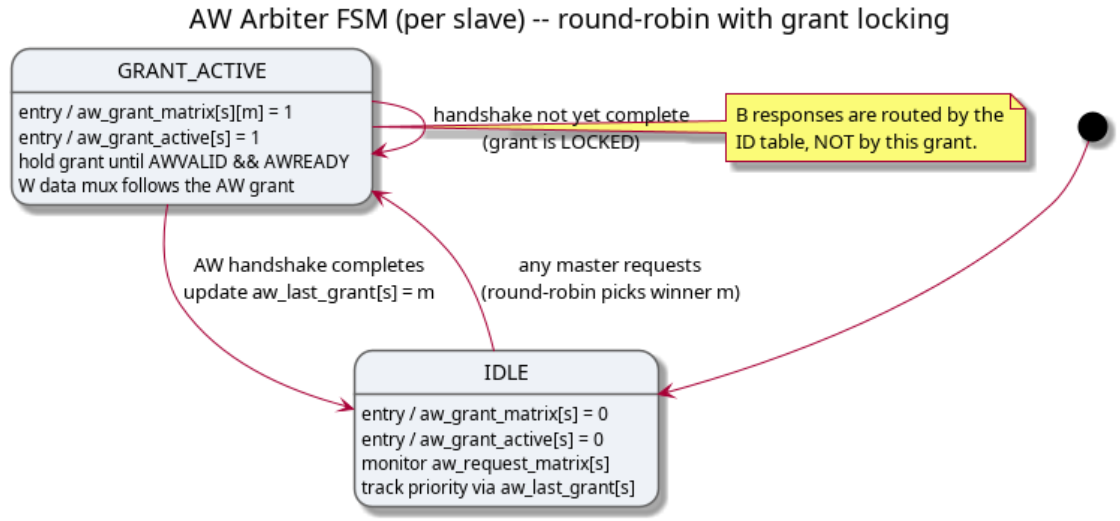
17.2 AW Channel Arbiter FSM

Purpose: Arbitrate write address requests from multiple masters to a single slave

States: 2 (IDLE, GRANT_ACTIVE)

Algorithm: Round-robin with grant locking

17.2.1 Figure 5.3: AW Arbiter FSM



AW Arbiter FSM

State Descriptions:

IDLE State: - **Entry Actions:** - `aw_grant_matrix[s] = 0` - No grant issued - `aw_grant_active[s] = 0` - Arbiter inactive - **Behavior:** - Monitor `aw_request_matrix[s]` for master requests - Track round-robin priority via `aw_last_grant[s]` - Remain in IDLE until at least one master requests access

GRANT_ACTIVE State: - **Entry Actions:** - `aw_grant_matrix[s][m] = 1` - Issue grant to winning master - `aw_grant_active[s] = 1` - Mark arbiter as active - **Behavior:** - Hold grant until AW handshake completes (`AWVALID && AWREADY`) - W channel data multiplexing follows AW grant - B channel response routes by bridge_id FIFO position (no ID table, not grant-based)

State Transitions:

From	To	Condition	Description
IDLE	GRANT_ACTIVE	`| aw_request_matrix[s]| > 0`	At least one master requesting access
GRANT_ACTIVE	IDLE	`m_axi_awvalid[s] &&`	AW handshake

From	To	Condition	Description
		`m_axi_awready[s]`	complete

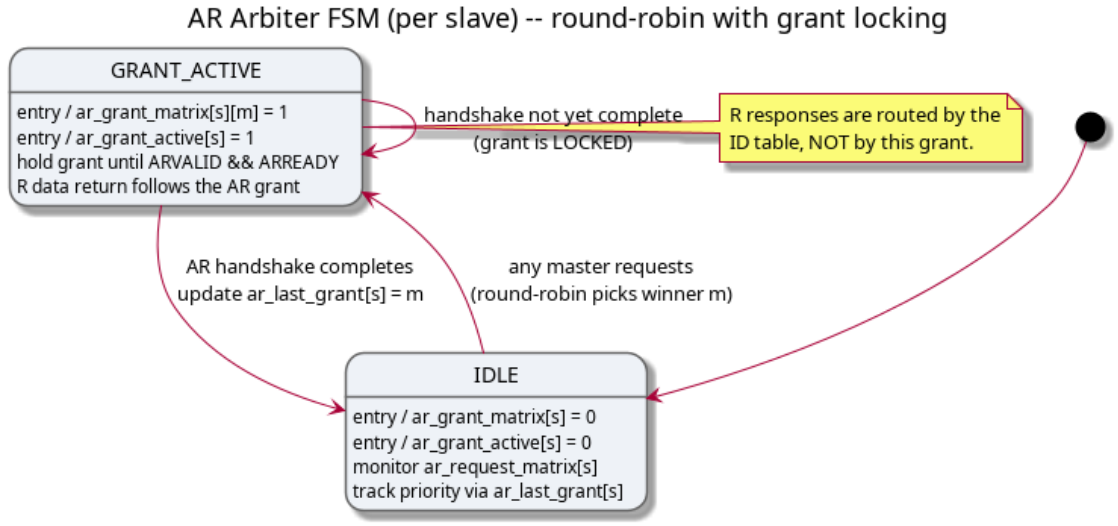
17.3 AR Channel Arbiter FSM

Purpose: Arbitrate read address requests from multiple masters to a single slave

States: 2 (IDLE, GRANT_ACTIVE)

Algorithm: Round-robin with grant locking (same as AW)

17.3.1 Figure 5.4: AR Arbiter FSM



AR Arbiter FSM

State Descriptions:

IDLE State: - **Entry Actions:** - `ar_grant_matrix[s] = 0` - No grant issued - `ar_grant_active[s] = 0` - Arbiter inactive - **Behavior:** - Monitor `ar_request_matrix[s]` for master requests - Track round-robin priority via `ar_last_grant[s]` - Independent from AW arbiter state

GRANT_ACTIVE State: - **Entry Actions:** - `ar_grant_matrix[s][m] = 1` - Issue grant to winning master - `ar_grant_active[s] = 1` - Mark arbiter as active - **Behavior:** - Hold grant until AR handshake completes (`ARVALID && ARREADY`) - R channel data multiplexing follows AR grant - R channel response routes by bridge_id FIFO position (no ID table)

State Transitions:

From	To	Condition	Description
IDLE	GRANT_ACTIVE	`| ar_request_matrix[s]| > 0`	At least one master requesting read

From	To	Condition	Description
GRANT_ACTIVE	IDLE	`m_axi_arvalid[s] && m_axi_arready[s]`	AR handshake complete

17.4 Round-Robin Arbitration Algorithm

Algorithm Description:

Both AW and AR arbiters use the same fair round-robin arbitration algorithm — learn it once and you've learned both:

```
Pseudocode:
1. Start search from (last_grant[s] + 1) % NUM_MASTERS
2. Search cyclically through all masters
3. Select first master with pending request
4. Update last_grant[s] = winning_master
5. Issue grant
```

Example with 4 Masters:

```
Initial state: last_grant[0] = 0

Request Pattern:
  Master 0: request
  Master 1: request
  Master 2: no request
  Master 3: request

Arbitration Sequence:
  Grant 1: Master 1 (start from master 1, first requester)
          last_grant = 1

  Grant 2: Master 3 (start from master 2, find master 3)
          last_grant = 3

  Grant 3: Master 0 (start from master 0, first requester)
          last_grant = 0

  Grant 4: Master 1 (start from master 1, first requester)
          last_grant = 1
```

Fairness Properties:

1. **No Starvation** - Every requesting master will eventually receive grant
2. **Priority Rotation** - Priority shifts after each grant
3. **Predictable Latency** - Maximum wait time = (NUM_MASTERS - 1) × grant_duration
4. **Equal Opportunity** - All masters treated equally

17.5 Grant Locking Mechanism

Purpose: Prevent grant changes mid-transaction

AW Grant Locking:

```
// AW grant locked until address handshake completes
always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        aw_grant_active[s] <= 1'b0;
        aw_grant_matrix[s] <= '0;
    end else begin
        if (aw_grant_active[s]) begin
            // GRANT_ACTIVE state: Hold grant until handshake
            if (m_axi_awvalid[s] && m_axi_awready[s]) begin
                aw_grant_active[s] <= 1'b0;  // Return to IDLE
                aw_grant_matrix[s] <= '0;
            end
        end else if (|aw_request_matrix[s]) begin
            // IDLE state: Arbitrate if requests pending
            aw_grant_matrix[s] <=
round_robin_select(aw_request_matrix[s]);
            aw_grant_active[s] <= 1'b1;  // Enter GRANT_ACTIVE
        end
    end
end
```

AR Grant Locking:

```
// AR grant locked until address handshake completes
always_ff @(posedge aclk or negedge aresetn) begin
    if (!aresetn) begin
        ar_grant_active[s] <= 1'b0;
        ar_grant_matrix[s] <= '0;
    end else begin
        if (ar_grant_active[s]) begin
            // GRANT_ACTIVE state: Hold grant until handshake
            if (m_axi_arvalid[s] && m_axi_arready[s]) begin
                ar_grant_active[s] <= 1'b0;  // Return to IDLE
                ar_grant_matrix[s] <= '0;
            end
        end else if (|ar_request_matrix[s]) begin
            // IDLE state: Arbitrate if requests pending
            ar_grant_matrix[s] <=
round_robin_select(ar_request_matrix[s]);
            ar_grant_active[s] <= 1'b1;  // Enter GRANT_ACTIVE
        end
    end
end
```

Why Grant Locking Matters:

1. **Protocol Compliance** - AXI4 spec requires stable AWID/ARID during handshake
2. **Data Integrity** - W channel must follow granted master's AW request
3. **Response Routing** - B/R channels need consistent transaction tracking
4. **Simplicity** - Single grant active per slave prevents mux conflicts

17.6 Independent Read/Write Arbitration

Here's the design decision that pays for itself: the AW and AR arbiters operate completely independently.

Benefits:

1. **No Head-of-Line Blocking:**
   - Read requests don't wait for write completion
   - Write requests don't wait for read completion
2. **Parallel Operation:**
   - Master 0 can write to Slave 0 (AW path)
   - Master 1 can read from Slave 0 (AR path)
   - Both transactions proceed simultaneously
3. **Resource Efficiency:**
   - Full utilization of read/write bandwidth
   - No artificial serialization

Example Scenario:

```
Time T0: Master 0 issues write to Slave 0
         - AW arbiter grants to Master 0
         - AW_GRANT_ACTIVE = 1
         - W data follows

Time T1: Master 1 issues read to Slave 0 (during Master 0 write)
         - AR arbiter grants to Master 1 (independent!)
         - AR_GRANT_ACTIVE = 1
         - R data returns

Result: Read and write happen in parallel - no blocking
```

17.7 FSM Instance Breakdown by Configuration

2×2 Configuration (2 masters, 2 slaves): - Slave 0 AW Arbiter: 1 FSM - Slave 0 AR Arbiter: 1 FSM - Slave 1 AW Arbiter: 1 FSM - Slave 1 AR Arbiter: 1 FSM - **Total: 4 FSMs**

4×4 Configuration (4 masters, 4 slaves): - 4 slaves × 2 arbiters per slave = **8 FSMs**

8×8 Configuration (8 masters, 8 slaves): - 8 slaves × 2 arbiters per slave = **16 FSMs**

Scalability: - FSM count scales linearly with NUM_SLAVES - Arbitration complexity scales with NUM_MASTERS (search time) - Synthesis impact minimal for up to 16×16 configurations

17.8 Performance Characteristics

Latency:

- **Best Case:** 1 clock cycle (IDLE → GRANT_ACTIVE → IDLE with immediate handshake)
- **Worst Case:** (NUM_MASTERS) clock cycles (round-robin search + contention)
- **Average Case:** ~(NUM_MASTERS / 2) clock cycles

Throughput:

- **Back-to-Back Grants:** Possible if different masters (no wait)
- **Same Master Repeated:** 1 cycle minimum between grants (FSM state change)

Fairness:

- **Guaranteed:** Every master gets grant within (NUM_MASTERS - 1) arbitration cycles
- **No Priority:** All masters treated equally (can be extended for QoS)

17.9 Comparison with Other Crossbar Arbiters

Feature	Bridge Arbiter	APB Crossbar	Commercial Tools
States	2 (IDLE, GRANT_ACTIVE)	1 (passthrough)	3-5 (complex)
Algorithm	Round-robin	N/A (APB is 1:1)	Priority, QoS, weighted
Grant Locking	Yes (until handshake)	N/A	Burst-aware locking
Read/Write Indep	Yes (2 FSMs/slave)	N/A	Yes
Complexity	Low	Minimal	High
Predictability	High (round-robin)	N/A	Medium (priority-based)

Simplicity Trade-off:

- **Advantages:**
  - Easy to verify (2-state FSM)
  - Predictable latency
  - Fair resource allocation
- **Limitations (future enhancements):**
  - No Quality-of-Service (QoS) support
  - No priority levels
  - No weighted arbitration

17.10 Future Enhancements (Phase 3+)

QoS Support:

```
// Master QoS priority field (AXI4 optional)
input logic [3:0] s_axi_awqos [NUM_MASTERS];

// Arbitration considers QoS priority
function automatic [NUM_MASTERS-1:0] qos_arbitrate(
    input logic [NUM_MASTERS-1:0] requests,
    input logic [3:0] qos [NUM_MASTERS]
);
    // Higher QoS value = higher priority
    // Round-robin among same QoS level
endfunction
```

Weighted Arbitration:

```
// Master weight configuration
parameter int MASTER_WEIGHTS [NUM_MASTERS] = '{1, 2, 1, 4};

// Grant frequency proportional to weight
// Master 3 gets 4× more grants than Master 0
```

Burst-Aware Locking:

```
// Lock grant until entire burst completes
// Currently: Lock until address handshake
// Enhanced: Lock until WLAST (write) or RLAST (read)
```

17.11 Related Documentation

See Also: - 3.1 - Module Structure - Generated RTL organization - 3.3 - Crossbar Core - Internal crossbar instantiation

Reference: - ARM AMBA AXI4 Specification (IHI 0022E) - Section A7 (Arbitration)

Next: 3.3 - Crossbar Core

---

18 Transaction Tracking FSMs

18.1 Overview

Several small FSMs track outstanding transactions so responses route correctly. They work alongside the per-slave in-order bridge_id FIFO – not a CAM, which was never built – each holding one piece of the transaction state.

18.2 Write Data Tracking FSM

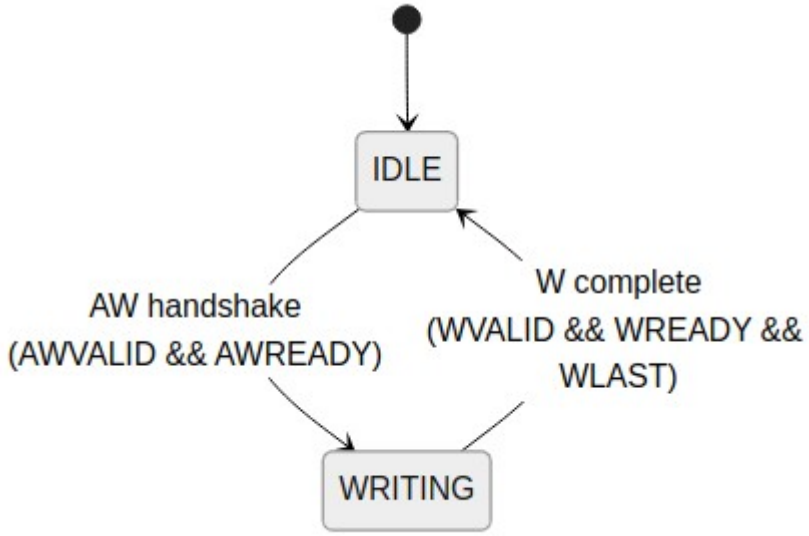
18.2.1 Purpose

The W channel carries no address of its own, so this FSM remembers which slave the AW handshake selected and steers the data beats to match.

18.2.2 States

State	Description
IDLE	Waiting for AW handshake
WRITING	Routing W beats to target slave

18.2.3 Figure 3.1: Write Data Tracking FSM



Write Data Tracking FSM

18.2.4 Implementation

```
typedef enum logic [0:0] {
    W_IDLE    = 1'b0,
    W_WRITING = 1'b1
} w_state_t;

w_state_t r_w_state;
logic [$clog2(NUM_SLAVES)-1:0] r_w_target;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_w_state <= W_IDLE;
```

```
        r_w_target <= '0;
    end else begin
        case (r_w_state)
            W_IDLE: begin
                if (awvalid && awready) begin
                    r_w_state <= W_WRITING;
                    r_w_target <= decoded_slave;
                end
            end
            W_WRITING: begin
                if (wvalid && wready && wlast) begin
                    r_w_state <= W_IDLE;
                end
            end
        endcase
    end
end
```

18.3 Response Pending Counter

Not built. There is no per-master outstanding-transaction counter and no flow control derived from one. Outstanding depth is bounded by the per-slave bridge_id FIFO, whose fullness gates the address handshake (bridge BUG-009, was BRIDGE-011). Searching the generated RTL for a response-pending counter returns nothing.

18.3.1 Purpose

Track number of outstanding transactions per master for flow control.

18.3.2 Implementation

```
logic [7:0] r_outstanding [NUM_MASTERS];

// Increment on AR/AW acceptance
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (int i = 0; i < NUM_MASTERS; i++)
            r_outstanding[i] <= '0;
    end else begin
        for (int m = 0; m < NUM_MASTERS; m++) begin
            logic inc = (ar_accepted[m] || aw_accepted[m]);
            logic dec = (r_delivered[m] || b_delivered[m]);

            case ({inc, dec})
                2'b10: r_outstanding[m] <= r_outstanding[m] + 1;
                2'b01: r_outstanding[m] <= r_outstanding[m] - 1;
                default: r_outstanding[m] <= r_outstanding[m];
            endcase
        end
    end
end

// Backpressure when outstanding limit reached
assign accept_ar[m] = (r_outstanding[m] < MAX_OUTSTANDING);
assign accept_aw[m] = (r_outstanding[m] < MAX_OUTSTANDING);
```

18.4 Burst Counter FSM

18.4.1 Purpose

Track burst beat count for responses spanning multiple cycles.

18.4.2 States

State	Description
IDLE	No burst in progress
COUNTING	Tracking burst beats

18.4.3 Implementation

```
logic [7:0] r_burst_count;
logic r_burst_active;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_burst_count <= '0;
        r_burst_active <= 1'b0;
    end else begin
        if (!r_burst_active && rvalid && rready) begin
            // Start new burst
            r_burst_active <= !rlast;
            r_burst_count <= rlast ? '0 : 8'd1;
        end else if (r_burst_active && rvalid && rready) begin
            // Continue burst
            r_burst_count <= r_burst_count + 1;
            r_burst_active <= !rlast;
        end
    end
end
```

18.5 Related Documentation

- Arbiter FSMs - Address channel arbitration
- ID Management - the in-order FIFO actually built; 01_cam_architecture.md documents an unbuilt CAM for response tracking

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

19 CAM Architecture

19.1 Overview

This chapter describes a design that was NEVER BUILT.

> **What the generated bridge actually does.** There is no CAM, no ID table and no ID injection. `bridge_cam.sv` exists in the tree and is instantiated in **zero** generated bridges. AXI IDs pass through UNTOUCHED and at equal width – `cpu_m_axi_awid` and `ddr_s_axi_awid` are both 4 bits in `bridge_2x2_rw`, with nothing prepended and nothing stripped.
>
> The originating master travels as a SIDEBAND signal (`xbar_bridge_id_aw` / `xbar_bridge_id_ar`) beside the transaction. Each slave adapter pushes it into an in-order FIFO on the address handshake, pops it on the response, and routes the response by FIFO POSITION – never by the returned BID/RID.
>
> Two consequences follow, and both are tracked: * each slave port REQUIRES responses in request order across all IDs; a slave that reorders between IDs misroutes here (bridge BUG-008 (was BRIDGE-010), now detected by a simulation-only check that compares the returned BID/RID against the FIFO head); * the FIFO is fixed-depth, so the address handshake is gated on it being not-full (bridge BUG-009, was BRIDGE-011).
>
> Out-of-order response routing is therefore NOT supported and NOT implemented.

> **Everything below this line describes a design that was never built.** It is retained because the mechanism that replaced it is easy to mistake for it.

The CAM existed to make out-of-order response routing possible: it would record which master owns each outstanding transaction ID, so a single lookup answers the routing question in one shot. Here's what was on the whiteboard.

19.2 Functional Description

19.2.1 What the CAM Was Meant to Do

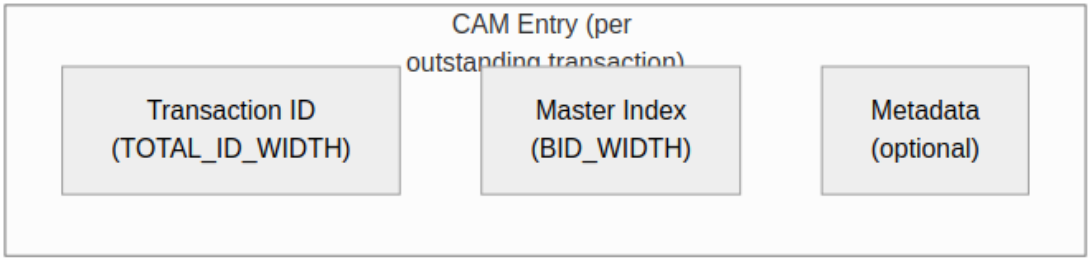
Transaction tracking. The CAM stores:

- Transaction ID (key)
- Originating master index (value)
- Transaction metadata (optional)

Response routing. When a response arrives:

1. Extract transaction ID from response
2. CAM lookup returns originating master
3. Route response to correct master

19.2.2 Figure 4.1: CAM Entry Format



CAM Entry Format

19.2.3 CAM Sizing

Depth scales with total outstanding transactions; width scales with everything the entry has to hold:

```
CAM Depth = MAX_OUTSTANDING × NUM_MASTERS
CAM Width = TOTAL_ID_WIDTH + BID_WIDTH + METADATA_WIDTH
```

19.2.4 Example Configuration

```
4 masters, 4-bit external ID, max 16 outstanding per master:
  BID_WIDTH = clog2(4) = 2
  TOTAL_ID_WIDTH = 4 + 2 = 6
  CAM Depth = 16 × 4 = 64 entries
  CAM Width = 6 + 2 + 8 = 16 bits
```

19.2.5 Insertion (AR/AW Acceptance)

On the AR handshake, the CAM writes the key (`{bid, arid}`) against the value (`master_idx`):

```
// On AR handshake
if (arvalid && arready) begin
    cam_write_en <= 1'b1;
    cam_write_id <= {bid, arid};  // Key
    cam_write_master <= master_idx;  // Value
end
```

19.2.6 Lookup (R/B Response)

When a response comes back from the slave, the lookup is one shot. A miss means the response belongs to no outstanding transaction — that's an error, full stop.

```
// On R response from slave
logic [BID_WIDTH-1:0] response_master;
logic cam_hit;

cam_lookup(slave_rid, response_master, cam_hit);

if (cam_hit) begin
    route_response_to(response_master);
end else begin
    // Error: Unknown transaction
end
```

19.2.7 Duplicate IDs: the Ordering Counter (Mode 2)

`bridge_cam` with `ALLOW_DUPLICATES = 1` accepts several outstanding transactions under one ID and retires them in order. Every entry carries a small ordering counter, and the entries holding one tag always carry the counts `0..k-1`: the oldest is 0, a newcomer takes `k`, and when the oldest retires the survivors step down by one. Two consequences shape the logic:

- The count a newcomer needs is `k`, which is simply how many entries match its tag: a popcount of the match vector, a few LUT levels. It is not a search for the largest existing count. That scan (`DEPTH` chained comparators, hanging off the crossbar's arbitrated ARID) was the bridge's critical path when first synthesized: 40 LUT levels and a 32 ns data path on an Artix-7 -1 (HAS 6.4), fixed 2026-09-11 by counting instead.
- The entry to retire is the one match with count 0, which is one-hot by construction, so its index and its stored master come out of OR reductions rather than a priority chain.

An allocate and a same-tag retire in one cycle take `k - 1`, because the survivors step down in that same cycle (MAS 2.3).

19.2.8 Deletion (Response Complete)

Entries free up when the transaction completes — the last R beat, or the B beat:

```
// On RLAST or B response
if ((rvalid && rready && rlast) || (bvalid && bready)) begin
    cam_delete_en <= 1'b1;
    cam_delete_id <= response_id;
end
```

19.2.9 Implementation Options

Two ways to build it, picked by depth.

19.2.9.1 Distributed RAM CAM

For small outstanding counts (< 16 entries), a parallel comparator array does the lookup combinationally — every entry compared at once, logic cost scaling with depth:

```
// Parallel comparator-based CAM
logic [DEPTH-1:0] valid;
logic [ID_WIDTH-1:0] id_table [DEPTH];
logic [BID_WIDTH-1:0] master_table [DEPTH];

// Parallel compare for lookup
logic [DEPTH-1:0] match;
for (genvar i = 0; i < DEPTH; i++) begin
    assign match[i] = valid[i] && (id_table[i] == lookup_id);
end
```

19.2.9.2 Block RAM CAM

For larger outstanding counts (> 16 entries), the parallel approach gets expensive, so the design falls back to a sequential search — one entry per cycle until a match or the end of the table:

```
// Sequential search CAM (saves logic)
logic [$clog2(DEPTH)-1:0] search_idx;
logic searching;

always_ff @(posedge clk) begin
    if (start_lookup) begin
        searching <= 1'b1;
        search_idx <= 0;
    end else if (searching) begin
        if (match_found || search_idx == DEPTH-1)
            searching <= 1'b0;
        else
            search_idx <= search_idx + 1;
```

```
    end
end
```

19.2.10 CAM Overflow Handling

Prevention. The CAM never overflows because it backpressures first:

```
// Backpressure when CAM full
assign ar_ready = !cam_full && downstream_ready;
assign aw_ready = !cam_full && downstream_ready;
```

Error response. If CAM overflow would occur:

1. Assert backpressure (ARREADY/AWREADY = 0)
2. Wait for responses to free entries
3. Never drop transactions

Worth repeating: none of this exists. The generated bridge gates the address handshake on the tracking FIFO being not-full instead (bridge BUG-009, was BRIDGE-011), and it routes responses strictly in order.

19.3 Related Modules

- ID Tracking - ID table implementation
- Response Routing - Using CAM for routing

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

20 Response Tracking

20.1 Overview

A multi-master fabric has to know which master a response belongs to. This bridge records it **positionally**: each slave adapter pushes the originating master's `bridge_id` onto an in-order FIFO at the address handshake and pops it on the response, routing by FIFO head. The AXI ID passes through untouched in both directions.

```
wr_fifo[wr_ptr[...]] <= xbar_bridge_id_aw;      // push on AW accept
assign bid_bridge_id  = wr_fifo[rd_ptr[...]];   // route by the HEAD
```

The consequence is a requirement the fabric does not check: **each slave port must return B/R in request order across ALL IDs.** AXI4 permits a slave to complete different-ID transactions out of order. Every generated slave adapter now carries a SIMULATION-ONLY check that compares the returned BID/RID against the FIFO head and $error()s on a mismatch (bridge BUG-008, was BRIDGE-010); it cannot fire in silicon – see bridge BUG-008 (was BRIDGE-010).

The rest of this page documents an **ID tracking table** design that was specified but never built: extended IDs formed by prepending a Bridge ID, per-slave lookup tables, out-of-order completion. `bridge_cam.sv` exists in the tree and is instantiated in zero generated bridges. It is kept because the positional scheme above is easy to mistake for it, and because several other pages once described it as real.

20.2 Functional Description

20.2.1 HISTORICAL – the unbuilt ID-table design

Everything below this line describes the design that was NOT implemented.

20.2.2 Not implemented: the per-slave ID table

What follows describes a design that was never built. It is retained because the mechanism that replaced it is easy to mistake for it.

The generated RTL has no ID tables and no CAM: `bridge_cam.sv` exists in the tree and is instantiated in zero generated bridges. IDs are pass-through – the slave port is declared at the same width as the master port, and nothing is prepended. Each slave adapter instead keeps an **in-order FIFO** of the originating master's `bridge_id`, pushed on the address handshake and popped on the response, and routes by FIFO POSITION rather than by the returned BID/RID. The consequence – each slave port must return B/R in request order across all IDs – is tracked as bridge BUG-008 (was BRIDGE-010).

20.2.3 Per-Slave ID Table (historical)

Each slave was to have its own ID tracking table:

20.2.4 Figure 4.2: ID Table Structure



ID Table Structure

20.2.5 Table Sizing

```
Entries = MAX_OUTSTANDING_PER_SLAVE
Width = TOTAL_ID_WIDTH + clog2(NUM_MASTERS) + 1 (valid)
```

20.2.6 ID Extension

20.2.6.1 Master-Side Extension

When a transaction enters the bridge, its ID would have been extended with the Bridge ID (master index):

```
// Extend ID with Bridge ID (master index)
logic [TOTAL_ID_WIDTH-1:0] extended_id;
assign extended_id = {master_bid, external_id};

// Example: Master 2, external ID = 4'hA
// master_bid = 2'b10, external_id = 4'b1010
// extended_id = 6'b10_1010
```

20.2.6.2 Slave-Side Presentation (historical design – not built)

In the unbuilt design an extended ID would have gone to the slave. In the RTL the slave receives the master's ID unchanged:

```
assign s_axi_arid = extended_id;  // 6 bits to slave
assign s_axi_awid = extended_id;  // 6 bits to slave
```

20.2.7 ID Extraction

20.2.7.1 Response Parsing

When the response returns from the slave, the extended ID splits back apart — the upper bits pick the master, the lower bits go home as the ID:

```
// Extract Bridge ID and external ID
logic [BID_WIDTH-1:0] bridge_id;
logic [ID_WIDTH-1:0] external_id;

assign bridge_id = slave_rid[TOTAL_ID_WIDTH-1:ID_WIDTH];
assign external_id = slave_rid[ID_WIDTH-1:0];

// Route to master based on bridge_id
assign m_rvalid[bridge_id] = s_rvalid;
assign m_rid[bridge_id] = external_id;  // Strip Bridge ID
```

20.2.8 Table Operations

20.2.8.1 Allocation (AR/AW Phase)

On the address handshake, find a free entry and claim it:

```
always_ff @(posedge clk) begin
    if (arvalid && arready) begin
        // Find free entry
```

```
        for (int i = 0; i < DEPTH; i++) begin
            if (!valid[i]) begin
                id_table[i] <= extended_arid;
                master_table[i] <= master_idx;
                valid[i] <= 1'b1;
                break;
            end
        end
    end
end
```

20.2.8.2 Lookup (R/B Phase)

The lookup is combinational across the whole table, producing a one-hot master select:

```
// Combinational lookup
logic [NUM_MASTERS-1:0] master_onehot;
always_comb begin
    master_onehot = '0;
    for (int i = 0; i < DEPTH; i++) begin
        if (valid[i] && (id_table[i] == response_id)) begin
            master_onehot[master_table[i]] = 1'b1;
        end
    end
end
```

20.2.8.3 Deallocation (Response Complete)

When the last beat lands, invalidate the matching entry:

```
always_ff @(posedge clk) begin
    if (rvalid && rready && rlast) begin
        // Find and invalidate matching entry
        for (int i = 0; i < DEPTH; i++) begin
            if (valid[i] && (id_table[i] == response_rid)) begin
                valid[i] <= 1'b0;
                break;
            end
        end
    end
end
```

20.2.9 Multi-ID Considerations

20.2.9.1 Same External ID, Different Masters

```
Master 0 issues ID=5 → Extended: 00_0101
Master 1 issues ID=5 → Extended: 01_0101
Master 2 issues ID=5 → Extended: 10_0101
```

```
All three can be outstanding simultaneously!
The extended ID ensures uniqueness.
```

That was the whole point of the extension — three masters can each have ID=5 outstanding at once, and the prepended Bridge ID keeps them distinct.

20.2.9.2 Same Extended ID (historical design – not built)

This cannot happen:

- Bridge ID is unique per master
- Extended ID = {unique BID, external ID}
- Same extended ID implies same master + same external ID
- AXI4 requires unique IDs per master for outstanding transactions

20.3 Design Notes

20.3.1 Resource Utilization

What the tables would have cost, for the record:

```
4 masters, 4-bit external ID, 16 outstanding per slave:

Per-slave table:
  Entries: 16
  Width: 6 + 2 + 1 = 9 bits
  Total: 16 × 9 = 144 bits

4 slaves:
  Total: 4 × 144 = 576 bits (~72 bytes)

Implementation:
  Distributed RAM: ~150 LEs
  Block RAM: 1 BRAM (minimum)
```

20.4 Related Modules

- CAM Architecture - CAM implementation details
- Response Routing - Using tables for routing

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

21 Width Converters

21.1 Overview

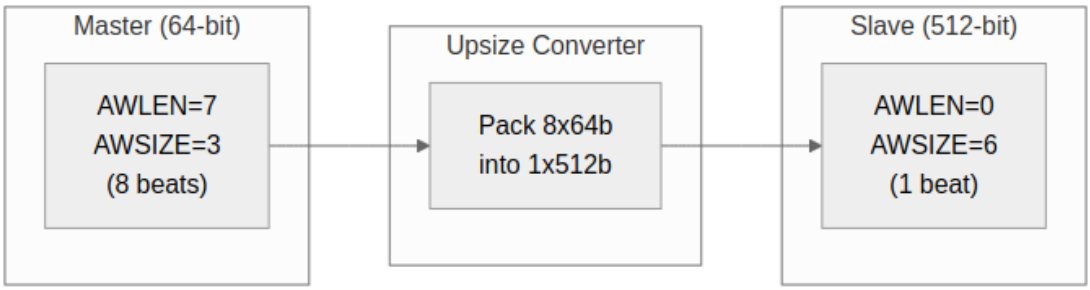
When a master and slave disagree on data width, a converter sits between them: an upsizer for narrow-to-wide, a downsizer for wide-to-narrow. The generator inserts them automatically wherever the configuration needs one — you never instantiate these by hand.

21.2 Functional Description

21.2.1 Upsize Converter

Purpose. Convert narrow master data to wide slave interface.

21.2.2 Figure 5.1: Upsize Converter (64-bit to 512-bit)



Upsize Converter

21.2.3 Implementation

The packing loop is simple in principle: fill a wide buffer one narrow beat at a time, then drain it. One thing before you read the sketch — there is no `width_upsize` module in the tree. The real converters are `axi_data_upsize` / `axi_data_dnsize` (beat packing) wrapped by `axi4_dwidth_converter_rd` / `axi4_dwidth_converter_wr`:

```
// NOTE: there is no `width_upsize` module. The real converters are
// `axi_data_upsize` / `axi_data_dnsize` (beat packing) wrapped by
// `axi4_dwidth_converter_rd` / `axi4_dwidth_converter_wr`. Sketch
only:
module width_upsize #(
    parameter IN_WIDTH = 64,
    parameter OUT_WIDTH = 512
) (
    input  logic clk, rst_n,

    // Narrow input
    input  logic                    s_valid,
    output logic                    s_ready,
    input  logic [IN_WIDTH-1:0]     s_data,
    input  logic                    s_last,

    // Wide output
    output logic                    m_valid,
    input  logic                    m_ready,
    output logic [OUT_WIDTH-1:0]    m_data,
    output logic                    m_last
```

```
);

    localparam RATIO = OUT_WIDTH / IN_WIDTH;

    logic [OUT_WIDTH-1:0] r_buffer;
    logic [$clog2(RATIO)-1:0] r_count;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_count <= '0;
            r_buffer <= '0;
        end else if (s_valid && s_ready) begin
            // Pack input into buffer
            r_buffer[r_count * IN_WIDTH +: IN_WIDTH] <= s_data;

            if (s_last || r_count == RATIO - 1) begin
                r_count <= '0;
            end else begin
                r_count <= r_count + 1;
            end
        end
    end

    // Output when buffer full or last beat
    assign m_valid = (r_count == RATIO - 1) || s_last;
    assign m_data = r_buffer;
    assign s_ready = !m_valid || m_ready;

endmodule
```

21.2.4 Burst Length Conversion

The burst math follows directly — eight narrow beats pack into one wide beat:

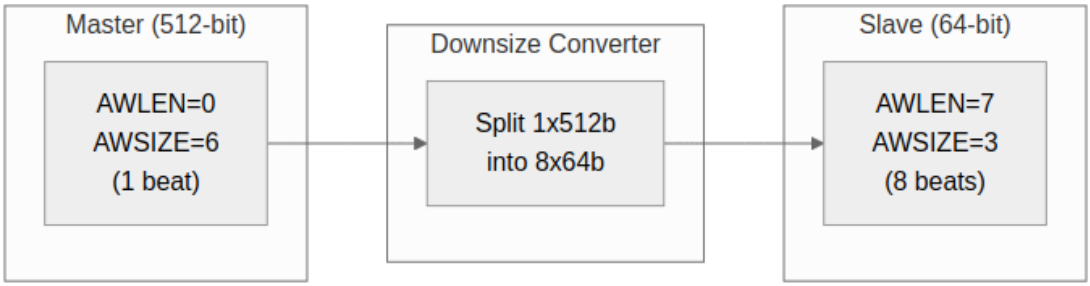
Table 5.3: Burst Length Conversion Examples

Master AWLEN	Master Beats	Slave AWLEN	Slave Beats
7 (8 beats)	8 × 64b	0 (1 beat)	1 × 512b
15 (16 beats)	16 × 64b	1 (2 beats)	2 × 512b
255 (256 beats)	256 × 64b	31 (32 beats)	32 × 512b

21.2.5 Downsize Converter

Purpose. Convert wide master data to narrow slave interface.

21.2.6 Figure 5.2: Downsize Converter (512-bit to 64-bit)



Downsize Converter

21.2.7 Implementation

The downsizer is the mirror image: latch one wide word, then slice it out one narrow beat at a time.

```
module width_downsize #(
    parameter IN_WIDTH = 512,
    parameter OUT_WIDTH = 64
) (
    input  logic clk, rst_n,

    // Wide input
    input  logic                    s_valid,
    output logic                    s_ready,
    input  logic [IN_WIDTH-1:0]     s_data,
    input  logic                    s_last,

    // Narrow output
    output logic                    m_valid,
    input  logic                    m_ready,
    output logic [OUT_WIDTH-1:0]    m_data,
    output logic                    m_last
);

    localparam RATIO = IN_WIDTH / OUT_WIDTH;

    logic [IN_WIDTH-1:0] r_buffer;
    logic [$clog2(RATIO)-1:0] r_count;
    logic r_active;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_count <= '0;
            r_buffer <= '0;
```

```
            r_active <= 1'b0;
        end else begin
            if (!r_active && s_valid) begin
                // Load new wide word
                r_buffer <= s_data;
                r_active <= 1'b1;
                r_count <= '0;
            end else if (r_active && m_ready) begin
                if (r_count == RATIO - 1) begin
                    r_active <= 1'b0;
                end else begin
                    r_count <= r_count + 1;
                end
            end
        end
    end

    // Output from buffer slice
    assign m_valid = r_active;
    assign m_data = r_buffer[r_count * OUT_WIDTH +: OUT_WIDTH];
    assign m_last = r_active && (r_count == RATIO - 1);
    assign s_ready = !r_active;

endmodule
```

21.2.8 Strobe Handling

Data isn't the only thing that rescales — the byte strobes pack and split right alongside it.

21.2.8.1 Upsize Strobe Packing

```
// Pack 8-byte strobes into 64-byte strobes
logic [7:0] in_strb;   // 64-bit input
logic [63:0] out_strb; // 512-bit output

always_ff @(posedge clk) begin
    if (s_valid && s_ready) begin
        out_strb[r_count * 8 +: 8] <= in_strb;
    end
end
```

21.2.8.2 Downsize Strobe Splitting

```
// Split 64-byte strobes into 8-byte strobes
logic [63:0] in_strb;  // 512-bit input
logic [7:0] out_strb;  // 64-bit output

assign out_strb = in_strb[r_count * 8 +: 8];
```

21.2.9 AXIL-to-Wider-Slave Master-Side Alignment Converters

An AXI4-Lite master talking to a wider AXI4 slave through the bridge has an alignment problem: AXIL single-beat transactions don't know where they land in a wide word. The bridge generator emits master-side alignment converters (`axil_to_axi4_wide_align_{rd,wr}.sv`) to handle partial-word alignment on the AXIL side while producing properly-aligned transactions on the wide slave side.

21.2.9.1 Use Case Example

A concrete write makes the lane math clear:

```
32-bit AXIL Master → Bridge → 64-bit AXI4 Slave

Master writes 32-bit word to offset 0x04:
  Data = 0xAABBCCDD (on bits [31:0])

The converter picks the lane from the address, then ROW-ALIGNS the
address:
  slot      = addr[2]                = 1      (SLOT_LSB=$clog2(4)=2,
SLOT_W=$clog2(2)=1)
  m_axi_wdata[63:32] = 0xAABBCCDD             (slot 1 -> the UPPER
half)
  m_axi_wstrb        = 0xF0                   (byte lanes 4..7)
  m_axi_awaddr       = 0x04 & ~0x7 = 0x00     (row_addr_mask clears
the lane bits)

  Slave sees: 0xAABBCCDD_00000000 at address 0x00, WSTRB 0xF0

The address the slave sees is 0x00, NOT 0x04: the lane bits move out
of the
address and into WSTRB. Byte lanes outside the strobe are don't-care,
shown
here as zero.
```

21.2.9.2 Architecture

Read Path: - Master issues AXIL read (no burst, single beat) - Converter passes address to slave (unchanged) - Slave returns 64-bit word - Converter extracts the relevant 32-bit slice for the master

Write Path: - Master issues AXIL write with address offset and 32-bit data - Converter computes byte-lane position within 64-bit word - Converter expands 32-bit strobe to 64-bit strobe (8 lanes) - Converter sends full 64-bit write to slave - Slave performs a 64-bit write with byte strobes

21.2.9.3 Modules

- **`axil_to_axi4_wide_align_rd.sv`**: Read path (address → data conversion)

- **`axil_to_axi4_wide_align_wr.sv`**: Write path (strobe expansion, data placement)

Note: These converters handle **width alignment**, not **protocol bridging**. Protocol conversion (AXIL→AXI4 burst restriction) is applied separately at the slave boundary (via `axi4_to_axil4_*` shims if needed).

21.3 Design Notes

21.3.1 Resource Utilization

Upsize Converter (64 to 512):

```
Registers: ~520 (buffer + control)
Logic: ~200 LEs (packing + control)
```

Downsize Converter (512 to 64):

```
Registers: ~520 (buffer + control)
Logic: ~150 LEs (selection + control)
```

The register counts are dominated by the data buffer — 512 bits of storage plus control, in either direction.

21.4 Related Modules

- Width Conversion Block - Block-level description
- APB Converters - Protocol conversion
- Generator-Emitted Conversion Shims - Master-side vs. slave-side shim placement

22 APB Converters

22.1 Overview

APB converters turn AXI4 transactions into APB transfers — the price a high-performance master pays to talk to a low-speed peripheral.

22.2 Functional Description

22.2.1 Conversion Requirements

22.2.1.1 Protocol Differences

The two protocols differ in ways that force real work on the converter:

Table 5.1: AXI4 vs APB Protocol Comparison

Feature	AXI4	APB
Channels	5 (AW, W, B, AR, R)	1 (combined)
Bursts	Up to 256 beats	None (single)
Pipelining	Yes	No
Min cycles	1 per beat	2 per transfer

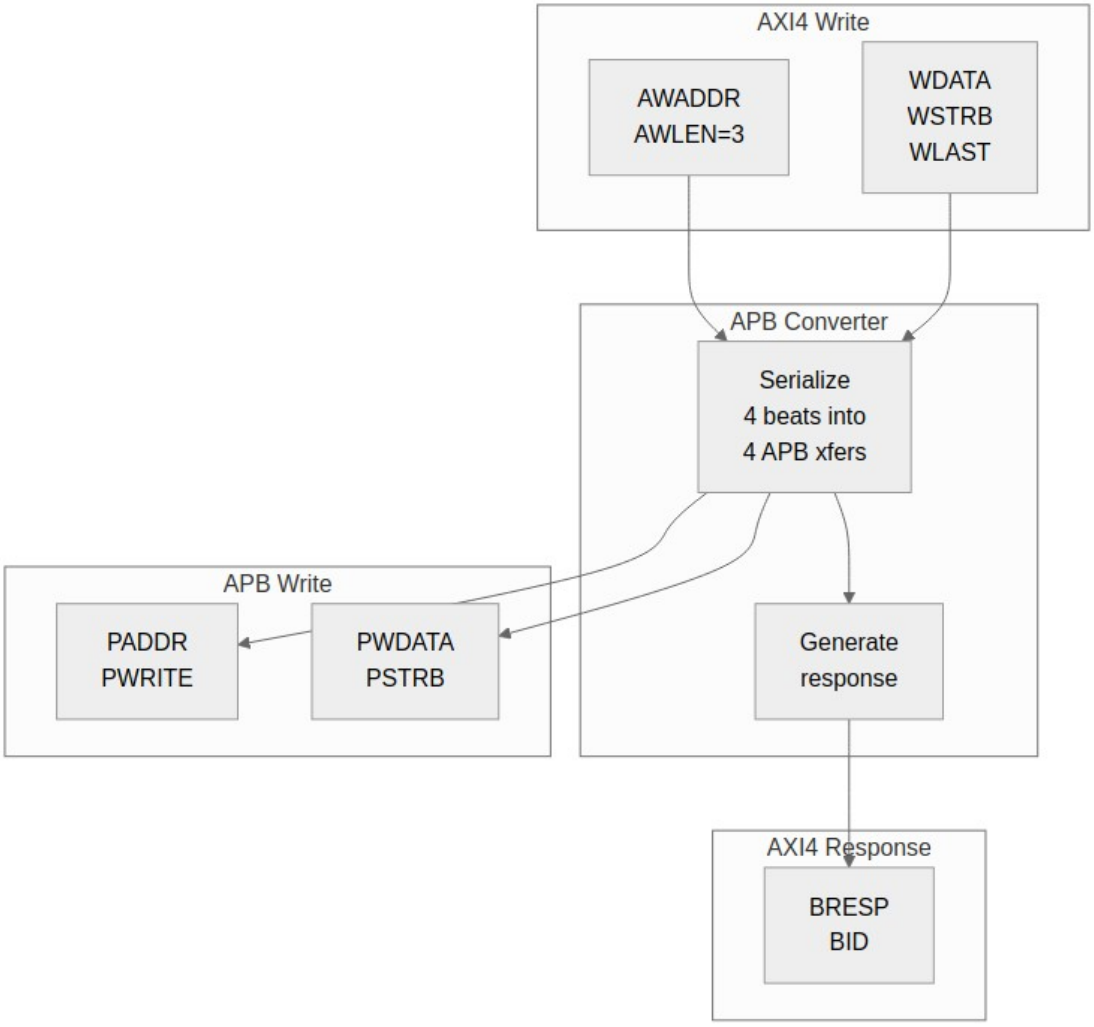
22.2.1.2 Conversion Strategy

The recipe, then:

1. Split AXI4 bursts into individual APB transfers
2. Serialize AW+W into APB write sequence
3. Convert AR into APB read sequence
4. Generate AXI4 responses from APB completions

22.2.2 Write Conversion

22.2.3 Figure 5.3: AXI4 Write to APB Write



AXI4 to APB Write

22.2.3.1 State Machine

The write path walks each beat through the two-cycle APB handshake:

```
typedef enum logic [2:0] {
    IDLE,
    AW_ACCEPT,
    W_ACCEPT,
    APB_SETUP,
    APB_ACCESS,
    B_GENERATE
} apb_wr_state_t;
```

```
apb_wr_state_t r_state;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_state <= IDLE;
    end else begin
        case (r_state)
            IDLE: if (awvalid) r_state <= AW_ACCEPT;

            AW_ACCEPT: begin
                if (awready) r_state <= W_ACCEPT;
            end

            W_ACCEPT: begin
                if (wvalid && wready) r_state <= APB_SETUP;
            end

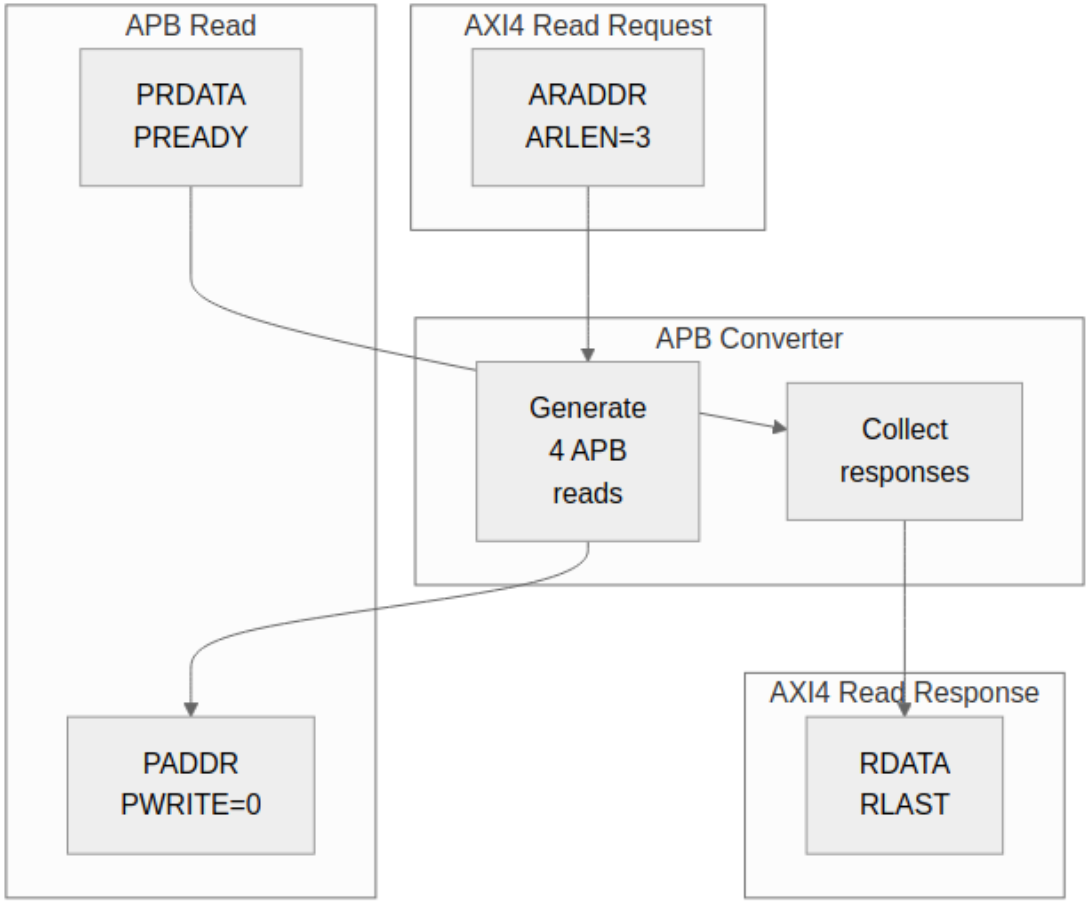
            APB_SETUP: r_state <= APB_ACCESS;

            APB_ACCESS: begin
                if (pready) begin
                    if (r_beat_count == r_awlen)
                        r_state <= B_GENERATE;
                    else
                        r_state <= W_ACCEPT;  // Next beat
                end
            end

            B_GENERATE: begin
                if (bready) r_state <= IDLE;
            end
        endcase
    end
end
```

22.2.4 Read Conversion

22.2.5 Figure 5.4: AXI4 Read to APB Read



AXI4 to APB Read

22.2.5.1 Read State Machine

The read path is the same shape, minus the W channel:

```
typedef enum logic [2:0] {
    IDLE,
    AR_ACCEPT,
    APB_SETUP,
    APB_ACCESS,
    R_GENERATE
} apb_rd_state_t;
```

22.2.6 Burst Handling

22.2.6.1 Burst to Single Conversion

Bursts don't exist on APB, so each beat becomes its own transfer with its own address:

```
AXI4 Burst (AWLEN=7, 8 beats):
  Beat 0: AWADDR + 0×SIZE
  Beat 1: AWADDR + 1×SIZE
  Beat 2: AWADDR + 2×SIZE
  ...
  Beat 7: AWADDR + 7×SIZE

APB Sequence:
  Transfer 0: PADDR = AWADDR + 0×SIZE
  Transfer 1: PADDR = AWADDR + 1×SIZE
  ...
  Transfer 7: PADDR = AWADDR + 7×SIZE
```

22.2.6.2 Address Calculation

Address generation honors all three burst types:

```
// Calculate APB address for each beat
logic [ADDR_WIDTH-1:0] apb_addr;
logic [7:0] beat_count;

always_comb begin
    case (r_burst_type)
        FIXED: apb_addr = r_base_addr;
        INCR:  apb_addr = r_base_addr + (beat_count << r_size);
        WRAP:  apb_addr = wrap_address(r_base_addr, beat_count,
r_size, r_len);
    endcase
end
```

22.2.7 Error Handling

22.2.7.1 APB Error to AXI4 Response

One error bit in, one of two response codes out:

Table 5.2: APB to AXI4 Error Mapping

APB PSLVERR	AXI4 Response
0 (OK)	OKAY (2'b00)
1 (Error)	SLVERR (2'b10)

22.2.7.2 Error Propagation

Errors latch on first sight and ride along to the final response — one bad beat fails the burst:

```
// Latch first error in burst
logic r_error_seen;

always_ff @(posedge clk) begin
    if (r_state == APB_ACCESS && pready && pslverr)
        r_error_seen <= 1'b1;
    if (r_state == IDLE)
        r_error_seen <= 1'b0;
end

// Final response reflects any errors
assign bresp = r_error_seen ? 2'b10 : 2'b00;
```

22.3 Timing

APB costs you on both axes, and it's inherent to the two-cycle transfer — not something you can pipeline around.

22.3.1 Throughput Impact

```
AXI4 alone: 1 beat per cycle (peak)
AXI4 to APB: 1 beat per 2+ cycles (minimum)

Throughput reduction: ~50% or more
```

22.3.2 Latency Impact

```
AXI4 read: 1-2 cycles (typical)
AXI4-to-APB read: 4+ cycles minimum
  - 1 cycle: AR accept
  - 1 cycle: APB setup
  - 1+ cycles: APB access (PREADY)
  - 1 cycle: R generate
```

22.4 Design Notes

22.4.1 Resource Utilization

```
Logic Elements: ~300-400 LEs
Registers: ~100-150 regs
Block RAM: 0

Breakdown:
- State machine: ~50 LEs, ~20 regs
- Address calc: ~100 LEs, ~50 regs
- Beat counter: ~30 LEs, ~8 regs
- Data buffering: ~100 LEs, ~DATA_WIDTH regs
```

22.5 Related Modules

- Protocol Conversion Block - Block description
- Width Converters - Data width conversion

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

23 Module Structure

23.1 Overview

The generator turns configuration files into parameterized SystemVerilog modules, and every topology gets the same internal structure. Parameterized at generation time, mind you — not at elaboration, as the next section makes clear. Once you've read one generated bridge, you can find your way around any of them.

23.2 Parameters

23.2.1 Compile-Time Parameters

> **Not built.** The generated top has no parameters and no `BID_WIDTH` / `TOTAL_ID_WIDTH` localparams: IDs are not extended, so there is nothing to widen. The package carries exactly two constants – `NUM_MASTERS` and `BRIDGE_ID_WIDTH = $clog2(NUM_MASTERS)` – and `BRIDGE_ID_WIDTH` sizes the SIDEBAND master id, not any AXI ID field. Per-port widths are baked into the port declarations; there is no `M0_DATA_WIDTH` localparam.

What the template would have emitted, had that design survived:

```
// Core parameters
parameter int NUM_MASTERS = 4;
parameter int NUM_SLAVES = 3;
parameter int ADDR_WIDTH = 32;
parameter int DATA_WIDTH = 64;
parameter int ID_WIDTH = 4;

// Derived parameters
localparam int BID_WIDTH = $clog2(NUM_MASTERS);
localparam int TOTAL_ID_WIDTH = ID_WIDTH + BID_WIDTH;
localparam int STRB_WIDTH = DATA_WIDTH / 8;
```

23.2.2 Per-Port Parameters

```
// Generated per-port widths
localparam int M0_DATA_WIDTH = 64;
localparam int M1_DATA_WIDTH = 256;
localparam int S0_DATA_WIDTH = 512;
localparam int S1_DATA_WIDTH = 32;  // APB
```

In the real output these widths are baked straight into the port declarations — there are no localparams to override.

23.3 Ports

23.3.1 Module Declaration

The generated top level has NO PARAMETER LIST. Every width is fixed for the configuration at generation time and the only shared constants live in the per-bridge package, so there is nothing to override at instantiation. This is `bridge_2x2_rw.sv` as emitted:

```
module bridge_2x2_rw
    import bridge_2x2_rw_pkg::*;
(
    // Clock and reset
    input  logic aclk,
    input  logic aresetn,

    // Master interfaces (M0-M3)
    // ... master port signals ...

    // Slave interfaces (S0-S2)
    // ... slave port signals ...
);
```

23.3.2 Port Organization

```
Module Ports:
├── Clock/Reset
│   ├── aclk
│   └── aresetn
├── Master Ports (per master)
│   ├── AW Channel (if rw or wr)
│   ├── W Channel (if rw or wr)
│   ├── B Channel (if rw or wr)
│   ├── AR Channel (if rw or rd)
│   └── R Channel (if rw or rd)
└── Slave Ports (per slave)
    ├── AW Channel
    ├── W Channel
    ├── B Channel
    ├── AR Channel
    └── R Channel
```

Note the conditional channels on the master side — a write-only master gets no AR/R, a read-only master gets no AW/W/B.

23.3.3 Master-Side Signals (External)

```
{prefix}_aw{signal}    - Write address channel
{prefix}_w{signal}     - Write data channel
{prefix}_b{signal}     - Write response channel
```

```
{prefix}_ar{signal}    - Read address channel
{prefix}_r{signal}     - Read data channel

Example (prefix = "cpu_m_axi"):
  cpu_m_axi_awvalid
  cpu_m_axi_awready
  cpu_m_axi_awaddr
  cpu_m_axi_wdata
  cpu_m_axi_bvalid
```

23.3.4 Slave-Side Signals (External)

```
{prefix}_aw{signal}    - Write address channel
{prefix}_w{signal}     - Write data channel
{prefix}_b{signal}     - Write response channel
{prefix}_ar{signal}    - Read address channel
{prefix}_r{signal}     - Read data channel

Example (prefix = "ddr_s_axi"):
  ddr_s_axi_awvalid
  ddr_s_axi_awready
  ddr_s_axi_awaddr
  ddr_s_axi_wdata
  ddr_s_axi_bvalid

CDC slave port (cdc = true, bridge TASK-006 (was BRIDGE-017)) adds two
clock pins for that port:
  {name}_aclk            - the port's own clock
  {name}_aresetn         - its active-low reset
```

A `cdc = true` slave port's adapter takes `s_aclk`/`s_aresetn` beside `aclk`/`aresetn`. Inside it the timing wrapper (and the monitor, when built) stay on `aclk`, driving `cdc_{name}_axi_*` nets; `axi4_cdc_wr` and `axi4_cdc_rd` carry those to the external port on `s_aclk`, one `gaxi_fifo_async` per channel. Nothing upstream of the wrapper knows the port is in another domain. Slave ports only, `protocol = "axi4"` only (HAS 4.5a).

23.4 Functional Description

23.4.1 Component Instantiation

Adapters are named after the port, not the index — and several modules you might go looking for (`master_adapter_rw`, a standalone `address_decoder`) don't exist anywhere in the tree. Address decode is inline in the crossbar and each master adapter. From `bridge_2x2_rw.sv`:

```
// Generated module internal structure

// Adapters are named after the PORT, not the index -- there is no
// master_adapter_rw / address_decoder module anywhere in the tree.
// From bridge_2x2_rw.sv:

axi4_subtractive_slave #(...) u_subtractive (...);   // unmapped-
address default
cpu_adapter            u_cpu_adapter (...);          // per-master,
named for the master
dma_adapter            u_dma_adapter (...);
bridge_2x2_rw_xbar     u_xbar (...);                 // one crossbar
ddr_adapter            u_ddr_adapter (...);          // per-slave,
named for the slave
sram_adapter           u_sram_adapter (...);
subtractive_adapter    u_subtractive_adapter (...);

// Address decode is INLINE in the crossbar and each master adapter --
// there is no separate decoder module to instantiate.
address_decoder u_m2_decoder (...);
address_decoder u_m3_decoder (...);

// Per-slave arbiters
arbiter_aw u_s0_aw_arb (...);
arbiter_ar u_s0_ar_arb (...);
arbiter_aw u_s1_aw_arb (...);
arbiter_ar u_s1_ar_arb (...);
arbiter_aw u_s2_aw_arb (...);
arbiter_ar u_s2_ar_arb (...);

// Width converters (as needed)
width_upsize_64_512 u_m0_s0_upsizer (...);

// Protocol converters (as needed)
axi4_to_apb4 u_s2_apb_conv (...);

// Response routing
response_router u_resp_router (...);
```

```
// Monitor system (when variants include "mon")
axi4_master_rd_mon u_m0_rd_mon (...);    // Per-port monitor wrappers
axi4_master_wr_mon u_m0_wr_mon (...);
axi4_slave_rd_mon  u_s0_rd_mon (...);
axi4_slave_wr_mon  u_s0_wr_mon (...);
// ... (repeat for remaining ports)

// Monitor aggregation tree (if multiple monitors)
monbus_arbiter u_mon_arb0 (...);         // Aggregate master-side
streams
monbus_arbiter u_mon_arb1 (...);         // Aggregate slave-side
streams

// Monitor AXIL group at bridge top
monbus_axil4_axil4_group #(
    .FIFO_DEPTH_ERR      (64),
    .FIFO_DEPTH_WRITE    (96),   // in BEATS, not packets
    .ADDR_WIDTH          (32),
    .FLUSH_TIMEOUT_CYCLES(1024),
    .NUM_PROTOCOLS       (3),
    .USE_COMPRESSION     (0)
) u_mon_axil_group (
    .axi_aclk         (aclk),
    .axi_aresetn      (aresetn),
    // the monbus_arbiter's output is the group's single input
    .monbus_valid     (mon_arb_monbus_valid),
    .monbus_ready     (mon_arb_monbus_ready),
    .monbus_packet    (mon_arb_monbus_packet),
    .monbus_timestamp (mon_arb_monbus_timestamp),
    // free-running time, fanned out to every wrapper's i_mon_time
    .mon_time_out     (mon_time_w),
    .s_axil_*         (s_mon_axil_*),   // slave read port (CPU
access)
    .m_axil_*         (m_mon_axil_*),   // master write port (bulk
DMA)
    .irq_out          (mon_irq_out)
);
```

23.4.2 Internal Signals

```
// Crossbar internal signals
xbar_m{N}_aw_{signal}  - Master N to crossbar AW
xbar_m{N}_ar_{signal}  - Master N to crossbar AR
xbar_s{N}_aw_{signal}  - Crossbar to slave N AW
xbar_s{N}_ar_{signal}  - Crossbar to slave N AR

// Arbitration signals
```

```
grant_aw_s{N}[M-1:0]   - AW grants for slave N
grant_ar_s{N}[M-1:0]   - AR grants for slave N

// ID tracking signals
wr_fifo/rd_fifo[...]   - per-slave bridge_id FIFO (no ID table exists)
```

23.4.3 Reset Style and the Emitted Filelist

Every generated adapter, crossbar and slave adapter opens with

```
`include "reset_defs.svh"
```

and writes its flops through the macro, never a raw `always_ff`:

```
`ALWAYS_FF_RST(aclk, aresetn,
    if (`RST_ASSERTED(aresetn)) begin
        ...
    end else begin
        ...
    end
)
```

Reset is **asynchronous on assertion in every build**. `ALWAYS_FF_RST` used to be conditional on `USE_ASYNC_RESET` and defaulted to synchronous, so `make lint` (which set the define) and simulation/synthesis (which did not) disagreed about what the design was. The define is now a no-op; passing it is harmless and changes nothing. Deassertion must still be synchronised externally.

Because the emitted RTL depends on that header, the generator also emits the `-f` that supplies it, so a generated filelist resolves standalone:

```
# Include directories
+incdir+$REPO_ROOT/rtl/amba/includes

# Reset macro header (`ALWAYS_FF_RST / `RST_ASSERTED)
-f $REPO_ROOT/rtl/amba/filelists/reset_defs.f

# Monitor packages (must precede any module that references them)
-f $REPO_ROOT/rtl/amba/filelists/monitor_pkgs.f
```

The `-f` matters as much as the `+incdir`: the header must be *compiled* ahead of the modules that expand the macro, not merely be findable. A consumer that hand-lists the generated `.sv` files instead of taking this filelist gets `Cannot find include file: 'reset_defs.svh'` — see `/GLOBAL_REQUIREMENTS.md` on resolving sources through filelists.

23.4.4 Generated Variants

When the `variants` list in the bridge TOML includes multiple entries, the generator produces one complete `.sv` file per variant:

```
variants = ["no", "mon"]

Generated files:
  bridge_4x3.sv              (variants = "no", no monitor)
  bridge_4x3_mon.sv          (variants = "mon", with monitor)
```

Each variant is a complete, standalone bridge module. Both can coexist in the same design or be selected at compile time.

23.4.5 Generated File Structure

23.4.5.1 Single-File Output

```
bridge_{name}.sv
├── Module declaration
├── Parameter section
├── Port declarations
├── Internal signal declarations
├── Master adapter instances
├── Address decoder instances
├── Arbiter instances
├── Converter instances
├── Response router instance
└── Debug signals (optional)
```

23.4.5.2 Multi-File Output (Optional)

```
bridge_{name}/
├── bridge_{name}.sv        - Top-level wrapper
├── master_adapter_m0.sv    - Master 0 adapter
├── master_adapter_m1.sv    - Master 1 adapter
├── address_decoder.sv      - Shared decoder
├── arbiter_aw.sv           - AW arbiter
├── arbiter_ar.sv           - AR arbiter
├── width_upsize_64_512.sv  - Width converter
├── axi4_to_apb4.sv          - Protocol converter
└── response_router.sv      - Response routing
```

23.5 Related Modules

- Signal Naming - Detailed naming conventions

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

24 Signal Naming

24.1 Overview

Signal names in generated RTL follow one convention, which is what makes pattern-based verification and painless integration possible. Learn the pattern once and you can drive any generated port without opening the file.

24.2 Ports

24.2.1 External Port Naming

Pattern:

```
{prefix}_{channel}{signal}
```

Components:

Component	Description	Example
prefix	User-defined port prefix	cpu_m_axi, ddr_s_axi
channel	AXI4 channel code	aw, w, b, ar, r
signal	Signal within channel	valid, ready, addr, data

24.2.2 Master Port Examples

```
// Master with prefix "cpu_m_axi"
input  logic        cpu_m_axi_awvalid,
output logic        cpu_m_axi_awready,
input  logic [31:0] cpu_m_axi_awaddr,
input  logic [3:0]  cpu_m_axi_awid,
input  logic [7:0]  cpu_m_axi_awlen,
input  logic [2:0]  cpu_m_axi_awsize,
input  logic [1:0]  cpu_m_axi_awburst,
input  logic        cpu_m_axi_awlock,
input  logic [3:0]  cpu_m_axi_awcache,
input  logic [2:0]  cpu_m_axi_awprot,
input  logic [3:0]  cpu_m_axi_awqos,
input  logic        cpu_m_axi_awuser,
```

24.2.3 Slave Port Examples

```
// Slave with prefix "ddr_s_axi"
output logic        ddr_s_axi_awvalid,
input  logic        ddr_s_axi_awready,
output logic [31:0] ddr_s_axi_awaddr,
output logic [3:0]  ddr_s_axi_awid,   // SAME width as the master port
-- IDs are pass-through
output logic [7:0]  ddr_s_axi_awlen,
output logic [2:0]  ddr_s_axi_awsize,
// ...
```

24.2.4 APB Port Naming

Pattern:

`{prefix}p{signal}`

APB Signal Examples:

```
// APB slave with prefix "uart_apb_"
output logic        uart_apb_psel,
output logic        uart_apb_penable,
output logic [11:0] uart_apb_paddr,
output logic        uart_apb_pwrite,
output logic [31:0] uart_apb_pwdata,
output logic [3:0]  uart_apb_pstrb,
output logic [2:0]  uart_apb_pprot,
input  logic [31:0] uart_apb_prdata,
input  logic        uart_apb_pslverr,
input  logic        uart_apb_pready,
```

24.3 Functional Description

24.3.1 Internal Signal Naming

24.3.1.1 Crossbar Signals

```
// Master to crossbar
logic        xbar_m0_awvalid;
logic        xbar_m0_awready;
logic [31:0] xbar_m0_awaddr;

// Crossbar to slave
logic        xbar_s0_awvalid;
logic        xbar_s0_awready;
logic [31:0] xbar_s0_awaddr;
```

24.3.1.2 Arbitration Signals

```
// Request/grant matrices
logic [NUM_MASTERS-1:0] aw_request_s0;  // AW requests to slave 0
logic [NUM_MASTERS-1:0] aw_grant_s0;    // AW grants from slave 0

// Arbiter state
logic aw_grant_active_s0;               // Grant locked
logic [$clog2(NUM_MASTERS)-1:0] aw_last_grant_s0;  // Round-robin
state
```

24.3.1.3 ID Signals

```
// bridge_id: an INTERNAL routing tag carried alongside the
transaction,
// never prepended to the AXI ID on the slave port
logic [TOTAL_ID_WIDTH-1:0] xbar_m0_awid;  // BID + external ID
logic [TOTAL_ID_WIDTH-1:0] xbar_s0_bid;   // Response with BID

// External IDs (to master)
logic [ID_WIDTH-1:0] cpu_m_axi_bid;       // ID passes through
unchanged
```

24.3.2 Debug Signal Naming

Pattern:

```
dbg_{category}_{description}   // NOT EMITTED -- see below
```

One caveat before you go looking for these in a waveform: they're illustrative only. The generator emits no `dbg_*` ports — the debug guide covers what you can actually reach.

```
// Arbitration debug
output logic [NUM_MASTERS-1:0] dbg_aw_grant_s0,   // illustrative only
```

```
output logic [NUM_MASTERS-1:0] dbg_ar_grant_s0,

// Transaction debug
output logic [7:0] dbg_outstanding_m0,
output logic [7:0] dbg_outstanding_m1,

// State machine debug
output logic [1:0] dbg_aw_state_s0,
output logic [1:0] dbg_ar_state_s0,
```

24.3.3 Verification Pattern Matching

24.3.3.1 Factory Pattern Support

The naming convention is what makes automatic BFM connection work:

```
# CocoTB/GAXI pattern matching
master = AXI4Master(
    dut=dut,
    prefix="cpu_m_axi_",  # Matches all cpu_m_axi_* signals
    clock=dut.aclk
)
```

24.3.3.2 Pattern Rules

1. Prefix must end with underscore or be directly followed by channel code
2. Channel codes are lowercase (aw, w, b, ar, r)
3. Signal names match AXI4 specification (valid, ready, addr, data, etc.)

24.4 Related Modules

- Module Structure - Overall RTL organization
- Verification - Pattern-based testing

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

25 Test Strategy

25.1 Overview

Bridge verification is CocoTB-based and parameterized: the same infrastructure covers every bridge topology and protocol combination.

25.2 Related Modules

- Debug Guide - Debugging failed tests
- Signal Naming - Pattern matching for BFMs

25.3 Testing

25.3.1 Test Categories

25.3.1.1 Unit Tests

Per-block verification:

```
Unit Test Coverage:
├── Master Adapter Tests
│   ├── bridge_id sideband tagging (IDs pass through unchanged)
│   ├── Channel routing
│   └── Backpressure handling
├── Address Decoder Tests
│   ├── Address mapping
│   ├── Multi-region support
│   └── Default slave routing
├── Arbiter Tests
│   ├── Round-robin fairness
│   ├── Grant/lock behavior
│   └── Multi-master contention
└── Converter Tests
    ├── Width up/down sizing
    ├── APB protocol conversion
    └── Burst handling
```

25.3.1.2 Integration Tests

Full bridge verification:

```
Integration Test Matrix:
├── 2x2 Configuration
│   ├── Basic read/write
│   ├── Concurrent transactions
│   └── Same-ID aliasing across masters (bridge_id keeps them apart)
├── 4x4 Configuration
│   ├── Full arbitration coverage
│   ├── Width conversion paths
│   └── Mixed protocol targets
└── NxM Configurations
    ├── Asymmetric topologies
    ├── Channel-specific masters
    └── Protocol mixing
```

25.3.2 Test Parameterization

25.3.2.1 Configuration Matrix

Which parameters get swept, and why:

Table 7.1: Test Parameter Matrix

Parameter	Test Values	Purpose
NUM_MASTERS	2, 4, 8	Scalability
NUM_SLAVES	2, 3, 4	Routing coverage
DATA_WIDTH	32, 64, 128, 256	Width paths
ID_WIDTH	4, 6, 8	ID space
OUTSTANDING	4, 8, 16	Depth stress

25.3.2.2 Protocol Combinations

```
# Test protocol matrix
PROTOCOL_COMBOS = [
    {"masters": ["axi4"], "slaves": ["axi4"]},
    {"masters": ["axi4"], "slaves": ["axi4", "apb"]},
    {"masters": ["axi4", "axil"], "slaves": ["axi4", "apb"]},
    {"masters": ["axi5"], "slaves": ["axi4", "axi5"]},         #
interop + native
    {"masters": ["axi5", "axi5"], "slaves": ["axi4", "axi5"]}, #
sideband through the arbiter
]
```

The AMBA5 fixtures in `bin/test_configs/`, each generated with its own tests and TB class:

Table 7.1a: AMBA5 Fixtures

Fixture	Shape	What it exercises
`bridge_1x2_{rd,wr}_axi5`	AXI5 master, AXI4 slaves	interop boundary; sideband terminates
`bridge_1x2_rd_axi5s`	AXI4 master, one AXI5 slave	AXI5 slave ports
`bridge_1x2_{rd,wr}_axi5n`	AXI5 both ends	native sideband values, poison (wr)
`bridge_1x2_wr_axi5a`	AXI5 both ends, `atomic`, write-only	store-class forwards; load/swap/compare DECERR at the filter
`bridge_1x2_rw_axi5a`	AXI5 both ends, `atomic`, rw	read-return atomics native: R data routed back by ID
`bridge_1x2_rw_axil5`, `bridge_1x2_rw_apb5`	AXI4 master, AXI5-Lite / APB5 slave	the Lite and APB5 shims

Fixture	Shape	What it exercises
`bridge_2x2_axi5`	two AXI5 masters, AXI5 + AXI4 slaves	sideband through arbitration
`bridge_1x2_rd_axi5w`	AXI5 32b master, 64b AXI4 + 32b AXI5 slaves	sideband across a width converter
`bridge_2x2_lite_req`	AXI4-Lite + AXI5-Lite masters, 64b AXI4 + AXI5-Lite slaves	Lite requesters (bridge TASK-004, was BRIDGE-014): sideband forwarding, the aligner, ID-less masters sharing a slave
`bridge_2x3_apb_req`	APB4 + APB5 masters, AXI4 + AXI5-Lite + APB4 slaves	APB requesters (bridge TASK-004, was BRIDGE-014): `apb{4,5}_to_axi4`, PSLVERR folding, APB in / APB out
`bridge_2x2_wb4`	WB4 + AXI4 masters, 64b AXI4 + WB4 slaves	Wishbone on both sides (bridge TASK-008, was BRIDGE-019): ERR folding both ways, SEL lanes, burst decomposition at the completer, two requesters at one Wishbone completer
`bridge_2x2_rw` (perf)	two 32b AXI4 masters, two 32b AXI4 slaves	`test_bridge_2x2_rw_perf` (bridge TASK-006, was BRIDGE-017): saturating read/write streams, one shared port with per-master shares, two parallel paths, loaded AW-to-B latency; every figure windowed and floored (HAS Tables 5.1a / 5.7a)
`bridge_2x2_rw_pipe`	as `bridge_2x2_rw` with	generated suite plus

Fixture	Shape	What it exercises
	`xbar_pipeline = true`	`test_bridge_2x2_rw_pipe_latency` (bridge TASK-006, was BRIDGE-017): asserts the registered crossbar adds exactly the documented stages on AW, AR and B/R (HAS Table 5.7a); `test_bridge_2x2_rw_perf` runs on it too (`BRIDGE_PERF_DUT`) to show the throughput floors still hold
`bridge_2x2_rw_qos`	as `bridge_2x2_rw` with `arbitration = "qos"`, `qos_aging_shift = 4`	generated suite plus `test_bridge_2x2_rw_qos_arb` (bridge TASK-006, was BRIDGE-017): under saturation the higher-AxQOS master takes >= 0.75 of a shared port in both orientations, the QoS-0 master's worst wait stays under the aging bound (`15 * 2**shift` plus a burst), equal QoS splits fairly, and the port stays >= 0.95 beat/cycle (MAS 2.3 policy paragraph)
`bridge_2x2_rw_qos_pipe`	`bridge_2x2_rw_qos` on the registered crossbar (`xbar_pipeline = true` as well)	generated suite; exists for the synthesis sweep (HAS 6.4), which showed the QoS compare, grant, mux and CAM allocate in one path and the crossbar's slave-side skid as the register that splits

Fixture	Shape	What it exercises
		it
`bridge_2x2_axi5_native`	two 128-bit AXI5 masters, two 128-bit AXI5 slaves, every port nsaid/trace/unique/poison/mte/chunking	generated suite with the AXI5 compliance checkers, plus `test_bridge_2x2_axi5_native_mte_chunk` (bridge TASK-007, was BRIDGE-018): Transfer writes from both masters land their per-beat tags in the slave BFM's tag store, Transfer reads return them, Match writes report BTAGMATCH right for matching and mismatching tags, Update writes follow WTAGUPDATE, chunked reads carry RCHUNKV/RCHUNKNUM per beat and unchunked reads do not; with the slave BFM emitting each burst's chunks in reverse, every burst still completes and reassembles by RCHUNKNUM – the fabric is order-agnostic
`bridge_1x2_rd_axi5c`	128-bit AXI5 chunking master, one AXI5 chunking slave and one AXI4 slave	generated suite plus `test_bridge_1x2_rd_axi5c_chunk` (bridge TASK-007, was BRIDGE-018): the chunk enable reaches the AXI5 port and its beats come back

Fixture	Shape	What it exercises
		chunked; to the AXI4 slave the enable terminates and the same read returns ordered data with RCHUNKV low
`bridge_2x2_rw_cdc`	as `bridge_2x2_rw`, `ddr` on its own clock (`cdc = true`), `sram` same-clock	generated suite on the CDC port at `BRIDGE_CDC_PERIOD_NS` (7 ns); `test_bridge_2x2_rw_cdc_ratio` (bridge TASK-006, was BRIDGE-017) sweeps 3/10/23 ns: both masters' streams land and read back, the port's beats are counted on the slave clock, the rate follows the slower clock (>= 0.85 beat/cycle), the same-clock neighbour is unaffected (HAS 4.5a)

25.3.3 Protocol-BFM-Only Testing with Memory-Backed Slaves

Modern bridge tests use **protocol BFMs only** (no direct DUT signal manipulation) with **memory-backed slave models**. If you find yourself poking a DUT signal in a test, stop — drive the protocol and check the memory instead. That includes AMBA5 sideband and atomics: the AXI5 BFMs take `nsaid`/`trace`/`unique`/`poison`/`atop` as transaction keyword arguments (`write_transaction`, `read_transaction`, `atomic_operation`), and the generated TB returns the echoed `trace` in the BFM's result.

The generated TB picks the BFM family from each port's protocol: AXI4 ports get the AXI4 BFMs, `axi5` ports the AXI5 BFMs (which declare every AMBA5 sideband field as optional, so one BFM binds to any feature subset), `axil5` ports the AXIL5 BFMs (`AXIL5Master*` on a master port), `apb5` slave ports `APB5Slave` and `apb`/`apb5` master ports `APBMaster`/`APB5Master`, all at the generator's 1-bit USER widths, and `wb4` ports `WB4Slave` (base-relative memory, out-of-range answers ERR) / `WB4Master`. Every AXI5 master port also gets an `AXI5ComplianceChecker` on the same prefix, armed in `setup_clocks_and_reset` and read by `tb.assert_compliance()`, which every generated test calls before it declares PASSED – a protocol violation on the AXI5 boundary fails the test that caused it even when the data still round-tripped.

Slave memory models are capped at 64 KB per slave (`SLAVE_MEM_CAP_BYTES`). A probe past the model is answered by the one out-of-range contract every slave BFM follows (RDS-DV `shared/memory_model.py`): SLVERR, nothing written, `0xDEADDEAD` read data, one warning. The boundary probe swallows that error only for probes past the seeded region – the error coming back from the slave the address decodes to is the routing evidence – and `master_write` raises on any error response, so a SLVERR write never passes through a helper unnoticed. The model's limit is not the design's: an address the bridge does not decode at all is the subtractive slave's DECERR.

25.3.3.1 BFM Instantiation and Slave Models

```
from CocoTBFramework.components.axi4 import AXI4Master, AXI4Slave
from TBClasses.shared.tbbase import TBBase
from TBClasses.shared.memory_model import MemoryModel

class BridgeTB(TBBase):
    def __init__(self, dut):
        super().__init__(dut)

        # Create masters using protocol BFMs
        self.masters = []
        for i in range(NUM_MASTERS):
            prefix = f"m{i}_axi_"
            master = AXI4Master(
                dut=dut,
                prefix=prefix,
                clock=dut.aclk,
                reset=dut.aresetn
            )
            self.masters.append(master)

        # Create slave memory models (NOT direct signal manipulation)
        self.slaves = []
        for i in range(NUM_SLAVES):
            prefix = f"s{i}_axi_"
            slave = MemoryModel(
                dut=dut,
                prefix=prefix,
                clock=dut.aclk,
                reset=dut.aresetn,
                size=0x100000  # 1 MB memory per slave
            )
            self.slaves.append(slave)
```

25.3.3.2 Transaction Generation and Assertion

```
async def cocotb_test_concurrent_writes(dut):
    """Test multiple masters writing simultaneously."""
    tb = BridgeTB(dut)
    await tb.setup_clocks_and_reset()
```

```
    # Generate transactions via BFMs only
    tasks = []
    for i, master in enumerate(tb.masters):
        addr = 0x1000 + (i * 0x100)
        data = 0xDEAD0000 + i
        # Drive via protocol BFM (no DUT signal poke)
        task = cocotb.start_soon(
            master.write(addr=addr, data=data, size=2)
        )
        tasks.append(task)

    # Wait for all writes to complete
    for task in tasks:
        await task

    # Verify via memory readback (not via routing logic check)
    for i, master in enumerate(tb.masters):
        addr = 0x1000 + (i * 0x100)
        readback = await master.read(addr=addr)
        assert readback == 0xDEAD0000 + i, \
            f"Master {i} readback mismatch at {addr:#x}"
```

25.3.3.3 Boundary Probe Testing

Address-window boundary probes catch address-decode errors at slave page edges:

```
async def test_boundary_probes(self):
    """Probe top, middle, bottom of each slave's address window."""

    for slave_idx, (base_addr, window_size) in
enumerate(self.slave_windows):
        # Payloads carry the slave index and the probe position, so a
readback
        # mismatch says BOTH which probe failed and whether the write
landed on
        # the wrong slave -- the failure a boundary probe is actually
hunting.
        bottom, middle, top = (0xB0000000 | (slave_idx << 8) | pos
                               for pos in (0x0, 0x1, 0x2))

        # Probe bottom (base address)
        await self.masters[0].write(base_addr + 0x000, data=bottom)

        # Probe middle (arbitrary offset)
        await self.masters[0].write(base_addr + window_size // 2,
data=middle)
```

```
        # Probe top (last valid address)
        await self.masters[0].write(base_addr + window_size - 1,
data=top)

        # Verify via readback
        rb_bottom = await self.masters[0].read(base_addr + 0x000)
        rb_middle = await self.masters[0].read(base_addr + window_size
// 2)
        rb_top = await self.masters[0].read(base_addr + window_size -
1)

        assert rb_bottom == bottom
        assert rb_middle == middle
        assert rb_top == top
```

25.3.4 Coverage Goals

25.3.4.1 Functional Coverage

```
Coverage Targets:
├── Protocol Coverage
│   ├── All burst types (FIXED, INCR, WRAP)
│   ├── All sizes (1, 2, 4, 8, ... bytes)
│   └── All response types
├── Routing Coverage
│   ├── Every master to every slave path
│   ├── Default slave routing
│   └── Error response paths
├── Arbitration Coverage
│   ├── All grant combinations
│   ├── Priority scenarios
│   └── Lock sequences
└── Corner Cases
    ├── Maximum outstanding
    ├── ID exhaustion
    └── Backpressure saturation
```

25.3.4.2 Code Coverage

```
Code Coverage Targets:
├── Line Coverage: >95%
├── Branch Coverage: >90%
├── FSM State Coverage: 100%
└── Toggle Coverage: >85%
```

25.3.5 Test Execution

25.3.5.1 Running the Suite

The bridge runs through the same four-line Makefile every val area uses (`make/tests.mk`): always a clean build first, then one of three levels.

```
cd projects/components/fabric-gen-ip/bridge/dv/tests
make clean-all && make run-all-gate-parallel        # smoke: 72 cells
make clean-all && make run-all-func-parallel        # development: 144
cells
make clean-all && make run-all-full-parallel        # sign-off: 216
cells
make clean-all && make run-all-full-parallel-waves  # same, with
dump.fst per cell
make run-bridge_2x2_rw-gate-parallel               # one file, by its
glob stem
```

`REG_LEVEL` (set by the target) selects the grid in each wrapper: GATE runs one `gate` cell per test, FUNC `gate+func`, FULL `gate+func+full`. Each cell exports `TEST_LEVEL` and a `SEED` (pinned per test node, so a rerun replays the same run) and the TB scales its work from the level profile in `dv/tbclasses/bridge_levels.py`. Sibling cells of one test log different `TEST_LEVEL=<x>` banners and different wall-clock – that is the evidence the grid is real. A raw `pytest` run works but is the FUNC subset with no clean, and a sub-second "passed" on a stale build is a fiction.

25.3.5.2 Test Naming Convention

Cocotb test functions follow the pattern: `cocotb_test_{N}x{M}_{variant}_{scenario}`

```
# Example: test_bridge_4x4_mon_capture.py
@cocotb.test()
async def cocotb_test_4x4_mon_capture(dut):
    """Monitor packet capture on 4x4 bridge with monitor enabled."""
    ...

# Pytest wrapper function
def test_bridge_4x4_mon_capture(request):
    """Wrapper to invoke cocotb_test_4x4_mon_capture."""
    run(
        python_search=[tests_dir],
        verilog_sources=verilog_sources,
        toplevel="bridge_4x4_mon",
        module=module,
        testcase="cocotb_test_4x4_mon_capture",
        ...
    )
```

25.3.5.3 Test Levels

What each depth does, per the profile in `bridge_levels.py`. Every count in the suite is read from here; nothing hardcodes one.

Table 7.2: Test Level Depth Profile

Level	Connectivity	Boundary probe	Arbitration	Monitor stress	Slaves
gate	1 offset/pair	boundary pages, low offset only	4 txn/master	128 reads/phase	prompt
func	4 offsets/pair	boundary pages, low/mid/high	8 txn/master	256 reads/phase	prompt
full	16 offsets/pair	every seeded page, low/mid/high	24 txn/master	512 reads/phase	24-cycle response delay

Measured on the 4x4 boundary probe: 4 s, 5 s and 40 s per cell. The FULL run's monitor stress count of 128 at gate is deliberate – the monbus err FIFO is 64 deep and the ERR_BP phase asserts it saturates, which at exactly 64 reads was a race against the drain pump.

Monitor Tests (when `variants` includes `"mon"`): - `test_bridge_1x2_rd_monitor_smoke`: Basic packet emission - `test_bridge_1x2_rd_monitor_capture`: Packet parsing and verification - `test_bridge_1x2_rd_monitor_error_inject`: SLVERR packet collection - `test_bridge_1x2_rd_monitor_irq`: IRQ assertion on error FIFO

26 Debug Guide

26.1 Overview

When a bridge test fails, start here. This guide lists the failure modes that actually occur and how to chase each one down.

26.2 Waveforms

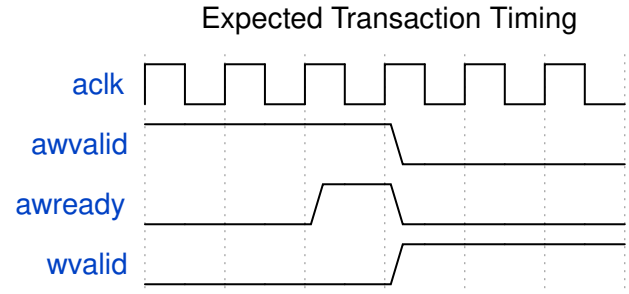
26.2.1 Generating Waveforms

```
# Enable VCD dump
WAVES=1 pytest test_bridge_2x2.py::test_basic -v

# View with GTKWave
gtkwave sim_build/dump.vcd
```

26.2.2 Key Signals to Observe

```
Waveform Signal Groups:
├── Clock/Reset
│   └── aclk, aresetn
├── Master 0 AW Channel
│   └── m0_axi_awvalid, m0_axi_awready, m0_axi_awaddr, m0_axi_awid
├── Slave 0 AW Channel
│   └── s0_axi_awvalid, s0_axi_awready, s0_axi_awaddr, s0_axi_awid
├── Arbitration (inside u_xbar)
│   └── ddr_aw_arb_gnt, ddr_aw_arb_locked, ddr_ar_arb_gnt,
ddr_ar_arb_locked
├── Target gating (inside the master adapter)
│   └── aw_gate_ok, ar_gate_ok, r_aw_active_target
└── Response
    └── s0_axi_bvalid, m0_axi_bvalid, ddr_axi_bid_bridge_id
```



Timing Analysis

26.3 Related Modules

- Test Strategy - Overall test approach
- Arbiter FSMs - FSM state details
- ID Tracking - ID table operation

26.4 Testing

26.4.1 Debug Signal Access

26.4.1.1 There are no dbg_* ports

The generator emits no debug ports. Earlier revisions of this chapter showed `dbg_aw_grant_s0`, `dbg_outstanding_m0`, `dbg_resp_master` and a `dbg_id_table_*` CAM view; none of them exist in any generated module, and every procedure written against them fails immediately with a cocotb `AttributeError`.

Debug instead by reaching into the real internal signals by hierarchical name. They are named after the SLAVE (`<slave>_...`) in the xbar and are plain `logic`, so cocotb can read them directly:

```
@cocotb.test()
async def test_with_debug(dut):
    xbar = dut.u_xbar                       # instance name in the
generated top
    grant   = xbar.ddr_aw_arb_gnt.value   # which master holds the AW
grant
    locked  = xbar.ddr_aw_arb_locked.value
    rr      = xbar.ddr_aw_arb_rr.value    # round-robin pointer
    routed  = xbar.ddr_axi_rid_bridge_id.value  # bridge_id the R beat
returns to
```

Table 7.3: Debug Signals That Exist

Category	Real signals	Purpose
Arbitration	`<slave>_aw_arb_gnt`, `_locked`, `_rr`, `_req`, `_pick`	Grant, lock and round-robin state, per slave, AW and AR
Response routing	`<slave>_axi_bid_bridge_id`, `<slave>_axi_rid_bridge_id`	The `bridge_id` a B/R beat is muxed to. This is the routing decision – the AXI ID is not used
Outstanding tracking	`aw_trk_wptr` / `aw_trk_rptr`, `ar_trk_*` (in the master adapter)	Depth of the in-order tracking FIFO
Target gating	`aw_gate_ok` / `ar_gate_ok`, `r_aw_active_target` (adapter)	Why a new AW/AR to a different slave is being held off

There is no outstanding-count output and no FSM state vector – the arbiters are two-state (`locked` / not) rather than an encoded FSM, so `_locked` plus `_gnt` is the whole state.

26.4.2 Common Issues

26.4.2.1 Issue: Transaction Timeout

Symptoms: - Test hangs waiting for response - `TimeoutError` after configured timeout

Debug Steps:

```
# 1. Check if request was accepted
await RisingEdge(dut.aclk)
print(f"AWVALID: {dut.m0_axi_awvalid.value}")
print(f"AWREADY: {dut.m0_axi_awready.value}")

# 2. Check arbitration state
print(f"AW grant (ddr): {dut.u_xbar.ddr_aw_arb_gnt.value}")
print(f"AW locked     : {dut.u_xbar.ddr_aw_arb_locked.value}")

# 3. Check the tracking FIFO and the single-outstanding-target gate
print(f"AW trk wptr/rptr: {dut.u_cpu_adapter.aw_trk_wptr.value}"
      f"/{dut.u_cpu_adapter.aw_trk_rptr.value}")
print(f"aw_gate_ok      : {dut.u_cpu_adapter.aw_gate_ok.value}")

# 4. Check slave response
print(f"BVALID: {dut.s0_axi_bvalid.value}")
print(f"BREADY: {dut.s0_axi_bready.value}")
```

Common Causes: - Arbiter locked by another master (`_arb_locked` high for someone else) - **A different slave is still outstanding.** `aw_gate_ok` low means the adapter is holding AWREADY down because every outstanding write must target one slave at a time. This is by design, not a fault – see the single-outstanding-target note in the PRD. - Slave not responding - Tracking FIFO full (`aw_trk_wptr` has lapped `aw_trk_rptr`)

26.4.2.2 Issue: Wrong Response Routing

Symptoms: - Response arrives at wrong master - BID mismatch errors

Debug Steps:

```
# The AXI ID passes through UNCHANGED -- there is no ID extension.
print(f"Master AWID : {dut.cpu_m_axi_awid.value}")
print(f"Slave  AWID : {dut.ddr_s_axi_awid.value}")   # same value

# Routing is by the bridge_id sideband, not the ID:
print(f"B routed to bridge_id:
{dut.u_xbar.ddr_axi_bid_bridge_id.value}")
```

```
print(f"R routed to bridge_id:
{dut.u_xbar.ddr_axi_rid_bridge_id.value}")
```

Common Causes: - **The slave returned responses out of order.** Routing pops an in-order FIFO, so a reordering slave sends one master's beats to another. Nothing detects it. This is the single most likely cause and it is a design constraint, not a bug in the bridge – see FR-2 in the PRD. - ID width mismatch between master and slave ports - (Historical, before bridge TASK-005 (was BRIDGE-016)) Two masters using the same AXI ID to one slave: their IDs aliased at the slave, and only the `bridge_id` sideband keeps the responses apart

26.4.2.3 Issue: Data Corruption

Symptoms: - Read data doesn't match written data - WSTRB/data alignment issues

Debug Steps:

```
# Check width conversion
# There are no M0_DATA_WIDTH / S0_DATA_WIDTH parameters: the generated
top
# has NO parameter list at all and widths are baked into the port
# declarations. Read the width off a port instead:
print(f"Master data width: {len(dut.cpu_m_axi_wdata.value)}")
print(f"Slave data width:  {len(dut.ddr_s_axi_wdata.value)}")

# Check strobe packing
print(f"Master WSTRB: {dut.m0_axi_wstrb.value}")
print(f"Slave WSTRB: {dut.s0_axi_wstrb.value}")

# Check address alignment
print(f"AWADDR: {dut.m0_axi_awaddr.value}")
print(f"AWSIZE: {dut.m0_axi_awsize.value}")
```

Common Causes: - Width converter byte lane error - Unaligned access handling - Burst boundary crossing

26.4.3 Assertion Failures

26.4.3.1 There are no built-in assertions

The generated RTL contains no `assert` statements of any kind – not in the top, the xbar or any adapter. An earlier revision of this section showed two protocol assertions as though they shipped; they never existed, and the signals they referenced (`s_awvalid`, `r_current_awid`) match no identifier in the generated design.

Protocol checking is the testbench's job here. Use the AXI4 BFMs and the monitor wrappers rather than expecting the DUT to flag a violation itself.

26.4.3.2 Handling Assertion Failures

1. **Locate assertion in RTL** - Search for error message

2. **Check signal history** - Review 10-20 cycles before failure
3. **Identify root cause** - Usually protocol or timing issue
4. **Fix test or RTL** - Depending on where bug exists

26.4.4 Performance Debugging

26.4.4.1 Throughput Issues

```
# Measure transaction throughput
start_time = cocotb.utils.get_sim_time('ns')

for i in range(100):
    await master.write(addr=0x1000 + i*4, data=[i])

end_time = cocotb.utils.get_sim_time('ns')
throughput = 100 / (end_time - start_time) * 1e9  # tx/sec
print(f"Throughput: {throughput:.2f} transactions/sec")
```

26.4.4.2 Latency Issues

```
# Measure single transaction latency
start = cocotb.utils.get_sim_time('ns')
await master.read(addr=0x1000)
latency = cocotb.utils.get_sim_time('ns') - start
print(f"Read latency: {latency} ns")
```